

HELPER WEB BOOK · FRONT-END + BACK-END · FOR ABSOLUTE
BEGINNERS & JUNIORS

Full-Stack Web Development The Complete Explained Guide

**HTML · CSS · JavaScript · Git · Bootstrap · Node · Express ·
SQL · Auth · React · Deploy · Roadmap**

Author: Mahmoud Mohamed Mahmoud Khattab

Phone: +20 114 122 5852 (201141225852)

Structured to follow the lesson order of:

- **Abdelrahman Gamal** — كامل بالعربي HTML & CSS كورس (81 videos)
- **Elzero Web School** — Learn HTML in One Video (4 h) · Learn CSS in One Video (11 h) · Learn JavaScript in One Video, Parts One & Two
- **Elzero Web School** — Learn CSS in Arabic (88 videos) · Learn JavaScript in Arabic (188 videos)
- **Abdelrahman Gamal** — React JS كامل بالعربي · كورس JavaScript كورس
- **Dr. Angela Yu** — The Complete Full-Stack Web Development Bootcamp (curriculum order)

How to use this book

- Badges tell you where to watch: **Gamal #** **Elzero** **Angela**
- Every example: code → result → → arrow notes that explain each piece of syntax.
- Start with the Roadmap chapter, then follow the parts in order. Type every example yourself.

Contents

1. [How to Read This Book](#)
2. [Before You Start: What Is the Web?](#)
3. [How to Study This Book — 16-Week Plan with the Video Courses](#)
4. [The Full-Stack Roadmap](#)
5. [Fundamentals Reference — How the Internet and the Browser Really Work](#)
6. [Fundamentals Reference — The Command Line \(Bash\) for Web Developers](#)
7. [Fundamentals Reference — DevTools Mastery and Debugging Workflow](#)
8. [Fundamentals Reference — Accessibility, SEO and Web Performance Basics](#)
9. [CSS Layout Patterns Gallery — 15 Layouts You Will Build Again and Again](#)
10. [What is HTML? How the Web Works](#)
11. [Creating website files & the skeleton](#)
12. [Tags, Elements & Attributes — Anatomy](#)
13. [The <head> section](#)
14. [Headings & Paragraphs](#)
15. [Comments](#)
16. [Text Formatting Tags](#)
17. [Images](#)
18. [Audio & Video](#)
19. [Tables](#)
20. [Lists](#)
21. [Semantic Elements](#)
22. [Links](#)
23. [Forms & Inputs](#)
24. [Radio, Checkbox, Select, Datalist, Textarea](#)
25. [Buttons & Validation](#)
26. [iframe vs embed vs object](#)
27. [Downloads, Entities & Block vs Inline](#)
28. [First HTML Project — Personal Page](#)
29. [HTML Reference A — Document Structure Tags](#)
30. [HTML Reference B — Text Content Tags](#)
31. [HTML Reference C — Inline Text Formatting Tags](#)
32. [HTML Reference D — Links, Images, Audio and Video](#)
33. [HTML Reference E — Tables](#)
34. [HTML Reference F — Forms and Every Input Type](#)

35. [HTML Reference G — Semantic Layout and Interactive Tags](#)
36. [What is CSS? Three ways to add it](#)
37. [CSS Syntax Anatomy & Priority](#)
38. [Selectors — All Kinds](#)
39. [The Box Model](#)
40. [Sizing: width, height, min/max, %, fit-content, border-radius](#)
41. [display & Hiding Elements](#)
42. [Text & Font Properties](#)
43. [Colors, Units, Backgrounds & Gradients](#)
44. [float, overflow, position, z-index](#)
45. [Pseudo-classes, Pseudo-elements, list & table style](#)
46. [transform, transition, animation](#)
47. [Flexbox — Complete](#)
48. [CSS Grid — Complete](#)
49. [Responsive Design & Media Queries](#)
50. [Bootstrap 5 Essentials](#)
51. [CSS Reference A — Getting Started, Syntax, Selectors, Cascade](#)
52. [CSS Reference B — Colours, Units and Backgrounds](#)
53. [CSS Reference C — The Box Model: Sizing, Padding, Border, Margin](#)
54. [CSS Reference D — Text and Fonts](#)
55. [CSS Reference E — Positioning: float, position, z-index](#)
56. [CSS Reference F — Pseudo-classes and Pseudo-elements](#)
57. [CSS Reference G — Transforms, Transitions and Animations](#)
58. [CSS Reference H — Flexbox, Complete](#)
59. [CSS Reference I — Grid, Complete](#)
60. [CSS Reference J — Responsive Design and Media Queries](#)
61. [JavaScript Basics: Variables, Types, Operators](#)
62. [Functions, Conditions, Loops](#)
63. [Arrays & Objects](#)
64. [DOM Manipulation & Events](#)
65. [ES6+, Async, fetch & JSON](#)
66. [jQuery — Quick Reference](#)
67. [Command Line, Git & GitHub](#)
68. [JavaScript Reference A — First Steps, Output, Data Types, Variables](#)
69. [JavaScript Reference B — Control Flow: Conditions and Loops](#)
70. [JavaScript Reference C — Functions](#)
71. [JavaScript Reference D — Strings, Numbers and Dates in Depth](#)

72. [JavaScript Reference E — Arrays and Objects in Depth](#)
73. [JavaScript Reference F — The DOM: Selecting, Changing, Creating](#)
74. [JavaScript Reference G — Events](#)
75. [JavaScript Reference H — Asynchronous JavaScript, fetch and APIs](#)
76. [JavaScript Reference I — Modern JavaScript: Classes, Modules, Browser APIs, Patterns](#)
77. [How Back-end Works: Node.js & NPM](#)
78. [Express.js — Server & Routing](#)
79. [EJS Templating](#)
80. [REST APIs — Build & Consume](#)
81. [Databases: SQL & PostgreSQL](#)
82. [Authentication & Security](#)
83. [Back-End Reference A — Node.js and npm from Zero](#)
84. [Back-End Reference B — Express.js Complete](#)
85. [Hello from Express](#)
86. [Back-End Reference C — EJS Templates and Server-Rendered Pages](#)
87. [Back-End Reference D — REST APIs and HTTP in Depth](#)
88. [Back-End Reference E — SQL and PostgreSQL](#)
89. [Back-End Reference F — Authentication and Security](#)
90. [React — Fundamentals & Hooks](#)
91. [Deployment](#)
92. [Full-Stack Capstone Project Plan](#)
93. [Cheat Sheets](#)
94. [React Reference A — Thinking in Components, JSX, Props](#)
95. [React Reference B — State, Events, Forms and Hooks](#)
96. [React Reference C — Routing, Data, Structure and a Full App](#)
97. [Walkthrough 1 — Building a Complete Restaurant Website Line by Line \(HTML + CSS\)](#)
98. [Walkthrough 2 — Building a Quiz App Step by Step \(JavaScript\)](#)
99. [Walkthrough 3 — Full-Stack Notes App: Express API + PostgreSQL + React, Step by Step](#)
100. [JavaScript Mini-Projects Gallery — 15 Small Apps with Full Code](#)
101. [HTML & CSS Component Cookbook — 20 UI Pieces with Code and Result](#)
102. [Tools Reference A — Git and GitHub Complete](#)
103. [Tools Reference B — Bootstrap 5 Complete](#)
104. [Tools Reference C — Deployment, Domains and Going Live](#)
105. [Projects Workbook — 12 Guided Projects from Beginner to Full Stack](#)
106. [Career Kit — Interview Questions, Cheat Sheets and What Next](#)
107. [Exercise Workbook A — HTML Drills \(30 exercises with solutions\)](#)
108. [Exercise Workbook B — CSS Drills \(30 exercises with solutions\)](#)

- 109. [Exercise Workbook C — JavaScript Drills \(40 exercises with solutions\)](#)
- 110. [Exercise Workbook D — SQL and Back-End Drills \(20 exercises with solutions\)](#)
- 111. [Glossary — 128 Terms Explained Simply \(English + Arabic\)](#)

1. How to Read This Book

This book is written for two kinds of readers:

- **Absolute beginners** — you have never written a line of code and maybe do not know what a "browser" or a "server" really is. Start from the very first page and do not skip anything.
- **Junior developers** — you know some HTML/CSS and want one clear, complete reference that goes all the way to the back end. Use the Table of Contents and the Roadmap, then jump to what you need.

The elements you will see on every page

Gamal #3 **Elzero CSS #07** **Angela §5**

→ **Video badges** tell you which lesson to watch for this topic: **red** = Abdelrahman Gamal playlist, **purple** = Elzero Web School, **violet** = Angela Yu's bootcamp section. Read the chapter, watch the video, then come back and do the exercises.

CODE — WHAT YOU TYPE

```
<p>Hello</p>
```

RESULT — WHAT YOU SEE IN THE BROWSER

Hello

→ Code is always on the left (dark box) and the browser result on the right (dashed box). Under them you will find **arrow notes (→)** that explain every piece of the code one by one.

Try it yourself

Blue boxes are exercises. Do them. Reading alone does not teach programming — typing does.

Solution — purple boxes give one correct answer. Compare with yours after you try.

Common mistake

Red boxes show errors beginners make, what they look like, and how to fix them.

? Quick check

Orange boxes are short quizzes. Answers are underneath, upside down in spirit — do not peek first.

Green boxes are tips and best practices from professionals.

Yellow boxes are important notes you must remember.

Red-bordered boxes are warnings about security or things that break your page.

A note about the symbols < and > and &

In HTML, the characters < (less than), > (greater than) and & (ampersand) have special jobs: they build tags. So when you want to display them as normal text on a page, you must write a special code called an **entity**:

You want to show this on the page	So you type this in your HTML	Name
<	<code>&lt;</code>	"less than"
>	<code>&gt;</code>	"greater than"
&	<code>&amp;</code>	"ampersand"
"	<code>&quot;</code>	"quotation"
©	<code>&copy;</code>	"copyright"

→ Special characters like the less-than sign have a typed code (shown in the table above). You only need those codes inside your HTML files; this book itself always uses normal English words.

2. Before You Start: What Is the Web?

Gamal #1

Elzero HTML 0:00-10:00

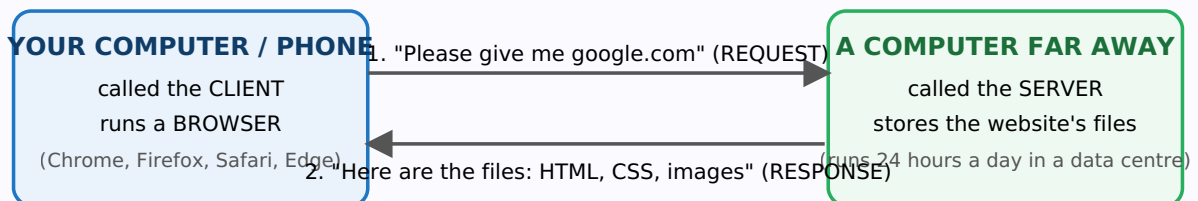
Angela \$1

If you already know what a browser, a server and a file are, skip to the Roadmap. If not, read this chapter slowly — everything else in the book stands on it.

1. The Internet vs. the Web

- **The Internet** is the physical network: cables under the ocean, Wi-Fi, phone towers, and millions of computers connected to each other.
- **The Web** (World Wide Web) is one service that runs on the Internet: the system of pages you open in a browser. Email, WhatsApp and online games also use the Internet but are not "the Web".

2. The two computers in every web page



This request/response conversation is called HTTP. It happens every time you open a page or click a link.

3. What the browser does with the files

- 1 The browser receives an **HTML file** — a plain text file. HTML describes what is on the page: "a heading, then a paragraph, then an image".
- 2 The HTML file says "also load **style.css**". CSS describes how it looks: colours, sizes, positions.
- 3 The HTML file says "also load **main.js**". JavaScript describes what happens when you click, type, or scroll.
- 4 The browser combines all three and paints the page you see. This is called **rendering**.

HTML

the skeleton (structure)
like the walls of a house

CSS

the look (style)
like paint and furniture

JavaScript

the behaviour (actions)
like electricity and doors that open

4. Front end and back end

	Front end	Back end
Where does it run?	In the user's browser (the client)	On the server
What is it made of?	HTML, CSS, JavaScript (and libraries like React, Bootstrap)	A server language (JavaScript with Node.js, Python, PHP, Java...) plus a database (PostgreSQL, MySQL, MongoDB)
What does it do?	Shows the page, reacts to clicks, sends forms	Saves data, checks passwords, sends emails, talks to other services
Can the user see the code?	Yes (right-click → View Source)	No — only the results it sends back
Job titles	Front-end developer	Back-end developer

A **full-stack developer** does both. This book takes you from zero to full-stack.

5. Files and folders — the part beginners are never taught

A website is just a **folder** on a computer containing **text files**. There is no magic.

- A file has a **name** and an **extension** after the dot: `index.html`, `style.css`, `main.js`, `logo.png`. The extension tells the computer what kind of file it is.
- Windows hides extensions by default. Turn them on: File Explorer → View → tick "File name extensions". Otherwise you may create `index.html.txt` by mistake and nothing works.
- Name files in **lowercase**, with **no spaces**: use `my-page.html` not `My Page.html`. Servers on Linux treat `Photo.jpg` and `photo.jpg` as different files.
- `index.html` is special: when someone opens a folder on a web server, the server shows `index.html` automatically. Your home page must have this name.

6. Your tools (2 minutes, no long installation guide)

- 1 A browser** — you already have one. Chrome is recommended because its Developer Tools (press `F12`) are the best for learning.
- 2 A code editor** — Visual Studio Code (free, code.visualstudio.com). Any text editor works, even Notepad, but VS Code colours your code and finds mistakes. Install the **Live Server** extension: it refreshes the browser every time you save.
- 3 Node.js** (nodejs.org, the LTS version) — you need it only from Part 5 (back end) onward. Skip it for now.

7. Your very first page — step by step

- 1** Create a folder on your Desktop called `my-first-site`.

- 2 Open VS Code → File → Open Folder → choose that folder.
- 3 Click the "New File" icon and name it `index.html` (exactly — lowercase, with the extension).
- 4 Type the code below. Type it, do not copy it.

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

BROWSER TAB SHOWS: MY FIRST PAGE

Hello, world!

This is my first web page.

- 5 Save with `Ctrl + S`. Right-click inside the file → "Open with Live Server" (or just double-click the file in your folder). Your browser opens and shows the page.
- 6 Change the text inside `<h1>`, save, and watch the browser update. **Congratulations — you are a web developer now.** Everything else is detail.

- **Line 1** `<!DOCTYPE html>` — tells the browser: "this file uses modern HTML (version 5)". Always the first line.
- `<html> ... </html>` — the whole page lives between these two tags.
- `<head>` — information about the page (its title, its CSS). Not shown inside the window.
- `<title>` — the text in the browser tab.
- `<body>` — everything the visitor actually sees.
- `<h1>` — a heading (h = heading, 1 = the biggest / most important).
- `<p>` — a paragraph.
- Notice every opening tag `<p>` has a closing tag `</p>` with a slash. The text between them is the **content**.

⚠ "I saved the file but the browser shows nothing / shows my code as text"

- The file is really called `index.html.txt`. Turn on file extensions (section 5) and rename it.
- You opened the file in the editor, not the browser. The browser address bar must show a path ending in `.html` or `localhost:5500`.
- You forgot to save. Look for a white dot on the file tab in VS Code — it means unsaved.

? Quick check

1. Which of the three languages decides the colour of a button?
2. Where does back-end code run — in the browser or on the server?
3. What is the special name for the home page file?
4. What is the difference between a tag and an element?

Answers: 1) CSS. 2) On the server. 3) index.html. 4) A tag is the marker like `<p>` or `</p>`; an element is the opening tag + content + closing tag together.

8. Glossary for absolute beginners

Browser

The program you use to open websites (Chrome, Firefox, Safari, Edge).

URL

The address of a page, e.g. `https://www.example.com/about.html`. `https` is the protocol (secure HTTP), `www.example.com` is the domain, `/about.html` is the path to the file.

Domain name

The human-readable name of a server (google.com). You rent it yearly from a registrar.

IP address

The real numeric address of a computer on the Internet, like 142.250.74.46. DNS converts domain names to IP addresses.

Hosting

Renting space on a server so your website files are online 24/7.

Tag

A marker in HTML written with angle brackets, like `<p>`.

Element

An opening tag, its content and its closing tag: `<p>Hello</p>`.

Attribute

Extra information inside an opening tag: `` — `src` is the attribute, `"cat.jpg"` is its value.

Syntax

The grammar rules of a language. A "syntax error" means you typed something the computer cannot understand — usually a missing bracket or quote.

Code editor / IDE

A text editor made for code (VS Code).

Developer Tools (DevTools)

The panel in the browser (F12) that shows the HTML, CSS, JavaScript errors, and network requests of any page.

Console

A tab inside DevTools where JavaScript prints messages and errors.

Render

What the browser does when it turns code into the visible page.

Responsive

A page that looks good on phones, tablets and desktops.

Framework / Library

Code written by others that you reuse: Bootstrap (CSS), React (JS), Express (Node).

API

Application Programming Interface — a way for one program to ask another for data. A weather API returns today's temperature as data your page can display.

Database

Organised storage for data (users, products, posts) on the server.

Deploy

Putting your website on a real server so anyone in the world can open it.

Bug

A mistake in code that makes the program behave wrongly. Debugging = finding and fixing bugs.

3. How to Study This Book — 16-Week Plan with the Video Courses

This book is a companion to three free video courses. Each week below lists what to watch, which chapters of this book to read, and what to build. Watching alone is not enough — the build column is where learning happens. If a week takes you two weeks, that is normal.

الكتاب ده مرافق للكورسات: شوف الفيديوهات، اقرأ الفصول، وابني المشروع. البناء هو اللي بيعلّم.

Week	Watch	Read in this book	Build
1	Gamal HTML #1-#14 · Elzero HTML one-video (first 2 h)	Beginner intro chapters · Fundamentals (how the web works) · HTML chapters + HTML Reference A-C	Profile page (Project 1, first half) · HTML drills 1-15
2	Gamal HTML #15-#27 · Elzero HTML one-video (rest)	HTML Reference D-G (forms, tables, media, semantic, a11y)	Finish Project 1 · HTML drills 16-30 · validate the page
3	Gamal CSS #1-#15 · Elzero CSS #01-#25	CSS chapters + CSS Reference A-C (selectors, cascade, colours, box model)	Style the profile (Project 2, part 1) · CSS drills 1-10
4	Gamal CSS #16-#30 · Elzero CSS #26-#45	CSS Reference D-G (text, position, pseudo, transitions)	CSS drills 11-20 · Component cookbook items 1-10
5	Gamal CSS #31-#47 · Elzero CSS #46-#64 (Flex + Grid)	CSS Reference H-J (Flexbox, Grid, responsive) · Layout patterns gallery	Finish Project 2 responsive · CSS drills 21-30 · Walkthrough 1 (restaurant site)
6	Angela: Bootstrap section · Elzero CSS #65-#88	Bootstrap chapter + Tools Reference B (Bootstrap) · Git chapter + Tools Reference A	TinDog (Project 3) · put Projects 1-3 on GitHub Pages
7	Elzero JS #001-#040 · Gamal JS (basics)	JS chapters + JS Reference A-B (types, variables, control flow)	JS drills 1-20 · DevTools chapter exercises
8	Elzero JS #041-#075 · Elzero JS one-video Part One	JS Reference C-E (functions, strings, arrays, objects)	JS drills 21-30 · Dicee + Drum Kit (Project 4)
9	Elzero JS #076-#131 (DOM, events, BOM)	JS Reference F-G (DOM, events) · Mini-projects 1-8	Simon (Project 5) · To-Do app (Project 6) · JS drills 31-36
10	Elzero JS #132-#188 · Elzero JS one-video Part Two	JS Reference H-I (async, fetch, classes, modules) · Mini-projects 9-15	Weather app (Project 7) · Quiz app (Walkthrough 2) · JS drills 37-40
11	Angela: Bash, Node, npm, Express	Command line chapter · Back-End Reference A-B	First Express server · REST API in memory · test with Postman
12	Angela: EJS, REST, APIs	Back-End Reference C-D	Blog with EJS (Project 8)
13	Angela: SQL, PostgreSQL	Back-End Reference E · SQL drills 1-15	Books/Products API with Postgres (Project 9)
14	Angela: Authentication & Security	Back-End Reference F · SQL/ back-end drills 16-20 · Security checklist	Secrets app (Project 10)
15	Angela: React (all) · Gamal React course	React chapters + React Reference A-C	Keeper + Products dashboard (Project 11) · Walkthrough 3 (full-stack notes app)
16	Angela: Deployment		

Week	Watch	Read in this book	Build
		Tools Reference C (deploy) · Accessibility/SEO/Performance chapter · Career kit	Capstone e-commerce (Project 12) deployed · portfolio + README · start interview questions

Daily routine that works: 20 minutes of video → pause and type every example → 20 minutes reading the matching reference entry → 40+ minutes building. Finish each day by committing to GitHub with a clear message. Keep a `notes.md` with one line per bug you solved.

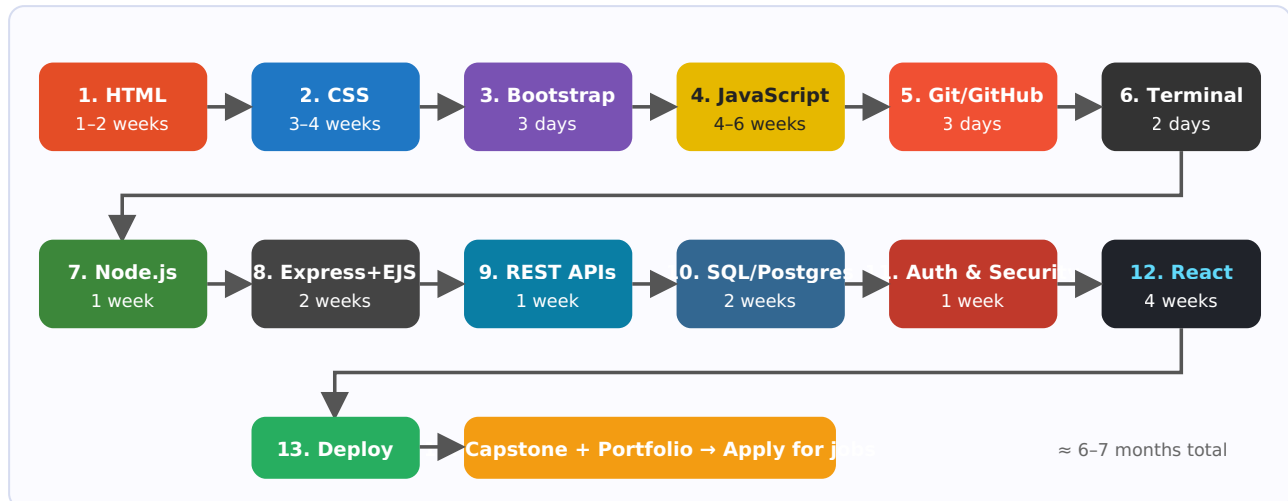
If you get stuck: read the error out loud → check the "Common error messages decoded" table in the Career Kit → search the exact message → ask in the Elzero or Gamal community groups with a minimal example. Never stay stuck more than an hour without asking.

Before you move on from any week, you should be able to:

- Explain the week's main idea to a friend in two minutes without notes.
- Rebuild the week's project from an empty folder in under two hours.
- Answer the quick-check questions at the end of the matching reference chapters.

4. The Full-Stack Roadmap

This is the order to learn in, how long each step takes at ~2 hours/day, which reference to watch, and what to build before moving on. **Do not skip the projects** — they are the learning.



Weeks 1-2 **HTML — Part 1 of this book**

Watch: [Gamal #1-27](#) or [Elzero one video](#) · [Angela §1-4](#)

Build: personal page (ch. 19), a recipe page with lists/images, a sign-up form with every input type. ☐ You're done when you can write a page from memory without Googling tags.

Weeks 3-6 **CSS — Part 2**

Watch: [Gamal CSS #1-47](#) · [Angela §5-13](#)

Build: style your personal page; a pricing table (Flexbox); a photo gallery (Grid); make both responsive. Angela's projects: CSS "Motivational Poster", "Flexbox Pricing Table", "Grid Mondrian", "Bootstrap TinDog" landing page. ☐ Done when you can rebuild any simple website layout you see.

Week 7 **Bootstrap 5 — ch. 34**

Build: a landing page with navbar, hero, cards, carousel, footer in one day.

Weeks 8-13 **JavaScript — Part 3**

Watch: [Angela §14-22](#) (JS basics → DOM → jQuery → advanced JS)

Build: Dice game, Drum Kit, Simon game (Angela classics), a to-do list, a weather app using fetch + a public API. ☐ Done when you can read JSON from an API and display it on the page.

Week 14 Command line + Git/GitHub — Part 4

Push every project you have to GitHub. Turn on GitHub Pages for one.

Weeks 15-22 Back-end — Part 5

Watch: **Angela §24-35** (Node, Express, EJS, APIs, PostgreSQL, Auth)

Build: To-do app with EJS; Blog with create/edit/delete; Book Notes app on Postgres; "Secrets" app with register/login (bcrypt + sessions + Google OAuth). □ Done when you can explain a request from browser → Express route → SQL → back.

Weeks 23-26 React — Part 6

Watch: **Angela §36**

Build: Keeper (Google Keep clone), then re-write your to-do front-end in React and connect it to your Express API.

Week 27+ Deploy + Capstone — ch. 49-50

Deploy front-end on Vercel/Netlify, API on Render/Railway, DB on Neon/Supabase. Write a README. Build your portfolio site listing 5+ projects. Start applying / freelancing.

Learning rules that actually work

- **Watch → pause → type → break it → fix it.** Never watch a full video passively.
- **One source at a time.** Use this book + one video course per topic. Don't hop between 5 tutorials.
- **Projects > certificates.** Employers open your GitHub, not your Udemy list.
- **Read the docs:** developer.mozilla.org (MDN) for HTML/CSS/JS, expressjs.com, react.dev, postgresql.org.
- **Don't learn React before you're comfortable in plain JS** — it's the #1 mistake.

5. Fundamentals Reference — How the Internet and the Browser Really Work

Before code, understand the machine you are programming. Everything in this book — HTML files, servers, APIs, deployment — makes more sense when you can picture what happens between typing an address and seeing a page. This chapter is plain-language, with diagrams.

Client, server, request, response

Angela §1 §24

Elzero HTML intro

Your browser is a **client**: it asks. A **server** is a computer that is always on and answers. The asking is a **request**; the answer is a **response**. Every page load, image, button click that loads data — all requests and responses.

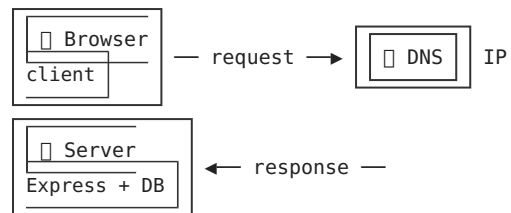
المتصفح عميل يطلب، السيرفر جهاز شغال دائماً بيرد. كل صفحة أو صورة بتشوفها كانت طلب ورد.

EXAMPLE

You type `https://nilegrill.com/menu` and press Enter:

1. DNS "nilegrill.com" → 104.21.55.7 (phone book: name → IP address)
2. TCP+TLS open a secure connection to 104.21.55.7 on port 443
3. HTTP GET /menu (with headers: browser type, accepted formats, cookies)
4. Server Express finds the route, queries the database, renders HTML
5. HTTP 200 OK + HTML text
6. Browser parses HTML → finds `<link>` and `<script>` and `<img>` → sends MORE requests for each
7. Browser builds DOM + CSSOM → layout → paint → runs JavaScript
8. You see the page (usually within 1–2 seconds)

RESULT



- IP address = the numeric address of a machine (like 104.21.55.7). Domain = the human name pointing to it.
- Port = door number on the machine: 80 http, 443 https, 3000 your dev server, 5432 Postgres.
- Localhost (127.0.0.1) = your own computer acting as server while you develop.

URL anatomy

Angela §1

Part	In Express	In the browser
path /menu/grills	req.path, route params req.params	location.pathname
query ?sort=price	req.query.sort	new URLSearchParams(location.search).get("sort")
fragment #kofta	not sent	location.hash; scrolls to id="kofta"
host	req.hostname	location.host

EXAMPLE

`https://www.nilegrill.com:443/menu/grills?sort=price&page=2#kofta`

| | | | | | |
 scheme host port path query string fragment
 (protocol) (domain) (default) ?key=value&... (jump to id, never sent to server)

Front-end vs back-end vs full-stack

Angela §1

Gamal intro

	Front-end	Back-end
Runs on	User's browser	Server
Languages	HTML, CSS, JavaScript	JavaScript (Node), Python, PHP, Java, Go...
Concerns	Layout, interaction, accessibility, performance	Data, security, business rules, APIs
Tools in this book	React, Bootstrap, Vite	Express, PostgreSQL, EJS
Can the user see the code?	Yes — always (View Source)	No
Where secrets live	Never here	Here (env vars)

- Full-stack = comfortable on both sides, able to build and deploy a complete product alone. That is the goal of this book.

Static vs dynamic sites, SPA vs SSR

Angela §28 §33

Type	How pages are made	Examples	Host
Static	Fixed HTML/CSS/JS files	Portfolio, landing page, docs	GitHub Pages, Netlify
Server-rendered (SSR)	Server builds HTML per request from data (EJS)	Blogs, shops, dashboards	Render, VPS
Single-page app (SPA)	One HTML shell; JS (React) fetches JSON and renders	Gmail-like apps, dashboards	Netlify + API
Hybrid (Next.js)	Server renders first view, React takes over	Most modern products	Vercel

How the browser renders: DOM, CSSOM, render tree, reflow, repaint

Elzero JS #105

Angela §17

EXAMPLE

```
HTML text → parse → DOM tree ┌
                                │ → Render tree → Layout (positions/sizes) → Paint (pixels)
                                └
→ Composite
CSS text → parse → CSSOM ───┐
```

JavaScript can change the DOM or CSSOM at any time → browser re-runs layout (reflow) and/or paint (repaint).

Changing transform/opacity is cheap (composite only) → smooth animations.

Changing width/top/margin in a loop is expensive (reflow each time) → batch changes, use classes.

- Scripts without defer block parsing while they download and run — that is why scripts go at the end or use defer.
- DevTools → Performance tab records exactly these phases when a page is slow.

HTTP methods, headers, cookies, caching — the essentials

Angela §29

Concept	Plain meaning	Where you meet it
GET / POST / PUT / PATCH / DELETE	read / create / replace / edit / remove	forms, fetch, Express routes
Status code	3-digit result: 2xx ok, 3xx redirect, 4xx your mistake, 5xx server's mistake	Network tab, res.status()
Headers	Key: value metadata on requests/responses	Content-Type, Authorization, Set-Cookie
Cookie	Small data the server asks the browser to store and send back	sessions, preferences
Cache	Browser keeps a copy to avoid re-downloading	Cache-Control header, 304 responses, hard refresh Ctrl+Shift+R
HTTPS / TLS	Encrypted connection; padlock; required for modern APIs	Free via Let's Encrypt / hosts
Stateless	Each request stands alone; server remembers nothing unless you add cookies/tokens	sessions vs JWT

Files, paths and where things live

Gamal HTML #3

Elzero HTML

Half of beginner bugs are wrong paths. A path is relative to the file that contains it (HTML for src/href; the CSS file for url()).

EXAMPLE

```
project/
├─ index.html      href="css/style.css"      src="images/logo.png"
├─ about/
│   └─ team.html   href=" ../css/style.css"  src=" ../images/logo.png"
├─ css/
│   └─ style.css    url("../images/bg.jpg")  ← relative to the CSS file
└─ images/
```

```
./  this folder      ../  parent folder      /  site root (works on a server, not from file://)
```

⚠ Common mistake

Uppercase/lowercase matters on Linux servers: `Logo.PNG` $\neq$ `logo.png`. Works on Windows, breaks after deployment. Use lowercase, no spaces, hyphens.

? Chapter quiz

1. What does DNS do?
2. Which port does HTTPS use by default?
3. Name the two trees the browser builds before layout.
4. Why can a page work locally but show missing images after deploying?

Answers: 1) Translates a domain name into an IP address. 2) 443. 3) DOM and CSSOM. 4) Case-sensitive file names or wrong relative paths on the Linux server.

6. Fundamentals Reference — The Command Line (Bash) for Web Developers

Angela's course has a whole section on the command line because every other tool — npm, git, node, deployment — lives there. You need about twenty commands. This chapter is the complete list with what each one does, plus the keyboard tricks that make it fast.

Opening a terminal and understanding the prompt

Angela §23

Windows: use Git Bash (installed with Git) or Windows Terminal with PowerShell; macOS: Terminal or iTerm; Linux: any terminal. VS Code has one built in (Ctrl+`). The prompt shows where you are; commands act on that folder.

الترمينال نافذة بتكتب فيها أوامر. كل أمر بيتنفذ في الفولدر اللي انت واقف فيه.

EXAMPLE

```
mahmoud@laptop:~/projects/nile-grill$ _
#      ^ user   ^ machine ^ current folder (~ = your home)  $ = ready for a command
```

Moving around and looking

Angela §23

EXAMPLE

```
pwd          # print working directory: where am I?
ls           # list files          ls -la → all files incl. hidden, with details
cd projects  # change directory (into)
cd ..        # up one level
cd ~         # home
cd -         # back to previous folder
cd /         # root
clear        # clean the screen (Ctrl+L)
tree -L 2    # folder tree (may need install)
```

- Press **Tab** to auto-complete names; press it twice to see options.
- Up/Down arrows cycle through previous commands. `history` lists them; `!!` repeats the last one.
- Paths with spaces need quotes: `cd "My Projects"`.

Creating, copying, moving, deleting

Angela §23

EXAMPLE

```
mkdir images                # make folder
mkdir -p src/components    # nested in one go
touch index.html           # create empty file (or update its date)
cp style.css backup.css    # copy file
cp -r src src-backup       # copy folder (recursive)
mv old.html new.html       # rename
mv file.js src/            # move
rm file.js                 # delete file (no recycle bin!)
rm -r folder               # delete folder and contents
rm -rf node_modules        # force, no questions – be careful what you type
cat file.txt               # print a file
less file.txt              # scroll a long file (q to quit)
head -20 log.txt   tail -f log.txt # first 20 lines / follow a growing log
echo "hello" > file.txt     # write (overwrite)  >> appends
code .                     # open current folder in VS Code
open . / explorer .       # open in Finder / Explorer
```

⚠ Common mistake

`rm -rf /` or `rm -rf ~` destroys your system or home folder. Never run `rm -rf` on anything you did not just type carefully; there is no undo.

Finding things and chaining commands

Angela §23

EXAMPLE

```
find . -name "*.css"        # files by name under current folder
grep -rn "TODO" src         # search text in files, recursive, with line numbers
grep -i "error" server.log  # case-insensitive
wc -l file.txt              # count lines

# pipes and chains
cat access.log | grep 404 | wc -l # count 404 lines
npm run build && npm run deploy    # second only if first succeeded
npm test || echo "tests failed"   # second only if first failed
command ; command                 # run both regard
command &                         # run in background
Ctrl+C                            # stop the running command (dev server)
Ctrl+D                            # exit shell / end input
```

Processes, ports and environment

Angela §25

EXAMPLE

```
node -v npm -v git --version # versions
which node                   # where a command lives
ps aux | grep node           # running node processes
kill 12345                   # stop process by PID      kill -9 = force
lsof -i :3000                # who is using port 3000 (mac/linux)
netstat -ano | findstr :3000  # Windows equivalent, then taskkill /PID 1234 /F

export API_KEY=abc           # env var for this session (Windows PowerShell:
$env:API_KEY="abc")
echo $API_KEY / printenv     # read
PORT=4000 node index.js      # set just for one command
```

Permissions, sudo, ssh, curl

Angela §23 §35

EXAMPLE

```
chmod +x deploy.sh          # make a script executable
./deploy.sh                 # run it
sudo apt install postgresql # run as admin (Linux)   brew install postgresql (mac)
ssh user@203.0.113.5        # log into a remote server
scp file.zip user@host:~/    # copy to a server
curl https://api.github.com/users/octocat # HTTP request from terminal
curl -X POST -H "Content-Type: application/json" -d '{"a":1}' http://localhost:3000/api
curl -I https://example.com # headers only
wget https://.../file.zip    # download
unzip file.zip / tar -xzf file.tar.gz
```

A tiny shell script

Angela §23

EXAMPLE

```
#!/bin/bash
# new-project.sh - scaffold a static site
name=$1 # first argument
if [ -z "$name" ]; then echo "Usage: ./new-project.sh name"; exit 1; fi
mkdir -p "$name/{css,js,images}"
cat > "$name/index.html" <<EOF
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8"><meta name="viewport" content="width=device-width,
initial-scale=1"><title>$name</title><link rel="stylesheet" href="css/style.css"></head>
<body><h1>$name</h1><script src="js/main.js" defer></script></body></html>
EOF
touch "$name/css/style.css" "$name/js/main.js"
cd "$name" && git init && echo "node_modules\n.env" > .gitignore
echo "Created $name"
```

- Run with `chmod +x new-project.sh && ./new-project.sh nile-grill`.
- package.json scripts are the JavaScript-world equivalent: put repeated commands there and run `npm run name`.

? Chapter quiz

1. Which command shows your current folder?
2. How do you go up one folder?
3. What does && do between two commands?
4. How do you stop a running dev server?

Answers: 1) pwd. 2) cd .. 3) Runs the second only if the first succeeded. 4) Ctrl+C.

7. Fundamentals Reference — DevTools Mastery and Debugging Workflow

The browser's developer tools are the most important learning tool you own: they show you the real DOM, the real CSS that applied, every network request, and let you pause JavaScript line by line. Elzero's very first JavaScript lessons are about DevTools for a reason. Open them with F12 or Ctrl+Shift+I (Cmd+Option+I on Mac).

Elements panel — inspect and live-edit HTML/CSS

Elzero JS #004

Gamal CSS #2

Angela §6

Right-click anything on a page → Inspect. The left shows the DOM; the right shows the CSS rules that apply, in cascade order, with overridden ones crossed out. Edit anything live to experiment before touching your file.

Elements بتورك ال HTML الحقيقي وال CSS اللي اتطبقت. جرب تعديل أي حاجة مباشرة قبل ما تغير ملفك.

Action	How
Pick an element	Ctrl+Shift+C then click on the page
Edit text or attribute	Double-click it in the DOM
Add / change a CSS property	Click inside the Styles pane; Tab to add; arrows ↑ ↓ change numbers (Shift = 10, Alt = 0.1)
Toggle a property	Checkbox next to it
Force a state	:hov button → check :hover / :focus / :active
Add a class	.cls button
See the box model	Computed tab → diagram with margin/border/padding/content numbers
Find which rule wins	Styles pane top-to-bottom; crossed-out = overridden (specificity/order)
Copy an element's selector or HTML	Right-click node → Copy → selector / outerHTML
Layout helpers	Grid / Flex badges next to elements toggle overlays
Which fonts rendered	Computed → Rendered Fonts at the bottom

- Changes in DevTools are NOT saved — copy the winning values into your CSS file.
- If your CSS is not applying at all: check the Network tab for a 404 on the CSS file, and Sources to see the file content the browser actually got (cache!).

Console panel

Elzero JS #007 #008

Angela §14

EXAMPLE

```

class="c">// type any JavaScript and press Enter; Shift+Enter for new line
document.title
$0                                class="c">// the element currently selected in Elements
$("h1")  $$(".card")             class="c">// shortcuts for querySelector / querySelectorAll (console only)
copy(someObject)                 class="c">// copy as JSON to clipboard
console.table(users)
console.dir(element)             class="c">// object view instead of HTML view
console.group("Cart"); console.log(a); console.log(b); console.groupEnd();
console.time("load"); await load(); console.timeEnd("load");  class="c">// measure
console.assert(x > 0, "x must be positive");
console.trace()                  class="c">// where was this called from
clear()                          class="c">// or Ctrl+L

class="c">// filter box: type "error" or -verbose; levels Errors/Warnings/Info
class="c">// "Preserve log" keeps messages across page reloads (useful for form submits/redirects)

```

- Click the file:line link at the right of any message to jump to the code in Sources.
- Red text = error with a stack trace. Read the FIRST line and the FIRST file:line that is yours.

Sources panel — breakpoints and stepping

Elzero JS #148

Angela §22

console.log is fine; the debugger is better. Pause the program at a line and look at every variable, then step forward one line at a time.

Action	How
Set a breakpoint	Sources → open file → click a line number (or write debugger; in code)
Run until it pauses	Trigger the code (click the button, reload)
Inspect variables	Hover over any name; Scope pane on the right; Watch to pin expressions
Step over (next line)	F10
Step into (enter function)	F11
Step out	Shift+F11
Resume	F8
Conditional breakpoint	Right-click line number → Add conditional → e.g. id === 42
Break on DOM change	Elements → right-click node → Break on → subtree modifications
Break on any exception	Pause on exceptions toggle (🚫 icon)
Break on event	Event Listener Breakpoints → Mouse → click
Edit and re-run	Edit in Sources, Ctrl+S applies live for this session

Network panel — every request the page makes

Angela §30

The truth about what your fetch actually sent and what the server actually returned.

Column / tab	Tells you
Name / Status	URL and status code (red = 4xx/5xx; grey = cancelled)
Type / Initiator	document, stylesheet, script, fetch/xhr, img — and which line started it
Size / Time	download size, how long; sort by Time to find slow ones
Headers tab	request URL, method, request headers (Authorization? Content-Type?), response headers (CORS? Set-Cookie?)
Payload tab	exactly what body you sent (JSON, form data)
Response / Preview tab	raw and pretty response — is it JSON or an HTML error page?
Filter	Fetch/XHR to see only API calls; type a URL fragment
Disable cache	Checkbox — do this while developing
Throttling	Simulate Slow 3G to see loading states
Copy as cURL / fetch	Right-click request → Copy — reproduce it in terminal or code
Waterfall	Visual timeline: what blocked what

- A fetch that 'does nothing' is almost always visible here as a red 4xx/5xx or a CORS error. Check here before changing code.
- Preserve log + Disable cache are the two checkboxes to leave on while developing.

Application, Performance, Lighthouse, device mode

Angela §35

Panel	Use
Application → Local Storage / Session / Cookies	See and edit stored values; clear site data with one click
Application → Manifest / Service Workers	PWA debugging
Device toolbar (Ctrl+Shift+M)	Phone/tablet sizes, touch, DPR, rotate, throttle network
Performance	Record; see long tasks, layout thrash, scripting vs rendering time
Lighthouse	One-click audit: Performance, Accessibility, Best Practices, SEO with fix suggestions
Rendering (More tools)	Paint flashing, layout shift regions, emulate prefers-color-scheme / reduced motion
Coverage (More tools)	Unused CSS/JS bytes
Memory	Leaks: detached DOM nodes, growing heap
Sensors	Fake geolocation, orientation
Console → Issues tab	Deprecations, cookie problems, CORS explanations

A debugging routine that always works

Elzero JS #148

Angela \$22

Symptom	First suspect
Nothing happens on click	Listener never attached: selector null, script ran before HTML, wrong event name, function called instead of passed
CSS not applied	Typo in selector/class, specificity lost, file 404, cache, !important elsewhere, wrong path
undefined is not a function / cannot read property	Value is not what you think — log it one line above
Works locally, not on server	Env vars missing, case-sensitive paths, CORS origin, PORT, database URL
Data shows old values	Cache (hard refresh), state not updated immutably (React), stale closure
Layout jumps / overflow on mobile	Missing viewport meta, fixed widths, images without max-width, long words without overflow-wrap

EXAMPLE

1. Reproduce — can you make the bug happen on purpose? What exact steps?
2. Read — the FULL error message and the first file:line that is your code. Google the message in quotes.
3. Isolate — comment out half; does it still break? Repeat (binary search). Or copy the piece to a fresh file.
4. Inspect — console.log the value right BEFORE the failing line. Is it what you assumed? (typeof, Array.isArray, JSON.stringify)
5. Check the pipes — Network tab: did the request go? What came back? Elements: is the DOM what you think?
6. Explain it — out loud, to a colleague or a rubber duck, line by line. You will usually find it by step 3 of the explanation.
7. Fix ONE thing, re-test. Do not change five things at once.
8. Write it down — a one-line note of cause and fix. The same bug will come back in another project.

? Chapter quiz

1. Which shortcut picks an element on the page for inspection?
2. Where do you see the body your fetch actually sent?
3. What key steps to the next line when paused in the debugger?
4. What does a crossed-out CSS property in the Styles pane mean?

Answers: 1) Ctrl+Shift+C. 2) Network → the request → Payload tab. 3) F10. 4) It was overridden by a more specific or later rule.

8. Fundamentals Reference — Accessibility, SEO and Web Performance Basics

Three qualities that separate a hobby page from professional work. Accessibility means everyone can use your site, including people using screen readers or only a keyboard. SEO means search engines understand and rank it. Performance means it loads fast on a cheap phone on 3G. All three are mostly about doing HTML properly.

Accessibility (a11y) — the rules that matter most

Angela §3 §4

Gamal HTML

إتاحة الموقع لكل الناس: اللي بيستخدم قارئ شاشة، اللي بيستخدم الكيبورد بس، اللي شايف ألوان بصعوبة.

Check	Rule of thumb
Colour contrast	Text $\geq 4.5:1$ against background (3:1 for large). Check in DevTools colour picker
Keyboard	Tab through the whole page: can you reach and operate everything? Is focus visible? Never outline: none without a replacement
Focus order	Follows the visual order; modals trap focus and return it on close
Motion	Respect @media (prefers-reduced-motion: reduce)
Text size	Use rem; page must work at 200% zoom
Links	Descriptive text: 'Read the menu', not 'click here'
Forms	Errors described in text next to the field, not only red colour
Tables	th with scope; caption
Video	Captions; do not autoplay with sound
Test	Lighthouse Accessibility; axe DevTools extension; try a screen reader (NVDA free, VoiceOver on Mac) for 10 minutes

EXAMPLE

```

<!-- 1. images: describe, or empty alt for decoration -->



<!-- 2. every input has a label -->
<label for="email">Email</label><input id="email" type="email">
<input type="search" aria-label="Search products">      <!-- when no visible label -->

<!-- 3. real buttons and links, not clickable divs -->
<button type="button">Open menu</button>      <!-- keyboard + screen reader friendly -->
<a href="/menu">Menu</a>                        <!-- navigation = link; action = button -->

<!-- 4. headings in order, one h1 -->
<h1>...</h1> <h2>...</h2> <h3>...</h3>

<!-- 5. landmarks -->
<header> <nav aria-label="Main"> <main> <aside> <footer>

<!-- 6. icon buttons need a name -->
<button aria-label="Close dialog">x</button>

<!-- 7. state for custom widgets -->
<button aria-expanded="false" aria-controls="menu">Menu</button>
<div role="alert">Saved!</div>      <!-- announced immediately -->

<!-- 8. skip link -->
<a href="#main" class="skip">Skip to content</a>

<!-- 9. language -->
<html lang="ar" dir="rtl">

```

SEO basics for developers

Angela \$3 \$35

Factor	What to do
Content in HTML	Real text in the page, not only in images or loaded by JS after the fact (SPAs: consider SSR/Next.js for public pages)
One h1, semantic headings	Search engines use them as outline
Clean URLs	/menu/grills not /page?id=17
Fast + mobile friendly	Core Web Vitals: LCP < 2.5 s, CLS < 0.1, INP < 200 ms
Internal links	Every page reachable in ≤ 3 clicks; descriptive anchor text
Images	alt text, descriptive file names (kofta-plate.jpg), compressed
sitemap.xml + robots.txt	Submit in Google Search Console
HTTPS	Ranking factor and required for trust
No duplicates	Canonical tag; one version www/non-www

EXAMPLE

```

<head>
  <title>Nile Grill – Egyptian Restaurant in Giza</title>           <!-- 50–60 chars,
  unique per page -->
  <meta name="description" content="Family grill restaurant in Giza..."> <!-- 150–160 chars -->
  <link rel="canonical" href="https://nilegrill.com/menu">
  <meta name="robots" content="index, follow">
  <!-- social previews (Open Graph) -->
  <meta property="og:title" content="Nile Grill">
  <meta property="og:description" content="Real Egyptian flavour...">
  <meta property="og:image" content="https://nilegrill.com/images/og.jpg">
  <meta property="og:url" content="https://nilegrill.com">
  <meta name="twitter:card" content="summary_large_image">
  <!-- structured data -->
  <script type="application/ld+json">
  { "@context": "https://schema.org", "@type": "Restaurant", "name": "Nile Grill", "address": "12
  Pyramids Road, Giza", "telephone": "+201141225852", "servesCuisine": "Egyptian" }
  </script>
</head>

```

Performance — the 80/20 list

Angela \$35

Technique	Why
Compress images (Squoosh/TinyPNG), right size, WebP/AVIF	Images are usually 60–80% of page weight
width/height on img	Prevents layout shift (CLS)
loading="lazy" below the fold	Do not download what is not visible
defer scripts; minimal JS on landing pages	JS blocks rendering and the main thread
Limit fonts; font-display: swap	Fonts block text painting
Minify + bundle (Vite does it)	Fewer, smaller files
Cache headers / CDN (Netlify, Cloudflare)	Repeat visits and global speed
Avoid layout thrash in JS	Batch DOM reads/writes; use classes and transforms
Server: gzip/brotli compression, DB indexes, pagination	API speed
Measure	Lighthouse, WebPageTest, Network throttling — before and after

EXAMPLE

```

<!-- images: the #1 win -->

<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
        <!-- hero: NOT lazy, high priority -->
</picture>

<!-- scripts & styles -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="stylesheet" href="css/style.css">
<script src="js/main.js" defer></script>
<script type="module" src="js/app.js"></script>      <!-- deferred by default -->
<link rel="preload" href="fonts/poppins.woff2" as="font" type="font/woff2" crossorigin>

```

Security basics for front-end developers

Angela §32

Rule	Reason
Never trust user input — validate on the server too	Client checks are bypassable
Escape output (textContent, EJS <%= %>, React does it)	XSS
No secrets in front-end code or Git	Anyone can read them
HTTPS everywhere	Encryption; required for many APIs
rel="noopener" on target=_blank links	Prevents the new tab controlling yours
Content-Security-Policy header (helmet)	Blocks injected scripts
Keep dependencies updated (npm audit)	Known vulnerabilities
httpOnly, secure, sameSite cookies	Token theft, CSRF
Rate limit login; generic error messages	Brute force, user enumeration

? Chapter quiz

1. Why should icon-only buttons have aria-label?
2. Name two meta tags that control how a link preview looks on social media.
3. Which image attributes prevent layout shift?
4. Where must validation always happen, even if the form validates in the browser?

Answers: 1) Screen readers need a text name; the icon has none. 2) og:title and og:image (Open Graph). 3) width and height. 4) On the server.

9. CSS Layout Patterns Gallery — 15 Layouts You Will Build Again and Again

Every website is made of the same dozen layout patterns. Here each one has the HTML shape, the complete CSS, and a picture of the result. Copy them into a scratch file, resize the browser, and watch how they behave. Then reproduce each from memory.

1. Centred content column

Gamal CSS #8

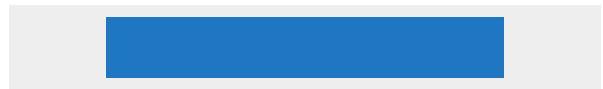
Elzero CSS #15

Angela §6

EXAMPLE

```
.container { max-width: 1100px; margin-inline: auto; padding-inline: 20px; }
```

RESULT



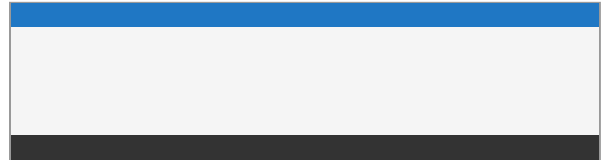
2. Sticky header + footer at bottom

Angela §8

EXAMPLE

```
body { min-height: 100vh; display: flex; flex-direction: column; margin: 0; }
header { position: sticky; top: 0; }
main { flex: 1; }
```

RESULT



3. Navbar: logo left, links right

Gamal CSS #41

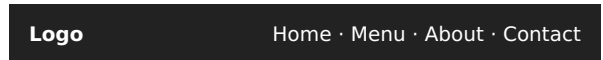
Elzero CSS #47

Angela §8

EXAMPLE

```
.nav { display: flex; align-items: center; justify-content: space-between; padding: 0 24px; height: 64px; }
.nav ul { display: flex; gap: 24px; list-style: none; }
```

RESULT



4. Hero: text left, image right, stacked on mobile

Gamal CSS #46

Angela §9

EXAMPLE

```
.hero { display: grid; grid-template-columns: 1fr 1fr; gap: 40px; align-items: center; min-height: 70vh; }
@media (max-width: 800px) { .hero { grid-template-columns: 1fr; text-align: center; } }
```

RESULT

Big headline
text text
[CTA](#)

5. Responsive card grid (no media queries)

Gamal CSS #46

Elzero CSS #63

Angela \$9

EXAMPLE

```
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 24px; }
```

RESULT



6. Card with image, body and footer pinned to bottom

Angela \$8

EXAMPLE

```
.card { display: flex; flex-direction: column; border-radius: 12px; overflow: hidden; box-shadow: 0 4px 16px rgba(0,0,0,.08); }
.card img { aspect-ratio: 16/9; object-fit: cover; }
.card-body { padding: 16px; flex: 1; }
.card-footer { padding: 12px 16px; border-top: 1px solid #eee; margin-top: auto; }
```

RESULT



7. Sidebar + main (holy grail)

Gamal CSS #46

Elzero CSS #60

Angela \$9

EXAMPLE

```
.layout { display: grid; grid-template-columns: 260px 1fr; min-height: 100vh; }
@media (max-width: 900px) { .layout { grid-template-columns: 1fr; } .sidebar { order: 2; } }
```

RESULT



8. Dashboard: header, sidebar, content, scrollable area

Angela \$9

EXAMPLE

```
.app { display: grid; grid-template-columns: 240px 1fr; grid-template-rows: 60px 1fr; height: 100vh; grid-template-areas: "side head" "side main"; }
.side { grid-area: side; } .head { grid-area: head; } .main { grid-area: main; overflow: auto; padding: 24px; }
```

RESULT



9. Split screen 50/50 full height

Angela §9

EXAMPLE

```
.split { display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; }
.split > * { display: grid; place-items: center; padding: 40px; }
@media (max-width: 700px) { .split { grid-template-columns: 1fr; } }
```

RESULT



10. Feature list: icon + text rows

Angela §8

EXAMPLE

```
.feature { display: flex; gap: 16px; align-items: flex-start; }
.feature .icon { flex: 0 0 48px; height: 48px; border-radius: 12px; display: grid; place-items: center; background: #eaf2fb; }
```

RESULT



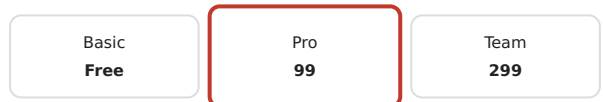
11. Pricing table with highlighted middle plan

Angela §11

EXAMPLE

```
.plans { display: grid; grid-template-columns: repeat(3, 1fr); gap: 24px; align-items: center; }
.plan { padding: 32px; border: 1px solid #eee; border-radius: 16px; text-align: center; }
.plan.featured { transform: scale(1.05); border-color: var(--red); box-shadow: 0 16px 40px rgba(0,0,0,.12); }
@media (max-width: 800px) { .plans { grid-template-columns: 1fr; } .plan.featured { transform: none; } }
```

RESULT



12. Masonry-like gallery with a featured tile

Elzero CSS #61 #62

Angela §9

EXAMPLE

```
.gallery { display: grid; grid-template-columns: repeat(4, 1fr); grid-auto-rows: 160px; gap: 12px; }
.gallery img { width: 100%; height: 100%; object-fit: cover; }
.gallery img:nth-child(1) { grid-column: span 2; grid-row: span 2; }
.gallery img:nth-child(6) { grid-column: span 2; }
```

RESULT



13. Sticky sidebar table of contents

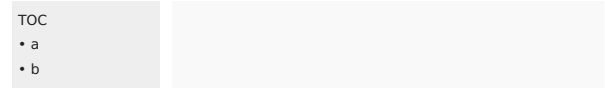
Elzero CSS #30

Angela §9

EXAMPLE

```
.docs { display: grid; grid-template-columns: 220px 1fr; gap: 40px; }
.toc { position: sticky; top: 80px; align-self: start; max-height: calc(100vh - 100px); overflow: auto; }
```

RESULT



14. Form: two-column fields that stack

Angela §4 §9

EXAMPLE

```
.form { display: grid; gap: 16px; grid-template-columns: 1fr 1fr; }
.form .full { grid-column: 1 / -1; }
@media (max-width: 600px) { .form { grid-template-columns: 1fr; } }
```

RESULT



15. Footer with 4 columns → 2 → 1

Angela §9

EXAMPLE

```
.footer-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 32px; }
@media (max-width: 900px) { .footer-grid { grid-template-columns: 1fr 1fr; } }
@media (max-width: 500px) { .footer-grid { grid-template-columns: 1fr; } }
```

RESULT



? Chapter quiz

1. Which pattern needs zero media queries to be responsive?
2. How do you pin a card footer to the bottom when bodies have different lengths?
3. Which two properties make the sticky TOC work?
4. What does `grid-column: 1 / -1` mean?

Answers: 1) The auto-fit/minmax card grid. 2) Card as flex column, body flex: 1 (or footer margin-top: auto). 3) position: sticky with top, plus align-self: start. 4) Span from the first to the last grid line — full width.

10. What is HTML? How the Web Works

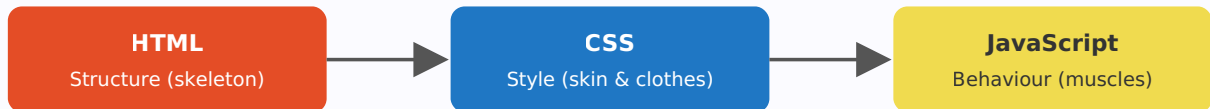
Gamal #1

Elzero 0:00

Angela \$1

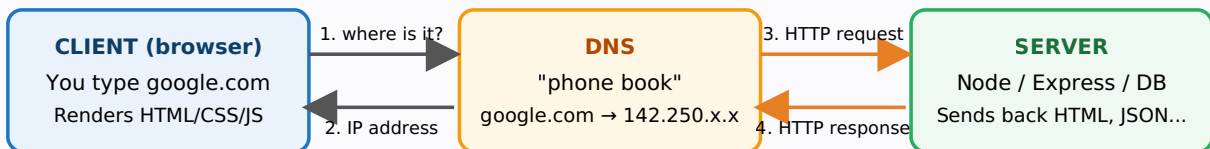
HTML = HyperText Markup Language. Not a programming language — a markup language: it describes the **structure** and **content** of a page (this is a heading, this is an image).

HTML لغة "وصف" لهيكل الصفحة — مش لغة برمجة. بتقول للمتصفح: ده عنوان، دي فقرة، دي صورة.



A web page = HTML (what) + CSS (how it looks) + JS (what it does)

How the web works (client ↔ server)



Front-end = everything in the browser. Back-end = everything on the server. Full-stack = both.

Emmet shortcut: in an empty `.html` file type `!` then `Tab` → the whole HTML skeleton is written for you. Press `F12` in the browser to inspect any site's HTML/CSS.

11. Creating website files & the skeleton

Gamal #2

Elzero

Project folder structure

```

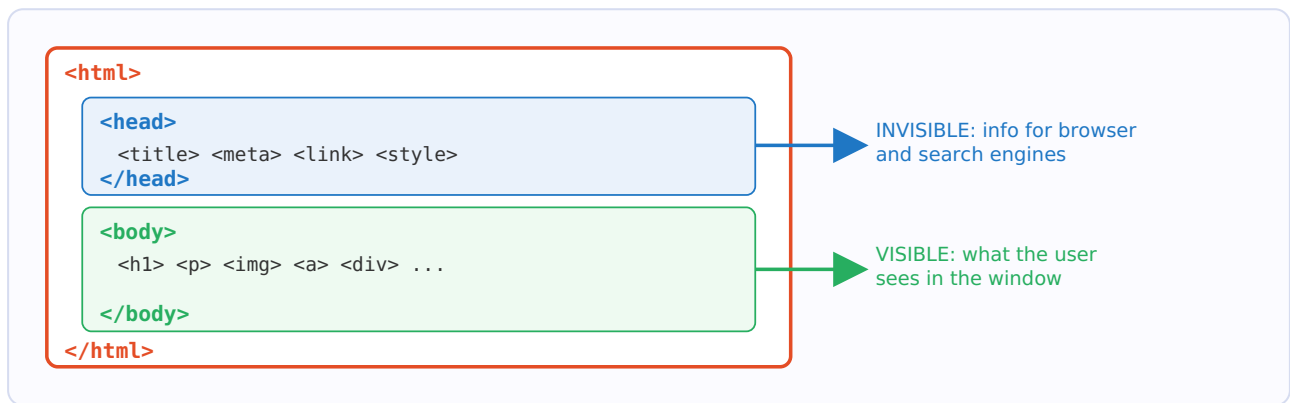
my-website/      ← project folder (any name, no spaces)
├── index.html    ← the home page. Browsers open index.html by default
├── about.html    ← other pages
├── css/
│   └── style.css ← styles
├── js/
│   └── main.js   ← scripts
└── images/
    └── logo.png
  
```

The HTML skeleton (boilerplate)

```

<!-- 1 --> <!DOCTYPE html>
<!-- 2 --> <html lang="en">
<!-- 3 --> <head>
<!-- 4 -->   <meta charset="UTF-8">
<!-- 5 -->   <meta name="viewport" content="width=device-width, initial-scale=1.0">
<!-- 6 -->   <title>My First Page</title>
<!-- 7 --> </head>
<!-- 8 --> <body>
<!-- 9 -->   <h1>Hello World</h1>
<!--10 --> </body>
<!--11 --> </html>
  
```

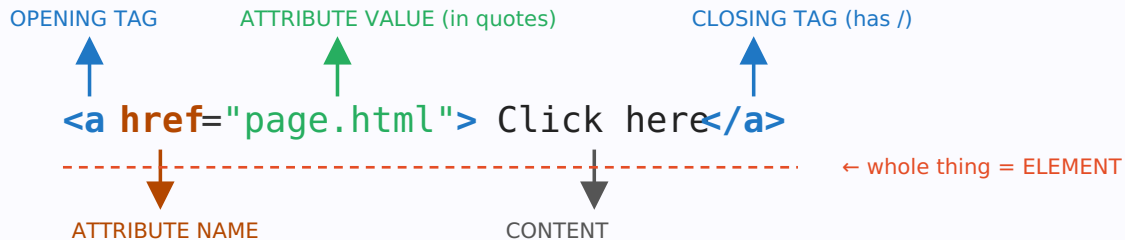
- **Line 1** — `<!DOCTYPE html>`: tells the browser "this is HTML5". Always first line. Not a tag, no closing.
- **Line 2** — `<html lang="en">`: root element, wraps everything. `lang="ar"` for Arabic pages (helps screen readers & Google).
- **Line 3-7** — `<head>`: information about the page. Nothing here is visible on the page itself.
- **Line 4** — `charset="UTF-8"`: character encoding — required for Arabic letters, emojis, symbols to show correctly.
- **Line 5** — **viewport**: makes the page responsive on phones (page width = device width, zoom = 100%).
- **Line 6** — `<title>`: text shown in the browser tab & Google results.
- **Line 8-10** — `<body>`: everything the user sees goes here.



12. Tags, Elements & Attributes — Anatomy

Gamal #3 #4

Elzero



- **Tag** = a keyword inside angle brackets: `<p>`. Tags are case-insensitive but always write lowercase.
- **Element** = opening tag + content + closing tag.
- **Attribute** = extra information written inside the opening tag as `name="value"`. Multiple attributes are separated by spaces.
- **Self-closing (void) elements** have no content and no closing tag: `<br>`, `<hr>`, `<img>`, `<input>`, `<meta>`, `<link>`.
- **Nesting**: elements go inside each other and must close in reverse order — `<p><b>text</b></p>` ❌ `<p><b>text</p></b>` ❌

Global attributes (work on any tag)

Attribute	Purpose	Example
<code>id</code>	Unique name for ONE element (used by CSS, JS, links #)	<code><div id="header"></code>
<code>class</code>	Name shared by MANY elements (used by CSS). Can have several separated by spaces	<code><p class="note big"></code>
<code>style</code>	Inline CSS	<code><p style="color:red"></code>
<code>title</code>	Tooltip on hover	<code><abbr title="HyperText">HT</abbr></code>
<code>lang</code>	Language of the content	<code><p lang="ar"></code>
<code>dir</code>	Text direction: <code>ltr</code> / <code>rtl</code>	<code><p dir="rtl">مرحبا</p></code>
<code>hidden</code>	Hide the element	<code><p hidden></code>
<code>data-*</code>	Custom data for JS	<code><li data-price="20"></code>

13. The <head> section

Gamal #5

Elzero

```

<head>
  <meta charset="UTF-8">                                encoding (Arabic support)
  <meta name="viewport" content="width=device-width, initial-scale=1">  mobile
  <meta name="description" content="Best bakery in Giza">  Google result text (SEO)
  <meta name="keywords" content="bakery, bread, cake">
  <meta name="author" content="Ahmed">
  <meta http-equiv="refresh" content="30">                reload page every 30 s
  <title>Giza Bakery | Home</title>                        tab title
  <link rel="icon" href="images/favicon.ico">              small tab icon
  <link rel="stylesheet" href="css/style.css">              connect external CSS
  <style> body { margin: 0 } </style>                      internal CSS
  <script src="js/main.js" defer></script>                connect JS (defer = run after HTML
loads)
</head>

```

meta = "metadata" = data about data. Each `<meta>` has a `name` (what kind) and `content` (the value). Only `<title>` is visible (in the tab).

14. Headings & Paragraphs

Gamal #6

Elzero

```
<h1>Main title (only ONE per page)</h1>
<h2>Section title</h2>
<h3>Sub-section</h3>
<h4>Smaller</h4>
<h5>Smaller</h5>
<h6>Smallest</h6>
<p>A paragraph of text. Extra    spaces
    and new lines are        ignored.</p>
<p>Line one<br>Line two</p>
<hr>
<pre>  keeps    spaces
    and lines</pre>
```

RESULT

Main title (only ONE per page)

Section title

Sub-section

Smaller

Smaller

Smallest

A paragraph of text. Extra spaces and new lines
are ignored.

Line one

Line two

```
  keeps    spaces
and lines
```

- **h1 → h6**: importance, not size. Use **one** **h1** per page and don't skip levels (h1 → h2 → h3). Change size with CSS, never by picking a different h-number.
- **<p>**: block of text. The browser collapses many spaces/newlines into one space (whitespace collapsing).
- **
: line break (self-closing). **<hr>: horizontal rule (thematic break).
- **<pre>**: preformatted — keeps spaces and line breaks exactly, monospace font.

15. Comments

Gamal #8

```
<!-- This is a comment. The browser ignores it. -->
<p>Visible text</p>
<!--
  Multi-line comment.
  <p>This paragraph is disabled</p>
-->
```

- Starts with `<!--` and ends with `-->`. VS Code shortcut: `Ctrl` + `/`.
- Use to explain code, mark sections (`<!-- Start Header -->`), or temporarily disable code.
- Comments ARE visible in "View Source" — never put passwords in them.

16. Text Formatting Tags

Gamal #9 Elzero

Tag	Meaning	Code	Result
<code></code>	Bold (visual only)	<code>bold</code>	bold
<code></code>	Important (semantic bold) prefer	<code>warning</code>	warning
<code><i></code>	Italic (visual only)	<code><i>italic</i></code>	<i>italic</i>
<code></code>	Emphasis (semantic italic) prefer	<code>really</code>	<i>really</i>
<code><u></code>	Underline	<code><u>under</u></code>	<u>under</u>
<code><s></code> / <code></code>	Strikethrough / deleted	<code>\$50 \$30</code>	\$50 \$30
<code><ins></code>	Inserted text	<code><ins>new</ins></code>	<u>new</u>
<code><mark></code>	Highlight	<code><mark>key</mark></code>	key
<code><small></code>	Smaller text	<code><small>fine print</small></code>	fine print
<code><sub></code>	Subscript	<code>H<sub>2</sub>O</code>	H ₂ O
<code><sup></code>	Superscript	<code>x<sup>2</sup></code>	x ²
<code><code></code>	Inline code	<code><code>let x</code></code>	<code>let x</code>
<code><kbd></code>	Keyboard key	<code><kbd>Ctrl</kbd></code>	Ctrl
<code><abbr></code>	Abbreviation with tooltip	<code><abbr title="World Health Organization">WHO</abbr></code>	<u>WHO</u>
<code><q></code>	Short inline quote (adds quotes)	<code><q>hi</q></code>	"hi"
<code><blockquote></code>	Long quote block (indented)	<code><blockquote cite="url">...</blockquote></code>	indented text
<code></code>	Generic inline container (no style itself — hook for CSS)	<code>word</code>	word
<code><div></code>	Generic block container (no style itself)	<code><div>section</div></code>	section (new line)

b vs strong / i vs em: look identical, but `strong` / `em` carry meaning (screen readers stress them, Google notices). Use the semantic one when the text really is important.

17. Images

Gamal #10 **Elzero**

```

```

- **src** (source) — path/URL of the image. **Required.**
- **alt** (alternative text) — shown if the image fails to load, read by screen readers, used by Google. **Always write it.**
- **width / height** — in pixels (no "px"). Set only one to keep the ratio; setting both prevents layout jumping.
- **title** — tooltip on hover. **loading="lazy"** — load only when scrolled into view (faster page).

Paths (very important)

Path	Meaning
cat.jpg	Same folder as the HTML file
images/cat.jpg	Inside the "images" folder (relative)
../cat.jpg	Go UP one folder, then find cat.jpg
/images/cat.jpg	From the site root (absolute path)
https://site.com/cat.jpg	External URL

Figure with caption & responsive images

```
<figure>
  
  <figcaption>The Great Pyramids, Giza</figcaption>
</figure>

<!-- different image per screen size -->
<picture>
  <source media="(max-width: 600px)" srcset="small.jpg">
        fallback
</picture>
```

Formats: JPG photos PNG transparency/logos SVG icons, scalable WebP modern, small GIF simple animation

18. Audio & Video

Gamal #11 #12

Elzero

```
<audio controls autoplay muted loop preload="auto">
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support audio.      fallback text
</audio>

<video controls width="640" poster="thumb.jpg">
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  <track src="subs_ar.vtt" kind="subtitles" srclang="ar" label="Arabic">
</video>
```

- **controls** — show play/pause/volume bar. Boolean attribute (no value needed).
- **autoplay** — start automatically. Browsers block it unless **muted** is also present.
- **loop** — repeat forever. **preload** — **none** | **metadata** | **auto**.
- **poster** — image shown before the video plays.
- **Multiple <source>** — browser picks the first format it supports. **type** = MIME type.
- **<track>** — subtitles file (.vtt).

YouTube video? Use the iframe embed code from YouTube "Share → Embed" (see chapter 17).

19. Tables

Gamal #13 Elzero

```
<table border="1">
  <caption>Prices</caption>
  <thead>
    <tr>
      <th>Item</th>
      <th>Price</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Bread</td>
      <td>5</td>
    </tr>
    <tr>
      <td>Cake</td>
      <td>40</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Total</td>
      <td>45</td>
    </tr>
  </tfoot>
</table>
```

RESULT

Item	Price
Bread	5
Cake	40
Total	45

- **table** — the whole table
- **tr** — table **row**
- **th** — table **header** cell (bold, centered)
- **td** — table **data** cell
- **thead / tbody / tfoot** — semantic groups
- **caption** — table title

Merging cells

```
<tr>
  <td colspan="2">Spans 2 columns</td>
</tr>
<tr>
  <td rowspan="2">Spans 2 rows</td>
  <td>A</td>
</tr>
<tr>
  <td>B</td>
</tr>
```

RESULT

Spans 2 columns	
Spans 2 rows	A
	B

- **colspan** = merge horizontally (columns).
- rowspan** = merge vertically (rows).

20. Lists

Gamal #14

Elzero

```

<!-- Unordered (bullets) -->
<ul>
  <li>HTML</li>
  <li>CSS
    <ul>                                nested list
      <li>Flexbox</li>
      <li>Grid</li>
    </ul>
  </li>
</ul>

<!-- Ordered (numbers) -->
<ol type="A" start="3" reversed>
  <li>Wake up</li>
  <li>Code</li>
</ol>

<!-- Description list -->
<dl>
  <dt>HTML</dt>
  <dd>Structure of page</dd>
  <dt>CSS</dt>
  <dd>Styling of page</dd>
</dl>

```

RESULT

- HTML
- CSS
 - Flexbox
 - Grid

1. Wake up
2. Code

HTML

Structure of page

CSS

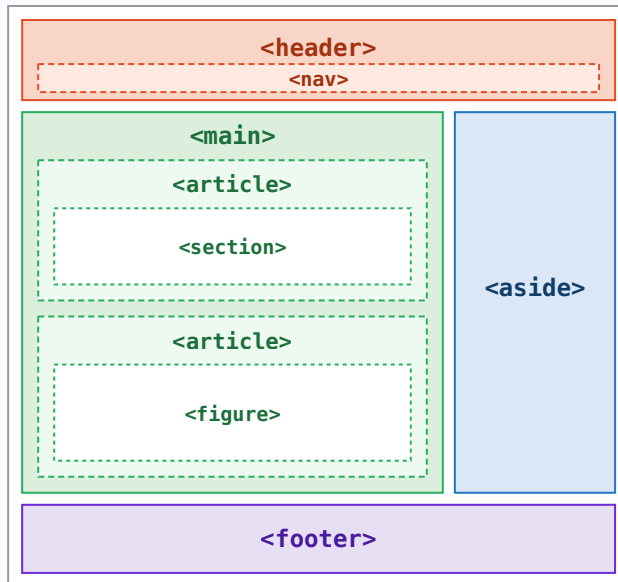
Styling of page

- **ul** = unordered list, **ol** = ordered list, **li** = list item
- **type** on ol: **1 A a I i**
- **start** = first number; **reversed** = count down
- **dl/dt/dd** = description list / term / definition
- Navigation menus are almost always **** inside **<nav>**

21. Semantic Elements

Gamal #15 Elzero

Semantic = the tag name describes its meaning. `<div>` says nothing; `<header>` says "this is the header". Better for SEO, accessibility and readable code.



Tag	Use for
<code><header></code>	Top of page/section: logo, title, nav
<code><nav></code>	Main navigation links
<code><main></code>	The unique main content (one per page)
<code><section></code>	A thematic group, usually with a heading
<code><article></code>	Self-contained content: blog post, product card, comment
<code><aside></code>	Side content: ads, related links, sidebar
<code><footer></code>	Bottom: copyright, contact, links
<code><figure></code> + <code><figcaption></code>	Image/diagram with caption
<code><time datetime="2026-09-07"></code>	Dates
<code><address></code>	Contact information
<code><details></code> + <code><summary></code>	Click-to-expand accordion (no JS!)
<code><div></code>	Only when no semantic tag fits (pure layout box)

```
<body>
  <header>
    <h1>My Blog</h1>
    <nav><ul><li><a href="/">Home</a></li><li><a href="about.html">About</a></li></ul></nav>
  </header>
  <main>
    <article>
      <h2>First post</h2>
      <p>Published <time datetime="2026-09-07">7 Sep 2026</time></p>
    </article>
    <aside>Related posts...</aside>
  </main>
  <footer>&copy; 2026 Ahmed</footer>
</body>
```

22. Links

Gamal #16 Elzero

```
<!-- external -->
<a href="https://google.com" target="_blank" rel="noopener">Google</a>
<!-- another page in my site -->
<a href="about.html">About us</a>
<!-- jump to a section on same page (bookmark) -->
<a href="#contact">Go to contact</a>
...
<section id="contact">...</section>
<!-- email / phone / whatsapp -->
<a href="mailto:me@mail.com?subject=Hello">Email me</a>
<a href="tel:+201001234567">Call</a>
<a href="https://wa.me/201001234567">WhatsApp</a>
<!-- image as a link -->
<a href="index.html"></a>
<!-- download -->
<a href="cv.pdf" download="Ahmed-CV.pdf">Download CV</a>
```

- **a** = anchor. **href** = hypertext reference = destination.
- **target="_blank"** opens in a new tab. Add **rel="noopener"** for security. Default is **_self**.
- **#id** jumps to the element with that id. **href="#"** alone jumps to top.
- **download** forces download instead of opening; optional value = new file name.
- Link states you can style in CSS: **a:link**, **a:visited**, **a:hover**, **a:active**.

23. Forms & Inputs

Gamal #17 #18 #19 #20

Elzero



```
<form action="/register" method="post">           action = where to send, method = how

  <label for="uname">Username</label>             for = id of the input (click label → focus
input)
  <input type="text" id="uname" name="username"
    placeholder="e.g. ahmed99" required minlength="3" maxlength="15">

  <label for="pw">Password</label>
  <input type="password" id="pw" name="password" required>

  <input type="email" name="email" autocomplete="email">
  <input type="number" name="age" min="18" max="99" step="1" value="18">
  <input type="hidden" name="user_id" value="42">    sent but not visible

  <button type="submit">Register</button>
</form>
```

- **name** — THE most important attribute: it's the key sent to the server (`username=ahmed99`). No name → not sent.
- **id** — connects to `<label for>`. **value** — default/initial value.
- **placeholder** — grey hint text (disappears when typing). Not a replacement for label!
- **required**, **minlength/maxlength**, **min/max/step**, **pattern** — browser validation without JS.
- **disabled** (grey, not sent) vs **readonly** (can't edit, IS sent). **autofocus** — cursor starts here.

All input types

type=	What you get	Notes
text	Single-line text	default
password	Hidden characters	
email	Text + validates "@"	mobile shows @ keyboard; multiple allowed
number	Numeric spinner	min max step
tel	Phone	mobile shows number pad; use pattern
url	Validates a URL	
search	Text with clear (x) button	
date	Calendar picker	value format 2026-09-07
time	Time picker	14:30
datetime-local	Date + time	
month / week	Month or week picker	
color	Color picker	value #ff0000
range	Slider	min max step
file	Upload button	accept="image/*" multiple; form needs enctype="multipart/form-data"
checkbox	☐ many choices	ch. 15
radio	☐ one choice	ch. 15
hidden	Invisible data	
submit / reset / button	Buttons	ch. 16
image	Image acting as submit	src alt

24. Radio, Checkbox, Select, Datalist, Textarea

Gamal #21 #22 #24

Elzero

```

<!-- RADIO: choose ONE. Same name = one group -->
-->
<fieldset>
  <legend>Gender</legend>
  <input type="radio" id="m" name="gender"
value="male" checked>
  <label for="m">Male</label>
  <input type="radio" id="f" name="gender"
value="female">
  <label for="f">Female</label>
</fieldset>

<!-- CHECKBOX: choose MANY -->
<input type="checkbox" id="h" name="skills"
value="html" checked>
<label for="h">HTML</label>
<input type="checkbox" id="c" name="skills"
value="css">
<label for="c">CSS</label>

```

RESULT

Gender

☒ Male ☐ Female☒ HTML ☐ CSS

- **radio**: same **name** → only one can be selected
- **checkbox**: independent; each checked one is sent
- **checked** = selected by default
- **value** is what gets sent (not the label text!)
- **fieldset/legend** group related inputs with a titled border

```

<!-- SELECT: dropdown -->
<label for="city">City</label>
<select id="city" name="city">
  <option value="">-- choose --</option>
  <optgroup label="Egypt">
    <option value="giza" selected>Giza</option>
  </optgroup>
  <option value="cairo">Cairo</option>
</select>

<!-- multiple: hold Ctrl to pick many -->
<select name="langs" multiple size="3">...</select>

<!-- DATALIST: text input WITH suggestions -->
<input list="browsers" name="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
</datalist>

<!-- TEXTAREA: multi-line -->
<textarea name="msg" rows="4" cols="30"
placeholder="Your message"
maxlength="200"></textarea>

```

RESULT

City Chrome

- **select** = dropdown list; **option** = a choice; **selected** = default
- **optgroup** groups options under a label
- **select vs datalist**: select = pick only from list; datalist = type freely + get suggestions
- **input list="x"** must match **datalist id="x"**
- **textarea** has a closing tag; default text goes between the tags, not in value
- rows/cols = visible size (better set with CSS)

25. Buttons & Validation

Gamal #23 #24

Elzero

```

<button type="submit">Send</button>           sends the form (default type inside a form!)
<button type="reset">Clear</button>           resets all fields
<button type="button">Click</button>         does nothing alone – for JavaScript
<button disabled>Wait...</button>
<button> Save</button>  button can contain HTML

<!-- old style – avoid -->
<input type="submit" value="Send">

```

Common bug: a `<button>` inside a form with no `type = submit`. If you only want JS, write `type="button"`.

Built-in validation attributes

```

<input type="text" required>                  cannot be empty
<input type="text" minlength="3" maxlength="10">  length limits
<input type="number" min="1" max="5">           value range
<input type="tel" pattern="01[0-9]{9}" title="Egyptian mobile: 01xxxxxxxxx">
                                                    regex; title = error hint
<form novalidate>                             turn browser validation OFF

```

CSS can react to validation: `input:valid { border-color: green }` `input:invalid { border-color: red }`

26. iframe vs embed vs object

Gamal #25

```

<!-- IFRAME: a web page inside your page (YouTube, Google Maps, other sites) -->
<iframe src="https://www.youtube.com/embed/qfPUMV9J5yw"
        width="560" height="315" title="HTML course"
        allowfullscreen loading="lazy"></iframe>

<!-- EMBED: external content like PDF, plugin media -->
<embed src="book.pdf" type="application/pdf" width="100%" height="500">

<!-- OBJECT: same idea, but has fallback content -->
<object data="book.pdf" type="application/pdf" width="100%" height="500">
  <p>Can't display PDF. <a href="book.pdf">Download</a></p>
</object>

```

	iframe	embed	object
Best for	Other HTML pages, YouTube, maps	PDF / media	PDF / media
Source attr	src	src	data
Closing tag	Yes (empty)	No (void)	Yes
Fallback content	No	No	Yes ☐

Many sites block being put inside an iframe (X-Frame-Options). YouTube works only with the `/embed/ID` URL, not the normal watch URL.

27. Downloads, Entities & Block vs Inline

Gamal #26

Elzero

Download files

```
<a href="files/report.pdf" download>Download report</a>
<a href="files/photo.jpg" download="my-photo.jpg">Save photo</a>  rename on download
```

→ Works only for files on your own domain (same-origin). External URLs will just open.

HTML entities (special characters)

Write	Get	Write	Get	Write	Get
<	<	>	>	&	&
"	"	©	©	®	®
 	non-breaking space	♥	♥	→	→
€	€	£	£	😀	👍

Block vs Inline elements

`<div> / <p> / <h1>` — BLOCK: full width, new line

next block always starts below

`<span>`

`<a>`

`<strong>`

`<img>`

INLINE: only as wide as content, sit side by side
width / height / top-bottom margin are IGNORED on inline

Block	Inline
div, p, h1-h6, ul, ol, li, table, form, header, footer, section, article, nav, main, aside, blockquote, pre, hr	span, a, strong, em, b, i, u, img, input, label, button, select, textarea, code, small, sub, sup, br
Starts on new line, takes full width	Flows within text, width = content
Can contain block + inline	Should contain only inline
width/height/margin all work	width/height ignored (use <code>display:inline-block</code>)

→ You change this with CSS `display` (chapter 28). `img`, `input`, `button` behave as inline-block: inline flow but accept width/height.

28. First HTML Project — Personal Page

Gamal #7

Type this, save as `index.html`, open with Live Server. It uses everything from Part 1.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ahmed | Web Developer</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    
    <h1>Ahmed Mohamed</h1>
    <p>Front-end developer from <strong>Giza, Egypt</strong></p>
    <nav>
      <ul>
        <li><a href="#about">About</a></li>
        <li><a href="#skills">Skills</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About me</h2>
      <p>I started learning web development in <time datetime="2026-09">September 2026</time>.</p>
    </section>

    <section id="skills">
      <h2>Skills</h2>
      <table>
        <tr><th>Skill</th><th>Level</th></tr>
        <tr><td>HTML</td><td>Good</td></tr>
        <tr><td>CSS</td><td>Learning</td></tr>
      </table>
    </section>

    <section id="contact">
      <h2>Contact me</h2>
      <form action="/send" method="post">
        <label for="name">Name</label>
        <input id="name" name="name" type="text" required>
        <label for="email">Email</label>
        <input id="email" name="email" type="email" required>
        <label for="msg">Message</label>
        <textarea id="msg" name="message" rows="4"></textarea>
        <button type="submit">Send</button>
      </form>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Ahmed · <a href="mailto:ahmed@mail.com">ahmed@mail.com</a></p>
  </footer>
</body>
</html>

```


Challenge: add a `<figure>` with your city photo, an ordered list of your goals, and a YouTube `<iframe>` of your favourite lesson.

29. HTML Reference A — Document Structure Tags

These tags build the frame of every page. You will type them in every project you ever make, so learn them until you can write them from memory.

<!DOCTYPE html>

Gamal #2 **Elzero HTML 12:00** **Angela §2**

A declaration (not a tag) that tells the browser which version of HTML the file uses. `<!DOCTYPE html>` means HTML5, the modern standard. Without it the browser enters 'quirks mode' and renders old-fashioned, unpredictable layouts.

سطر يقول للمتصفح: الملف ده HTML5. لازم يكون أول سطر في الملف.

SYNTAX

```
<!DOCTYPE html>
```

- Must be the very first line, before `<html>`.
- It is not case sensitive, but write it exactly like this by habit.
- It has no closing tag because it is not an element.

⚠ Common mistake

Writing it after `<html>` or forgetting it. Symptom: CSS widths and margins behave strangely in some browsers.

<html>

Gamal #2 **Elzero HTML** **Angela §2**

The root element. Every other element lives inside it. It takes the `lang` attribute that tells search engines and screen readers what language the page is written in.

SYNTAX

```
<html lang="en"> ... </html>
<html lang="ar" dir="rtl"> ... </html>  <!-- Arabic page, right-to-left -->
```

Attribute	Meaning	Values
lang	Language of the page	en, ar, fr, de, es ...
dir	Text direction	ltr (default) or rtl

- Only two things go directly inside `<html>`: one `<head>` and one `<body>`.

→ `lang` matters: Google uses it, and browsers offer translation based on it.

<head>

Gamal #5

Elzero HTML

Angela \$2

A container for information **about** the page: its title, character set, links to CSS files, and meta data for search engines and social networks. Nothing inside `<head>` is displayed in the page area.

ال head فيه معلومات عن الصفحة، مش محتوى بيظهر للزائر.

SYNTAX

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page title</title>
  <link rel="stylesheet" href="style.css">
</head>
```

→ Allowed children: `title`, `meta`, `link`, `style`, `script`, `base`.

→ Exactly one `<head>` per page, and it comes before `<body>`.

<title>

Gamal #5

Elzero HTML

Angela \$2

The name of the page. It appears in the browser tab, in bookmarks, in the browser history, and as the blue clickable headline in Google search results.

EXAMPLE

```
<head>
  <title>Giza Bakery | Fresh Bread Daily</
title>
</head>
```

RESULT

Giza Bakery | Fresh Bread Daily ×

(this is how the browser tab looks)

→ Required — a page without a title is invalid HTML.

→ Keep it under about 60 characters so Google does not cut it.

→ Pattern used by most sites: Page name | Site name.

Try it yourself

Give your first page the title About Me | [Your Name]. Save and look at the browser tab.

Solution

```
<title>About Me | Mahmoud</title>
```

<meta>

Gamal #5

Elzero HTML

Angela \$2

Metadata = data about the page. Each `<meta>` tag is one fact: the character encoding, the description for Google, the author, how the page should scale on phones. It is a void (self-closing) element.

SYNTAX

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Fresh bread and cakes in Giza, delivered daily.">
<meta name="keywords" content="bakery, bread, cake, giza">
<meta name="author" content="Mahmoud Mohamed Mahmoud Khattab">
<meta name="robots" content="index, follow">
<meta http-equiv="refresh" content="10">
<!-- social sharing preview (Open Graph) -->
<meta property="og:title" content="Giza Bakery">
<meta property="og:image" content="https://site.com/cover.jpg">
```

Meta	What it does	Beginner rule
charset="UTF-8"	Character encoding — lets Arabic letters, emojis and symbols display correctly	Always include, first line in head
viewport	Tells phones to use their real width and not zoom out	Always include or your page is tiny on mobile
description	Grey text under your link in Google	Write 1-2 sentences, 150 characters max
keywords	Old SEO tag, Google ignores it now	Optional
author	Who wrote the page	Optional
robots	Whether search engines may index the page	Default is index; use noindex for private pages
http-equiv="refresh"	Reload (or redirect) after N seconds	Rarely used; annoying for users
og:* (Open Graph)	Title/image shown when shared on Facebook, WhatsApp, LinkedIn	Add for public sites

⚠ Common mistake

Forgetting `charset` and then seeing Arabic text as `Ø§Ù„Ø¹Ø±Ø²ÙŠØ©`. Fix: add `<meta charset="UTF-8">` as the first line inside head and save the file as UTF-8 (VS Code default).

<link>

Gamal #5

Elzero CSS #02

Angela \$5

Connects an external file to the page. Its main job is to load a CSS stylesheet. It also loads the favicon (the small tab icon) and web fonts. Void element — no closing tag.

SYNTAX

```
<link rel="stylesheet" href="css/style.css">
<link rel="icon" href="favicon.ico">
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Cairo&display=swap" rel="stylesheet">
```

- **rel** = relationship: what is this file to my page? **stylesheet**, **icon**, **preconnect** ...
- **href** = the path or URL of the file.
- You may have many **<link>** tags. Later stylesheets override earlier ones when rules have equal weight.

⚠ Common mistake

Writing **src** instead of **href** (**<link src=...>** does nothing). Images and scripts use **src**; links and anchors use **href**.

<style>

Elzero CSS #02

Angela §5

Holds CSS rules written directly inside the HTML file (internal CSS). Useful for small tests and one-page demos. For real projects, put CSS in a separate file and use **<link>**.

EXAMPLE

```
<head>
  <style>
    body { background: #f4f4f4; }
    h1 { color: darkred; }
  </style>
</head>
<body>
  <h1>Styled from the head</h1>
</body>
```

RESULT

Styled from the head

<script>

Elzero JS #005

Angela §14

Adds JavaScript to the page, either written between the tags or loaded from a file with **src**. Where you put it matters: the browser stops and runs the script the moment it meets the tag.

SYNTAX

```

<!-- external file (recommended) -->
<script src="js/main.js" defer></script>

<!-- inline -->
<script>
  console.log("Hello from JavaScript");
</script>

```

Where / how	Behaviour	Use when
At the end of body	Runs after all HTML above it exists	Classic, always works
In head with <code>defer</code>	Downloads early, runs after HTML is parsed, in order	Best modern default
In head with <code>async</code>	Runs as soon as downloaded, any order	Independent scripts like analytics
In head with nothing	Blocks page; elements below do not exist yet	Avoid
<code>type="module"</code>	Enables import/export; behaves like defer	Modern apps

⚠ Common mistake

Putting `<script>` in the head without `defer`, then `document.querySelector("button")` returns `null` because the button has not been created yet.

<body>

Gamal #2

Elzero HTML

Angela §2

Contains everything the visitor sees and interacts with: text, images, links, forms, videos. There is exactly one body per page.

EXAMPLE

```

<body>
  <h1>Welcome</h1>
  <p>Everything visible goes here.</p>
</body>

```

RESULT

Welcome

Everything visible goes here.

- Old attributes like `bgcolor` and `text` on body are obsolete — use CSS.
- Common CSS on body: `margin:0`, `font-family`, `background`, `line-height`.

? Chapter quiz

1. Which element holds the visible content?
2. Where does the page's title appear?
3. What does the viewport meta tag do?
4. Which attribute does `<link>` use to point to the file — `src` or `href`?

Answers: 1) `body`. 2) In the browser tab and Google results. 3) Makes the page use the real width of a phone screen instead of zooming out. 4) `href`.

30. HTML Reference B — Text Content Tags

Text is 90% of the web. These tags give text structure: headings, paragraphs, lists, quotes. Structure is not decoration — screen readers, search engines and CSS all depend on it.

<h1> to <h6> — headings

Gamal #6

Elzero HTML

Angela §2

Six levels of headings. **h1** is the main title of the page; **h2** are the main sections; **h3** are sub-sections inside an **h2**, and so on. Think of them as the outline of a document, like chapter and section numbers in a book.

العناوين من h1 لـ h6. h6 هو عنوان الصفحة الرئيسي (واحد بس)، و h2 للأقسام، و h3 للأقسام الفرعية.

EXAMPLE

```
<h1>Learn Web Development</h1>
<h2>Part 1: HTML</h2>
<h3>Chapter 1: Tags</h3>
<h3>Chapter 2: Attributes</h3>
<h2>Part 2: CSS</h2>
<h4>Level four</h4>
<h5>Level five</h5>
<h6>Level six</h6>
```

RESULT

**Learn Web
Development**

Part 1: HTML

Chapter 1: Tags

Chapter 2: Attributes

Part 2: CSS

Level four

Level five

Level six

- Use **one h1** per page. It should say what the page is about.
- Do not skip levels: h1 → h2 → h3. Never jump h1 → h4.
- Never choose a heading level because of its size. Size is CSS's job: `h2 { font-size: 40px }` is fine.
- Headings are block elements: each starts on a new line and has default margins.

⚠ Common mistake

Using `<h3>` because 'h2 looked too big'. Search engines read this as bad structure. Choose the level by meaning and fix the size in CSS.

✍ Try it yourself

Write the heading outline for a recipe page: the dish name, then sections 'Ingredients' and 'Steps', and under Steps two sub-sections 'Preparation' and 'Cooking'.

Solution

```
<h1>Koshari</h1>
<h2>Ingredients</h2>
<h2>Steps</h2>
<h3>Preparation</h3>
<h3>Cooking</h3>
```

<p> — paragraph

Gamal #6 **Elzero HTML** **Angela \$2**

A block of text. The browser puts space above and below each paragraph. Inside a paragraph, several spaces or line breaks in your code are collapsed into one single space — this is called whitespace collapsing.

EXAMPLE

```
<p>This is the first paragraph.    Extra
spaces
  and line breaks in the code
  are ignored.</p>
<p>This is the second paragraph.</p>
```

RESULT

This is the first paragraph. Extra spaces and line breaks in the code are ignored.

This is the second paragraph.

- Paragraphs cannot contain other block elements (no `<div>` or `<p>` inside a `<p>`). The browser will silently close your paragraph early and your CSS will look broken.
- To force a line break inside a paragraph use `<br>`.

⚠ Common mistake

Using empty `<p></p>` or many `<br>` tags to create vertical space. Use CSS `margin` instead.

**
 — line break**

Gamal #6 **Elzero HTML**

Forces the text to continue on the next line. Void element — no closing tag. Correct use: addresses, poems, song lyrics — places where the line break is part of the content.

EXAMPLE

```
<p>Mahmoud Mohamed Mahmoud Khattab<br>
12 Pyramids Road<br>
Giza, Egypt</p>
```

RESULT

Mahmoud Mohamed Mahmoud Khattab
 12 Pyramids Road
 Giza, Egypt

⚠ Common mistake

Using `<br><br><br>` to push things down. That is layout, and layout belongs to CSS (margin, padding).

<hr> — horizontal rule

Gamal #6 **Elzero HTML**

A thematic break: a horizontal line that separates two topics inside a page, for example between scenes in a story or between sections of a form. Void element.

EXAMPLE

```
<p>End of the introduction.</p>
<hr>
<p>Start of the main content.</p>
```

RESULT

End of the introduction.

Start of the main content.

Tip: Style it with CSS: `hr { border: 0; border-top: 2px solid #ddd; margin: 30px 0; }`

<pre> — preformatted text

Gamal #9 **Elzero HTML**

Shows text exactly as typed: every space and every line break is kept, in a monospace font. Used for code listings, ASCII art, and anything where spacing matters.

EXAMPLE

```
<pre>
function hi() {
    return "spaces kept";
}
</pre>
```

RESULT

```
function hi() {
    return "spaces kept";
}
```

- Combine with `<code>` for code: `<pre><code>...</code></pre>`.
- Inside pre you still need the entity code for a less-than sign (see the entities table).

<blockquote> and <q> — quotations

Gamal #9 **Elzero HTML**

`<blockquote>` is for a long quotation that stands on its own as a block (browsers indent it).
`<q>` is for a short quote inside a sentence (browsers add quotation marks automatically).
`cite` holds the source URL.

EXAMPLE

```
<blockquote cite="https://example.com/speech">
  The best way to learn to code is to write
  code every day.
</blockquote>
<p>As my teacher said, <q>practice beats
theory</q>.</p>
```

RESULT

The best way to learn to code is to
write code every day.

As my teacher said, “practice beats theory”.

<abbr>, <address>, <time>

Gamal #9

Elzero HTML

Small semantic tags that add meaning to text. `<abbr>` marks an abbreviation and shows the full form as a tooltip. `<address>` wraps contact information for the page or an article. `<time>` marks a date or time in a machine-readable way.

EXAMPLE

```
<p>I study <abbr title="HyperText Markup
Language">HTML</abbr>.</p>
<address>
  Written by <a
href="mailto:m@site.com">Mahmoud</a><br>
  Giza, Egypt
</address>
<p>Published on <time datetime="2026-09-07">7
September 2026</time></p>
```

RESULT

I study HTML.

Written by [Mahmoud](#)

Giza, Egypt

Published on 7 September 2026

→ `datetime` uses the ISO format YYYY-MM-DD or YYYY-MM-DDTHH:MM. Computers read that; humans read the text between the tags.

, , — lists

Gamal #14

Elzero HTML

Angela \$3

Lists group related items. `<ul>` = unordered list (bullets), when order does not matter. `<ol>` = ordered list (numbers), when order matters like recipe steps. Every item is an `<li>` (list item). Navigation menus are almost always lists.

ul قائمة نقاط (الترتيب مش مهم)، ol قائمة مرقمة (الترتيب مهم)، وكل عنصر جواها li.

Attribute (on ol)	Effect	Example
type	Numbering style: 1, A, a, I, i	<code><ol type="A"></code> → A. B. C.
start	First number	<code><ol start="5"></code> → 5. 6. 7.
reversed	Count down	<code><ol reversed></code> → 3. 2. 1.
value (on li)	Set the number of one item	<code><li value="10"></code>

EXAMPLE

```
<h3>Shopping</h3>
<ul>
  <li>Bread</li>
  <li>Eggs</li>
  <li>Milk</li>
</ul>

<h3>Make tea</h3>
<ol>
  <li>Boil water</li>
  <li>Add tea</li>
  <li>Wait 3 minutes</li>
</ol>
```

RESULT

Shopping

- Bread
- Eggs
- Milk

Make tea

1. Boil water
2. Add tea
3. Wait 3 minutes

→ Only `<li>` may be a direct child of `ul/ol`. Text or `div`s directly inside a `ul` is invalid.

→ To nest a list, put the inner `<ul>` **inside** an `<li>`, not between two `li` tags.

⚠ **Common mistake**

Nesting wrongly: `<ul><li>A</li><ul>...</ul></ul>`. Correct: `<ul><li>A <ul>...</ul> </li></ul>`.

🔧 **Try it yourself**

Create a nested list: 'Front end' with items HTML, CSS, JavaScript; and 'Back end' with items Node, SQL.

Solution

```
<ul>
  <li>Front end
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Back end
    <ul>
      <li>Node</li>
      <li>SQL</li>
    </ul>
  </li>
</ul>
```

<dl>, <dt>, <dd> — description list**Gamal #14** **Elzero HTML**

A list of terms and their descriptions, like a glossary or a product's specification table. **dl** = description list, **dt** = term, **dd** = description (details).

EXAMPLE

```
<dl>
  <dt>HTML</dt>
  <dd>Gives the page its structure.</dd>
  <dt>CSS</dt>
  <dd>Gives the page its style.</dd>
</dl>
```

RESULT**HTML**

Gives the page its structure.

CSS

Gives the page its style.

<div> — generic block container**Gamal #15** **Elzero HTML** **Angela §7**

A box with no meaning of its own. It groups other elements so you can style or position them together with CSS. It is a block element: it starts on a new line and stretches to the full width of its parent. If a semantic tag (header, nav, section, article...) fits, use that instead.

div صندوق عام مالوش معنى — بنستخدمه عشان نجمع عناصر ونعمل لها تنسيق سوا.

EXAMPLE

```
<div class="card">
  <h2>Product</h2>
  <p>Price: 20 EGP</p>
</div>
<div class="card">
  <h2>Another product</h2>
  <p>Price: 35 EGP</p>
</div>
```

RESULT

Product

Price: 20 EGP

Another product

Price: 35 EGP

- Give divs a `class` so CSS can find them: `.card { border: 1px solid #ccc }.`
- 'Div soup' (dozens of nested divs) is a sign you should use semantic tags and CSS Grid/Flexbox instead.

 — generic inline container

Gamal #15 **Elzero HTML**

The inline cousin of div. It wraps a piece of text inside a line without breaking the line, so you can colour or style just that piece.

EXAMPLE

```
<p>The price is <span class="price">20 EGP</span> only today.</p>
<style>.price { color: red; font-weight: bold; }</style>
```

RESULT

The price is **20 EGP** only today.

- span has no default styling at all — invisible until you add CSS.
- Because it is inline, `width` and `height` do not apply unless you set `display: inline-block`.

? Chapter quiz

1. How many h1 tags should a page have?
2. What happens to five spaces in a row inside a paragraph?
3. Which list tag would you use for the steps of a recipe?
4. When should you use a div instead of a section?

Answers: 1) One. 2) They collapse to one space. 3) ol (ordered list). 4) Only when the group has no meaning and is purely for styling/layout.

31. HTML Reference C — Inline Text Formatting Tags

Formatting Tags

These tags change or add meaning to a few words inside a sentence. Some are purely visual (bold, italic); others carry meaning that screen readers and search engines use. Prefer the meaningful ones.

 and — bold

Gamal #9 Elzero HTML Angela \$2

Both display bold text. `<strong>` means 'this is important' (semantic). `<b>` means 'make this bold with no special importance' (visual, e.g. product names in a review). Screen readers may stress `strong`.

EXAMPLE

```
<p><strong>Warning:</strong> do not turn off
the power.</p>
<p>The <b>Samsung A54</b> costs 12,000 EGP.</p>
```

RESULT

Warning: do not turn off the power.
The **Samsung A54** costs 12,000 EGP.

 and <i> — italic

Gamal #9 Elzero HTML

Both display italic text. `<em>` = emphasis, changes the meaning of the sentence when spoken. `<i>` = alternate voice: a foreign word, a technical term, a ship name, a thought.

EXAMPLE

```
<p>I <em>really</em> need coffee.</p>
<p>The word <i>habibi</i> means my dear.</p>
```

RESULT

I *really* need coffee.
The word *habibi* means my dear.

<u>, <s>, , <ins>, <mark>, <small>

Gamal #9 Elzero HTML

A family of tags that decorate text: underline, strikethrough, deleted and inserted (for showing edits), highlight, and fine print.

Tag	Meaning	Default look
u	Unarticulated annotation (misspelling, proper name in Chinese)	underline
s	No longer relevant	strikethrough
del	Removed in an edit (cite, datetime attributes)	strikethrough
ins	Added in an edit	underline
mark	Highlighted for reference	yellow background
small	Side comment, fine print, copyright	smaller text

EXAMPLE

```
<p><u>Underlined</u> – avoid, looks like a link.</p>
<p>Old price <del>500 EGP</del> new price <ins>350 EGP</ins></p>
<p><s>No longer accurate</s></p>
<p>Search result: the <mark>keyword</mark> is highlighted.</p>
<p><small>Terms and conditions apply.</small></p>
```

RESULT

Underlined — avoid, looks like a link.

Old price ~~500 EGP~~ new price 350 EGP

~~No longer accurate~~

Search result: the **keyword** is highlighted.

Terms and conditions apply.

<sub> and <sup> — subscript and superscript

Gamal #9 **Elzero HTML**

Lower or raise text for chemical formulas, footnotes, exponents and ordinals.

EXAMPLE

```
<p>Water is H<sub>2</sub>O.</p>
<p>E = mc<sup>2</sup></p>
<p>The 1<sup>st</sup> of June<sup>[1]</sup></p>
```

RESULT

Water is H₂O.

E = mc²

The 1st of June^[1]

<code>, <kbd>, <samp>, <var>

Gamal #9 **Elzero HTML**

Tags for technical writing: a piece of code, a key the user presses, sample output from a program, and a variable name.

EXAMPLE

```
<p>Type <code>npm install</code> and press <kbd>Enter</kbd>.</p>
<p>The program prints <samp>Done!</samp></p>
<p>Let <var>x</var> be the width.</p>
```

RESULT

Type `npm install` and press `Enter`.

The program prints `Done!`

Let `x` be the width.

<bdo> and <bdi> — text direction

Gamal CSS #13

Rarely needed but useful for Arabic sites. `<bdo dir="rtl">` forces a direction. `<bdi>` isolates a piece of text (like a user name) so its direction does not mess up the surrounding sentence.

EXAMPLE

```
<p>User <bdi>محمود</bdi> scored 90 points.</p>
<p><bdo dir="rtl">This text is reversed</bdo></p>
```

RESULT

User محمود scored 90 points.

This text is reversed

Character entities

Gamal #9

Elzero HTML

Some characters cannot be typed directly because HTML uses them for tags, or because they are not on your keyboard. An entity starts with `&` and ends with `;`. In the table below, the left column shows what you type and the right column what the visitor sees.

بعض الرموز زي > و < و & ليها استخدام خاص في HTML، فعشان تظهر كنص لازم تكتبها بصيغة خاصة تبدأ بـ & وتنتهي بـ ;

You type	Visitor sees	Name
<	<	less than
>	>	greater than
&	&	ampersand
"	"	double quote
'	'	apostrophe
 	(a space that never breaks the line)	non-breaking space
©	©	copyright
®	®	registered
™	™	trademark
€	€	euro
£	£	pound
¥	¥	yen
→	→	right arrow
←	←	left arrow
♥	♥	heart
×	×	multiplication / close icon
…	...	ellipsis
—	—	em dash
م	␣	any character by its number

EXAMPLE

```
<p>Use the &lt;p>&gt; tag for paragraphs.</p>
<p>Tom &amp; Jerry &copy; 2026</p>
<p>Price: 5&nbsp;EGP</p>
```

RESULT

Use the <p> tag for paragraphs.

Tom & Jerry © 2026

Price: 5 EGP

⚠ Common mistake

Writing a bare **&** in text such as 'Tom & Jerry'. Browsers usually forgive it, but 'AT&T' followed by certain letters can be misread. Always write **&**.

? Chapter quiz

1. Which tag should you use for a warning that is important: `b` or `strong`?
2. Write the entity for the copyright symbol.
3. What does ` ` do?

Answers: 1) `strong`. 2) `©` 3) Inserts a space that will never be used as a line-break point — keeps '5 EGP' together.

32. HTML Reference D — Links, Images, Audio and Video

Links are what make the web a web. Images and media make it alive. This chapter covers every attribute you will use for them.

<a> — anchor (link)

Gamal #16

Elzero HTML

Angela \$3

Creates a clickable link. The `href` attribute (hypertext reference) says where it goes: another website, another page in your site, a place on the same page, an email address, a phone number, or a file to download.

الـ a بتعمل لينك. href بتحدد اللينك رايع فين.

SYNTAX

```
<a href="URL" target="_blank" rel="noopener" title="tooltip" download>Link text</a>
```

Attribute	Meaning	Values / notes
href	Destination	URL, path, #id, mailto:, tel:
target	Where to open	_self (default, same tab), _blank (new tab)
rel	Relationship / security	noopener (with _blank), nofollow (tell Google not to count it)
download	Download instead of open	optional value = new file name; same-origin files only
title	Tooltip on hover	any text

EXAMPLE

```
<a href="https://developer.mozilla.org">MDN
  (external)</a><br>
<a href="about.html">About page (same site)</a><br>
<a href="#pricing">Jump to pricing (same
  page)</a><br>
<a href="mailto:hi@site.com?
  subject=Hello">Email us</a><br>
<a href="tel:+201141225852">Call us</a><br>
<a href="https://wa.me/
  201141225852">WhatsApp</a><br>
<a href="files/menu.pdf" download>Download
  menu</a><br>
<a href="https://google.com" target="_blank"
  rel="noopener">Opens in new tab</a>
```

RESULT

[MDN \(external\)](https://developer.mozilla.org)
[About page \(same site\)](#)
[Jump to pricing \(same page\)](#)
[Email us](#)
[Call us](#)
[WhatsApp](#)
[Download menu](#)
[Opens in new tab](#)

- Link text should describe the destination: 'Read the pricing page', not 'click here'.
- A link can wrap anything: an image, a whole card, a heading.

- To link to a section on the same page, give that section an `id` and use `href="#thatid"`.
`href="#"` alone scrolls to the top.
- Relative paths: `about.html` (same folder), `pages/about.html` (subfolder), `../about.html` (parent folder), `/about.html` (site root).

⚠ Common mistake

Forgetting the `https://` in an external link: `href="google.com"` looks for a file called `google.com` inside your own site.

✎ Try it yourself

Build a small navigation: Home (`index.html`), Services (`services.html`), Contact (a link that jumps to a section with `id 'contact'` on the same page).

Solution

```
<nav>
  <a href="index.html">Home</a>
  <a href="services.html">Services</a>
  <a href="#contact">Contact</a>
</nav>
...
<section id="contact">...</section>
```

 — image

Gamal #10

Elzero HTML

Angela §3

Embeds an image. Void element. `src` is the path to the image file; `alt` is text that describes the image for people who cannot see it (blind users with screen readers, or when the file fails to load). Both are required.

img بتحت صورة. src مكان الصورة، alt وصف للصورة لو مطهرتش أو لضعاف البصر.

SYNTAX

```

```

Attribute	Meaning	Notes
src	Image file path or URL	required
alt	Text alternative	required; empty <code>alt=""</code> for purely decorative images
width / height	Size in pixels (number only, no px)	setting both prevents the page from jumping while loading
title	Tooltip	optional
loading="lazy"	Load only when scrolled near	use for images below the first screen
srcset / sizes	Different files for different screen sizes	advanced responsive images

EXAMPLE

```

```

RESULT



[pyramids.jpg 300×200]

If the file is missing the browser shows: The three pyramids of Giza at sunset

- Image formats: **JPG** for photos, **PNG** for logos/transparency, **SVG** for icons (scales perfectly), **WebP** modern and small, **GIF** for simple animations.
- Make every image responsive with CSS: `img { max-width: 100%; height: auto; }`.
- Compress images before uploading (squoosh.app). A 5 MB photo makes your page slow.

⚠ Common mistake

Path is wrong. If your HTML is in the main folder and the image is in `images/`, the path is `images/cat.jpg` — not `cat.jpg`, and not `C:\Users\...\cat.jpg` (that only works on your computer).

<figure> and <figcaption>

Gamal #10

Elzero HTML

Wraps an image (or chart, code block, video) together with its caption, so the two are linked as one unit.

EXAMPLE

```
<figure>
  
  <figcaption>Figure 1: Sales growth in 2026</figcaption>
</figure>
```

RESULT

[chart.png]

Figure 1: Sales growth in 2026

<picture> — responsive image sources

Gamal #10

Lets the browser choose between several image files depending on the screen width or the format it supports. The `<img>` inside is the fallback and is required.

EXAMPLE

```
<picture>
  <source media="(max-width: 600px)" srcset="hero-small.jpg">
  <source type="image/webp" srcset="hero.webp">
  
</picture>
```

- The browser reads sources top to bottom and uses the first that matches.
- Use it to send a small image to phones and a large one to desktops — faster mobile pages.

<audio>

Gamal #11

Elzero HTML

Plays sound. Add `controls` to show the play/pause bar. Provide several `<source>` files so every browser finds a format it can play.

Attribute	Effect
controls	Show the player bar (without it the audio is invisible)
autoplay	Start automatically — browsers block it unless muted
loop	Repeat forever
muted	Start with sound off
preload	none / metadata / auto — how much to download before play

EXAMPLE

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>
```

RESULT

▶ 0:00 / 3:24

<video>

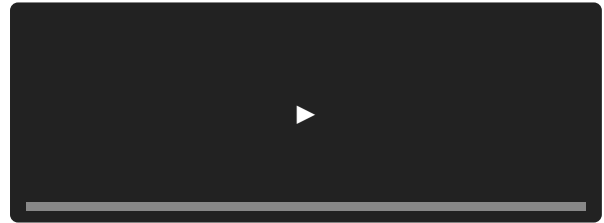
Gamal #12 **Elzero HTML**

Plays video. Same attributes as audio, plus `width`, `height` and `poster` (the image shown before play). `<track>` adds subtitles.

EXAMPLE

```
<video controls width="480"
poster="thumbnail.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <source src="lesson.webm" type="video/webm">
  <track src="subs-ar.vtt" kind="subtitles"
  srclang="ar" label="Arabic">
  Your browser does not support video.
</video>
```

RESULT



- Background hero videos: `<video autoplay muted loop playsinline>` — muted is what allows autoplay.
- Videos are heavy. For anything longer than a few seconds, upload to YouTube and embed with an `iframe` (next entry).

<iframe> — a page inside your page

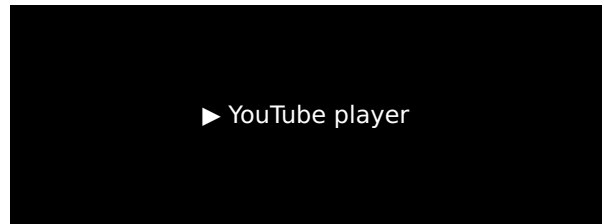
Gamal #25 **Elzero HTML**

Embeds another web page in a rectangle: a YouTube video, a Google Map, a form. The outer page cannot read what is inside it (security).

EXAMPLE

```
<iframe src="https://www.youtube.com/embed/
qfPUMV9J5yw"
width="560" height="315"
title="HTML course by Elzero"
allowfullscreen loading="lazy"></
iframe>
```

RESULT



- YouTube: click Share → Embed and copy the code. The URL must contain `/embed/`.
- Google Maps: Share → Embed a map.
- Many sites forbid being embedded; you will see a blank box. That is their choice, not your bug.
- Always add `title` for accessibility.

<embed> and <object>

Gamal #25

Older tags for embedding external content such as PDF files. `<object>` can show fallback content if the file cannot be displayed; `<embed>` cannot.

EXAMPLE

```
<object data="menu.pdf" type="application/pdf" width="100%" height="500">
  <p>Cannot display the PDF. <a href="menu.pdf">Download it</a>.</p>
</object>
```

→ For PDFs, a simple download link is often the better user experience on mobile.

<svg> and <canvas> — drawing

Elzero HTML

`<svg>` draws vector shapes with tags (circles, paths) that scale without blur — every icon in this book's diagrams is SVG. `<canvas>` is a blank bitmap you draw on with JavaScript (games, charts).

EXAMPLE

```
<svg width="120" height="60">
  <circle cx="30" cy="30" r="25"
  fill="tomato"/>
  <rect x="65" y="5" width="50" height="50"
  fill="steelblue" rx="8"/>
</svg>
```

RESULT



→ Icons: copy SVG code from heroicons.com or fontawesome and paste it inline; colour it with CSS `fill: currentColor`.

? Chapter quiz

1. Which attribute makes a link open in a new tab?
2. Why is alt required on images?
3. What does poster do on a video?
4. Which URL form does YouTube require inside an iframe?

Answers: 1) `target="_blank"` (add `rel="noopener"`). 2) For screen readers, for when the image fails to load, and for search engines. 3) Shows an image before the video starts. 4) `https://www.youtube.com/embed/VIDEO_ID`

33. HTML Reference E — Tables

Tables are for data that has rows and columns: prices, schedules, results. They are not for page layout — that is what CSS Grid is for.

<table>, <tr>, <th>, <td>

Gamal #13

Elzero HTML

Angela §3

A table is built row by row. `<table>` wraps everything; each `<tr>` is a table row; inside a row each `<th>` is a header cell (bold, centered) and each `<td>` is a data cell.

table الجدول، tr صف، th خلية عنوان، td خلية بيانات.

EXAMPLE

```
<table>
  <tr>
    <th>Day</th>
    <th>Open</th>
    <th>Close</th>
  </tr>
  <tr>
    <td>Saturday</td>
    <td>9:00</td>
    <td>22:00</td>
  </tr>
  <tr>
    <td>Friday</td>
    <td>13:00</td>
    <td>23:00</td>
  </tr>
</table>
```

RESULT

Day	Open	Close
Saturday	9:00	22:00
Friday	13:00	23:00

(borders added with CSS: `td, th { border: 1px solid #999 }`; by default tables have no lines)

- Every row must have the same number of cells (or use `colspan` to make up the difference).
- The old `border="1"` attribute works but is obsolete; use CSS.
- Add `scope="col"` or `scope="row"` on `th` for screen readers.

<thead>, <tbody>, <tfoot>, <caption>

Gamal #13

Elzero HTML

Semantic groups inside a table: the header rows, the body rows, and a footer (totals).

`<caption>` is the table's title, placed right after the opening table tag.

EXAMPLE

```

<table>
  <caption>Monthly expenses</caption>
  <thead>
    <tr><th>Item</th><th>EGP</th></tr>
  </thead>
  <tbody>
    <tr><td>Rent</td><td>4000</td></tr>
    <tr><td>Food</td><td>2500</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>6500</td></tr>
  </tfoot>
</table>

```

RESULT

Monthly expenses

Item	EGP
Rent	4000
Food	2500
Total	6500

- When printing a long table, browsers repeat thead on every page.
- CSS can style them separately: `thead { background: #333; color: #fff }.`

colspan and rowspan — merging cells

Gamal #13

Elzero HTML

`colspan="n"` makes a cell as wide as n columns. `rowspan="n"` makes a cell as tall as n rows. When you merge, remove the cells that the merged one now covers.

EXAMPLE

```

<table>
  <tr>
    <th colspan="2">Contact</th>
  </tr>
  <tr>
    <td rowspan="2">Mahmoud</td>
    <td>0103 411 7684</td>
  </tr>
  <tr>
    <td>m@site.com</td>
  </tr>
</table>

```

RESULT

Contact	
Mahmoud	0103 411 7684
	m@site.com

⚠ Common mistake

Keeping the extra cell after adding rowspan. Count: if row 1 has a cell with rowspan=2, then row 2 needs one cell fewer.

🔧 Try it yourself

Make a 3-column timetable (Time, Saturday, Sunday) with two rows, where the 10:00 row says 'Free' across both days using colspan.

Solution

```
<table>
  <tr><th>Time</th><th>Saturday</th><th>Sunday</th></tr>
  <tr><td>9:00</td><td>HTML</td><td>CSS</td></tr>
  <tr><td>10:00</td><td colspan="2">Free</td></tr>
</table>
```

<colgroup> and <col>**Elzero HTML**

Apply styles to whole columns without repeating a class on every cell.

EXAMPLE

```
<table>
  <colgroup>
    <col style="background:#fff8e1">
    <col span="2">
  </colgroup>
  <tr><th>Name</th><th>Q1</th><th>Q2</th></tr>
  <tr><td>Sales</td><td>10</td><td>12</td></tr>
</table>
```

RESULT

Name	Q1	Q2
Sales	10	12

Making tables usable on phones**Gamal CSS #33**

A wide table does not fit on a phone. The simplest fix is to let the table scroll sideways inside a wrapper.

EXAMPLE

```
<div style="overflow-x: auto;">
  <table>...</table>
</div>
```

→ Other options: hide less important columns with a media query, or turn each row into a card with CSS `display: block`.

? Chapter quiz

1. Which tag is a table row?
2. What is the difference between th and td?
3. A cell should be two columns wide — which attribute?

Answers: 1) tr. 2) th is a header cell (bold, centered, meaningful as a label); td is a normal data cell. 3) colspan="2".

34. HTML Reference F — Forms and Every Input Type

Forms are how users send data to you: sign up, log in, search, comment, pay. Every input type is covered here with its attributes. When you reach the back-end part of this book you will receive this data on the server.

<form>

Gamal #17 #18

Elzero HTML

Angela \$4

The container for all the fields. **action** is the URL that receives the data. **method** is how it is sent: **get** puts the data in the URL (good for searches, bookmarkable), **post** hides it in the request body (required for passwords, uploads, anything that changes data).

form بتجمع الحقول. action الرابط اللي هيستقبل البيانات، method طريقة الإرسال (get في الرابط، post مخفية).

SYNTAX

```
<form action="/register" method="post" enctype="multipart/form-data" autocomplete="on" novalidate>
  ...fields...
  <button type="submit">Send</button>
</form>
```

Attribute	Meaning	When
action	URL that handles the data	Leave empty to send to the same page
method	get or post	post for anything private or that changes data
enctype	How to encode the data	multipart/form-data is required when uploading files
autocomplete	Let browser fill known values	on (default) / off
novalidate	Turn off browser validation	When you validate with JavaScript yourself
target	Where the response opens	_blank etc., rarely used

- Pressing Enter inside a text field submits the form.
- Without a back end, submitting just reloads the page. Part 5 teaches you how to receive the data.
- Test what a GET form sends by looking at the URL after submit: **?name=Ahmed&age=25**.

<label>

Gamal #17

Elzero HTML

Angela \$4

The visible name of a field. Connecting it to the input with `for` = the input's `id` means clicking the label focuses the input (bigger click target on phones) and screen readers read the label when the user reaches the field.

EXAMPLE

```
<!-- method 1: for + id (preferred) -->
<label for="email">Email address</label>
<input type="email" id="email" name="email">

<!-- method 2: wrap the input -->
<label>
  Phone
  <input type="tel" name="phone">
</label>
```

RESULT

Email address

Phone

⚠ Common mistake

Using only `placeholder` instead of a label. Placeholder disappears when typing and is too light to read; it is not a label.

<input> — the attributes every type shares

Gamal #17 #19

Elzero HTML

Angela \$4

A single-line field. Its `type` decides what it looks like and what it accepts. Void element.

The most important attribute is `name`: it is the key the server receives. No name → the field is not sent at all.

SYNTAX

```
<input type="text" name="username" id="username" value="" placeholder="e.g. ahmed99"
      required minlength="3" maxlength="20" pattern="[a-z0-9]+" autofocus
      autocomplete="username">
```

Attribute	Meaning
type	Kind of field (see table below)
name	Key sent to the server. Required for the data to be sent
id	Unique identifier; connects to label and to JavaScript
value	Initial value (or the data sent for checkboxes/radios/buttons)
placeholder	Grey hint text shown when empty
required	Cannot be submitted empty
minlength / maxlength	Length limits for text
min / max / step	Range and increments for number, range, date
pattern	A regular expression the value must match
readonly	Cannot be edited, but IS sent
disabled	Greyed out, cannot be edited, NOT sent
autofocus	Cursor starts in this field
autocomplete	Hint for the browser: email, name, tel, new-password, off
list	id of a datalist to suggest values
multiple	Allow several values (email, file)
accept	Allowed file types for type=file, e.g. image/*
checked	Pre-selected (checkbox, radio)
size	Visible width in characters

Text-like inputs: text, password, email, tel, url, search, number

Gamal #19 **Elzero HTML**

They all look like a text box, but the type changes validation and the keyboard shown on phones (an @ key for email, a number pad for tel).

EXAMPLE

```

<label for="u">Username</label>
<input type="text" id="u" name="username"
required>

<label for="p">Password</label>
<input type="password" id="p" name="password"
minlength="8" required>

<label for="e">Email</label>
<input type="email" id="e" name="email"
placeholder="you@example.com">

<label for="t">Mobile</label>
<input type="tel" id="t" name="phone"
pattern="01[0-9]{9}"
title="11 digits starting with 01">

<label for="w">Website</label>
<input type="url" id="w" name="site">

<label for="s">Search</label>
<input type="search" id="s" name="q">

<label for="a">Age</label>
<input type="number" id="a" name="age"
min="18" max="99" step="1" value="18">

```

RESULT

Username

Password

Email

Mobile

Website

Search

Age

- email and url are validated by the browser on submit: 'abc' in an email field shows an error bubble.
- number: use `step="0.01"` for prices, `step="any"` to allow any decimal.
- tel is NOT validated automatically (phone formats vary) — add a `pattern`.
- password hides characters but sends them as plain text — that is why the form must use POST and the site must use HTTPS.

Choice inputs: checkbox and radio

Gamal #21

Elzero HTML

Angela \$4

Radio buttons let the user pick exactly **one** option from a group; all radios in the group share the same `name`. **Checkboxes** are independent yes/no switches; the user can tick many. For both, the `value` attribute is what gets sent, not the label text.

radio بتختار اختيار واحد بس من مجموعة (نفس الـ name). checkbox تقدر تختار أكثر من واحد.

EXAMPLE

```

<fieldset>
  <legend>Delivery</legend>
  <input type="radio" id="pick"
name="delivery" value="pickup" checked>
  <label for="pick">Pick up</label>
  <input type="radio" id="home"
name="delivery" value="home">
  <label for="home">Home delivery</label>
</fieldset>

<fieldset>
  <legend>Extras</legend>
  <input type="checkbox" id="c1"
name="extras" value="cheese">
  <label for="c1">Extra cheese</label>
  <input type="checkbox" id="c2"
name="extras" value="sauce" checked>
  <label for="c2">Sauce</label>
</fieldset>

<input type="checkbox" id="terms"
name="terms" required>
<label for="terms">I accept the terms</label>

```

RESULT

Delivery

☒ Pick up ☐ Home delivery

Extras

☐ Extra cheese ☒ Sauce

☐ I accept the terms

Sent: delivery=pickup&extras=sauce

- A radio group cannot be un-selected once chosen — give one option `checked` by default, or add a 'None' option.
- Several checkboxes may share a name (extras) — the server receives an array.
- `<fieldset>` + `<legend>` group related choices with a title; screen readers announce the legend.

⚠ Common mistake

Giving radios different names. Then the user can select all of them at once, because each is its own group.

🔧 Try it yourself

Make a 'Size' radio group (S, M, L — M selected by default) and a required 'Subscribe to newsletter' checkbox.

Solution

```
<fieldset>
  <legend>Size</legend>
  <input type="radio" id="s" name="size" value="S"> <label for="s">S</label>
  <input type="radio" id="m" name="size" value="M" checked> <label for="m">M</label>
  <input type="radio" id="l" name="size" value="L"> <label for="l">L</label>
</fieldset>
<input type="checkbox" id="news" name="newsletter" required>
<label for="news">Subscribe to newsletter</label>
```

Date and time inputs

Gamal #20

Elzero HTML

Native pickers that return values in a fixed machine format regardless of the user's country, which saves you from parsing '07/09/2026' vs '09/07/2026'.

type	Value format sent	Example
date	YYYY-MM-DD	2026-09-07
time	HH:MM	14:30
datetime-local	YYYY-MM-DDTHH:MM	2026-09-07T14:30
month	YYYY-MM	2026-09
week	YYYY-Www	2026-W36

EXAMPLE

```
<input type="date" name="birthday"
min="1950-01-01" max="2026-12-31">
<input type="time" name="appointment"
min="09:00" max="18:00" step="900">
<input type="datetime-local" name="meeting">
<input type="month" name="card-expiry">
<input type="week" name="week">
```

RESULT

dd / mm / yyyy

-- : --

dd/mm/yyyy, --:--

----- ▾

- step on time is in seconds: 900 = 15-minute steps.
- The picker's look is decided by the browser and cannot be fully styled; libraries like flatpickr exist if you need a custom look.

Other inputs: color, range, file, hidden

Gamal #19

Elzero HTML

Special-purpose inputs. `file` needs the form to have `method="post"` and `enctype="multipart/form-data"`. `hidden` sends a value the user does not see (an id, a token).

EXAMPLE

```
<label for="col">Theme colour</label>
<input type="color" id="col" name="theme"
value="#1f77c4">

<label for="vol">Volume</label>
<input type="range" id="vol" name="volume"
min="0" max="100" step="10" value="50">

<label for="cv">Upload CV</label>
<input type="file" id="cv" name="cv"
accept=".pdf,.docx">
<input type="file" name="photos"
accept="image/*" multiple>

<input type="hidden" name="product_id"
value="42">
```

RESULT

Theme colour 

Volume 

Upload CV No file chosen

(hidden input — nothing visible)

- range gives no visible number; show it with a little JavaScript: `vol.oninput = () => out.textContent = vol.value`.
- `accept` only filters the file dialog — always check the file type on the server too.

<textarea> — multi-line text

Gamal #24 **Elzero HTML** **Angela §4**

For messages, comments, addresses — anything longer than one line. Unlike input, it has a closing tag, and its default text goes between the tags, not in a value attribute.

EXAMPLE

```
<label for="msg">Message</label>
<textarea id="msg" name="message" rows="4"
cols="40"
placeholder="Write your message..."
maxlength="500" required></textarea>
```

RESULT

Message

Write your message...

- rows/cols set the visible size; better: CSS `width: 100%; height: 120px`.
- Stop the user resizing with CSS `resize: none` or allow only vertical: `resize: vertical`.
- Whitespace between the tags becomes content — keep `<textarea></textarea>` tight together.

<select>, <option>, <optgroup> — drop-down list

Gamal #22 **Elzero HTML** **Angela §4**

A list the user opens and picks from. Each `<option>` has a `value` (sent to the server) and text (shown to the user). `selected` marks the default. `<optgroup>` puts a bold label above a group of options.

EXAMPLE

```
<label for="city">City</label>
<select id="city" name="city" required>
  <option value="">-- Choose a city --</option>
  <optgroup label="Greater Cairo">
    <option value="giza" selected>Giza</option>
    <option value="cairo">Cairo</option>
  </optgroup>
  <optgroup label="Coast">
    <option value="alex">Alexandria</option>
  </optgroup>
</select>

<!-- many choices at once: hold Ctrl -->
<select name="langs" multiple size="3">
  <option>Arabic</option>
  <option>English</option>
  <option>French</option>
</select>
```

RESULT

City

Arabic

English

French

- If an option has no value attribute, its text is sent.
- A first empty option with `value=""` plus `required` on the select forces the user to choose.
- Styling native selects is limited; the arrow cannot be changed without tricks.

<datalist> — text input with suggestions

Gamal #22Elzero HTML

Combines a free text field with a list of suggestions. The user can pick a suggestion or type anything else. Link them with the input's `list` attribute = the datalist's `id`.

	select	datalist
User can type anything?	No	Yes
Shows all options	Yes, always	Only matching ones while typing
Best for	Fixed short list	Long list or when custom values are allowed

EXAMPLE

```
<label for="browser">Favourite browser</label>
<input type="text" id="browser"
name="browser" list="browsers">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

RESULT

Favourite browser

Chrome

<button>

Gamal #23 Elzero HTML Angela \$4

A clickable button. Its `type` decides what it does: `submit` sends the form (the default inside a form!), `reset` clears the form, `button` does nothing by itself and is meant for JavaScript. A button can contain text, icons or images.

button زرار. type=submit بيعت الفورم، reset يمسح الحقول، button مش بيعمل حاجة لوحده (لجافاسكريبت).

EXAMPLE

```
<button type="submit">Create account</button>
<button type="reset">Clear</button>
<button type="button"
onclick="alert('Hi')">Say hi</button>
<button type="submit" disabled>Please
wait...</button>
<button type="submit" formaction="/save-
draft">Save as draft</button>
```

RESULT



- `<input type="submit" value="Send">` is the old way; `<button>` is more flexible (can hold an icon).
- `formaction` on a button overrides the form's action — useful for 'Save' vs 'Publish'.
- A link styled like a button is still a link: use `<a>` to go somewhere, `<button>` to do something.

⚠ Common mistake

A button inside a form with no type attribute is a submit button. If you only wanted to run JavaScript, the page reloads unexpectedly. Always write `type="button"` for JS-only buttons.

<fieldset>, <legend>, <output>, <progress>, <meter>

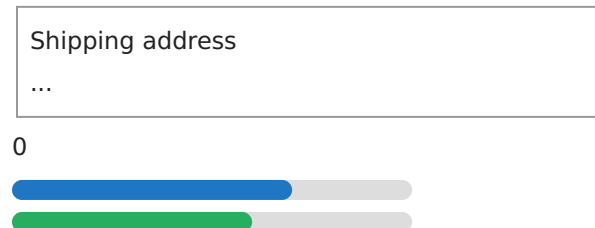
Gamal #21 Elzero HTML

Grouping and feedback elements. `fieldset` draws a box around related fields with `legend` as its title. `output` displays a calculation result. `progress` shows how far a task is. `meter` shows a measurement within a range (disk usage, password strength).

EXAMPLE

```
<fieldset>
  <legend>Shipping address</legend>
  ...
</fieldset>
<output name="total">0</output>
<progress value="70" max="100">70%</progress>
<meter value="0.6" min="0" max="1" low="0.3"
high="0.8">60%</meter>
```

RESULT



Browser validation — the attributes and how they look

Gamal #24 **Elzero HTML**

Before the form is sent, the browser checks the rules you wrote as attributes. If a rule fails, the form is not sent and a bubble appears next to the first bad field. No JavaScript needed.

Attribute	Rule	Browser message (Chrome)
required	Not empty	Please fill in this field.
type=email	Contains @ and a domain	Please include an '@' in the email address.
minlength / maxlength	Length	Please lengthen this text to 2 characters or more.
min / max	Range	Value must be less than or equal to 10.
pattern	Regex match	Please match the requested format. + your title text
step	Multiple of	Please enter a valid value.

EXAMPLE

```
<form>
  <input type="text" name="name" required
minlength="2" placeholder="Name">
  <input type="email" name="email" required>
  <input type="number" name="qty" min="1"
max="10" value="1">
  <input type="text" name="zip" pattern="[0-9]
{5}" title="5 digits">
  <button>Send</button>
</form>
```

RESULT

Name

⚠ Please fill in this field.

- Style valid/invalid fields with CSS: `input:invalid { border-color: red }` and `input:valid { border-color: green }`. Use `:user-invalid` to wait until the user has touched the field.
- Browser validation is for convenience. The server must validate again — anyone can send data without your form.

Complete example — a registration form

Gamal #17-#24 **Angela §4**

Everything from this chapter together. Type it and submit it; watch the URL (change method to get to see the data).

EXAMPLE

```

<form action="/register" method="post">
  <fieldset>
    <legend>Account</legend>
    <label for="name">Full name</label>
    <input type="text" id="name" name="name" required minlength="3" autocomplete="name">

    <label for="email">Email</label>
    <input type="email" id="email" name="email" required autocomplete="email">

    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" autocomplete="new-
password">
  </fieldset>

  <fieldset>
    <legend>Profile</legend>
    <label for="dob">Date of birth</label>
    <input type="date" id="dob" name="dob" max="2008-12-31">

    <label for="city">City</label>
    <select id="city" name="city">
      <option value="giza">Giza</option>
      <option value="cairo">Cairo</option>
    </select>

    <p>Level:</p>
    <input type="radio" id="l1" name="level" value="beginner" checked> <label for="l1">Beginner</
label>
    <input type="radio" id="l2" name="level" value="junior"> <label for="l2">Junior</label>

    <label for="bio">About you</label>
    <textarea id="bio" name="bio" rows="3" maxlength="300"></textarea>

    <input type="checkbox" id="terms" name="terms" required>
    <label for="terms">I agree to the terms</label>
  </fieldset>

  <button type="submit">Create account</button>
  <button type="reset">Clear</button>
</form>

```

- Every input has a `name` — check this first when data is missing on the server.
- Every input has a `label` — check this first when the form is hard to use on a phone.
- Later, in CSS, you will lay this out with Grid; in Part 5 you will read `req.body.name` on the server.

? Chapter quiz

1. Which attribute decides the key name the server receives?
2. GET or POST for a login form? Why?
3. What is the difference between disabled and readonly?
4. A button inside a form has no type — what does it do?
5. How do you connect a label to its input?

Answers: 1) name. 2) POST — the password must not appear in the URL or browser history. 3) disabled is not sent and cannot be edited; readonly is sent but cannot be edited. 4) It submits the form. 5) label for="x" and input id="x".

35. HTML Reference G — Semantic Layout and Interactive Tags

Semantic tags describe what a part of the page IS. Google, screen readers and other developers understand your page immediately. Use them instead of div wherever they fit.

<header>

Gamal #15

Elzero HTML

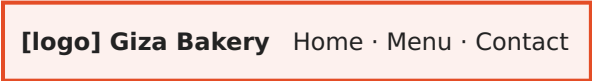
Angela \$3

The introductory part of the page or of a section: logo, site name, main navigation, or an article's title and date. A page may have several headers (one for the page, one per article).

EXAMPLE

```
<header>
  
  <h1>Giza Bakery</h1>
  <nav>...</nav>
</header>
```

RESULT



<nav>

Gamal #15

Elzero HTML

Angela \$3

A block of major navigation links: the main menu, a table of contents, breadcrumbs. Not every group of links needs nav — footer links usually do not.

EXAMPLE

```
<nav aria-label="Main">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/menu.html">Menu</a></li>
    <li><a href="/contact.html">Contact</a></li>
  </ul>
</nav>
```

RESULT



→ Menus are lists of links: nav → ul → li → a. CSS later removes the bullets and puts them in a row.

<main>

Gamal #15

Elzero HTML

Angela \$3

The unique content of this page — what makes it different from other pages. Exactly one per page. It should not contain the header, nav or footer that repeat on every page. Screen-reader users can jump straight to it.

EXAMPLE

```
<body>
  <header>...</header>
  <main>
    <h1>Our Menu</h1>
    ...
  </main>
  <footer>...</footer>
</body>
```

<section>

Gamal #15 **Elzero HTML** **Angela §3**

A thematic grouping of content, almost always with its own heading: 'About us', 'Features', 'Pricing', 'Testimonials'. If you cannot give it a heading, it is probably a div.

EXAMPLE

```
<section id="features">
  <h2>Why choose us</h2>
  <p>...</p>
</section>
<section id="pricing">
  <h2>Pricing</h2>
  ...
</section>
```

RESULT

Why choose us

...

Pricing

...

<article>

Gamal #15 **Elzero HTML** **Angela §3**

A self-contained piece that would make sense on its own or in an RSS feed: a blog post, a news story, a product card, a forum comment. Test: could you copy it to another site and still understand it? Then it is an article.

EXAMPLE

```
<article>
  <header>
    <h2>How I learned CSS in 30 days</h2>
    <p>By Mahmoud · <time
datetime="2026-09-07">7 Sep 2026</time></p>
  </header>
  <p>...</p>
  <footer>Tags: css, learning</footer>
</article>
```

RESULT

How I learned CSS in 30 days

By Mahmoud · 7 Sep 2026

...

Tags: css, learning

→ section vs article: a section groups related things; an article is one complete thing. Articles can contain sections and sections can contain articles.

<aside>

Gamal #15 Elzero HTML

Content related to the main content but separate from it: a sidebar with related posts, an advertisement, an author biography, a pull-quote.

EXAMPLE

```
<aside>
  <h3>Related posts</h3>
  <ul>
    <li><a href="#">Flexbox in 10 minutes</a></li>
  </ul>
</aside>
```

RESULT

Related posts

- Flexbox in 10 minutes

<footer>

Gamal #15 Elzero HTML Angela §3

The end of the page or of a section: copyright, contact links, sitemap, social icons. Like header, there can be one per article too.

EXAMPLE

```
<footer>
  <p>&copy; 2026 Giza Bakery · <a
href="privacy.html">Privacy</a></p>
  <address>Call <a href="tel:
+201141225852">0103 411 7684</a></address>
</footer>
```

RESULT

© 2026 Giza Bakery · Privacy
Call 0103 411 7684

Putting the layout together

Gamal #15 Angela §3

A typical page skeleton. Memorise this shape; almost every website is a variation of it. In the CSS part you will place these boxes with Grid.

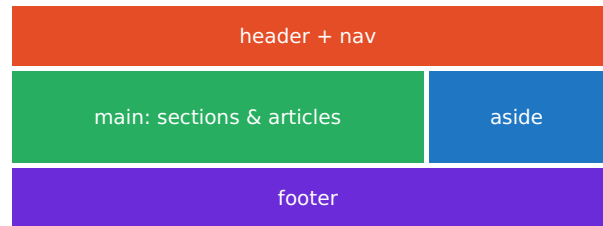
EXAMPLE

```
<body>
  <header>
    <h1>Site name</h1>
    <nav>...</nav>
  </header>

  <main>
    <section id="hero">...</section>
    <section id="posts">
      <article>...</article>
      <article>...</article>
    </section>
    <aside>...</aside>
  </main>

  <footer>...</footer>
</body>
```

RESULT



Try it yourself

Convert this div-only markup into semantic tags: `<div class="top">` logo + menu, `<div class="content">` with three blog posts, `<div class="side">` with ads, `<div class="bottom">` with copyright.

Solution — top → header (with nav inside for the menu); content → main containing three article elements; side → aside; bottom → footer.

<details> and <summary> — accordion without JavaScript

Gamal #15

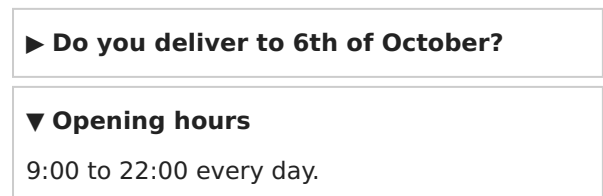
Elzero HTML

A disclosure widget: the summary is always visible; clicking it reveals the rest. Perfect for FAQs. Add `open` to start expanded.

EXAMPLE

```
<details>
  <summary>Do you deliver to 6th of October?</summary>
  <p>Yes, delivery takes 45 minutes and costs 20 EGP.</p>
</details>
<details open>
  <summary>Opening hours</summary>
  <p>9:00 to 22:00 every day.</p>
</details>
```

RESULT



<dialog> — modal window

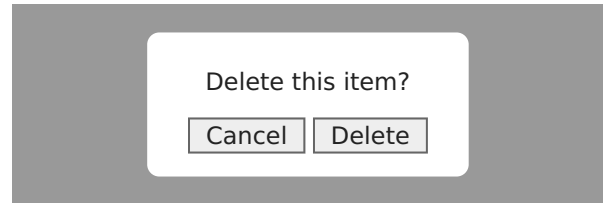
Elzero HTML

A native pop-up box. Open it with a little JavaScript (`dialog.showModal()`); the browser handles the dark backdrop, focus and the Escape key.

EXAMPLE

```
<dialog id="confirm">
  <p>Delete this item?</p>
  <form method="dialog">
    <button value="cancel">Cancel</button>
    <button value="ok">Delete</button>
  </form>
</dialog>
<button onclick="confirm.showModal()">Delete</button>
```

RESULT



→ A form with `method="dialog"` closes the dialog when submitted; the button's value becomes `dialog.returnValue`.

<template> and <slot>

Elzero JS

Advanced: `<template>` holds HTML that is not rendered until JavaScript clones it — useful for repeating a card for each item from an API. You will use it in the JavaScript part.

EXAMPLE

```
<template id="card-tpl">
  <article class="card">
    <h3></h3>
    <p></p>
  </article>
</template>
<script>
  const tpl = document.querySelector("#card-tpl");
  const clone = tpl.content.cloneNode(true);
  clone.querySelector("h3").textContent = "Title from data";
  document.body.append(clone);
</script>
```

Global attributes summary

Gamal #4

Elzero HTML

Attributes that work on every element.

Attribute	Purpose	Example
id	Unique name of ONE element; used by CSS (#id), JavaScript, links (#id) and labels	<code><section id="pricing"></code>
class	Group name shared by many elements; space-separated list; used by CSS (.class) and JS	<code><p class="note important"></code>
style	Inline CSS (avoid in real projects)	<code><p style="color:red"></code>
title	Tooltip text on hover	<code><abbr title="..."></code>
lang	Language of this element's text	<code><p lang="ar"></code>
dir	Text direction ltr / rtl / auto	<code><p dir="rtl"></code>
hidden	Hide the element completely	<code><div hidden></code>
tabindex	Keyboard focus order; 0 = focusable, -1 = not in tab order	<code><div tabindex="0"></code>
data-*	Custom data for JavaScript, read via element.dataset	<code><li data-id="7" data-price="20"></code>
contenteditable	User can edit the text in place	<code><p contenteditable></code>
draggable	Can be dragged (drag-and-drop API)	<code></code>
aria-*	Accessibility info for screen readers	<code><button aria-label="Close">x</button></code>
role	Tells assistive tech what the element acts as	<code><div role="alert"></code>

- id must be unique in the page and cannot contain spaces. class names: lowercase with hyphens (`main-menu`).
- One element can have many classes: `class="btn btn-primary large"` — this is how Bootstrap works.

Accessibility basics (a11y)

Angela §3

Accessibility means people with disabilities can use your site: blind users with screen readers, people who cannot use a mouse, colour-blind users. Most of it is free if you write correct HTML.

Do this	Because
Use semantic tags (nav, main, button) instead of divs	Screen readers announce them and users can jump between them
Write meaningful alt text on images	Blind users hear it; empty alt="" for decoration
Every input has a label	Users hear what the field is for
Headings in order h1 → h2 → h3	Users navigate by headings
Links say where they go, not 'click here'	Screen readers can list all links out of context
Colour contrast at least 4.5:1 for text	Readable for low vision; check with DevTools
Everything works with the keyboard (Tab, Enter, Escape)	Not everyone uses a mouse
Use button for actions, a for navigation	Correct keyboard behaviour for free
Set lang on html	Screen readers pick the right voice

? Chapter quiz

1. How many main elements per page?
2. Article or section for a single product card in a list?
3. Which tag makes an FAQ accordion with no JavaScript?
4. What is the difference between id and class?

Answers: 1) One. 2) article — each card is self-contained. 3) details + summary. 4) id names one unique element; class names a group and can be reused.

36. What is CSS? Three ways to add it

[Gamal CSS intro](#)[Angela §5](#)

CSS = Cascading Style Sheets. It describes how HTML elements **look**: colors, sizes, spacing, layout, animation. "Cascading" = rules flow down and later/more specific rules win.

```
/* 1. INLINE – style attribute. Highest priority, hard to maintain. Avoid. */
<p style="color: red; font-size: 20px;">Hi</p>

/* 2. INTERNAL – <style> inside <head>. OK for one-page demos. */
<head>
  <style>
    p { color: red; }
  </style>
</head>

/* 3. EXTERNAL – separate .css file. ☐ The professional way (one file styles all pages). */
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

- `rel="stylesheet"` tells the browser the relationship of the linked file. `href` is the path.
- You can link several CSS files; later ones override earlier ones on equal specificity.

37. CSS Syntax Anatomy & Priority

Gamal CSS #1 #2

Angela \$5

SELECTOR (which elements) VALUE DECLARATION = property: value;

`h1 { color: blue; font-size: 32px; }`

↑ ↓ ↓ ↓

PROPERTY ; ends each declaration { } = DECLARATION BLOCK

Selector + block = one RULE (ruleset)

```

/* comments look like this */
h1, h2, .title {           comma = apply to ALL of these */
  color: #0f3d68;
  margin: 0 0 10px;        multiple values separated by spaces */
}

```

Priority — who wins when rules conflict?



Specificity is counted as (IDs, classes, tags). Higher wins. Equal? → the LAST rule in the file wins.

```

p           { color: blue; }    /* 0,0,1 */
.text       { color: green; }   /* 0,1,0 → wins over p */
#intro      { color: red; }     /* 1,0,0 → wins over .text */
div p.text  { color: gold; }    /* 0,1,2 → loses to #intro */
p { color: black !important; } /* beats everything – use almost never */

```

Rule of thumb: style with **classes**. Avoid IDs for styling and **!important** entirely — they make CSS impossible to override later.

38. Selectors — All Kinds

Gamal CSS #3 #4 #35 #36 #37 #38

Angela §6

Selector	Example	Selects
Universal	<code>* { }</code>	Every element (used for resets)
Type / tag	<code>p { }</code>	All <code><p></code>
Class	<code>.card { }</code>	All elements with <code>class="card"</code>
ID	<code>#header { }</code>	The one element with <code>id="header"</code>
Group	<code>h1, h2, p { }</code>	All of them
Chained	<code>p.note.big { }</code>	<code><p></code> that has BOTH classes <code>note</code> and <code>big</code>
Descendant (space)	<code>nav a { }</code>	Every <code><a></code> anywhere inside <code><nav></code> (any depth)
Child (>)	<code>ul > li { }</code>	Only <code></code> that are DIRECT children of <code></code>
Adjacent sibling (+)	<code>h2 + p { }</code>	The first <code><p></code> immediately after an <code><h2></code>
General sibling (~)	<code>h2 ~ p { }</code>	All <code><p></code> after <code><h2></code> (same parent)
Attribute	<code>[type="email"] { }</code>	Elements with that attribute value
Attribute starts / ends / contains	<code>a[href^="https"]</code> <code>a[href\$=".pdf"]</code> <code>a[href*="google"]</code>	<code>^=</code> starts with, <code>\$=</code> ends with, <code>*=</code> contains
Pseudo-class	<code>a:hover</code> <code>li:first-child</code>	A state or position (ch. 29)
Pseudo-element	<code>p::first-line</code> <code>.box::before</code>	A part of an element (ch. 29)

```
<div class="parent">
```

```
<p> child A </p>
```

```
<p> child B </p>
```

```
<section>
```

```
<p> grandchild </p>
```

`.parent > p` → child A, child B only

`.parent p` → A, B AND grandchild

`p + p` → child B (right after A)

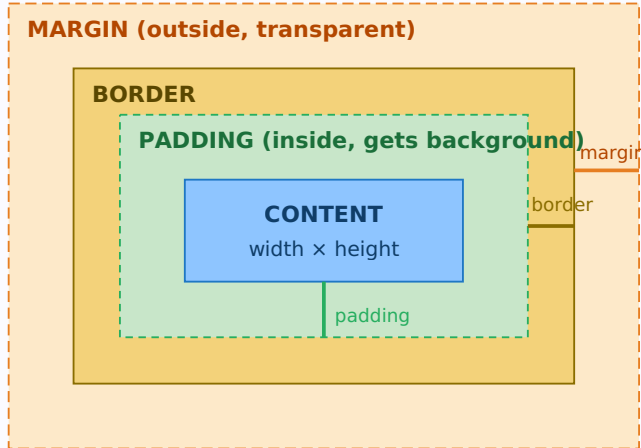
`p:first-child` → child A

39. The Box Model

Gamal CSS #5 #6

Angela \$7

Every element is a rectangular box made of 4 layers. Understanding this fixes 80% of "why is there a gap?" problems.



```
.box {
  width: 200px;
  height: 100px;
  padding:
    20px; /* space inside */
  border: 5px solid black;
  margin: 30px; /* space
    outside */
}

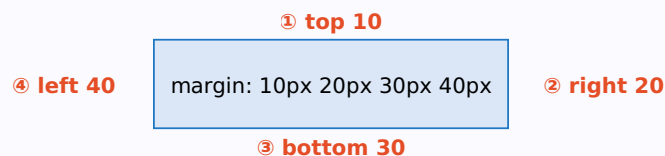
/* Total width on screen (default
content-box):
    200 + 20*2 + 5*2 = 250px (+ margin
    60 for spacing) */

/* ☐ Make width mean the WHOLE box: */
* { box-sizing: border-box; margin: 0;
padding: 0; }

/* now .box is exactly 200px wide
including padding+border */
```

Shorthand order — clockwise from TOP

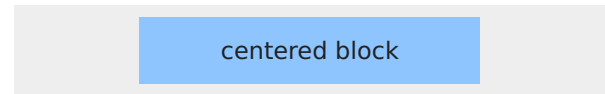
```
margin: 10px; /* all four sides */
margin: 10px 20px; /* top&bottom left&right */
margin: 10px 20px 30px; /* top left&right bottom */
margin: 10px 20px 30px 40px; /* top right bottom left (clockwise v) */
margin-top: 10px; padding-left: 5px; /* single side */
border: 2px dashed red; /* width style color */
border-bottom: 1px solid #ccc;
border-style: solid | dashed | dotted | double | none;
```



margin: auto — centering a block

```
.container {  
  width: 80%;           /* MUST have a width */  
  max-width: 1100px;  
  margin: 0 auto;       /* top/bottom 0, left/  
right auto */  
}
```

RESULT



→ **auto** splits the leftover horizontal space equally on both sides. Works on block elements with a width. Does NOT center text (use **text-align: center**) and does NOT center vertically (use Flexbox).

Margin collapse: vertical margins of two stacked blocks don't add up — the larger one wins (30px + 20px = 30px gap, not 50). Horizontal margins never collapse.

40. Sizing: width, height, min/max, %, fit-content, border-radius

Gamal CSS #7 #8 #9 #10

```
.a { width: 300px; height: 150px; } /* fixed */
.b { width: 50%; } /* 50% of PARENT's width */
.c { height: 50%; } /* works ONLY if parent has an explicit height! */
.d { width: 100vw; height: 100vh; } /* viewport = browser window size */
.e { width: fit-content; } /* as wide as its content, no more */
.f { width: min-content; } /* narrowest possible (longest word) */
.g { width: max-content; } /* one long line, never wraps */

/* Responsive pattern: fluid but with limits */
img { max-width: 100%; height: auto; } /* never overflow its container */
.card { width: 90%; min-width: 250px; max-width: 600px; }
.hero { min-height: 400px; } /* at least 400, grows with content */
```

- % is always relative to the **parent**. Height % needs a parent with a set height (or use `vh`).
- **min-width** / **max-width** override width. `min` wins over `max` if they conflict.
- Prefer `min-height` over `height` for text boxes — content can never overflow.

border-radius

```
.r1 { border-radius: 10px; } /* all corners */
.r2 { border-radius: 50%; } /* circle (if square) */
.r3 { border-radius: 20px 0; } /* top-left+bottom-right / others */
.r4 { border-radius: 10px 20px 30px 40px; } /* TL TR BR BL (clockwise) */
.r5 { border-top-left-radius: 30px; }
.pill { border-radius: 999px; } /* pill button */
```

RESULT



41. display & Hiding Elements

Gamal CSS #11 #43 #44

Angela §7

	block	inline	inline-block
New line?	Yes	No	No
Width by default	100% of parent	content	content
width/height respected?	☐	☐	☐
top/bottom margin&padding?	☐	☐ (padding draws but doesn't push)	☐
Examples	div p h1	span a strong	img button input

```

{
display:
block; }
.inline-
{
display:
inline-block; }
.none
{
display:
none; }
.flex
{
display:
flex; } //
*
ch.
31
*/
.grid
{
display:
grid; } //
*
ch.
32
*/
/
*
classic:
make
nav
links
sit
in
a
row
with
padding
*/
nav
a
{
display:
inline-block;
padding:
10px
15px; }

```

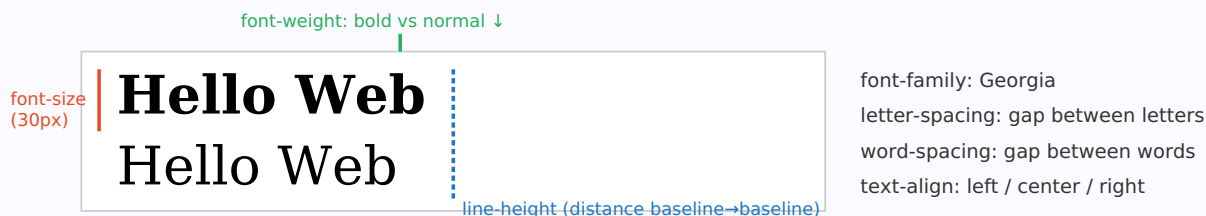
Three ways to hide — they are NOT the same

Property	Takes space?	Clickable?	Animatable?	Use when
<code>display: none</code>	☐ removed from layout	☐	☐	Toggle menus, modals
<code>visibility: hidden</code>	☐ leaves an empty gap	☐	☐	Keep layout stable
<code>opacity: 0</code>	☐	☐ still there!	☐ smooth fade	Fade animations

42. Text & Font Properties

Gamal CSS #12-#24

Angela §5-6



```
p {
  /* ---- FONT ---- */
  font-family: "Segoe UI", Arial, sans-serif;  list of fallbacks; last = generic family (serif,
  sans-serif, monospace, cursive) */
  font-size: 16px;          px fixed | 1.2rem (× root size) | 1.2em (× parent size) | 120% */
  font-weight: 700;         100 thin ... 400 normal ... 700 bold ... 900 black (font must include it) */
  font-style: italic;       normal | italic | oblique */
  font-variant: small-caps;
  line-height: 1.6;         unitless = multiply font-size (best). 1.5–1.7 for body text */
  /* shorthand: style variant weight size/line-height family (size + family REQUIRED) */
  font: italic small-caps 700 16px/1.6 Arial, sans-serif;

  /* ---- TEXT ---- */
  color: #333;
  text-align: center;
  left | right | center | justify (aligns INLINE content inside a block) */
  direction: rtl;          right-to-left for Arabic. Better: <html dir="rtl"> */
  text-transform: uppercase; uppercase | lowercase | capitalize */
  text-decoration: underline wavy red;
  none (remove link underline) | underline | line-through | overline */
  text-indent: 30px;        first-line indent */
  letter-spacing: 2px;
  word-spacing: 5px;
  text-shadow: 2px 2px 4px rgba(0,0,0,.4);  x-offset y-offset blur color */
  white-space: nowrap;      normal | nowrap (one line) | pre (keep spaces) | pre-wrap */
  overflow-wrap: break-word; break very long words/URLs */
  text-overflow: ellipsis;  "..." - needs overflow:hidden + white-space:nowrap */
}
```

```
/* Google Fonts (put in <head>) */
<link href="https://fonts.googleapis.com/css2?
family=Cairo:wght@400;700&display=swap"
rel="stylesheet">
/* then */
body { font-family: 'Cairo', sans-serif; }

/* Own font file */
@font-face {
  font-family: "MyFont";
  src: url("fonts/myfont.woff2");
}
```

TEXT-SHADOW VS BOX-SHADOW

text-shadow on letters

box-shadow on the box

→ **box-shadow: x y blur spread color;** add **inset** for inner shadow. Several shadows separated by commas.

43. Colors, Units, Backgrounds & Gradients

Gamal CSS #25 #26 #27 #34

Angela \$5

Colors

Format	Example	Notes
Name	<code>red tomato steelblue</code>	140 names
HEX	<code>#ff0000 #f00 #ff000080</code>	RR GG BB (00-ff); 8 digits adds alpha
RGB / RGBA	<code>rgb(255 0 0) rgba(255,0,0,.5)</code>	0-255 each; alpha 0-1 = transparency
HSL / HSLA	<code>hsl(0 100% 50%)</code>	hue 0-360°, saturation, lightness — easiest to tweak
Special	<code>transparentcurrentColor</code>	

Units

Unit	Relative to	Use for
<code>px</code>	nothing (absolute)	borders, small fixed things
<code>%</code>	parent element	widths, fluid layouts
<code>em</code>	font-size of the element (parent for font-size)	padding/margin that scale with text — compounds when nested △
<code>rem</code>	root (html) font-size — 16px default	□ font sizes, spacing. Predictable
<code>vw</code> / <code>vh</code>	1% of viewport width / height	full-screen sections, hero
<code>vmin</code> / <code>vmax</code>	smaller / larger of vw, vh	
<code>ch</code>	width of "0" character	readable line length: <code>max-width: 65ch</code>
<code>fr</code>	fraction of free space (Grid only)	grid columns

```
html { font-size: 62.5%; } /* trick: 1rem = 10px, so 1.6rem = 16px */
h1 { font-size: clamp(1.5rem, 4vw, 3rem); } /* min, preferred, max → fluid text */
.w { width: calc(100% - 40px); } /* math with mixed units */
```

Backgrounds

```
.hero {
  background-color: #0f3d68;
  background-image: url("images/bg.jpg");
  background-repeat: no-repeat;          repeat | repeat-x | repeat-y | no-repeat */
  background-position: center top;       x y: left/center/right, px, % */
  background-size: cover;                cover (fill, crop) | contain (fit, may leave gaps) | 100%
  auto */
  background-attachment: fixed;          parallax effect */
  /* shorthand: color image repeat attachment position / size */
  background: #0f3d68 url("bg.jpg") no-repeat center / cover;
}
/* multiple layers: first = on top */
.overlay { background: linear-gradient(rgba(0,0,0,.6), rgba(0,0,0,.6)), url(bg.jpg) center/
cover; }
```

```
/* GRADIENTS are images, not colors */
background: linear-gradient(to right, red,
blue);
background:
linear-gradient(45deg, #f39c12 0%, #e74c3c
50%, #8e44ad 100%);
background: radial-gradient(circle at center,
#fff, #1f77c4);
background:
conic-gradient(red, yellow, green, red);
background: repeating-linear-gradient(45deg,
#ccc 0 10px, #fff 10px 20px);
```

RESULT



→ direction: **to right**, **to bottom left**, or
degrees (0deg = to top, 90deg = to right). Color
stops take % positions.

44. float, overflow, position, z-index

Gamal CSS #28 #29 #30 #31

Angela §8

float (old layout tool — today used for wrapping text around images)

```
img { float: left; margin: 0 15px 10px 0; }
.clear { clear: both; } /* stop wrapping */
/* parent collapses around floats → fix: */
.parent::after { content: ""; display: block;
clear: both; }
/* or simply */ .parent { display: flow-
root; }
```

Text wraps around the floated blue box on the left, continuing beneath it when the box ends. This was how columns were built before Flexbox and Grid existed. Use Flex/Grid for layout now.

overflow — what happens when content is bigger than the box

```
.box { height: 80px; overflow: visible; } /*
default: spills out */
.box { overflow: hidden; } /* cut off */
.box { overflow: scroll; } /* always
scrollbars */
.box { overflow: auto; } /* scrollbars
only if needed */
.box { overflow-x: auto; overflow-y: hidden; }
```

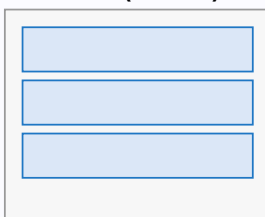
visible text
spills out of
the box and
keeps going

hidden text is
cut at the
edge of the

auto shows a
scrollbar so
you can scroll

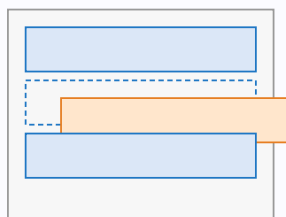
position

static (default)



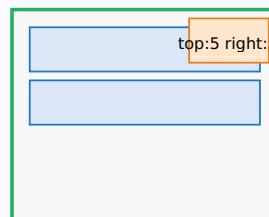
normal flow; top/left ignored

relative



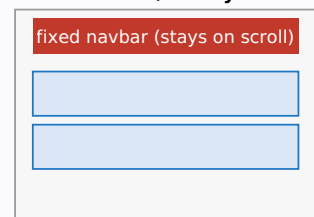
moved from its own spot;
original space is kept

absolute



removed from flow; placed
relative to nearest POSITIONED
ancestor (green = position: relative)

fixed / sticky



fixed = relative to viewport
sticky = normal until you scroll
past it, then sticks (needs top:)

```
.parent { position: relative; }           /* becomes the reference box */  
.badge { position: absolute; top: -8px; right: -8px; } /* pinned to parent's corner */  
.navbar { position: fixed; top: 0; left: 0; width: 100%; }  
th      { position: sticky; top: 0; }      /* sticky table header */  
/* center anything absolutely */  
.modal { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }
```

z-index — stacking order

```
.modal { position: fixed; z-index: 1000; } /* higher = on top */  
.overlay { position: fixed; z-index: 999; }
```

- Works ONLY on positioned elements (anything except static) and flex/grid children.
- Default is `auto` (0). Can be negative. Children can't escape their parent's stacking context.

45. Pseudo-classes, Pseudo-elements, list & table style

Gamal CSS #32 #33 #35 #36

Pseudo-classes (single colon) — state or position

```
/* interaction */
a:hover      { color: orange; }      /* mouse over */
a:active     { color: red; }         /* being clicked */
a:visited    { color: purple; }
input:focus  { outline: 2px solid blue; } /* keyboard/cursor inside */
input:checked + label { font-weight: bold; }
input:disabled, input:required, input:valid, input:invalid, input:placeholder-shown { }
/* position among siblings */
li:first-child { } li:last-child { } li:only-child { }
li:nth-child(2) { } /* the 2nd */
li:nth-child(odd) { } li:nth-child(even) { } /* zebra stripes */
li:nth-child(3n) { } /* every 3rd: 3,6,9 */
li:nth-child(3n+1) { } /* 1,4,7 */
li:nth-last-child(2) { } /* 2nd from the end */
p:first-of-type { } p:nth-of-type(2) { } /* count only <p> siblings */
/* logic */
li:not(.active) { }
div:has(> img) { } /* parent selector (modern) */
:root { --main: #1f77c4; } /* = html; used for CSS variables */
.btn { color: var(--main); }
```

Pseudo-elements (double colon) — a part of an element

```
p::first-letter { font-size: 2em; }
p::first-line   { font-weight: bold; }
::selection     { background: yellow; }
input::placeholder { color: #aaa; }
/* ::before / ::after insert content – MUST have content: */
.price::before { content: "$"; }
a[target=_blank]::after { content: " ↗"; }
.btn::after {
  content: ""; /* empty but present */
  position: absolute; inset: 0;
  background: rgba(255,255,255,.2);
}
```

RESULT

D cap with ::first-letter. Very common in articles and magazines.

Price: **\$49** (the \$ came from ::before)

Link [Docs](#) ↗

list-style & table styling

```
ul { list-style-type: square; }      /* disc | circle | square | none | decimal | lower-alpha |  
upper-roman */  
ul { list-style-image: url(check.svg); }  
ul { list-style-position: inside; } /* bullet inside the text box */  
nav ul { list-style: none; padding: 0; margin: 0; } /* menu reset */  
  
table { border-collapse: collapse; width: 100%; } /* single borders instead of double */  
th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }  
th { background: #0f3d68; color: #fff; }  
tr:nth-child(even) { background: #f5f7fa; }  
tr:hover { background: #e8f0fe; }  
td { vertical-align: middle; }
```

46. transform, transition, animation

Gamal CSS #39 #40 #41 #42

transform — move/rotate/scale without affecting layout

```
.a { transform: translate(20px, -10px); } /*
move x,y */

.b { transform: rotate(15deg); }

.c { transform: scale(1.2); }           /*
120% */

.d { transform: skew(10deg); }

.e { transform: rotate(45deg) scale(.8); } /*
combine (order matters) */

.f { transform-origin: top left; }      /*
pivot point (default center) */

/* new individual properties (no overwrite
issue) */

.g { translate: 10px 0; rotate: 10deg; scale:
1.1; }
```

RESULT

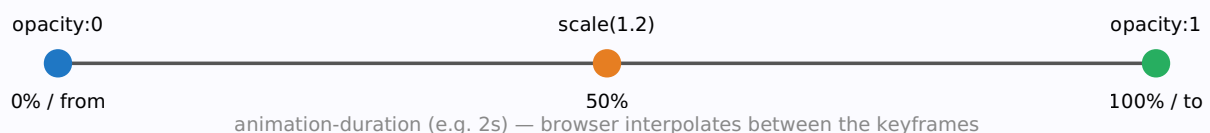


transition — animate a change smoothly (needs a trigger like :hover)

```
.btn {
  background: #1f77c4;
  transition: background .3s ease, transform .2s;
  /*          property    duration timing    delay(optional) – or just: all .3s */
}

.btn:hover { background: #0f3d68; transform: translateY(-3px); }
/* timing functions: ease | linear | ease-in | ease-out | ease-in-out | cubic-bezier(...) */
```

animation — runs by itself, with keyframes




```

@keyframes pop {
  0% { opacity: 0; transform: scale(.5); }
  50% { transform: scale(1.2); }
  100% { opacity: 1; transform: scale(1); }
}
.card {
  animation-name: pop;
  animation-duration: 1s;
  animation-timing-function: ease-out;
  animation-delay: .2s;
  animation-iteration-count: infinite; /* or a number */
  animation-direction: alternate; /* normal | reverse | alternate */
  animation-fill-mode: forwards; /* keep the final state */
  animation-play-state: running; /* paused */
  /* shorthand: name duration timing delay count direction fill */
  animation: pop 1s ease-out .2s infinite alternate forwards;
}
/* loading spinner */
@keyframes spin { to { transform: rotate(360deg); } }
.spinner { border: 4px solid #eee; border-top-color: #1f77c4; border-radius: 50%; width: 40px;
height: 40px; animation: spin 1s linear infinite; }

```

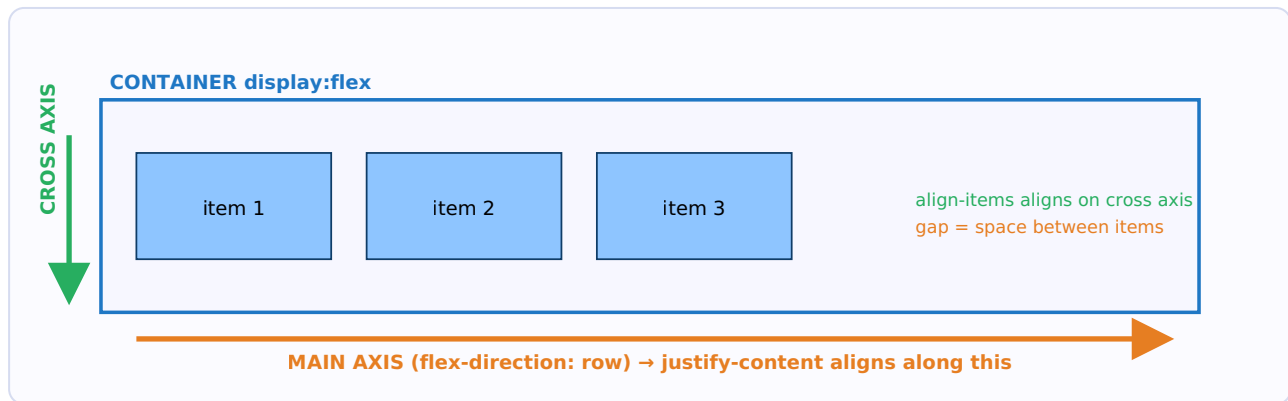
transition = A→B when something changes. animation = a timeline that can loop on its own. Animate **transform** and **opacity** for smooth 60fps; avoid animating width/height/margin.

47. Flexbox — Complete

Gamal CSS #45

Angela \$10

Flexbox lays out items in **one dimension** (a row OR a column). Set `display: flex` on the **parent (container)**; its direct children become **flex items**.

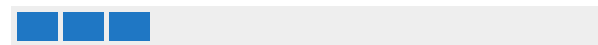


Container properties

```
.container {
  display: flex;           /* or inline-flex */
  flex-direction: row;     /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;         /* nowrap (default, squeeze) | wrap (new line) */
  flex-flow: row wrap;     /* shorthand of the two */
  justify-content: space-between; /* MAIN axis: flex-start | center | flex-end | space-between |
space-around | space-evenly */
  align-items: center;      /* CROSS axis: stretch (default) | flex-start | center | flex-end |
baseline */
  align-content: center;    /* multiple LINES when wrapping (same values as justify) */
  gap: 20px;               /* row-gap column-gap */
}
```

JUSTIFY-CONTENT

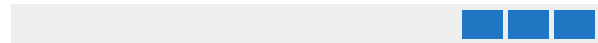
flex-start



center



flex-end



space-between



space-around

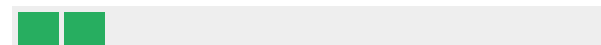


space-evenly



ALIGN-ITEMS (CONTAINER HEIGHT 50PX)

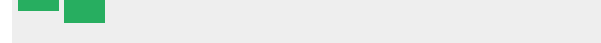
stretch



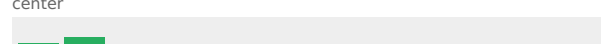
flex-start



center



flex-end



Item properties

```
.item {
  flex-grow: 1;      /* how much extra space to take (0 = don't grow). 2 = twice as much as 1 */
  flex-shrink: 1;    /* can it shrink when space is tight? 0 = never */
  flex-basis: 200px; /* starting size before grow/shrink (auto = content/width) */
  flex: 1 1 200px;   /* shorthand grow shrink basis. flex:1 = 1 1 0 (equal columns) */
  align-self: flex-end; /* override align-items for THIS item */
  order: -1;         /* visual order (default 0); lower first */
}
```

FLEX-GROW 1 / 2 / 1



ORDER: ITEM C MOVED FIRST



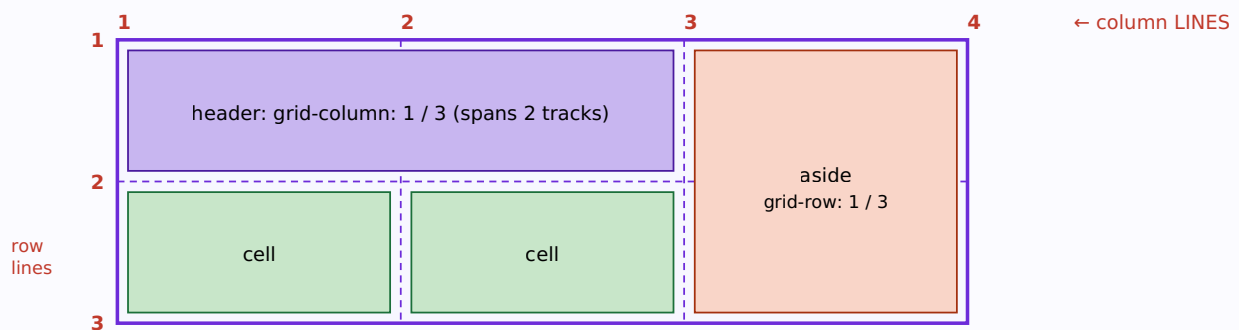
```
/* † The 3 patterns you'll use daily † */
/* 1. perfectly center anything */
.center { display: flex; justify-content: center; align-items: center; }
/* 2. navbar: logo left, links right */
.nav { display: flex; justify-content: space-between; align-items: center; }
/* 3. responsive cards */
.cards { display: flex; flex-wrap: wrap; gap: 20px; }
.card { flex: 1 1 250px; } /* min 250, grow to fill */
```

48. CSS Grid — Complete

Gamal CSS #46

Angela §11

Grid lays out in **two dimensions** (rows AND columns) at once. Flexbox = components; Grid = page layouts.



Container

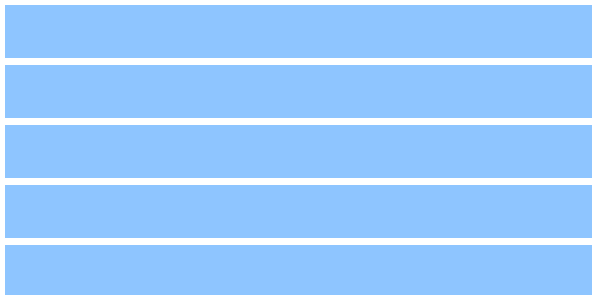
```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr 1fr; /* 3 columns: fixed + two equal shares of the rest */
  grid-template-columns: repeat(3, 1fr); /* same as 1fr 1fr 1fr */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* + responsive without media queries */
  grid-template-rows: 80px auto 60px;
  grid-auto-rows: minmax(100px, auto); /* size of rows created automatically */
  gap: 20px 10px; /* row-gap column-gap */
  justify-items:
  center; /* items horizontally inside their cell: start | center | end | stretch */
  align-items: center; /* items vertically inside their cell */
  justify-content: center; /* the whole grid inside container (if grid is smaller) */
  place-items: center; /* align-items + justify-items */
}
```

Items — placing by line numbers or names

```
.header { grid-column: 1 /
3; } /* from line 1 to line 3 */
.header { grid-column: 1 / span 2; } /* same,
"span 2 tracks" */
.aside { grid-column: 3; grid-row: 1 / 3; }
.full { grid-column: 1 /
-1; } /* -1 = last line → full width */
.item { grid-area: 1 / 1 / 3 / 2; } /* row-
start / col-start / row-end / col-end */
.item { justify-self: end; align-self:
start; }
```

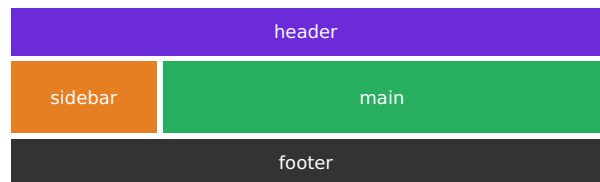
```
/* † Named areas – draw your layout in ASCII
*/
.page {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}
header { grid-area: header; }
aside { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }
/* on mobile just redraw: */
@media (max-width:600px) { .page { grid-
template-columns: 1fr;
  grid-template-areas: "header" "main"
"sidebar" "footer"; } }
```

REPEAT(AUTO-FIT, MINMAX(80PX,1FR))



→ as many 60px+ columns as fit, stretched equally. Shrink the window → fewer columns, automatically.

NAMED AREAS RESULT



Use Flexbox when...

Items in one row or column; navbar; centering; button groups; unknown number of small items

Use Grid when...

Whole-page layout; rows + columns must align; card galleries; dashboards; overlapping items

49. Responsive Design & Media Queries

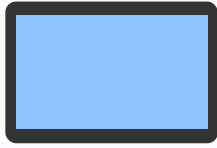
Gamal CSS #47

Angela \$9

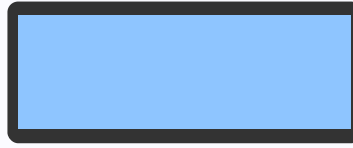
Mobile-first: write CSS for phones, then ADD rules for bigger screens with min-width



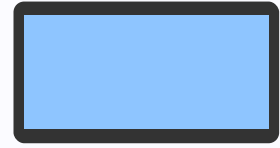
< 600px



600-899px



900-1199px



> 1200px

```
/* 1. REQUIRED in <head> or nothing works on phones */
<meta name="viewport" content="width=device-width, initial-scale=1">

/* 2. Base styles = mobile */
.cards { display: grid; grid-template-columns: 1fr; gap: 1rem; }
.nav-links { display: none; }

/* 3. Media queries: "when the screen is at least X, also apply this" */
@media (min-width: 600px) { .cards { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 992px) { .cards { grid-template-columns: repeat(3, 1fr); }
  .nav-links { display: flex; } .burger { display: none; } }
@media (min-width: 1200px) { .container { max-width: 1140px; } }

/* other queries */
@media (max-width: 599px) { } /* desktop-first style */
@media (min-width: 600px) and (max-width: 991px) { }
@media (orientation: landscape) { }
@media (prefers-color-scheme: dark) { body { background: #111; color: #eee; } }
@media print { nav { display: none; } }
```

Responsive toolkit checklist

- **Fluid widths:** %, fr, `max-width` instead of fixed px.
- **Images:** `img { max-width:100%; height:auto; }`
- **Text:** rem units, `clamp()` for headings.
- **Layout:** Flexbox `flex-wrap` + `flex: 1 1 250px`, or Grid `auto-fit/minmax` — often no media query needed.
- **Test:** DevTools `F12` → toggle device toolbar `Ctrl+Shift+M`.

```
/* Responsive form (Gamal's example) */
form { display: grid; gap: 1rem; max-width: 600px; margin: auto; }
input, textarea { width: 100%; padding: .75rem; border: 1px solid #ccc; border-radius: 6px; }
@media (min-width: 600px) {
  form { grid-template-columns: 1fr 1fr; }
  textarea, button { grid-column: 1 / -1; }
}
```

50. Bootstrap 5 Essentials

Angela §12-13

Bootstrap is a CSS framework: pre-made classes and components so you can build a responsive page fast. Learn CSS first — Bootstrap is just CSS someone else wrote.

```
<!-- 1. Add via CDN in <head> (CSS) and before </body> (JS) -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

<!-- 2. The 12-column grid: container → row → col -->
<div class="container">
  <div class="row">
    <div class="col-12 col-md-6 col-lg-4">A</div>    full on phone, half on ≥768, third on ≥992
    <div class="col-12 col-md-6 col-lg-4">B</div>
    <div class="col-12 col-md-12 col-lg-4">C</div>
  </div>
</div>
```

col-4 (4 of 12)	col-4	col-4
-----------------	-------	-------

Breakpoints: (xs) <576 · sm ≥576 · md ≥768 · lg ≥992 · xl ≥1200 · xxl ≥1400 — a breakpoint class applies from that size UP

Utility classes you'll use constantly

Group	Classes	Meaning
Spacing	m-3 p-2 mt-4 mb-0 px-5 py-1 mx-auto	m=margin p=padding; t r b l x y; scale 0-5 (0, .25, .5, 1, 1.5, 3 rem)
Flex	d-flex justify-content-between align-items-center flex-column gap-3	same as CSS flex
Display	d-none d-md-block d-lg-flex	responsive show/hide
Text	text-center text-primary fw-bold fs-3 text-uppercase	
Colors	bg-dark text-white bg-primary text-muted	primary secondary success danger warning info light dark
Sizing	w-100 h-50 vh-100 mw-100	
Borders	border rounded rounded-circle shadow	

Components (copy from getbootstrap.com/docs)

```
<button class="btn btn-primary btn-lg">Primary</button>
<button class="btn btn-outline-danger">Outline</button>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">Brand</a>
    <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#menu"><span
class="navbar-toggler-icon"></span></button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
      </ul>
    </div>
  </div>
</nav>

<div class="card" style="width:18rem">
  
  <div class="card-body">
    <h5 class="card-title">Title</h5>
    <p class="card-text">Text</p>
    <a href="#" class="btn btn-primary">Go</a>
  </div>
</div>
```

Other components:

[alert](#)
[badge](#)
[carousel](#)
[modal](#)
[accordion](#)
[dropdown](#)
[form-control](#)
[table table-striped](#)
[spinner-border](#)

. Angela's project: **TinDog** landing page.

51. CSS Reference A — Getting Started, Syntax, Selectors, Cascade

CSS (Cascading Style Sheets) decides how HTML looks. This chapter explains how to connect CSS, how a rule is written, how to choose which elements a rule applies to (selectors), and which rule wins when two disagree (the cascade).

Three ways to add CSS

Gamal CSS #1

Elzero CSS #02

Angela §5

CSS can be written inline on one element, inside a `<style>` tag, or in a separate `.css` file linked from the head. Real projects use the external file: one file styles every page, and the browser caches it.

تقدر تكتب CSS جوه العنصر (inline)، أو جوه `<style>` في الـ head، أو في ملف منفصل. css. (الأفضل).

Method	Good for	Bad because
Inline	Quick tests, emails	Mixes content and style; hard to change 100 elements
Internal	One-page demos	Not shared between pages
External	Everything real	Nothing — it is the standard

EXAMPLE

```

<!-- 1. inline: only this one element -->
<p style="color: red;">Red text</p>

<!-- 2. internal: this page only -->
<head>
  <style>
    p { color: red; }
  </style>
</head>

<!-- 3. external: all pages (best) -->
<head>
  <link rel="stylesheet" href="css/style.css">
</head>

```

⚠ Common mistake

The CSS file does not apply. Check: (1) the href path is correct relative to the HTML file, (2) the file is really named `.css`, (3) you saved both files, (4) DevTools → Network tab shows the file loaded with status 200.

Anatomy of a rule

Gamal CSS #1

Elzero CSS #02

Angela §5

A rule = a selector + a declaration block. The block is inside curly braces and contains declarations. Each declaration is a property, a colon, a value, and a semicolon.

EXAMPLE

```
/* selector { property: value; } */
h1 {
  color: darkblue;      /* declaration 1 */
  font-size: 32px;      /* declaration 2 */
  text-align: center;    /* declaration 3 */
}
```

RESULT

A centred dark-blue heading

- The last semicolon is optional but always write it — forgetting it when you add a line later is the #1 CSS bug.
- Comments are `/* like this */`. There is no `//` comment in CSS.
- Property names are lowercase with hyphens: `background-color`, not `backgroundColor` (that is the JavaScript name).
- Whitespace does not matter; write one declaration per line for readability.

Simple selectors: type, class, id, universal

Gamal CSS #3

Elzero CSS #03 #04

Angela §6

A selector chooses which elements the rule styles. The four basics: by tag name, by class (a reusable label you add to elements), by id (a unique label), and the universal selector that matches everything.

السيكتور يحدد الرول هتطبق على مين: اسم التاج، الكلاس (.), الآي دي (#)، أو الكل (*).

Selector	Written as	Matches	Reusable?
Type	<code>p</code>	All p elements	—
Class	<code>.note</code>	Elements with class="note"	Yes, on many elements
ID	<code>#logo</code>	The single element with id="logo"	No, one per page
Universal	<code>*</code>	Every element	—

EXAMPLE

```
/* CSS */
p      { color: gray; }      /* every <p>
*/
.note   { background: yellow; } /* every
element with class="note" */
#logo   { width: 120px; }     /* the one
element with id="logo" */
*       { box-sizing: border-box; } /*
everything */

/* HTML */
<p>Normal paragraph</p>
<p class="note">Highlighted paragraph</p>

```

RESULT

Normal paragraph

Highlighted paragraph

- An element can have several classes: `class="note big"` matches both `.note` and `.big`.
- `p.note` (no space) = a p that has class note. `p .note` (with space) = a .note inside a p. The space changes everything.
- Prefer classes for styling. Use ids for JavaScript hooks and link targets.

 Try it yourself

Write CSS that makes all h2 elements green, elements with class 'warning' bold and red, and the element with id 'footer' have grey text.

Solution

```
h2 { color: green; }
.warning { color: red; font-weight: bold; }
#footer { color: gray; }
```

Grouping and combinators

Gamal CSS #4 #37

Elzero CSS #13 #14

Angela \$6

Combine selectors to be more precise. A comma groups several selectors under one rule. A space, `>`, `+` and `~` describe relationships between elements.

EXAMPLE

```
h1, h2, h3 { font-family: Georgia; } /*
group: all three */
nav a { color: white; } /*
descendant: any <a> inside <nav>, any depth */
ul > li { margin: 4px; } /*
child: only <li> directly inside <ul> */
h2 + p { font-weight: bold; } /*
adjacent sibling: the <p> right after an <h2>
*/
h2 ~ p { color: #444; } /*
general sibling: every <p> after an <h2> (same
parent) */
```

RESULT

h2 + p → only the first paragraph after the heading is bold.

h2 ~ p → all paragraphs after the heading get the colour.

ul > li → nested lists inside an li are NOT matched (not direct children).

- Descendant (space) is the one you will use most: `.card img`, `footer a`.
- Keep selectors short. `body div.container ul li a` is slow to read and hard to override; `.menu-link` is better.

Attribute selectors

Gamal CSS #38

Elzero CSS

Select elements by the attributes they carry — very useful for form inputs and links.

EXAMPLE

```
[required] { border-left: 3px solid
red; } /* has the attribute */
[type="email"] { background:
#eef; } /* exact value */
a[href^="https://"] { color:
green; } /* starts with */
a[href$=".pdf"] { font-weight:
bold; } /* ends with */
a[href*="youtube"] { color:
red; } /* contains */
[class~="card"] { } /* has "card" as
one word in the class list */
[lang]="en"] { } /* en or en-US, en-
GB */
[data-status="done"] { text-decoration: line-
through; }
```

RESULT

required input

`https link` · **report.pdf** · `youtube link`

`data-status="done"` task

The cascade: which rule wins?

Gamal CSS #2

Elzero CSS #43

Angela §6

When two rules set the same property on the same element, the browser picks one. Order of importance: (1) **!important** beats all, (2) inline style, (3) specificity of the selector, (4) if still equal, the rule written last wins.

لما رولين يتعارضوا: important! يكسب، بعدين inline، بعدين الأكثر تحديداً (specificity)، ولو متساويين اللي مكتوب آخر يكسب.

Selector part	Adds to	Example	Score
Inline style attribute	(top)	style="..."	1,0,0,0
#id	IDs column	#intro	1,0,0
.class, [attr], :pseudo-class	classes column	.text, :hover	0,1,0
tag, ::pseudo-element	tags column	p, ::before	0,0,1
*, combinators (space, >, +, ~)	nothing		0,0,0

EXAMPLE

```
p          { color: blue; }      /* specificity 0,0,1 */
.text      { color: green; }    /* 0,1,0 → beats p */
#intro     { color: red; }      /* 1,0,0 → beats .text */
div p.text { color: gold; }     /* 0,1,2 → loses to #intro (an id beats any number of classes) */
p { color: black !important; } /* beats everything above */
```

- Compare column by column from the left: 1,0,0 beats 0,9,9.
- `:not(x)`, `:is(x)` and `:has(x)` take the specificity of x.
- Avoid `!important`. The one legitimate use: utility classes like `.hidden { display:none !important }`.

⚠ Common mistake

'My CSS is not working!' — 99% of the time another rule with higher specificity wins. Open DevTools → Elements → Styles: crossed-out declarations lost; the winner is shown above them.

Inheritance

Elzero CSS #22

Angela §6

Some properties automatically pass from parent to children: text properties like `color`, `font-family`, `font-size`, `line-height`, `text-align`. Box properties like `margin`, `padding`, `border`, `background`, `width` do not inherit.

EXAMPLE

```
body { font-family: Arial; color: #333; } /*
every text on the page inherits these */
.box { border: 1px
solid; } /* children do NOT
get a border */

/* force the behaviour */
a { color: inherit; } /* take parent
colour instead of default blue */
.reset { all: initial; } /* back to browser
defaults */
h1 { font-size: unset; } /* inherit if
inheritable, else initial */
```

RESULT

body sets Arial + #333 → this span inherits the font and colour but not the border rule

- Set fonts and colours once on `body` and let inheritance do the work.
- Form controls (input, button, select) do NOT inherit font by default; add `button, input, select, textarea { font: inherit; }` to your reset.

Naming conventions and organising a stylesheet

Elzero CSS #04

Good names make CSS maintainable. Use lowercase, hyphens, and names that describe purpose, not appearance. Order your file from general to specific.

EXAMPLE

```
/* good names: what it IS */
.site-header { }
.product-card { }
.btn-primary { }

/* bad names: how it LOOKS (breaks when design changes) */
.red-text { }
.left-box { }
.big { }

/* BEM style (Block__Element--Modifier) used in many teams */
.card { }
.card__title { }
.card__image { }
.card--featured { }

/* suggested file order */
/* 1. variables 2. reset 3. base (body, headings, links)
4. layout (header, footer, grid) 5. components (cards, buttons)
6. utilities (.text-center) 7. media queries */
```

CSS variables (custom properties)

Elzero CSS #45

Store a value once and reuse it everywhere. Change the variable → the whole site updates. Define them on `:root` (the html element) so they are available everywhere; read them with `var()`.

EXAMPLE

```
:root {
  --primary: #1f77c4;
  --text: #222;
  --radius: 8px;
  --space: 16px;
}

.btn {
  background: var(--primary);
  border-radius: var(--radius);
  padding: var(--space);
}

.card { color: var(--text, black); } /*
  fallback value if the variable is missing */

/* dark mode by changing variables only */
body.dark { --text: #eee; --primary:
#8ec5ff; }
```

RESULT

var(--primary)

dark mode

- Variable names start with two hyphens and are case sensitive.
- They can be changed from JavaScript: `document.documentElement.style.setProperty('--primary', 'red')`.
- They cascade and inherit like normal properties — you can override them inside a component.

Vendor prefixes

Elzero CSS #38

Years ago, browsers shipped experimental properties with a prefix: `-webkit-` (Chrome, Safari), `-moz-` (Firefox), `-ms-` (old Edge/IE). Today almost nothing needs them. Tools called autoprefixers add them automatically when required.

EXAMPLE

```
/* old way */
.box {
  -webkit-transition: all .3s;
  -moz-transition: all .3s;
  transition: all .3s; /* standard last */
}

/* still needed occasionally */
.text { -webkit-line-clamp: 3; }
input::-webkit-input-placeholder { color: gray; }
```

- Check caniuse.com to see if a property still needs a prefix.
- Write the un-prefixed standard version last so it wins when supported.

? Chapter quiz

1. What are the three parts of a declaration?
2. Which wins: #id or twenty classes?
3. Does padding inherit?
4. Where do you define global CSS variables?

Answers: 1) property, colon, value (ending with a semicolon). 2) The id. 3) No — only text-related properties inherit. 4) On :root.

52. CSS Reference B — Colours, Units and Backgrounds

Every visual property needs values: a colour, a length, an image. This chapter explains every way to write them, and then covers backgrounds and gradients in full.

Colour values

Gamal CSS #16

Elzero CSS #17

Angela \$5

Colours can be written by name, as hexadecimal, as RGB, or as HSL. All of them can include transparency (alpha). Pick one style per project; hex for design hand-offs, HSL when you want to compute lighter or darker variants easily.

الألوان تكتب بالاسم، أو hex (#ff0000)، أو rgb، أو hsl. كلهم ينفع يبقى فيهم شفافية.

HSL part	Meaning	Values
Hue	Position on the colour wheel	0 red, 60 yellow, 120 green, 180 cyan, 240 blue, 300 magenta
Saturation	How vivid	0% grey → 100% pure
Lightness	How bright	0% black, 50% normal, 100% white

EXAMPLE

```
.a { color: tomato; } /* 140
named colours */
.b { color: #ff6347; } /* hex:
#RRGGBB, 00-ff each */
.c { color: #f64; } /* short
hex = #ff6644 */
.d { color: #ff634780; } /* 8
digits: last two = alpha (80 = 50%) */
.e { color: rgb(255, 99, 71); } /* red
green blue 0-255 */
.f { color: rgba(255, 99, 71, 0.5); } /* +
alpha 0-1 */
.g { color: hsl(9, 100%, 64%); } /* hue 0-
360°, saturation, lightness */
.h { color: hsla(9, 100%, 64%, .5); }
.i { color: transparent; }
.j { border-color: currentColor; } /* the
element's own text colour */
```

RESULT



All seven are the same tomato colour written differently (two are 50% transparent).

- Hex is the most common in the wild. Learn to read it: `#000` black, `#fff` white, `#f00` red, `#0f0` green, `#00f` blue, `#888` grey.
- HSL is easiest to adjust by hand: for a hover state, just lower the lightness by 10%.
- Contrast matters: dark grey text (`#333`) on white is readable; light grey (`#bbb`) is not. DevTools' colour picker shows the contrast ratio.

Length units

Gamal CSS #7 #34

Elzero CSS #24

Angela \$7

Lengths are used for sizes, spacing, positions. Absolute units (px) are fixed. Relative units (% , em, rem, vw, vh) change with something else — that flexibility is what makes responsive design possible.

Unit	Relative to	Typical use	Note
px	Nothing (1/96 inch)	Borders, shadows, small fixed things	Does not scale with user font settings
%	Parent element's same property	Widths, fluid layouts	% height needs a parent with explicit height
rem	Root font-size (html), default 16px	Font sizes, spacing, most things	Predictable — the modern default
em	Element's own font-size (parent's, for font-size itself)	Padding on buttons that should scale with text	Compounds when nested — can surprise you
vw / vh	1% of viewport width / height	Full-screen sections, hero text	100vw includes the scrollbar width on desktop
vmin / vmax	Smaller / larger of vw, vh	Square elements that fit any screen	
dvh / svh / lvh	Dynamic / small / large viewport height	Mobile browsers where the address bar hides	Use dvh instead of vh on mobile
ch	Width of the '0' character	Text line length (max-width: 65ch)	
fr	Fraction of free space	Grid columns only	
deg, turn	Angles	Gradients, rotate	90deg = 0.25turn
s, ms	Time	Transitions, animations	1s = 1000ms

EXAMPLE

```

.a { width: 300px; }      /* pixels: fixed */
.b { width: 50%; }       /* half of the PARENT's width */
.c { font-size: 1.5rem; } /* 1.5 × the root (html) font size = 24px by default */
.d { padding: 1em; }     /* 1 × THIS element's font size */
.e { height: 100vh; }    /* full viewport (window) height */
.f { width: 50vw; }      /* half viewport width */
.g { max-width: 65ch; }   /* 65 characters wide – perfect reading width */
.h { font-size: clamp(1rem, 2.5vw, 2rem); } /* min, preferred, max */
.i { width: calc(100% - 40px); } /* maths between units */

```

→ Golden rule: **rem for sizes and spacing, % or fr for layout widths, px only for borders.**

→ Trick: `html { font-size: 62.5%; }` makes 1rem = 10px, so 1.6rem = 16px — easy mental maths.

- If the user sets a larger default font in their browser, rem/em respect it, px ignores it (accessibility).

⚠ Common mistake

`height: 100%` does nothing. The parent has no set height, so 100% of 'auto' is auto. Use `100vh` or give the parent a height.

opacity and the box shadow family

Elzero CSS #29 #40

Gamal CSS #17

`opacity` makes the whole element (including its children and text) see-through. `box-shadow` draws one or more shadows around the box. `text-shadow` does the same for letters.

EXAMPLE

```
.faded { opacity: 0.5; } /* 0
invisible → 1 solid */
.card { box-shadow: 0 4px 12px
  rgba(0,0,0,.15); }
/*          x y blur
colour          */
.inset { box-shadow: inset 0 2px 4px
  rgba(0,0,0,.3); }
.multi { box-shadow: 0 1px 2px
  rgba(0,0,0,.2),
          0 8px 24px
  rgba(0,0,0,.1); } /* several, comma-
separated */
.spread { box-shadow: 0 0 0 4px
  gold; } /* 4th value = spread →
outline ring */
h1 { text-shadow: 2px 2px 4px
  rgba(0,0,0,.4); }
```

RESULT



Shadow

- Soft, realistic shadows: small y offset, large blur, low-alpha black.
- `opacity` also affects children; to fade only the background use `rgba` on the background colour instead.

background-color and background-image

Gamal CSS #25

Elzero CSS #05

Angela \$5

The background fills the content and padding area of the box (and under the border). You can set a colour, an image, or both — the colour shows through transparent parts of the image and while the image loads.

EXAMPLE

```
.hero {
  background-color: #0f3d68;           /
  * fallback colour */
  background-image: url("images/hero.jpg"); /
  * path relative to the CSS file! */
}
.icon { background-image: url("data:image/
svg+xml;..."); } /* inline SVG data */
```

RESULT



⚠ Common mistake

The image path is relative to the **CSS file**, not the HTML. If your CSS is in `css/style.css` and the image in `images/`, write `url("../images/hero.jpg")`.

background-repeat, position, size, attachment

Gamal CSS #25

Elzero CSS #05 #06

Angela \$5

Control how the image tiles, where it sits, how big it is, and whether it scrolls with the page.

background-size	Meaning	Use
cover	Scale to fill the whole box; crops what does not fit	Hero photos
contain	Scale so the whole image is visible; may leave empty space	Logos, product images
auto	Natural size	Patterns, icons
100% auto	Full width, height keeps ratio	Banners

EXAMPLE

```
.box {
  background-repeat: no-repeat; /* repeat (default) | repeat-x |
  repeat-y | no-repeat | space | round */
  background-position: center top; /* x y –
  keywords, px, or % ; "right 20px bottom 10px"
  */
  background-size: cover; /* cover
  | contain | 100% auto | 300px 200px */
  background-attachment:
  fixed; /* scroll (default) | fixed
  (parallax) | local */
}
```

RESULT



background shorthand and multiple backgrounds

Gamal CSS #25

Elzero CSS #06

Write all background properties in one line. Layer several images by separating them with commas — the first one is on top. A common trick: a dark gradient over a photo so white text stays readable.

EXAMPLE

```
/* color image repeat attachment position /
size */
.hero { background: #0f3d68 url("hero.jpg")
no-repeat fixed center / cover; }

/* two layers: dark overlay on top of a photo
*/
.hero {
  background:
    linear-gradient(rgba(0,0,0,.55),
    rgba(0,0,0,.55)),
    url("hero.jpg") center / cover no-repeat;
  color: white;
}
```

RESULT

Readable white text on any photo

- In the shorthand, size must come after position with a slash: `center / cover`.
- The shorthand resets every background property you did not mention.

Gradients

Gamal CSS #26 #27

Elzero CSS #55

A gradient is an image generated by the browser: a smooth blend between colours. Use it wherever an image is accepted (background-image, border-image, mask). Three kinds: linear (in a straight line), radial (from a centre outward), conic (around a centre like a pie).

الجريدينت صورة يي عملها المتصفح: تدرج بين لونين أو أكثر. linear خطي، radial دائري، conic زي الفطيرة.

Direction value	Meaning
to right / to left / to top / to bottom	Toward that side (default: to bottom)
to bottom right	Diagonal toward a corner
45deg	Angle: 0deg = to top, 90deg = to right, 180deg = to bottom
Colour stops	<code>red 0%, blue 100%</code> — positions where each colour is pure

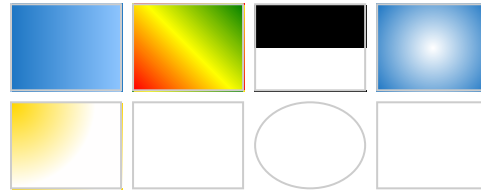
EXAMPLE

```

.l1 { background: linear-gradient(to right,
#1f77c4, #8ec5ff); }
.l2 { background: linear-gradient(45deg, red,
yellow, green); }
.l3 { background: linear-gradient(to bottom,
#000 0%, #000 50%, #fff 50%); } /* hard stop
= two colours */
.r1 { background: radial-gradient(circle,
white, #1f77c4); }
.r2 { background: radial-gradient(circle at
top left, gold, transparent 60%); }
.c1 { background: conic-gradient(red, yellow,
lime, aqua, blue, magenta, red); } /* colour
wheel */
.c2 { background: conic-gradient(#1f77c4 0
70%, #eee 70%); border-radius: 50%; } /* 70%
pie chart */
.rp { background: repeating-linear-
gradient(45deg, #ccc 0 10px, #fff 10px
20px); } /* stripes */

```

RESULT



Tip: Generators such as cssgradient.io let you design a gradient visually and copy the code.

Try it yourself

Create a button background that goes from purple (`#6c2bd9`) on the left to blue (`#1f77c4`) on the right, and a separate 'sale' badge that is a circle with a red-to-orange radial gradient.

Solution

```

.btn { background: linear-gradient(to right, #6c2bd9, #1f77c4); color: white; }
.badge { width: 60px; height: 60px; border-radius: 50%;
background: radial-gradient(circle, orange, red); }

```

filter and backdrop-filter

Elzero CSS #54

Apply image-editor effects to any element: blur, brightness, grayscale, drop shadows.

`backdrop-filter` applies the effect to whatever is behind a semi-transparent element — the 'frosted glass' look.

EXAMPLE

```
img:hover { filter: brightness(1.2)
contrast(1.1); }
.gray     { filter: grayscale(100%); }
.blur     { filter: blur(4px); }
.logo     { filter: drop-shadow(0 4px 6px
rgba(0,0,0,.3)); } /* follows transparent
shapes, unlike box-shadow */
.sepia    { filter: sepia(80%) hue-
rotate(20deg) saturate(1.5) invert(0)
opacity(.9); }

.glass {
  background: rgba(255,255,255,.2);
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255,255,255,.4);
}
```

RESULT



original · grayscale · blur · brightness

? Chapter quiz

1. Write pure blue in hex, rgb and hsl.
2. What is 2rem if the html font size is 16px?
3. Which background-size fills the box completely even if it must crop?
4. How do you put a dark overlay over a background photo?

Answers: 1) #0000ff, rgb(0,0,255), hsl(240,100%,50%). 2) 32px. 3) cover. 4) Two background layers: a semi-transparent linear-gradient first, then the url() image.

53. CSS Reference C — The Box Model: Sizing, Padding, Border, Margin

Every element is a rectangular box. Understanding the four layers of that box — content, padding, border, margin — solves most 'why is there a gap' and 'why does it overflow' problems. Read this chapter twice.

The four layers

Gamal CSS #5

Elzero CSS #41

Angela §7

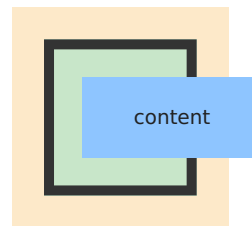
From inside out: **content** (text/image, sized by width/height), **padding** (space inside, between content and border, shows the background), **border** (the line), **margin** (space outside, transparent, pushes neighbours away).

أي عنصر صندوق من 4 طبقات: المحتوى، ال padding (مسافة داخلية)، ال border (الإطار)، ال margin (مسافة خارجية بتبعد الجيران).

EXAMPLE

```
.box {
  width: 200px;           /* content width */
  height: 100px;          /* content height */
  padding: 20px;          /* inside */
  border: 5px solid #333; /* line */
  margin: 30px;           /* outside */
  background: lightblue; /* fills content + padding */
}
```

RESULT



orange = margin · green = padding · blue = content · black = border

- DevTools → Elements → Computed shows this exact diagram for any element with real numbers. Use it constantly.
- Background covers content + padding, not margin.
- Margin is transparent; you can see the parent's background through it.

box-sizing — make width mean the whole box

Gamal CSS #5

Elzero CSS #41

Angela §7

By default (`content-box`) width sets only the content, and padding and border are added on top, so a 200px box with 20px padding is really 240px wide. With `border-box` , width includes padding and border — what everyone expects. Every project should set border-box globally.

EXAMPLE

```

/* default: content-box */
.a { width: 200px; padding: 20px; border: 5px solid; }
/* visible width = 200 + 40 + 10 = 250px */

/* border-box: width is the visible width */
.b { box-sizing: border-box; width: 200px; padding: 20px; border: 5px solid; }
/* visible width = 200px; content shrinks to 150px */

/* put this at the top of every stylesheet */
*, *::before, *::after { box-sizing: border-box; }

```

RESULT

content-box:

width:150 → looks 186

border-box:

width:150 → looks 150

⚠ Common mistake

Two 50% columns side by side wrap onto two lines. Cause: padding added on top of 50% + 50% = more than 100%. Fix: border-box.

width, height, min/max

Gamal CSS #7 #9

Elzero CSS #15

Angela §7

`width` and `height` set the size. `min-*` and `max-*` set limits that override them. Combining a fluid width with a max-width is the basis of responsive containers.

Value	Meaning
auto	Default. Block: fill the parent's width. Height: fit the content
fit-content	As wide as content, but not wider than the parent
min-content	The narrowest possible (longest word)
max-content	As wide as it would be on one line

EXAMPLE

```
.fixed { width: 300px; height: 200px; }
.fluid { width: 100%; } /*
as wide as parent */
.limited { width: 90%; max-width:
1100px; } /* fluid, but never wider than
1100 */
.card { min-width: 250px; } /*
never narrower than 250 */
.hero { min-height: 60vh; } /*
at least 60% of screen, grows with content */
img { max-width: 100%; height:
auto; } /* never overflow, keep ratio */
.square { width: 200px; aspect-ratio: 1 /
1; } /* height follows automatically */
.video { aspect-ratio: 16 / 9; }
```

RESULT

width 70%, max 200px

1:1

- Prefer `min-height` to `height` for anything containing text; fixed height makes text overflow when it grows or the screen shrinks.
- If min and max conflict, min wins.

padding

Gamal CSS #5

Elzero CSS #07

Angela §7

Space between the content and the border, inside the element. It increases the clickable/tappable area of buttons and links and keeps text away from edges. Padding cannot be negative.

EXAMPLE

```
.a { padding: 10px; } /*
all four sides */
.b { padding: 10px 20px; } /*
top&bottom left&right */
.c { padding: 10px 20px 30px; } /*
top left&right bottom */
.d { padding: 10px 20px 30px 40px; } /*
top right bottom left - clockwise from top */
.e { padding-left: 1rem; padding-block:
2rem; } /* single side / logical (top+bottom)
*/
.btn { padding: .6em
1.2em; } /* em = scales with the
button's font size */
```

RESULT

10px

10 20

10 20 30 40

- Memory aid for four values: **TRouBLE** — Top, Right, Bottom, Left (clockwise).
- Percent padding is relative to the parent's **width**, even for top/bottom — the old trick for aspect-ratio boxes.

border and outline

Gamal CSS #5

Elzero CSS #09 #10

Angela \$7

border draws a line around the box and takes up space. **outline** draws a line outside the border and takes no space — browsers use it for keyboard focus rings. Never remove focus outlines without replacing them.

border-style values	Look
solid	Continuous line
dashed / dotted	Dashes / dots
double	Two lines (needs width $\geq$ 3px)
groove / ridge / inset / outset	3D effects (rare today)
none / hidden	No border

EXAMPLE

```
.a { border: 2px solid #333; } /* width style colour - style is required */
.b { border-bottom: 1px dashed gray; }
.c { border-style: dotted; border-width: 3px; border-color: red green blue gold; } /* per side */
.d { border: none; }
.e { border-top: 4px solid var(--primary); } /* accent line on cards */

/* outline: no layout shift, can be offset */
button:focus-visible { outline: 3px solid #1f77c4; outline-offset: 2px; }
.debug * { outline: 1px solid red; } /* see every box while debugging */
```

RESULT



border-radius — rounded corners

Gamal CSS #10

Elzero CSS #39

Angela \$7

Rounds the corners of the box (and its background and border). One value rounds all corners; four values go clockwise from top-left. 50% on a square makes a circle. A very large value on a wide element makes a pill.

EXAMPLE

```
.card { border-radius: 12px; }
.avatar { width: 80px; height: 80px; border-radius: 50%; } /* circle */
.pill { border-radius: 999px; }
.tab { border-radius: 8px 8px 0 0; } /* TL TR BR BL */
.leaf { border-radius: 40px 0 0 0; } /* TL&BR / TR&BL */
.blob { border-radius: 60% 40% 30% 70% / 60% 30% 70% 40%; } /* horizontal / vertical radii */
.one { border-top-left-radius: 20px; }
```

RESULT



→ Add `overflow: hidden` to a rounded container if a child image sticks out of the corners.

margin

Gamal CSS #5 #6

Elzero CSS #08 #44

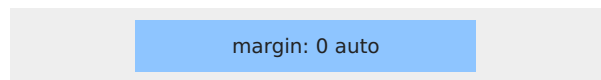
Angela \$7

Space outside the border that separates the element from its neighbours. Same shorthand as padding. Margin can be negative (pulls elements closer or over each other) and can be `auto` (absorb the free space — the classic way to centre a block horizontally).

EXAMPLE

```
.a { margin: 16px; }
.b { margin: 0 auto; width: 800px; } /*
centre horizontally: needs a width */
.c { margin-top: -20px; } /*
overlap upward */
.d { margin-bottom: 2rem; }
.stack > * + * { margin-top: 1rem; } /*
space between siblings, none above the first */
.e { margin-inline: auto; } /*
logical: left+right */
```

RESULT



- Only block-level elements with a width can be centred with `margin: auto`. It does not centre inline text (use `text-align`) or centre vertically (use Flexbox or Grid).
- Browser default margins: `body` 8px, `h1-h6` and `p` have top/bottom margins, `ul` has left padding. A reset removes them so you control spacing.

Margin collapse — the surprise

Gamal CSS #5

Elzero CSS #44

When two **vertical** margins touch (a block's bottom margin and the next block's top margin, or a parent's and its first child's), they do not add up: the larger one is used. Horizontal margins never collapse. Flex and grid items never collapse.

EXAMPLE

```
h2 { margin-bottom: 30px; }
p { margin-top: 20px; }
/* gap between h2 and p is 30px, NOT 50px */

/* parent + first child: the child's top
margin "leaks" outside the parent */
.parent { background: gold; }
.child { margin-top: 40px; } /* the gold
box moves down instead of the child inside it
*/

/* fixes: give the parent padding, a border,
overflow:auto, or display:flow-root */
.parent { padding-top: 1px; } /* or
*/ .parent { display: flow-root; }
```

RESULT

h2 margin-bottom 30 + p margin-top 20 →

h2

30px (not 50)

p

Tip: Use margins in one direction only (for example only margin-bottom) and you will rarely be surprised by collapse.

display — block, inline, inline-block, none

Gamal CSS #11 #43 #44

Elzero CSS #11 #12

Angela §7

The display property decides how an element participates in layout. Block elements stack vertically and fill the width. Inline elements flow inside text and ignore width/height. Inline-block flows inline but accepts width/height and vertical padding/margin. None removes the element completely.

display بتحدد العنصر بيتصرف إزاي: block يأخذ سطر لوحده، inline جوّه النص، inline-block جوّه النص بس بيقبل width و height بيختفي خالص.

	block	inline	inline-block
Starts new line	yes	no	no
width / height	respected	ignored	respected
top/bottom margin & padding	respected	padding draws but does not push lines; margin ignored	respected
Default for	div p h1 ul li section	span a strong em img*	img button input select (behave like it)

EXAMPLE

```
span { display: block; }           /* now
takes its own line */
li   { display: inline-block; padding: 10px
15px; } /* horizontal menu */
.hide { display: none; }           /* gone,
takes no space */
.flex { display: flex; }           /* see
Flexbox chapter */
.grid { display: grid; }           /* see
Grid chapter */
.contents { display:
contents; } /* element disappears but
children stay */
```

RESULT

block:

full width

next line

inline: a b c flow with text

inline-block: 50×24 50×24

→ Inline-block elements have a small gap between them caused by the whitespace in your HTML. Flexbox has no such gap — another reason to prefer it.

Hiding: display none vs visibility hidden vs opacity 0

Gamal CSS #44

Elzero CSS #12

Three ways to hide with different consequences. Choose based on whether the space should remain and whether you need an animation.

	display: none	visibility: hidden	opacity: 0
Takes up space	No	Yes (empty gap)	Yes
Receives clicks	No	No	Yes — invisible but clickable!
Read by screen readers	No	No	Yes
Can be animated	No	No (only toggled)	Yes, smooth fade
Use for	Menus, modals, tabs	Keep layout stable	Fade effects (add pointer-events:none)

EXAMPLE

```
.menu.closed { display: none; }
.placeholder { visibility: hidden; } /* reserve the space */
.fade-out    { opacity: 0; pointer-events: none; transition: opacity .3s; }
/* accessible hide: invisible but still read by screen readers */
.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
```

overflow — content bigger than the box

Gamal CSS #29

Elzero CSS #16

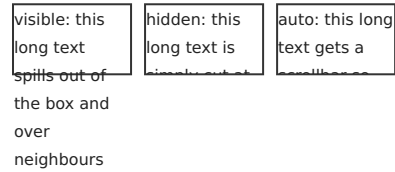
Angela \$8

Decides what happens when content does not fit in a box with a fixed size: spill out, cut off, or scroll.

EXAMPLE

```
.box { height: 100px; }
.a { overflow:
visible; } /* default: spills out over
neighbours */
.b { overflow: hidden; } /* cut off, no
scroll */
.c { overflow: scroll; } /* scrollbars
always */
.d { overflow:
auto; } /* scrollbars only when needed
*/
.e { overflow-x: auto; overflow-y:
hidden; } /* horizontal carousel */
.f { overflow:
clip; } /* like hidden but cannot be
scrolled by JS either */
```

RESULT



visible: this
long text
spills out of
the box and
over
neighbours

hidden: this
long text is
simply cut off

auto: this long
text gets a
scrollbar

- `overflow: hidden` on a parent also clips absolutely-positioned children and box-shadows — a frequent surprise.
- `text-overflow: ellipsis` for the '...' effect requires `overflow: hidden` and `white-space: nowrap`.

Complete example — a card component

Gamal CSS #5-#11

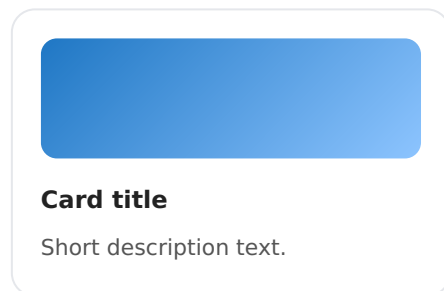
Angela §7

Everything in this chapter applied to one reusable card.

EXAMPLE

```
.card {
  box-sizing: border-box;
  width: 100%;
  max-width: 320px;
  padding: 1.5rem;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,.08);
  background: #fff;
  margin: 0 auto 1.5rem;
  overflow: hidden;
}
.card img { width: 100%; height: 160px;
object-fit: cover; border-radius: 8px; }
.card h3 { margin: 1rem 0 .5rem; }
.card p { margin: 0; color: #555; }
```

RESULT



Try it yourself

Build a 'notification badge': a 24×24px red circle with white centred number, positioned at the top-right corner of a 60×60px grey square (hint: position relative on the square, absolute on the badge — see Reference E).

Solution

```
.icon { position: relative; width: 60px; height: 60px; background: #ddd; border-radius: 8px; }  
.badge { position: absolute; top: -8px; right: -8px; width: 24px; height: 24px; border-radius: 50%; background: red; color: white; display: flex; align-items: center; justify-content: center; font-size: 12px; }
```

? Chapter quiz

1. What is the visible width of a content-box element with width 100px, padding 10px, border 2px?
2. Which margins collapse: vertical or horizontal?
3. How do you centre a block horizontally?
4. Which hiding method keeps the space but hides the element?

Answers: 1) 124px. 2) Vertical. 3) Give it a width and margin: 0 auto. 4) visibility: hidden.

54. CSS Reference D — Text and Fonts

Most of a web page is text. These properties control the typeface, its size and weight, spacing, alignment and decoration. Good typography alone makes a site look professional.

font-family and web fonts

Gamal CSS #19

Elzero CSS #23

Angela §5

The typeface. Give a comma-separated list: the browser uses the first font that is installed on the visitor's computer, and the last item should be a generic family as a safety net. To use a font the visitor may not have, load it from Google Fonts or your own files.

نوع الخط. بتكتب قائمة: المتصفح يستخدم أول خط موجود على جهاز الزائر، وآخر واحد لازم يكون عائلة عامة زي -sans-serif.

Generic family	Look	Common fonts
sans-serif	No strokes at letter ends	Arial, Helvetica, Roboto, Segoe UI, Cairo
serif	Small strokes (feet)	Times New Roman, Georgia, Amiri
monospace	Fixed width	Consolas, Courier New, Fira Code
cursive	Handwriting	Comic Sans, Brush Script
system-ui	The OS default	San Francisco, Segoe UI, Roboto

EXAMPLE

```
body { font-family: "Segoe UI", Roboto,
Helvetica, Arial, sans-serif; }
code { font-family: "Fira Code", Consolas,
monospace; }
h1 { font-family: Georgia, "Times New
Roman", serif; }

/* Google Fonts: 1) in <head> */
<link href="https://fonts.googleapis.com/css2?
family=Cairo:wght@400;700&display=swap"
rel="stylesheet">
/* 2) in CSS */
body { font-family: "Cairo", sans-serif; }

/* your own font file */
@font-face {
  font-family: "MyBrand";
  src: url("../fonts/mybrand.woff2")
  format("woff2");
  font-weight: 400;
  font-display: swap; /* show fallback text
until the font loads */
}
```

RESULT

sans-serif — clean, screens

serif — has small feet, print/elegant

monospace — every letter same width, code

cursive — handwriting

- Quote font names that contain spaces: `"Times New Roman"`.
- For Arabic, good free Google Fonts: Cairo, Tajawal, Almarai, Amiri, Noto Naskh Arabic.
- Limit to two fonts per site (one for headings, one for body) and only the weights you use — each weight is a download.

font-size

Gamal CSS #20

Elzero CSS #24

Angela \$5

How big the letters are. Browsers default to 16px for body text. Use rem so sizes scale with the user's preference and are easy to reason about. Headings get bigger sizes; small print smaller.

EXAMPLE

```
html { font-size: 16px; } /* the root;
1rem = 16px */
body { font-size: 1rem; }
h1 { font-size: 2.5rem; } /* 40px */
h2 { font-size: 2rem; } /* 32px */
small { font-size: .875rem; } /* 14px */
.lead { font-size:
1.25em; } /* 1.25 × the parent's size */
h1 { font-size: clamp(1.75rem, 1rem + 3vw,
3.5rem); } /* fluid: grows with the screen,
within limits */
.kw { font-size: larger; } /* keywords:
xx-small ... xx-large, smaller, larger */
```

RESULT

2rem 1.5rem

1rem .875rem .75rem

- A common type scale: 0.75 / 0.875 / 1 / 1.25 / 1.5 / 2 / 2.5 / 3 rem.
- Never go below 16px for body text on mobile — iPhones zoom into inputs smaller than 16px.

⚠ Common mistake

em compounding: `li { font-size: 1.2em }` makes a nested list $1.2 \times 1.2 = 1.44 \times$ bigger, then $1.73 \times \dots$ Use rem to avoid this.

font-weight

Gamal CSS #23

Elzero CSS #25

Angela \$5

Thickness of the strokes, from 100 (thin) to 900 (black). `normal` = 400, `bold` = 700. The font file must actually include that weight; otherwise the browser fakes bold (ugly) or falls back to the nearest available weight.

EXAMPLE

```
p { font-weight: 400; } /* normal */
strong { font-weight: 700; } /* bold */
h1 { font-weight: 800; }
.thin { font-weight: 300; }
.rel { font-weight:
bolder; } /* relative to parent */
```

RESULT

300 light · 400 normal · **600 semibold**
· **700 bold** · **900 black**

- When loading Google Fonts, request the weights you use: `wght@300;400;700`.
- Variable fonts include every weight in one file; then any number 100–900 works.

font-style, font-variant, font shorthand

Gamal CSS #21 #22 #24

Elzero CSS #25

`font-style` gives italic. `font-variant` gives small caps. The `font` shorthand sets everything in one line — size and family are required, in that order at the end.

EXAMPLE

```
em { font-style:
italic; } /* normal | italic | oblique */
.caps { font-variant: small-caps; }
.nums { font-variant-numeric: tabular-
nums; } /* digits same width – tables of
prices */

/* shorthand: style variant weight size/line-
height family */
body { font: italic small-caps 700 16px/1.6
"Cairo", sans-serif; }
body { font: 1rem/1.6 system-ui, sans-
serif; } /* minimal valid form */
button { font: inherit; } /* form controls
do not inherit by default */
```

RESULT

italic · SMALL CAPS TEXT · 1234 / 9876
tabular

⚠ Common mistake

Writing `font: bold 16px;` (no family) — invalid, the whole line is ignored. Both size and family are mandatory in the shorthand.

line-height

Gamal CSS #15

Elzero CSS #20

Angela \$5

The height of each line of text — the distance from one baseline to the next. Bigger line-height = more breathing room. Use a unitless number (a multiplier of the font-size) so it scales correctly when the font size changes.

EXAMPLE

```
body { line-height:
1.6; } /* unitless: 1.6 × font-size */
h1 { line-height: 1.1; } /* headings
tighter */
.btn { line-height: 40px; height:
40px; } /* old trick: vertically centre one
line of text */
.bad { line-height: 24px; } /* px:
children with bigger font will overlap */
```

RESULT

line-height 1.1: lines are tight and hard to read for long paragraphs of text

line-height 1.7: comfortable spacing that is easy to read in long paragraphs

- Recommended: 1.5–1.7 for body text, 1.1–1.3 for large headings.
- Extra line-height is split equally above and below the letters — this is why text seems to have a top gap.

color and text-align

Gamal CSS #12

Elzero CSS #17 #18

Angela \$5

`color` sets the text colour (it inherits, so set it on `body`). `text-align` aligns the inline content inside a block: left, right, centre, or justified. It aligns text and inline elements, not the block itself.

EXAMPLE

```
body      { color: #222; }
.muted    { color: #6b7280; }
.center    { text-align: center; }
.right     { text-align: right; } /* or
"end" — respects RTL */
.justify   { text-align: justify; } /*
straight edges both sides; can create rivers
of space */
.start     { text-align: start; } /* left
in LTR, right in RTL for bilingual sites */
```

RESULT

left (default in English)

center

right

justify stretches the spaces so both edges are straight
lines like this

⚠ Common mistake

Using `text-align: center` to centre a box. It centres the text inside the box; the box stays where it was. Centre boxes with `margin auto`, Flexbox or Grid.

text-decoration, text-transform, text-indent

Gamal CSS #14

Elzero CSS #19

Angela \$5

Decoration adds lines (underline, strike-through). Transform changes letter case without touching the HTML. Indent pushes the first line of a paragraph.

EXAMPLE

```
a          { text-decoration:
none; }      /* remove underline */
a:hover     { text-decoration: underline; }
.del         { text-decoration: line-through
red; }
.fancy       { text-decoration: underline wavy
#1f77c4; text-underline-offset: 4px; }
/* shorthand: line style colour thickness */

.upper       { text-transform:
uppercase; } /* HELLO */
.lower       { text-transform:
lowercase; } /* hello */
.cap         { text-transform:
capitalize; } /* Hello World */
p            { text-indent: 2em; }
```

RESULT

underline · ~~strike~~ · wavy

UPPERCASE · Capitalize Each Word

Indented first line of a paragraph...

→ `text-transform` keeps the source text; copying the text copies the original case. Screen readers may read UPPERCASE letters one by one — avoid on long text.

letter-spacing, word-spacing, white-space, word-break

Gamal CSS #18

Elzero CSS #20 #21

Fine control of spacing and wrapping. `white-space` decides whether text wraps and whether extra spaces are kept. `overflow-wrap` lets a very long word (URL) break so it does not overflow.

white-space	Collapses spaces?	Wraps lines?	Keeps newlines?
normal (default)	yes	yes	no
nowrap	yes	no	no
pre	no	no	yes
pre-wrap	no	yes	yes
pre-line	yes	yes	yes

EXAMPLE

```
h1      { letter-spacing: -0.02em; } /*
tighten big headings */
.badge  { letter-spacing: .1em; text-
transform: uppercase; }
p       { word-spacing: 2px; }

.nowrap { white-space: nowrap; } /*
never wrap: one long line */
.pre    { white-space:
pre; }   /* keep spaces and newlines like
<pre> */
.prewrap { white-space: pre-
wrap; }  /* keep them AND wrap */
.url     { overflow-wrap: break-word; } /*
break long words if needed */
.hard    { word-break: break-all; } /*
break anywhere, even mid-word */
.hy      { hyphens: auto; } /*
add hyphens when breaking (needs lang attr) */
```

RESULT

Default:

averyveryveryverylongwordthatoverflows
break-word:
averyveryveryverylongwor
dthatwraps

Text overflow with ellipsis (...) and line clamp

Elzero CSS #21

Gamal CSS #18

Cut long text and show three dots. One line needs three properties together. Several lines use line-clamp.

EXAMPLE

```

/* one line */
.title {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

/* three lines */
.excerpt {
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

```

RESULT

A very long product title ...

Two-line clamp cuts the paragraph after exactly two lines and adds the dots automatically at the end.

direction and writing for Arabic (RTL)

Gamal CSS #13

Elzero CSS #18

Arabic, Hebrew and Persian are written right-to-left. Set the direction on the html element with the `dir` attribute (better than CSS because it applies before CSS loads). Use logical properties (start/end) instead of left/right so one stylesheet works for both directions.

Physical (avoid)	Logical (prefer)
margin-left	margin-inline-start
margin-right	margin-inline-end
padding-top / bottom	padding-block-start / end
left: 0	inset-inline-start: 0
text-align: left	text-align: start
border-left	border-inline-start

EXAMPLE

```
/* HTML: <html lang="ar" dir="rtl"> – preferred */
/* CSS alternative */
body { direction: rtl; }

/* logical properties: flip automatically with direction */
.card { margin-inline-start: 1rem; } /* margin-left in LTR, margin-right in RTL */
.card { padding-inline: 1rem 2rem; } /* start end */
.card { border-inline-start: 4px solid; }
.nav { text-align: start; }
.icon { inset-inline-end: 10px; } /* right in LTR, left in RTL */

unicode-bidi: isolate; /* keep mixed-direction text tidy (bdi element) */
```

RESULT

هذا نص عربي يبدأ من اليمين، ويحتوي على كلمة English في المنتصف.

cursor, user-select, pointer-events, caret-color

Elzero CSS #26 #56

Small interaction properties: which mouse pointer to show, whether text can be selected, whether an element reacts to the mouse at all, and the colour of the typing cursor.

EXAMPLE

```
button, a, .clickable { cursor: pointer; } /* hand */
.disabled { cursor: not-allowed; }
.loading { cursor: wait; } /* also: text, move, grab, grabbing, zoom-in, help, crosshair, none */
.custom { cursor: url("pen.png") 4 4, auto; }

.logo { user-select: none; } /* cannot be highlighted */
.code { user-select: all; } /* one click selects everything */

.overlay { pointer-events: none; } /* clicks pass through to what is behind */
input { caret-color: #1f77c4; } /* the blinking text cursor */
```

RESULT

☐ default · ☐ pointer · ☐ wait · ☐ not-allowed · ☐ grab · ☒ move

Lists: list-style

Gamal CSS #32

Elzero CSS #32

Control or remove the bullets and numbers of ul/ol. Menus remove them entirely; articles may customise them.

EXAMPLE

```
ul { list-style-type: square; } /* disc |
circle | square | none */
ol { list-style-type: upper-roman; } /*
decimal | lower-alpha | upper-alpha | lower-
roman | arabic-indic ... */
ul { list-style-image: url("check.svg"); }
ul { list-style-position: inside; } /*
bullet inside the text block (default outside)
*/
ul { list-style: none; padding: 0; margin:
0; } /* menu reset */

/* modern: custom bullet with ::marker */
li::marker { color: #1f77c4; font-size:
1.2em; }
ul.check li::before { content: "✓ "; color:
green; }
```

RESULT

- square I. Roman ✓ custom
- bullets II. numbers ✓ marker

Tables: styling

Gamal CSS #33

Elzero CSS #33

Tables have their own properties. The most important: collapse the double borders, add padding to cells, stripe the rows, and highlight the row under the mouse.

EXAMPLE

```
table { border-collapse: collapse; width:
100%; } /* separate (default, double lines)
| collapse */
th, td { padding: .75rem 1rem; text-align:
start; border-bottom: 1px solid #e5e7eb; }
thead th { background: #0f3d68; color: #fff;
position: sticky; top: 0; }
tbody tr:nth-child(even) { background:
#f8fafc; }
tbody tr:hover { background: #e8f0fe; }
td.num { text-align: end; font-variant-
numeric: tabular-nums; }
table { table-layout: fixed; } /* equal
columns, faster; auto = fit content */
caption { caption-side: bottom; color: #666; }
td.empty { background: #fafafa; } /* empty-
cells: hide also exists */
```

RESULT

Item	EGP
Bread	5
Cake	40
Tea (hover)	10

? Chapter quiz

1. Which unit should you use for line-height and why?
2. What is required in the font shorthand?
3. How do you remove the underline from links?
4. Name the three properties needed for a single-line ellipsis.

Answers: 1) A unitless number, so it scales with each element's font size. 2) font-size and font-family, in that order at the end. 3) text-decoration: none. 4) white-space: nowrap; overflow: hidden; text-overflow: ellipsis.

55. CSS Reference E — Positioning: float, position, z-index

Normal flow stacks blocks top to bottom. These properties take elements out of the flow or move them: float for wrapping text around images, position for pinning, overlapping and sticky headers, z-index for deciding what is on top.

float and clear

Gamal CSS #28

Elzero CSS #27

Angela \$8

Float pushes an element to the left or right and lets the following text wrap around it — the newspaper photo effect. Before Flexbox it was abused for whole layouts; today use it only for wrapping. `clear` stops the wrapping.

float بتزق العنصر يمين أو شمال والنص يلف حوالیه. زمان كانوا بيعملوا بيه ال layout كله، دلوقتي استخدم Flexbox وGrid.

EXAMPLE

```
img.left { float: left; margin: 0 1rem 1rem 0; }
img.right { float: right; margin: 0 0 1rem 1rem; }
.after { clear: both; } /* left | right | both: start below the floats */

/* the collapsed-parent problem: a parent with only floated children has height 0 */
.clearfix::after { content: ""; display: block; clear: both; }
/* modern fix */
.parent { display: flow-root; }
```

RESULT

The blue box is floated left, so this text flows around it on the right side and continues underneath once the box ends. This is exactly how magazines place pictures inside articles.

⚠ Common mistake

Floated children make the parent's background disappear because the parent's height collapses to zero. Add `display: flow-root` to the parent.

position: static and relative

Gamal CSS #30

Elzero CSS #30

Angela \$8

`static` is the default: the element sits in normal flow and the offset properties (top, right, bottom, left) do nothing. `relative` keeps the element's original space but lets you nudge it

with offsets — and, crucially, makes it the reference point for absolutely positioned children.

EXAMPLE

```
.nudge { position: relative; top: 4px; left: 10px; } /* moved 4 down, 10 right; its old space stays */
.parent { position: relative; } /* most common use: anchor for absolute children (no offsets needed) */
```

RESULT



The orange box moved but its dashed original space is kept; neighbours do not move.

position: absolute

Gamal CSS #30

Elzero CSS #30

Angela \$8

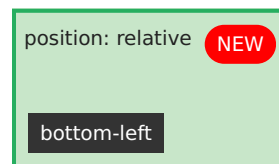
Removes the element from the flow (neighbours act as if it does not exist) and places it using top/right/bottom/left relative to the nearest ancestor that has a position other than static. If there is none, it is placed relative to the page. Use it for badges, tooltips, close buttons, overlays.

EXAMPLE

```
.card { position: relative; } /* the container becomes the reference */
.badge { position: absolute; top: 8px; right: 8px; }
.close { position: absolute; top: 0; right: 0; }
.overlay { position: absolute; inset: 0; background: rgba(0,0,0,.5); } /* inset = top right bottom left = 0 → cover parent */

/* centre an absolute element */
.center { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }
```

RESULT



- Always ask: 'absolute relative to what?' If it flies to the corner of the page, the parent is missing `position: relative`.
- Absolute elements shrink to fit their content unless you give them a width or both left and right.
- `inset: 0` is shorthand for `top:0; right:0; bottom:0; left:0`.

Try it yourself

Put a small 'Sale' label in the top-left corner of a product image.

Solution

```
.product { position: relative; display: inline-block; }
.product .label { position: absolute; top: 10px; left: 10px;
                  background: crimson; color: #fff; padding: 4px 8px; font-size: 12px; }
```

position: fixed

Gamal CSS #30

Elzero CSS #30

Angela §8

Like absolute, but the reference is always the browser window, and the element stays in place when the page scrolls. Navbars, 'back to top' buttons, cookie banners, chat bubbles.

EXAMPLE

```
.navbar { position: fixed; top: 0; left: 0;
          right: 0; height: 60px; z-index: 100; }
body { padding-top: 60px; } /* the navbar covers the top — push
                             content down */
.to-top { position: fixed; bottom: 20px;
          right: 20px; }
.modal-backdrop { position: fixed; inset: 0;
                  background: rgba(0,0,0,.6); }
```

RESULT

fixed navbar — stays while content scrolls

page content...
more...
more...



→ A fixed element inside a parent with `transform`, `filter` or `perspective` becomes relative to that parent instead of the window — a known gotcha.

position: sticky

Gamal CSS #30

Elzero CSS #30

A hybrid: the element scrolls normally until it reaches the offset you set (e.g. `top: 0`), then it sticks there — until its parent scrolls out of view. Perfect for section headers, table headers and sidebars.

EXAMPLE

```
th { position: sticky; top: 0;
      background: #fff; } /* table header stays
                           visible */
.sidebar { position: sticky; top: 80px;
           align-self: start; } /* in a flex/grid row */
h2.section { position: sticky; top: 0; }
```

RESULT

Sticky header (scroll me)

row 1
row 2

⚠ Common mistake

Sticky does not stick. Causes: you forgot `top`; a parent has `overflow: hidden/auto`; the parent is not taller than the sticky element.

z-index and stacking

Gamal CSS #31

Elzero CSS #31

When elements overlap, z-index decides which is in front: higher numbers on top. It only works on positioned elements (relative, absolute, fixed, sticky) and on flex/grid children. Elements with a z-index create a stacking context: children cannot escape above their parent's level.

EXAMPLE

```
.dropdown { position: absolute; z-index: 10; }
.navbar   { position: fixed; z-index: 100; }
.modal    { position: fixed; z-index: 1000; }
.toast     { position: fixed; z-index: 1100; }
.behind    { position: relative; z-index:
-1; }      /* behind its parent's background */
```

RESULT



- Without z-index, later elements in the HTML are painted on top of earlier ones.
- Use a scale for your project (10, 100, 1000) and keep it in variables: `--z-modal: 1000`.
- 'My z-index: 9999 does not work' — the element's parent has a lower z-index and creates a stacking context; you cannot escape it. Move the element or raise the parent.

object-fit and object-position (images and video)

Angela \$7

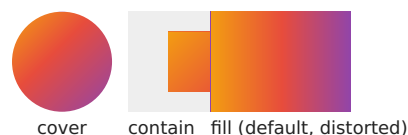
When an image is forced into a box with a different shape, object-fit decides whether it stretches, is cropped, or is letter-boxed — the same idea as background-size but for real `img` elements.

Value	Effect
fill	Stretch to fill (default) — distorts
cover	Fill and crop, keeps ratio
contain	Fit inside, keeps ratio, may leave gaps
none	Natural size, cropped
scale-down	Smaller of none and contain

EXAMPLE

```
img.avatar { width: 80px; height: 80px;
object-fit: cover; border-radius: 50%; }
img.logo   { width: 200px; height: 80px;
object-fit: contain; }
img.hero   { width: 100%; height: 400px;
object-fit: cover; object-position: center
top; }
```

RESULT



? Chapter quiz

1. Absolute elements are positioned relative to what?
2. Which position value keeps an element in place while scrolling?
3. Why might z-index: 9999 still be behind another element?
4. What is inset: 0 short for?

Answers: 1) The nearest ancestor with a position other than static (or the page). 2) fixed (sticky, after it reaches its offset). 3) Its parent creates a stacking context with a lower z-index. 4) top:0; right:0; bottom:0; left:0.

56. CSS Reference F — Pseudo-classes and Pseudo-elements

Pseudo-classes select elements in a particular state or position (hovered, first child, checked) without adding classes in HTML. Pseudo-elements style a part of an element or insert decorative content before/after it. Together they remove a huge amount of HTML clutter.

Link and interaction states

Gamal CSS #35

Elzero CSS #34

Angela \$6

States that change as the user interacts. For links, write them in the order Link–Visited–Hover–Active ("LoVe HAtE") so each overrides the previous correctly.

EXAMPLE

```
a:link { color: #1f77c4; } /* not yet visited */
a:visited { color: #6c2bd9; }
a:hover { color: #e67e22; text-decoration: underline; } /* mouse over */
a:active { color: red; } /* while the mouse button is down */
a:focus-visible { outline: 2px solid #1f77c4; outline-offset: 2px; } /* keyboard focus only */

button:hover { background: #0f3d68; }
button:active { transform: translateY(1px); }
input:focus { border-color: #1f77c4; box-shadow: 0 0 0 3px rgba(31,119,196,.2); }
.card:hover { box-shadow: 0 8px 24px rgba(0,0,0,.15); transform: translateY(-4px); }
```

RESULT



- `:hover` does not exist on touch screens — never hide important actions behind hover only.
- `:focus` fires for mouse clicks too; `:focus-visible` only for keyboard — use it for outlines so mouse users do not see rings.

Form states

Gamal CSS #35

Elzero CSS #34

Style inputs according to their validation and interaction state, with no JavaScript.

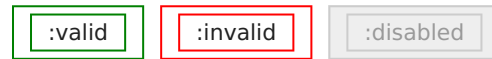
EXAMPLE

```

input:required      { border-left: 3px solid
#e67e22; }
input:optional      { }
input:valid          { border-color: green; }
input:invalid        { border-color: red; }
input:user-invalid   { border-color:
red; } /* only after the user interacted
*/
input:disabled       { background: #eee;
cursor: not-allowed; }
input:read-only      { background: #fafafa; }
input:placeholder-shown { border-style:
dashed; }
input:checked + label { font-weight: bold;
color: #1f77c4; } /* checkbox/radio ticked
*/
input:indeterminate  { }
input:in-range / :out-of-range { }
select:focus, textarea:focus { outline: 2px
solid #1f77c4; }

```

RESULT



☒ **:checked + label**

Tip: `:invalid` is red before the user has typed anything, which feels aggressive. `:user-invalid` waits until they leave the field.

Structural: first, last, nth, only, empty

Gamal CSS #35

Elzero CSS #34

Angela §6

Select by position among siblings. `nth-child(n)` takes a number, `odd`, `even`, or a formula `an+b`. The `-of-type` variants count only siblings of the same tag.

EXAMPLE

```
li:first-child      { border-top: 0; }
li:last-child      { border-bottom: 0; }
li:only-child      { }
li:nth-child(3)    { color:
red; }             /* the 3rd */
tr:nth-child(odd)   { background:
#f5f5f5; }         /* 1,3,5 zebra */
tr:nth-child(even)  { }
li:nth-child(3n)    { }             /* 3,6,9 every third */
li:nth-child(3n+1)  { }             /* 1,4,7 */
li:nth-child(-n+3)  { }             /* the first
three */
li:nth-child(n+4)   { }             /* from the
4th onward */
li:nth-last-child(2) { }            /* second from the end */
p:first-of-type     { font-size: 1.2em; } /
* first <p> even if an <h2> comes before it */
img:nth-of-type(2)  { }
p:empty             { display: none; } /
* no content at all */
```

RESULT

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

li:nth-child(3n) → 3, 6, 9 highlighted

⚠ Common mistake

`p:first-child` does not match if the first child of the parent is an `h2`. `first-child` means 'is the first child', not 'is the first p'. Use `p:first-of-type`.

Logical: not, is, where, has

Gamal CSS #35

Elzero CSS #34

Combine and invert selectors. `:has()` is the long-awaited 'parent selector': style an element based on what is inside it.

EXAMPLE

```
li:not(:last-child) { margin-bottom: 8px; }
input:not([type="submit"]) { width: 100%; }
:is(h1, h2, h3) a    { color: inherit; }    /* instead of h1 a, h2 a, h3 a */
:where(ul, ol) li    { }                  /* like :is but adds zero specificity – easy to override */

.card:has(img)        { padding: 0; }        /* card that contains an image */
form:has(:invalid) button { opacity: .5; }    /* disable-looking button while form invalid */
label:has(+ input:focus) { color: #1f77c4; } /* label whose next sibling input is focused */
body:has(.modal.open) { overflow: hidden; } /* stop page scroll when modal open */
```

- `:has()` works in all modern browsers since 2023.
- Specificity of `:not/:is/:has` = that of the most specific argument; `:where` is always 0.

Others: root, target, lang, dir, fullscreen, empty

Elzero CSS #34

Less common but handy pseudo-classes.

EXAMPLE

```
:root          { --primary: #1f77c4; }      /* = html, higher specificity */
section:target { background: #fffbe6; }      /* the element whose id is in the URL #hash – CSS-
only tabs */
:lang(ar)      { font-family: Cairo; }
:dir rtl       { }
video:fullscreen { }
:focus-within  { box-shadow: 0 0 0 2px #1f77c4; } /* parent with a focused child – highlight
whole form group */
@media (hover: hover) { .btn:hover { } }      /* apply hover only on devices with a mouse */
```

::before and ::after

Gamal CSS #36

Elzero CSS #36 #37

Angela \$6

Insert a virtual child at the start or end of an element's content. They need the `content` property (even empty) to appear. Used for icons, decorative lines, quotes, counters, overlays — anything visual that does not belong in the HTML. They are inline by default.

before:: و after:: يضيفوا عنصر وهمي قبل أو بعد محتوى العنصر. لازم تكتب content عشان يظهر.

EXAMPLE

```

.price::before { content: "EGP "; color:
#888; }
a[target="_blank"]::after { content: " ↗"; }
blockquote::before { content: open-quote;
font-size: 3em; color: #ccc; }

/* decorative underline under headings */
h2 { position: relative; padding-bottom:
8px; }
h2::after {
  content: "";
  position: absolute; left: 0; bottom: 0;
  width: 60px; height: 3px; background:
#1f77c4;
}

/* dark overlay on an image card */
.card { position: relative; }
.card::after { content: ""; position:
absolute; inset: 0; background: linear-
gradient(transparent, rgba(0,0,0,.7)); }

/* attribute value as content: tooltip */
[data-tip]:hover::after { content: attr(data-
tip); position: absolute; background: #333;
color: #fff; padding: 4px 8px; }

/* automatic numbering */
h2 { counter-increment: section; }
h2::before { content: counter(section) ". "; }

```

RESULT

EGP 49 · [Docs](#) ↗ **Heading**

- The content string cannot contain HTML; it is plain text. For icons use a unicode character, an SVG background, or an icon font.
- ::before/:after do not work on replaced elements (img, input, video) because they have no content box.
- One colon (:before) still works for old browsers; two colons is the modern standard.

Try it yourself

Add a red asterisk after every label that has class 'required'.

Solution

```
label.required::after { content: " *"; color: red; }
```

Other pseudo-elements: first-letter, first-line, selection, placeholder, marker

Gamal CSS #36

Elzero CSS #35

Style a part of the element's own content.

EXAMPLE

```
p.intro::first-letter { font-size: 3em; float: left; line-height: 1; margin-right: 6px; } /* drop cap */

p::first-line { font-weight: bold; }
::selection { background: #1f77c4; color: #fff; } /* highlighted text */
input::placeholder { color: #9ca3af; font-style: italic; }
li::marker { color: #1f77c4; }
::backdrop { background: rgba(0,0,0,.6); } /* behind <dialog> */
::file-selector-button { padding: 6px 12px; border-radius: 6px; } /* the "Choose file" button */
```

RESULT

Drop caps come from `::first-letter`. `This is ::selection` styling.

? Chapter quiz

1. In what order should link states be written?
2. Which pseudo-element needs the content property?
3. `p:first-child` vs `p:first-of-type` — what is the difference?
4. How do you style a form's button when any field inside the form is invalid?

Answers: 1) `:link`, `:visited`, `:hover`, `:active`. 2) `::before` and `::after`. 3) `first-child` must be the very first child element; `first-of-type` is the first `p` among siblings. 4) `form:has(:invalid) button { ... }`

57. CSS Reference G — Transforms, Transitions and Animations

Motion makes interfaces feel alive and explains change: a menu sliding in, a button growing under the mouse, a card fading in. Transform moves/rotates/scales, transition animates a change smoothly, animation runs a timeline on its own.

transform: translate, rotate, scale, skew

Gamal CSS #39 #40

Elzero CSS #65-#68

Visually move, rotate, resize or slant an element without affecting the layout around it (neighbours do not move). Transforms are cheap for the browser, so they are the right way to animate movement.

transform يتحرك أو تلف أو تكبير أو تميل العنصر من غير ما تأثير على اللي حوالیه.

EXAMPLE

```
.a { transform: translate(20px,
-10px); } /* move right 20, up 10 */
.b { transform:
translateX(50%); } /* % = of the
element's OWN size */
.c { transform:
rotate(15deg); } /* clockwise;
negative = counter */
.d { transform:
scale(1.1); } /* 110%; scale(2,
.5) = x, y */
.e { transform: skewX(10deg); }
.f { transform: rotate(45deg) scale(.8)
translateY(-4px); } /* several: applied
right-to-left */
.g { transform-origin: top
left; } /* pivot point; default
center */
.h { transform: none; }

/* modern individual properties (do not
overwrite each other) */
.i { translate: 10px 0; rotate: 10deg; scale:
1.1; }

/* 3D */
.flip { transform: perspective(600px)
rotateY(180deg); }
```

RESULT



translate · rotate · scale · skew · combined

→ Centring trick: `top:50%; left:50%; transform: translate(-50%, -50%)` — the percentages in translate refer to the element itself.

- A transformed element becomes the containing block for fixed children and creates a stacking context.

transition

Gamal CSS #41

Elzero CSS #42

Angela \$6

When a property changes (because of `:hover`, a class added by JavaScript, etc.), a transition animates from the old value to the new one over time instead of jumping. Put the transition on the element's normal state so it animates both in and out.

Timing function	Feel
ease (default)	Slow start, fast, slow end
linear	Constant speed (progress bars, spinners)
ease-in	Slow start — things leaving
ease-out	Slow end — things arriving ✓ most UI
ease-in-out	Both
cubic-bezier(x1,y1,x2,y2)	Custom curve (cubic-bezier.com)
steps(4)	Jump in 4 steps (sprite animations)

EXAMPLE

```
.btn {
  background: #1f77c4;
  transform: translateY(0);
  transition: background .3s ease, transform
.2s ease-out;
  /*      property  duration timing
(delay optional) */
}
.btn:hover { background: #0f3d68; transform:
translateY(-3px); }

.card { transition: all .3s; } /*
every animatable property (lazy but slightly
slower) */
.menu { transition: opacity .25s, visibility
0s linear .25s; } /* hide after fade */
.menu.open { opacity: 1; visibility: visible;
transition-delay: 0s; }

/* long-hand */
transition-property: opacity;
transition-duration: 300ms;
transition-timing-function: cubic-bezier(.4,
0, .2, 1);
transition-delay: 100ms;
```

RESULT



- Good durations: 150–300ms for hover, 300–500ms for panels. Longer feels slow.

- Not everything can transition: `display`, `height: auto` (use `max-height` or `grid-template-rows: 0fr → 1fr`), `background-image`.
- Animate `transform` and `opacity` for smooth 60fps; animating width/height/top/left forces layout recalculation.

@keyframes and animation

Gamal CSS #42

Elzero CSS

An animation runs by itself (no hover needed) following a timeline you define with `@keyframes`: what the element looks like at 0%, 50%, 100%. Then attach it with the animation properties.

EXAMPLE

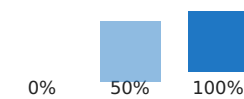
```
/* 1. define the timeline */
@keyframes fade-up {
  from { opacity: 0; transform:
  translateY(20px); } /* from = 0% */
  to { opacity: 1; transform:
  translateY(0); } /* to = 100% */
}

@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.1); }
}

/* 2. attach it */
.hero h1 {
  animation-name: fade-up;
  animation-duration: .8s;
  animation-timing-function: ease-out;
  animation-delay: .2s;
  animation-iteration-count: 1; /*
number or infinite */
  animation-direction: normal; /*
reverse | alternate | alternate-reverse */
  animation-fill-mode:
  both; /* keep first frame during
delay and last frame after end */
  animation-play-state: running; /*
paused */
}

/* shorthand: name duration timing delay count
direction fill */
.hero
h1 { animation: fade-up .8s ease-out .2s 1
normal both; }
.heart { animation: pulse 1s ease-in-out
infinite; }
.card:hover { animation-play-state: paused; }
```

RESULT



fade-up keyframes: the browser draws every frame in between.

- Without `fill-mode: both/forwards` the element jumps back to its original style when the animation ends.

- Percent keyframes can list several stops: `0%, 100% { }`.
- Respect users who get motion-sick: `@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }`.

Ready-made animations you will reuse

Gamal CSS #42

Copy these into your project.

EXAMPLE

```
/* spinner */
@keyframes spin { to { transform: rotate(360deg); } }
.spinner { width: 40px; height: 40px; border: 4px solid #eee; border-top-color: #1f77c4; border-radius: 50%; animation: spin 1s linear infinite; }

/* skeleton loading shimmer */
@keyframes shimmer { from { background-position: -200% 0; } to { background-position: 200% 0; } }
.skeleton { background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%); background-size: 200% 100%; animation: shimmer 1.5s infinite; }

/* shake on error */
@keyframes shake { 10%, 90% { transform: translateX(-2px); } 20%, 80% { transform: translateX(4px); } 30%, 50%, 70% { transform: translateX(-6px); } 40%, 60% { transform: translateX(6px); } }
.error { animation: shake .5s; }

/* bounce */
@keyframes bounce { 0%, 100% { transform: translateY(0); } 50% { transform: translateY(-12px); } }

/* typing cursor */
@keyframes blink { 50% { opacity: 0; } }
.cursor::after { content: "|"; animation: blink 1s step-end infinite; }

/* staggered list: each item a bit later */
.list li { animation: fade-up .5s both; }
.list li:nth-child(1) { animation-delay: .05s; }
.list li:nth-child(2) { animation-delay: .10s; }
.list li:nth-child(3) { animation-delay: .15s; }
```

RESULT



typing|

Try it yourself

Make a notification dot (10px red circle) that pulses forever: it grows to 1.5× and fades to 0 opacity, then restarts, over 1.2 seconds.

Solution

```
@keyframes ping { 0% { transform: scale(1); opacity: 1; } 100% { transform: scale(1.5); opacity: 0; } }
.dot { width: 10px; height: 10px; border-radius: 50%; background: red; animation: ping 1.2s ease-out infinite; }
```

Scroll-driven effects and smooth scrolling

Angela §6

Small touches that make a site feel polished: smooth anchor scrolling, snap points for carousels, and revealing elements as they scroll into view (with a few lines of JavaScript using IntersectionObserver, taught in the JS part).

EXAMPLE

```
html { scroll-behavior: smooth; }           /* anchor links glide instead of jump */
section { scroll-margin-top: 80px; }        /* leave room for the fixed navbar when jumping to #id */

/* carousel that snaps */
.carousel { display: flex; overflow-x: auto; scroll-snap-type: x mandatory; gap: 1rem; }
.carousel > * { flex: 0 0 80%; scroll-snap-align: start; }

/* reveal on scroll: JS adds .visible when the element enters the viewport */
.reveal { opacity: 0; transform: translateY(30px); transition: .6s ease-out; }
.reveal.visible { opacity: 1; transform: none; }
```

? Chapter quiz

1. Which two properties are cheapest to animate?
2. Where should the transition property be written — on the normal state or the :hover state?
3. What does animation-fill-mode: forwards do?
4. Can you transition from height: 0 to height: auto?

Answers: 1) transform and opacity. 2) On the normal state, so it animates in both directions. 3) Keeps the final keyframe's styles after the animation ends. 4) No — use max-height, or grid-template-rows 0fr→1fr, or JavaScript.

58. CSS Reference H — Flexbox, Complete

Flexbox arranges items in one direction — a row or a column — and distributes space between them. It solves centring, navbars, card rows, form layouts and most component-level layout. Set `display:flex` on the PARENT; the CHILDREN become flex items.

The mental model: container, items, two axes

Gamal CSS #45

Elzero CSS #46

Angela §10

A flex container has a **main axis** (the direction items flow: horizontal for row) and a **cross axis** (perpendicular). `justify-content` distributes items along the main axis; `align-items` aligns them on the cross axis. Change `flex-direction` to column and the axes swap.

الحاوية لديها محورين: الرئيسي (اتجاه العناصر) والعرضي. `justify-content` يوزع على الرئيسي، `align-items` يحدد على العرضي.

EXAMPLE

```
.container { display:
flex; } /* that is all – children are now in
a row */
```

RESULT

before (block):

1

2

3

after display:flex:

1

2

3

- Only direct children become flex items. Grandchildren are not affected.
- Flex items shrink to their content width by default and stretch to the container's height.

flex-direction, flex-wrap, flex-flow

Gamal CSS #45

Elzero CSS #46

Angela §10

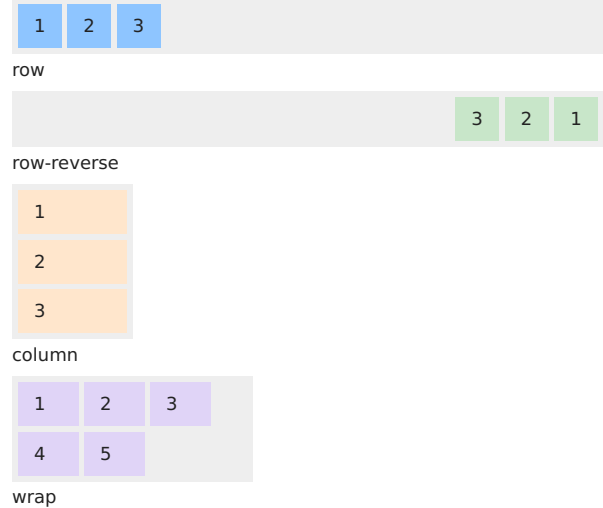
Direction sets the main axis. Wrap allows items to move to a new line when they run out of room (by default they squeeze onto one line).

EXAMPLE

```
.row { flex-direction:
row; } /* default: left-right */
.row-rev { flex-direction: row-reverse; }
.col { flex-direction:
column; } /* top-bottom; main axis is
now vertical */
.col-rev { flex-direction: column-reverse; }

.wrap { flex-wrap: wrap; } /* nowrap
(default) | wrap | wrap-reverse */
.both { flex-flow: row wrap; } /*
shorthand */
```

RESULT



⚠ Common mistake

Items overflow the container on small screens. You forgot `flex-wrap: wrap` — nowrap is the default.

justify-content (main axis)

Gamal CSS #45

Elzero CSS #47

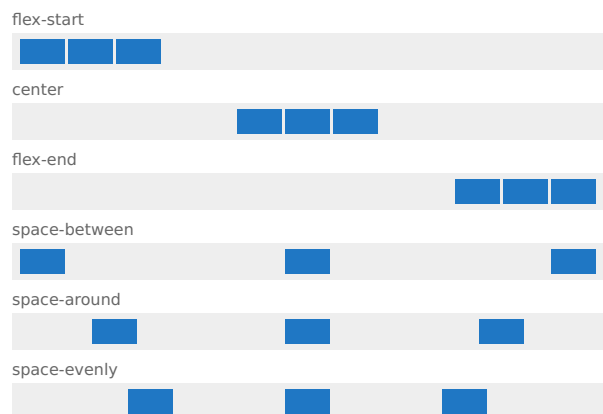
Angela \$10

How free space is distributed along the main axis (horizontally in a row). Only matters when items do not fill the line.

EXAMPLE

```
.c { justify-content: flex-start; } /*
default: packed at start */
.c { justify-content: center; }
.c { justify-content: flex-end; }
.c { justify-content: space-between; } /*
first and last touch edges, equal gaps between
*/
.c { justify-content: space-around; } /*
equal space around each item (half at the
edges) */
.c { justify-content: space-evenly; } /*
truly equal gaps everywhere */
```

RESULT



align-items (cross axis) and align-content

Gamal CSS #45

Elzero CSS #48 #49

Angela \$10

`align-items` aligns items on the cross axis (vertically in a row). `align-content` aligns the lines when items wrap onto several lines — it does nothing with a single line.

EXAMPLE

```
.c { align-items: stretch; } /* default:
items fill container height */
.c { align-items: flex-start; } /* top */
.c { align-items: center; } /* vertical
centre */
.c { align-items: flex-end; } /* bottom */
.c { align-items: baseline; } /* text
baselines line up */

.c { flex-wrap: wrap; align-content: space-
between; } /* space between the wrapped lines
*/
```

RESULT

stretch



flex-start



center



flex-end



→ Memory aid: **justify** = main axis (usually horizontal), **align** = cross axis (usually vertical).
In a column they swap.

gap

Elzero CSS #46

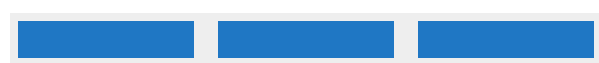
Angela §10

Space between items, without margins on the items themselves and without space at the outer edges. Works in Flexbox and Grid.

EXAMPLE

```
.c { gap:
16px; } /* rows and columns */
.c { gap: 8px 24px; } /* row-gap column-
gap */
.c { column-gap: 1rem; }
```

RESULT



Item properties: flex-grow, flex-shrink, flex-basis, flex

Gamal CSS #45

Elzero CSS #50 #51

Angela §10

These go on the children. **flex-basis** is the starting size. **flex-grow** says how much of the leftover space this item takes (0 = none). **flex-shrink** says whether it may shrink when space is short (1 = yes). The **flex** shorthand sets all three.

على الأولاد: flex-grow قد إيه ياخذ من المساحة الفاضية، flex-shrink يصغر ولا لأ، flex-basis الحجم اللي بيدأ بيه.

EXAMPLE

```
.item { flex-grow: 0; flex-shrink: 1; flex-basis: auto; } /* defaults */

.a { flex: 1; } /* = 1 1 0% → all .a items share space EQUALLY (ignores content size) */
.b { flex: 2; } /* takes twice as much extra space as flex:1 */
.c { flex: 0 0 200px; } /* fixed 200px, never grows or shrinks */
.d { flex: 1 1 250px; } /* start at 250, grow and shrink → responsive cards */
.e { flex: auto; } /* = 1 1 auto → share space but respect content size */
.f { flex: none; } /* = 0 0 auto */

img { flex-shrink: 0; } /* images: never squash */
```

RESULT

flex:1	flex:2	flex:1
0 0 70px	flex:1 fills the rest	

- `flex: 1` on every item = equal columns regardless of content. This is the answer to 'make three equal columns'.
- Sidebar + content: sidebar `flex: 0 0 250px`, content `flex: 1`.
- A long word can stop an item shrinking; add `min-width: 0` to the item.

align-self and order

Gamal CSS #45

Elzero CSS #52

Angela \$10

Per-item overrides. `align-self` aligns one item differently from the rest. `order` changes the visual order without touching the HTML (lower numbers first; default 0).

EXAMPLE

```
.item-2 { align-self: flex-end; } /* auto | flex-start | center | flex-end | stretch | baseline */
.logo { order: -1; } /* move to the front */
.cta { order: 1; } /* move to the end */
.item { margin-left: auto; } /* push THIS item (and those after) to the far right very useful */
```

RESULT

1		3	auto
	2		

item 2 has `align-self: flex-end`; the orange item has `margin-left: auto`.

- `margin: auto` on a flex item absorbs free space — the easiest way to push a login button to the right end of a navbar.
- Order changes only the visual order; screen readers and Tab key follow the HTML order. Do not use it for meaningful reordering.

The patterns you will use every day

Gamal CSS #45

Elzero CSS #53

Angela \$10

Memorise these five. They cover 80% of layout problems.

EXAMPLE

```
/* 1. centre anything horizontally AND vertically */
.center { display: flex; justify-content: center; align-items: center; min-height: 100vh; }

/* 2. navbar: logo left, links right, all vertically centred */
.nav { display: flex; align-items: center; gap: 2rem; padding: 1rem; }
.nav .links { display: flex; gap: 1rem; margin-left: auto; }

/* 3. responsive card row: as many per line as fit, min 250px each */
.cards { display: flex; flex-wrap: wrap; gap: 1.5rem; }
.card { flex: 1 1 250px; }

/* 4. sticky footer: footer at the bottom even on short pages */
body { min-height: 100vh; display: flex; flex-direction: column; }
main { flex: 1; }

/* 5. media object: image beside text */
.media { display: flex; gap: 1rem; align-items: flex-start; }
.media img { flex: 0 0 80px; }
.media .body { flex: 1; }
```

RESULT



Try it yourself

Build a pricing row: three cards side by side, equal width, 20px apart, the middle one slightly bigger (scale 1.05). On screens narrower than 700px they should stack vertically.

Solution

```
.pricing { display: flex; gap: 20px; align-items: center; }
.plan    { flex: 1; padding: 2rem; border: 1px solid #ddd; border-radius: 12px; }
.plan.featured { transform: scale(1.05); border-color: #1f77c4; }
@media (max-width: 700px) { .pricing { flex-direction: column; } .plan.featured { transform: none; } }
```

Practice game

Elzero CSS #53

Play **Flexbox Froggy** (flexboxfroggy.com) — 24 levels that drill every property. Elzero solves it on video (CSS #53). Do not move to Grid until you finish it.

? Chapter quiz

1. Which property aligns items along the main axis?
2. What does flex: 1 expand to?
3. How do you push one navbar item to the far right?
4. Items overflow instead of wrapping — what is missing?

Answers: 1) justify-content. 2) flex-grow 1, flex-shrink 1, flex-basis 0%. 3) margin-left: auto on that item. 4) flex-wrap: wrap.

59. CSS Reference I — Grid, Complete

Grid lays out in two dimensions at once: rows AND columns. It is the tool for page layouts, dashboards, galleries and any design where things must line up in both directions. Flexbox = components; Grid = the page.

Vocabulary: lines, tracks, cells, areas, gaps

Gamal CSS #46

Elzero CSS #57

Angela §11

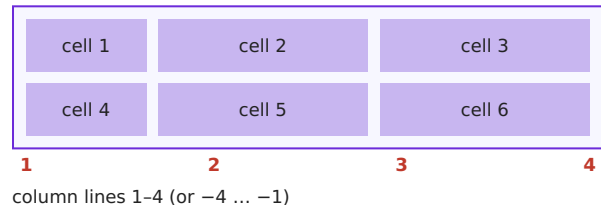
A grid is defined by numbered **lines** (starting at 1, and -1 from the end). The space between two lines is a **track** (a column or a row). One row $\times$ one column is a **cell**. A rectangle of cells is an **area**. The **gap** is the gutter between tracks.

الجريد شبكة من خطوط مرقمة. المسافة بين خطين = track (عمود أو صف). خلية = تقاطع صف وعمود. = area. مستطيل من خلايا.

EXAMPLE

```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr
  1fr; /* 3 columns → 4 vertical lines */
  grid-template-rows: 80px
  auto; /* 2 rows → 3 horizontal lines */
  gap: 16px;
}
```

RESULT



grid-template-columns and rows; fr, repeat, minmax

Gamal CSS #46

Elzero CSS #57 #58 #63

Angela §11

Define the tracks. Use any length, `auto`, or `fr` (a fraction of the free space — the Grid super-unit). `repeat()` avoids repetition; `minmax()` gives a track a floor and a ceiling.

auto-fit vs auto-fill	Behaviour when there are few items
auto-fit	Empty tracks collapse; items stretch to fill the row
auto-fill	Empty tracks are kept; items stay at min size, leaving space

EXAMPLE

```

.g1 { grid-template-columns: 1fr 1fr 1fr; }           /* three equal columns */
.g2 { grid-template-columns: repeat(3, 1fr); }        /* same */
.g3 { grid-template-columns: 2fr 1fr; }              /* 2/3 and 1/3 */
.g4 { grid-template-columns: 250px 1fr; }            /* fixed sidebar + fluid content */
.g5 { grid-template-columns: repeat(4, minmax(0, 1fr)); } /* equal even with long content */
.g6 { grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); } /* + responsive, no media query */
.g7 { grid-template-columns: repeat(auto-fill, 120px); } /* as many 120px columns as fit, leaves space */
.g8 { grid-template-columns: [full-start] 1fr [content-start] minmax(0, 1100px) [content-end] 1fr [full-end]; } /* named lines */

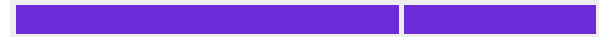
.rows { grid-template-rows: 60px 1fr 40px; }         /* header, content, footer */
.auto { grid-auto-rows: minmax(120px, auto); }       /* rows created automatically get this size */
.flow { grid-auto-flow: column; }                   /* new items go into new COLUMNS instead of rows; "dense" fills holes */

```

RESULT



repeat(3, 1fr)



2fr 1fr



50px 1fr



auto-fit minmax(50px,1fr)

- `1fr` is really `minmax(auto, 1fr)` — long unbreakable content can make a column wider. Use `minmax(0, 1fr)` for strictly equal columns.
- Rows are usually left to `auto` so they grow with content.

gap and alignment in Grid

Gamal CSS #46

Elzero CSS #59

Angela §11

`gap` works as in Flexbox. Alignment has two levels: items inside their cells (`justify-items` / `align-items`) and the whole grid inside its container when the grid is smaller (`justify-content` / `align-content`).

EXAMPLE

```
.g { gap: 24px
16px; } /* row-gap column-gap */

/* each item inside its cell */
.g { justify-items: start; } /*
horizontal: start | center | end | stretch
(default) */
.g { align-items: center; } /*
vertical */
.g { place-items: center; } /* both –
the famous 2-line centring: display:grid;
place-items:center */

/* the whole grid inside the container (when
tracks are fixed size and there is spare room)
*/
.g { justify-content: space-between; }
.g { align-content: center; }
.g { place-content: center; }

/* per item */
.item { justify-self: end; align-self: start;
place-self: center; }
```

RESULT



place-items: start · center · end (item inside its cell)

Placing items by line numbers

[Gamal CSS #46](#)
[Elzero CSS #61](#)
[Angela \\$11](#)

By default items fill cells in order. To place one deliberately, give it a start and end line for columns and/or rows. `span n` means 'cover n tracks'. Negative numbers count from the end.

EXAMPLE

```
.header { grid-column: 1 / 4; } /*
from line 1 to line 4 = 3 columns */
.header { grid-column: 1 / -1; } /*
full width no matter how many columns */
.header { grid-column: 1 / span 3; } /*
same as 1 / 4 */
.sidebar { grid-column: 1; grid-row: 2 /
4; } /* column 1, rows 2–3 */
.feature { grid-column: span 2; grid-row: span
2; } /* a 2x2 tile wherever it lands */

/* long-hand */
grid-column-start: 2; grid-column-end: 4;
grid-row-start: 1; grid-row-end: 3;
/* one property for all four: row-start / col-
start / row-end / col-end */
.item { grid-area: 1 / 2 / 3 / 4; }
```

RESULT



⚠ Common mistake

`grid-column: 1 / 3` spans TWO columns (from line 1 to line 3), not three. Think lines, not columns.

grid-template-areas — draw the layout

Gamal CSS #46

Elzero CSS #60 #62

Angela \$11

Name regions and draw the layout as text: each string is a row, each word a column. Then assign each element to a name. Rearranging the page for mobile means redrawing the picture — the most readable way to build page layouts.

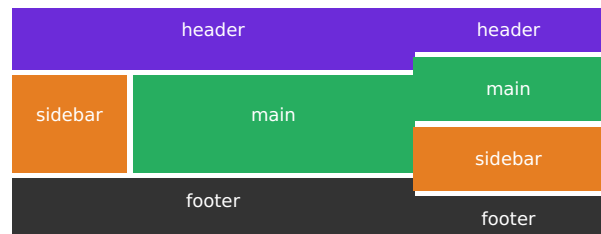
بتسمي المناطق وترسم التصميم كنص: كل سطر = صف، كل كلمة = عمود. للموبايل ترسم الصورة ثاني.

EXAMPLE

```
.page {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
  gap: 1rem;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}
header { grid-area: header; }
aside { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }

/* mobile: one column, sidebar after main */
@media (max-width: 768px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "main"
      "sidebar"
      "footer";
  }
}
/* a dot means an empty cell: "header ." */
```

RESULT



desktop → mobile by redrawing the areas

- Every row string must have the same number of words. Areas must be rectangles.
- Line names are created automatically: `header-start`, `header-end`.

Try it yourself

Draw a dashboard: a top bar across the full width, a narrow nav on the left spanning two rows, a large 'stats' panel, and beside it a 'chart' panel; below both, an 'activity' panel spanning the two right columns.

Solution

```
.dash {
  display: grid;
  grid-template-columns: 200px 1fr 1fr;
  grid-template-areas:
    "topbar topbar topbar"
    "nav stats chart"
    "nav activity activity";
}
```

Implicit grid, auto-placement and dense packing

Elzero CSS #63

When there are more items than cells, Grid creates extra rows automatically (the implicit grid) sized by `grid-auto-rows`. `grid-auto-flow: dense` lets later small items fill holes left by big ones — great for masonry-like galleries.

EXAMPLE

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill,
minmax(150px, 1fr));
  grid-auto-rows: 150px;
  grid-auto-flow: dense;
  gap: 8px;
}
.gallery .wide { grid-column: span 2; }
.gallery .tall { grid-row: span 2; }
.gallery img { width: 100%; height: 100%;
object-fit: cover; }
```

RESULT



Subgrid and nesting

Angela §11

A grid item can itself be a grid (nesting). With `subgrid`, the inner grid uses the parent's tracks so nested content lines up across cards (e.g., all card titles at the same height).

EXAMPLE

```
.cards { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
.card { display: grid; grid-template-rows: subgrid; grid-row: span 3; } /* image, title, text
rows align across cards */
```

Flexbox or Grid? and the practice game

Gamal CSS #46

Elzero CSS #64

Angela §11

Use both. Ask: is this one line of things (Flex) or a two-dimensional layout where rows and columns must align (Grid)?

Use Flexbox for	Use Grid for
Navbar, toolbar, button group	Whole page (header/sidebar/main/footer)
Centring one thing	Photo gallery, product grid
Items whose count is unknown and should flow	Dashboard with panels of different sizes
Content-sized items in a row	Forms with aligned labels and fields
Small components inside a Grid cell	Anything you can draw on paper as a table

→ Play **Grid Garden** (cssgridgarden.com); Elzero finishes it in CSS #64. Then rebuild a real site's layout (a news site, YouTube) with Grid + Flex.

? Chapter quiz

1. What does 1fr mean?
2. Write a column template that makes as many 200px-minimum columns as fit.
3. grid-column: 2 / 4 covers how many columns?
4. Which property lets you draw the layout with names?

Answers: 1) One fraction of the free space after fixed tracks are placed. 2) repeat(auto-fit, minmax(200px, 1fr)). 3) Two (lines 2 to 4). 4) grid-template-areas.

60. CSS Reference J — Responsive Design and Media Queries

Responsive design means one page that works on a 320px phone and a 2560px monitor. It is a mindset (fluid sizes, flexible layouts) plus one tool: media queries, which apply CSS only when a condition about the device is true.

The viewport meta tag — nothing works without it

Gamal CSS #47

Elzero HTML

Angela §9

Phones pretend to be 980px wide and zoom out unless you tell them to use their real width. This tag does that. It goes in the head of every page.

EXAMPLE

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width`: page width = screen width. `initial-scale=1`: no zoom.
- Never add `user-scalable=no` — it blocks pinch-zoom, which people with low vision need.

Mobile-first thinking

Gamal CSS #47

Angela §9

Write your base CSS for the smallest screen (one column, big touch targets), then add media queries with `min-width` that enhance the layout as the screen grows. It is easier to add columns than to remove them, and phones download less CSS.

ابدأ بتصميم الموبايل (عمود واحد)، وبعدئذٍ زود قواعد للشاشات الأكبر بـ `min-width`.

EXAMPLE

```

/* base = phone */
.cards { display: grid; grid-template-columns:
1fr; gap: 1rem; }
.nav-links { display: none; }
.burger { display: block; }

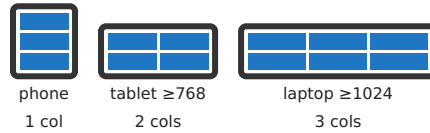
/* tablet and up */
@media (min-width: 768px) {
  .cards { grid-template-columns: repeat(2,
1fr); }
}

/* laptop and up */
@media (min-width: 1024px) {
  .cards { grid-template-columns: repeat(3,
1fr); }
  .nav-links { display: flex; }
  .burger { display: none; }
}

/* large desktop */
@media (min-width: 1280px) {
  .container { max-width: 1200px; }
}

```

RESULT



Media query syntax and features

Gamal CSS #47

Elzero CSS

Angela §9

A media query wraps rules that apply only when the condition is true. Conditions can be combined with `and`, listed with commas (or), and negated with `not`.

Common breakpoints (Bootstrap)	min-width	Device
sm	576px	large phones
md	768px	tablets
lg	992px	laptops
xl	1200px	desktops
xxl	1400px	large desktops

EXAMPLE

```

@media (min-width: 768px) { ... }           /* 768px and wider */
@media (max-width: 767px) { ... }          /* 767px and narrower */
@media (min-width: 768px) and (max-width: 1023px) { ... } /* only tablets */
@media (width >= 768px) { ... }            /* new range syntax, same meaning */
@media (768px <= width < 1024px) { ... }
@media screen and (orientation: landscape) { ... }
@media print { nav, footer, .ads { display: none; } body { font-size: 12pt; color: #000; } }
@media (prefers-color-scheme: dark) { :root { --bg: #111; --text: #eee; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }
@media (hover: hover) and (pointer: fine) { .card:hover { transform: translateY(-4px); } } /* mouse devices only */
@media (min-resolution: 2dppx) { .logo { background-image: url(logo@2x.png); } } /* retina */

/* also as a link attribute — loads only when matching */
<link rel="stylesheet" href="print.css" media="print">

```

- Do not design for devices; design for your content. Add a breakpoint where the layout starts to look wrong when you drag the window narrower.
- Put media queries at the end of the file, or right after the component they modify — be consistent.
- Media queries do not add specificity; a rule inside one still loses to a more specific rule outside it.

Fluid techniques that need no media query

Gamal CSS #47

Angela §9

Modern CSS can adapt on its own. Reach for these before writing a breakpoint.

EXAMPLE

```

/* fluid container */
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }

/* fluid typography */
h1 { font-size: clamp(1.75rem, 1rem + 3vw, 3.5rem); }
body { font-size: clamp(1rem, .95rem + .3vw, 1.125rem); }

/* fluid spacing */
section { padding-block: clamp(2rem, 6vw, 6rem); }

/* auto-wrapping grids */
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(250px, 100%), 1fr));
gap: 1rem; }

/* wrapping flex rows */
.row { display: flex; flex-wrap: wrap; gap: 1rem; }
.row > * { flex: 1 1 200px; }

/* responsive images and video */
img, video { max-width: 100%; height: auto; }
.video-wrap { aspect-ratio: 16 / 9; } .video-wrap iframe { width: 100%; height: 100%; }

/* readable line length */
p { max-width: 65ch; }

```

- `min(250px, 100%)` inside `minmax` prevents overflow on screens narrower than 250px.
- `clamp(min, preferred, max)`: the middle value usually mixes `rem` and `vw` so it scales but never becomes unreadable.

Container queries

Angela §9

Media queries look at the screen; container queries look at the size of the element's parent. A card can switch to a horizontal layout when its container is wide, regardless of the screen — perfect for reusable components.

EXAMPLE

```

.card-wrapper { container-type: inline-size; container-name: card; }

.card { display: grid; gap: 1rem; }
@container card (min-width: 400px) {
  .card { grid-template-columns: 150px 1fr; } /* image beside text */
}

/* container units: 1cqi = 1% of the container inline size */
.card h3 { font-size: clamp(1rem, 4cqi, 1.5rem); }

```

Responsive navigation (hamburger) — CSS-only version

Gamal CSS #47

Angela §9

A menu that is a horizontal bar on desktop and a hidden panel opened by a button on mobile. This version uses a hidden checkbox so it needs no JavaScript; the JS part of the book shows the class-toggle version.

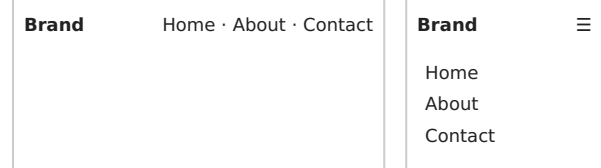
EXAMPLE

```
<nav class="nav">
  <a class="logo" href="/">Brand</a>
  <input type="checkbox" id="menu-toggle"
hidden>
  <label for="menu-toggle" class="burger"
aria-label="Menu">≡</label>
  <ul class="links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>

<style>
.nav { display: flex; align-items: center;
justify-content: space-between; padding: 1rem;
position: relative; }
.links { list-style: none; margin: 0; padding:
0; display: flex; gap: 1.5rem; }
.burger { display: none; font-size: 1.5rem;
cursor: pointer; }

@media (max-width: 767px) {
  .burger { display: block; }
  .links { display: none; position: absolute;
top: 100%; left: 0; right: 0;
flex-direction: column; background:
#fff; padding: 1rem; box-shadow: 0 8px 16px
rgba(0,0,0,.1); }
  #menu-toggle:checked ~ .links { display:
flex; } /* checkbox checked → show the
sibling list */
}
</style>
```

RESULT



Responsive form and table (Gamal's examples)

Gamal CSS #47 responsive form

Angela §9

Forms go from one column on phones to two on wider screens; tables scroll sideways or turn into cards.

EXAMPLE

```

/* form */
.form { display: grid; gap: 1rem; }
.form input, .form
textarea { width: 100%; padding: .75rem; font: inherit; border: 1px solid #ccc; border-radius:
8px; }
@media (min-width: 640px) {
  .form { grid-template-columns: 1fr 1fr; }
  .form .full { grid-column: 1 / -1; } /* textarea and button span both */
}

/* table → cards on mobile */
@media (max-width: 640px) {
  table, thead, tbody, tr, th, td { display: block; }
  thead { display: none; }
  tr { margin-bottom: 1rem; border: 1px solid #ddd; border-radius: 8px; padding: .5rem; }
  td { display: flex; justify-content: space-between; padding: .4rem 0; }
  td::before { content: attr(data-label); font-weight:
bold; } /* <td data-label="Price">40</td> */
}

```

Testing responsiveness

Gamal CSS #47

Angela \$9

Open DevTools (F12) and click the device toolbar icon (or press Ctrl+Shift+M). Choose a phone preset or drag the edge to any width. Check: nothing overflows horizontally, text is at least 16px, buttons are at least 44×44px, images scale, the menu works.

Check	How
Horizontal scrollbar on mobile?	Something is wider than the screen — find it with <code>* { outline: 1px solid red }</code> or DevTools
Text too small?	Body $\geq$ 16px; use rem
Touch targets	$\geq$ 44px tall; add padding to links
Images	max-width: 100%
Real device	Open your Live Server URL from your phone on the same Wi-Fi (e.g. 192.168.1.5:5500)

? Chapter quiz

1. Which tag is required for responsive design to work on phones?
2. Mobile-first uses min-width or max-width?
3. Write a fluid heading size between 1.5rem and 3rem.
4. What is the difference between a media query and a container query?

Answers: 1) The viewport meta tag. 2) min-width. 3) font-size: clamp(1.5rem, 1rem + 2.5vw, 3rem). 4) Media queries respond to the screen; container queries respond to the parent element's size.

61. JavaScript Basics: Variables, Types, Operators

Angela §14-16

JavaScript is the programming language of the browser (and, with Node, of the server). HTML is nouns, CSS is adjectives, JS is **verbs** — it makes things happen.

```
<!-- connect JS: put before </body> or in <head> with defer -->
<script src="js/main.js" defer></script>
<!-- Test in browser: F12 → Console tab -->
```

KEYWORD (declare)

ASSIGN (not "equals")

let **userName** = "Ahmed" ;

NAME (camelCase) VALUE (a string)

const = can't be reassigned □ default

let = can change later

var = old, avoid (function-scoped)

```
// ---- DATA TYPES ----
const name = "Ahmed";           // string (" " ' ' or ` ` template)
let age = 25;                   // number (no int/float distinction)
let isStudent = true;          // boolean
let nothing = null;            // intentionally empty
let notSet;                    // undefined (declared, no value)
const skills = ["html", "css"]; // array (list)
const user = { name: "Ahmed", age: 25 }; // object (key: value)
console.log(typeof age);       // "number"

// ---- STRINGS ----
const msg = `Hello ${name}, you are ${age}`; // template literal: backticks + ${}
name.length; name.toUpperCase(); name.slice(0, 3); name.includes("Ah");
"a" + "b";           // "ab" concatenation
"5" + 2;             // "52" string wins with +
"5" * 2;             // 10 (converted)
Number("42"); String(42); parseInt("42px"); parseFloat("3.5");

// ---- OPERATORS ----
10 % 3; // 1 remainder (modulo) – even check: n % 2 === 0
2 ** 3; // 8 power
age++; age += 5; age -= 1;
5 == "5"; // true – loose, converts types avoid
5 === "5"; // false – strict, checks type too □ always use === and !==
&& (and) || (or) ! (not)
const label = age >= 18 ? "adult" : "minor"; // ternary: condition ? yes : no
const city = user.city ?? "Giza"; // nullish default
Math.random(); Math.floor(4.7); Math.round(); Math.max(1, 9); // random 1–6:
Math.floor(Math.random()*6)+1
```

62. Functions, Conditions, Loops

Angela §15-16

keyword ↓ name ↓ PARAMETERS (placeholders) ↓ CALL ↓ ARGUMENTS (real values) ↓

```
function add(a, b) {           add(2, 3); // → 5
  return a + b;
}
```

return sends a value OUT and stops the function. No return → unde

```
// 3 ways to write a function
function greet(name = "friend") { return `Hi ${name}`; } // declaration (hoisted) + default
param
const greet2 = function(name) { return `Hi ${name}`; }; // expression
const greet3 = (name) => `Hi ${name}`; // arrow: implicit return when one
expression □ modern
const square = n => n * n; // one param → no parentheses needed
const log = () => { console.log("hi"); }; // no params → ()

// ---- CONDITIONS ----
if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else {
  grade = "C";
}
switch (day) {
  case "sat": case "sun": msg = "weekend"; break; // break or it falls through!
  default: msg = "work";
}
// falsy values: false, 0, "", null, undefined, NaN – everything else is truthy
if (user.name) { ... } // runs only if name is non-empty

// ---- LOOPS ----
for (let i = 0; i < 5; i++) { console.log(i); } // start; keep-going-while; step → 0 1 2 3 4
while (lives > 0) { lives--; } // unknown count – make sure it ends!
do { ... } while (cond); // runs at least once
for (const skill of skills) { } // each VALUE of an array □
for (const key in user) { user[key] } // each KEY of an object
skills.forEach((s, i) => console.log(i, s)); // array method
break; // exit loop      continue; // skip to next iteration
```

63. Arrays & Objects

Angela \$16, \$22

```
// ARRAY – ordered list, index starts at 0
const fruits = ["apple", "mango", "fig"];
fruits[0];           // "apple"
fruits.length;       // 3
fruits[fruits.length - 1]; // last → "fig" or fruits.at(-1)
// change (mutate)
fruits.push("kiwi"); // add to end
fruits.pop();         // remove last
fruits.unshift("x"); // add to start
fruits.shift();       // remove first
fruits.splice(1, 1);  // remove 1 item at index 1
fruits.sort(); fruits.reverse();
// find
fruits.indexOf("fig"); // 2 or -1
fruits.includes("fig"); // true
fruits.find(f => f.startsWith("m")); // "mango"
// transform (return NEW arrays) +
const nums = [1, 2, 3, 4];
nums.map(n => n * 2); // [2,4,6,8]
same length, changed
nums.filter(n => n % 2 === 0); // [2,4]
keep those passing test
nums.reduce((sum, n) => sum + n, 0); // 10
combine to one value
nums.some(n => n > 3); nums.every(n => n > 0);
fruits.join(", "); "a,b".split(",");
[...nums, 5]; // spread: copy + add
const [first, second] = fruits; // destructuring
```

```
// OBJECT – key/value pairs (like a dictionary)
const user = {
  name: "Ahmed",
  age: 25,
  skills: ["html", "css"],
  address: { city: "Giza" }, // nested
  greet() { return `Hi, I'm ${this.name}`; } // method
};
user.name; // dot notation
user["name"]; // bracket (dynamic key)
user.address.city; // "Giza"
user.email = "a@b.c"; // add property
delete user.age;
user.greet(); // "Hi, I'm Ahmed"
Object.keys(user); // ["name", ...]
Object.values(user); Object.entries(user);
const { name, age } = user; // destructuring
const copy = { ...user, age: 26 }; // spread
copy + override
user?.address?.zip; // optional chaining: no crash if missing

// ARRAY OF OBJECTS – the most common data shape (from APIs)
const products = [
  { id: 1, title: "Bread", price: 5 },
  { id: 2, title: "Cake", price: 40 },
];
products.filter(p => p.price > 10).map(p => p.title); // ["Cake"]
const total = products.reduce((s, p) => s + p.price, 0);

// JSON – text format for sending objects
const text =
  JSON.stringify(user); // object → string
const obj =
  JSON.parse(text); // string → object
localStorage.setItem("user", text); // save in browser
JSON.parse(localStorage.getItem("user"));
```

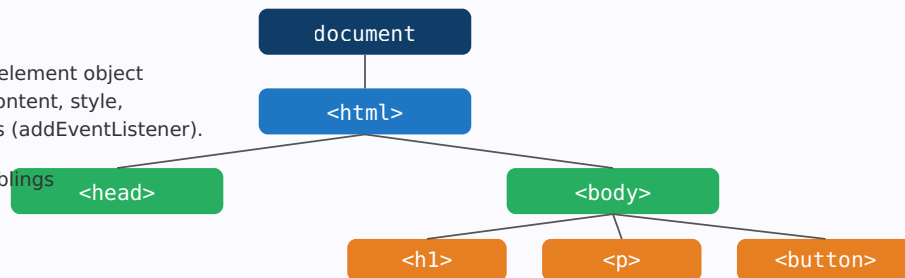
64. DOM Manipulation & Events

Angela §17-19

The **DOM** (Document Object Model) is the browser's live tree of your HTML. JS reads and changes it through the `document` object.

Every tag = a NODE / element object with properties (textContent, style, classList) and methods (addEventListener).

parent ↔ children ↔ siblings



```

// 1. SELECT
const title = document.querySelector("h1");           // first match, any CSS selector []
const btn   = document.querySelector("#save");
const items = document.querySelectorAll(".item");     // NodeList of all matches (use forEach)
document.getElementById("save"); document.getElementsByClassName("item"); // older

// 2. READ / CHANGE CONTENT
title.textContent = "New title";                       // plain text [] safe
title.innerHTML = "<em>New</em>";                     // parses HTML never with user input (XSS)
input.value;                                           // what user typed in an input
img.src = "new.jpg"; link.href; img.getAttribute("alt"); el.setAttribute("data-id", 5);
el.dataset.id;

// 3. CHANGE STYLE – prefer toggling classes
title.style.color = "red";                             // inline; CSS property names become camelCase:
background-color
title.classList.add("active");
title.classList.remove("hidden");
title.classList.toggle("dark");                       // add if missing, remove if present
title.classList.contains("active");

// 4. CREATE / REMOVE
const li = document.createElement("li");
li.textContent = "New task";
list.appendChild(li); // or list.append(li) / list.prepend(li)
li.remove();
list.insertAdjacentHTML("beforeend", `<li>${task}</li>`);
// navigate: el.parentElement el.children el.nextElementSibling el.closest(".card")
  
```


Events — react to the user

TARGET element

EVENT TYPE

HANDLER / callback — runs each time

```
btn.addEventListener( "click", (event) => { ... } );
```

event object: event.target (what was clicked), event.key, event.preventDefault() (stop form submit / link)

```
const form = document.querySelector("#todo-form");
form.addEventListener("submit", (e) => {
  e.preventDefault(); // stop page reload
  const text = form.querySelector("input").value.trim();
  if (!text) return;
  list.insertAdjacentHTML("beforeend", `<li>${text}</li> <button class="del">x</button></li>`);
  form.reset();
});
// event DELEGATION: one listener on the parent handles future children too +
list.addEventListener("click", (e) => {
  if (e.target.classList.contains("del")) e.target.closest("li").remove();
});
document.addEventListener("keydown", (e) => { if (e.key === "Escape") closeModal(); });
// common events: click dblclick mouseover mouseout input change submit keydown keyup load scroll
// resize focus blur
setTimeout(() => alert("hi"), 1000); // run once after 1s
const id = setInterval(tick, 1000); clearInterval(id); // repeat every 1s
```

Angela's projects here: Dicee (random images), Drum Kit (keydown + play audio + class animation with setTimeout), Simon Game (arrays, sequences). Build all three.

65. ES6+, Async, fetch & JSON

Angela \$22, \$30

Why async? JavaScript runs one thing at a time



Output order: 1, 3, 2. A Promise = "I'll give you the value later: pending → fulfilled (then) or rejected (catch)".

```

// PROMISE syntax
fetch("https://api.example.com/users")
  .then(res => res.json())           // parse body as JSON (also async)
  .then(data => render(data))
  .catch(err => console.error(err))
  .finally(() => hideSpinner());

// ASYNC / AWAIT - same thing, reads like normal code ☐
async function loadUsers() {
  try {
    const res = await fetch("https://api.example.com/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const users = await res.json();
    users.forEach(u => list.insertAdjacentHTML("beforeend", `<li>${u.name}</li>`));
  } catch (err) {
    console.error("Failed:", err.message);
  }
}
loadUsers();

// POST - send data
await fetch("/api/todos", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Learn JS", done: false }),
});

// Weather app example (Angela-style) - OpenWeather API
const url = `https://api.openweathermap.org/data/2.5/weather?q=Giza&appid=${KEY}&units=metric`;
const data = await (await fetch(url)).json();
temp.textContent = `${data.main.temp}°C`; // read the JSON structure from the API docs
  
```

ES6+ features summary

Feature	Example
let / const, arrow functions, template literals	covered above
Destructuring	<code>const {name, age} = user; const [a, b] = arr;</code>
Spread / rest	<code>[...a, ...b] function f(...args)</code>
Default params	<code>function f(x = 10)</code>
Modules	<code>export const x = 1; / import { x } from "./file.js";</code> — needs <code><script type="module"></code>
Classes	<code>class Dog { constructor(n){ this.name = n } bark(){ } }</code> <code>new Dog("Rex")</code>
Optional chaining / nullish	<code>a?.b?.c x ?? "default"</code>
Map / Set	<code>new Set([1,1,2])</code> → unique values

66. jQuery — Quick Reference

Angela §20

jQuery is an older library that shortened DOM code. Modern JS does everything it did; you'll still see it in old projects and Angela teaches it briefly. Include: `<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>`

Task	jQuery	Vanilla JS equivalent
Select	<code>\$("#h1")</code> <code>\$(".btn")</code>	<code>document.querySelector</code> <code>querySelectorAll</code>
Text / HTML	<code>\$("#h1").text("Hi")</code> <code>.html()</code>	<code>el.textContent = "Hi"</code> <code>innerHTML</code>
CSS	<code>\$("#h1").css("color", "red")</code>	<code>el.style.color = "red"</code>
Classes	<code>.addClass("x")</code> <code>.removeClass()</code> <code>.toggleClass()</code>	<code>el.classList.add/remove</code> <code>toggle</code>
Attributes	<code>\$("#a").attr("href", "x")</code>	<code>el.setAttribute("href", "x")</code>
Events	<code>\$("#button").click(fn)</code> <code>.on("keydown", fn)</code>	<code>el.addEventListener("keydown", fn)</code>
Add / remove	<code>.append("")</code> <code>.prepend()</code> <code>.remove()</code>	<code>insertAdjacentHTML</code> / <code>remove</code>
Effects	<code>.hide()</code> <code>.show()</code> <code>.toggle()</code> <code>.fadeIn()</code> <code>.slideUp()</code> <code>.animate({opacity:.5})</code>	CSS transitions + <code>classList</code>
Ready	<code>\$(document).ready(fn)</code>	<code><script defer></code> or <code>DOMContentLoaded</code>

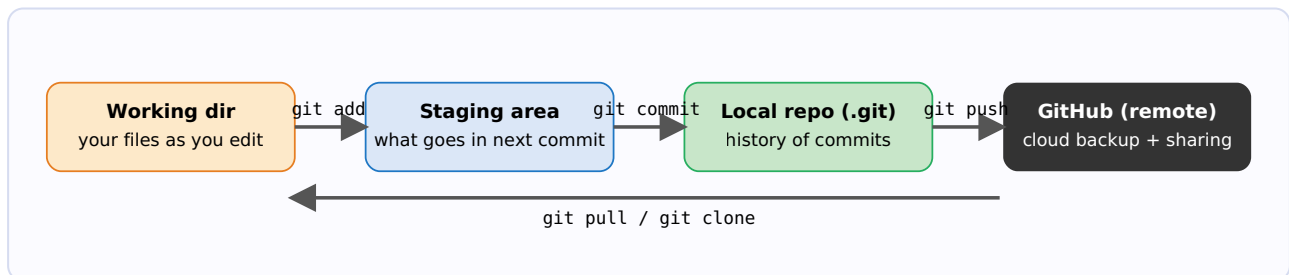
67. Command Line, Git & GitHub

Angela §21, §23

Terminal basics (bash / Git Bash on Windows)

Command	Meaning
<code>pwd</code>	print working directory (where am I?)
<code>ls</code> (<code>ls -la</code>)	list files (all + details)
<code>cd folder</code> <code>cd ..</code> <code>cd ~</code>	change directory / go up / go home
<code>mkdir name</code>	make directory
<code>touch index.html</code>	create empty file
<code>rm file</code> <code>rm -rf folder</code>	delete (⚠ no recycle bin)
<code>cp a b</code> <code>mv a b</code>	copy / move-rename
<code>cat file</code>	print file contents
<code>code .</code>	open current folder in VS Code
<code>Tab</code> / <code>↑</code> / <code>Ctrl+C</code>	autocomplete / previous command / stop running process

Git — the 4 areas



```

# one-time setup
git config --global user.name "Ahmed"
git config --global user.email "ahmed@mail.com"

# start tracking a project
git init                # create .git in this folder
git status              # what changed? (use constantly)
git add index.html      # stage one file      git add . → stage everything
git commit -m "Add homepage" # save a snapshot with a message
git log --oneline       # history
git diff                # see unstaged changes

# connect to GitHub (create empty repo on github.com first)
git remote add origin https://github.com/USER/REPO.git
git branch -M main
git push -u origin main    # first push; afterwards just: git push

# daily loop: edit → git add . → git commit -m "msg" → git push

# branches: work on a feature without breaking main
git checkout -b feature/navbar # create + switch
git checkout main              # switch back
git merge feature/navbar      # bring changes into main
git branch -d feature/navbar

# get someone's project / update
git clone https://github.com/USER/REPO.git
git pull

# undo
git checkout -- file.css      # discard changes in a file
git reset --soft HEAD~1      # undo last commit, keep changes
git revert <hash>             # safe undo of a pushed commit

```

.gitignore — a file listing what NOT to track: `node_modules/`, `.env`, `.DS_Store`. Never push secrets (API keys, passwords).

GitHub Pages: repo → Settings → Pages → Branch main → your site is live at `user.github.io/repo` (static sites only).

68. JavaScript Reference A — First Steps, Output, Data Types, Variables

JavaScript is the programming language of the web. It runs in every browser and, with Node.js, on servers. Unlike HTML and CSS, it is a real programming language: it stores values, makes decisions, repeats work, and reacts to the user. This chapter starts from absolute zero.

What JavaScript is and where the code goes

Elzero JS #001-#005

Gamal JS lecture 1

Angela §14

JavaScript (JS) adds behaviour: when the user clicks, typing validation, loading data without refresh, animations, games. The code lives in a .js file connected with a script tag, or between script tags. The browser reads it top to bottom and executes each statement.

الجافاسكريبت لغة برمجة حقيقية بتخلي الصفحة تتفاعل: لما المستخدم يضغط، تتحقق من البيانات، تجيب داتا من غير تحديث.

EXAMPLE

```
<!-- index.html -->
<body>
  <h1 id="title">Hello</h1>
  <button id="btn">Click me</button>

  <!-- 1. external file, at the end of body so the HTML above exists -->
  <script src="js/main.js"></script>

  <!-- 2. or inline -->
  <script>
    console.log("Page loaded");
  </script>
</body>
```

- Statements end with a semicolon `;`. JavaScript can guess them, but always write them.
- JavaScript is case sensitive: `name` and `Name` are different.
- Comments: `// one line` and `/* many lines */`.
- Open the Console: F12 → Console. Every example in this chapter can be typed straight into it.

Output: console.log, alert, document.write, innerHTML

Elzero JS #007 #008

Angela §14

Ways to see a result. `console.log` prints to the DevTools console — you will use it hundreds of times a day for debugging. `alert` pops up a box (annoying; only for quick tests). Writing into the page is done through the DOM (a later chapter).

EXAMPLE

```
console.log("Hello, world");
class="c">// prints text
console.log(5 + 3);
class="c">// 8
console.log("Sum:", 5 + 3);
class="c">// several values: Sum: 8
console.error("Something failed");
class="c">// red
console.warn("Careful");
class="c">// yellow
console.table([{a:1,b:2},{a:3,b:4}]);
class="c">// nice table for arrays of objects
console.log("%cStyled", "color: red; font-size: 20px");

alert("Welcome!");
class="c">// blocking popup
const answer = prompt("Your name?");
class="c">// popup with text field → returns string or null
const ok = confirm("Delete?");
class="c">// OK/Cancel → true/false

document.write("<p>Avoid this</p>");
class="c">// erases the page if called after load
```

RESULT

```
> Hello, world
> 8
> Sum: 8
✖ Something failed
⚠ Careful
```

⚠ Common mistake

Writing `console.log(Hello)` without quotes → **ReferenceError: Hello is not defined.**
Text must be inside quotes; without quotes JavaScript thinks Hello is a variable.

Data types and typeof

Elzero JS #010

Gamal JS

Angela \$14

Every value has a type. Seven primitive types and one object type. `typeof` tells you the type of any value.

كل قيمة لها نوع: نص (string)، رقم (number)، صح/غلط (boolean)، فاضي (null/undefined)، وكائنات (object) ومصفوفات (array).

Type	Example	Used for
string	"hello", 'hi', `hi \${x}`	Text
number	42, 3.14, -7, NaN, Infinity	All numbers
boolean	true, false	Yes/no decisions
undefined	let x;	A variable with no value yet
null	let x = null;	Deliberately empty
object	{ name: "A", age: 5 }	Structured data
array (object)	[1, 2, 3]	Ordered lists
function (object)	function() {}	Reusable code

EXAMPLE

```

typeof "Ahmed"      class="c">// "string"   text, in quotes
typeof 42           class="c">// "number"   integers and decimals are the same type
typeof 3.14         class="c">// "number"
typeof true         class="c">// "boolean"   true or false only
typeof undefined    class="c">// "undefined" declared but no value yet
typeof null         class="c">// "object"    (a famous historical bug – null is "nothing on
purpose")
typeof 12345678901234567890n class="c">// "bigint"
typeof Symbol("id") class="c">// "symbol"
typeof {}           class="c">// "object"    key-value pairs
typeof []           class="c">// "object"    arrays are objects (use Array.isArray)
typeof function(){} class="c">// "function"

```

- **NaN** (Not a Number) is what you get from bad maths like `"abc" * 2`. Check with `Number.isNaN(x)`.
- **null vs undefined**: **undefined** = the language says 'nothing here yet'; **null** = you say 'nothing here on purpose'.

Variables: let, const, var

Elzero JS #011-#013

Gamal JS

Angela JS14

A variable is a named box that stores a value. **const** makes a box whose value cannot be replaced (use it by default). **let** makes a box you can change. **var** is the old way with confusing scope rules — avoid it.

المتغير صندوق باسم يخزن قيمة. **const** مينفعش تغير قيمته (استخدمه دائماً)، **let** ينفع تغيره، **var** قديم تجنبه.

	const	let	var
Can reassign	No	Yes	Yes
Scope	Block { }	Block { }	Function (leaks out of if/for blocks)
Use before declaration	Error	Error	undefined (hoisted)
Redeclare same name	Error	Error	Allowed (bugs)
Use it	Default	When value changes	Never

EXAMPLE

```

const siteName = "Giza Bakery";    class="c">// cannot be reassigned
let visitors = 0;                  class="c">// can change
visitors = visitors + 1;           class="c">// 1
visitors++;                       class="c">// 2

siteName = "Other";                class="c">// * TypeError: Assignment to constant variable

const user = { name: "Ahmed" };
user.name = "Sara";                class="c">// ✓ allowed: the box still holds the same object; we
changed what is inside it

let a = 5, b = 10;                 class="c">// several at once
let notYet;                       class="c">// undefined

```

- Names: letters, digits, _ and \$; cannot start with a digit; cannot be a reserved word (let, class, return...).
- Convention: **camelCase** for variables and functions (**firstName**), **PascalCase** for classes (**UserAccount**), **UPPER_SNAKE** for constants that never change (**MAX_SIZE**).
- Descriptive names: **totalPrice**, not **tp** or **x**.

Try it yourself

Declare a constant for your city, a variable for your age, then increase the age by one and print both.

Solution

```

const city = "Giza";
let age = 24;
age = age + 1;    class="c">// or age++
console.log(city, age);    class="c">// Giza 25

```

Strings: quotes, escaping, template literals, concatenation

Elzero JS #014 #015

Gamal JS

Angela \$14

Text values. Single and double quotes are the same. Backticks (template literals) can span lines and embed expressions with `${ }` — use them whenever you build text from variables.

EXAMPLE

```
const a = "double quotes";
const b = class="v">'single quotes';
const c = "He said \"hi\"";      class="c">//
// escape a quote with backslash → He said "hi"
const d = class="v">'It's
fineclass="v">';      class="c">// It's
fine
const e = "Line 1\nLine 2";      class="c">//
\n = new line, \t = tab, \\ = backslash

class="c">// concatenation with +
const name = "Ahmed";
const msg1 = "Hello " + name + ", you have "
+ 3 + " messages";

class="c">// template literal (backticks) –
cleaner ✓
const msg2 = class="v">`Hello ${name}, you
have ${1 + 2} messages`;
const multi = class="v">`Line 1
Line 2`;      class="c">// real line
breaks allowed
```

RESULT

```
Hello Ahmed, you have 3 messages
Hello Ahmed, you have 3 messages
Line 1
Line 2
```

⚠ Common mistake

Mixing quote types: `"It's"` works, but `'It's'` ends the string at the second quote → `SyntaxError`. Escape it or use the other quote type / backticks.

Numbers and arithmetic operators

Elzero JS #016-#020

Gamal JS

Angela \$14

Maths works as expected, plus a few operators you may not know. Beware: `+` with a string joins text instead of adding.

Useful Math	Result
Math.round(4.6)	5
Math.floor(4.9)	4 (down)
Math.ceil(4.1)	5 (up)
Math.trunc(-4.7)	-4 (cut decimals)
Math.abs(-5)	5
Math.max(3, 9, 2) / Math.min()	9 / 2
Math.sqrt(16) / Math.pow(2,3)	4 / 8
Math.random()	$0 \leq n < 1$
Math.floor(Math.random() * 6) + 1	dice: 1-6
Math.PI	3.14159...
(3.14159).toFixed(2)	"3.14" (a string)
Number.MAX_SAFE_INTEGER	9007199254740991

EXAMPLE

```

10 + 3    class="c">// 13    addition
10 - 3    class="c">// 7
10 * 3    class="c">// 30
10 / 3    class="c">// 3.3333333333333335
10 % 3    class="c">// 1    remainder (modulo) — is n even? n % 2 === 0
2 ** 10   class="c">// 1024 power

let x = 5;
x++; x--; class="c">// increment / decrement by 1
x += 10;  class="c">// x = x + 10 (also -=, *=, /=, %=, **=)

class="c">// the + trap
"5" + 3    class="c">// "53"    string wins → concatenation
"5" - 3    class="c">// 2      other operators convert to number
"5" * "2"   class="c">// 10
1 + 2 + "3" class="c">// "33"   left to right: 3 then "3"+"3"

class="c">// precedence: * / % before + -; use parentheses
2 + 3 * 4   class="c">// 14
(2 + 3) * 4  class="c">// 20

class="c">// decimals are approximate
0.1 + 0.2   class="c">// 0.30000000000000004 → use (0.1+0.2).toFixed(2) or work in cents

```

Type conversion

Elzero JS #021 #022

Gamal JS

Values from inputs, prompts and URLs are always strings. Convert them before doing maths. JavaScript also converts automatically in some situations (coercion) — know the rules to avoid surprises.

EXAMPLE

```

class="c">// string → number
Number("42")           class="c">// 42
Number("42px")         class="c">// NaN
Number("")             class="c">// 0
parseInt("42px")       class="c">// 42   reads digits until it cannot
parseFloat("3.5kg")    class="c">// 3.5
+"42"                 class="c">// 42   unary plus shortcut

class="c">// number → string
String(42)            class="c">// "42"
(42).toString()       class="c">// "42"
42 + ""              class="c">// "42"

class="c">// to boolean
Boolean(0)            class="c">// false
Boolean("hello")      class="c">// true
!!value              class="c">// shortcut

class="c">// FALSY values (become false): false, 0, -0, 0n, "", null, undefined, NaN
class="c">// everything else is TRUTHY, including "0", "false", [], {}

```

⚠ Common mistake

`const age = prompt("Age?"); age + 1` → "251" instead of 26. `prompt` returns a string. Use `Number(prompt(...))`.

Comparison and logical operators

Elzero JS #023-#026

Gamal JS

Angela \$15

Comparisons produce true or false. Always use the strict versions `===` and `!==` which also compare the type; the loose `==` converts types first and produces bizarre results.

Expression	Result	Why
<code>"" == 0</code>	true	both convert to 0 — loose is weird
<code>"" === 0</code>	false	different types — strict is sane
<code>[] == false</code>	true	array → "" → 0
<code>NaN === NaN</code>	false	use <code>Number.isNaN()</code>
<code>0 5</code>	5	0 is falsy
<code>0 ?? 5</code>	0	0 is not null/undefined

EXAMPLE

```

5 > 3      class="c">// true
5 >= 5     class="c">// true
5 < 3      class="c">// false
5 === 5    class="c">// true   strict equal (value AND type) ✓
5 === "5"  class="c">// false
5 == "5"   class="c">// true   loose equal converts → avoid
5 !== "5"  class="c">// true   strict not equal ✓
null == undefined class="c">// true (the one loose case people rely on)
"apple" < "banana" class="c">// true strings compare alphabetically

class="c">// logical
true && false class="c">// false AND: both must be true
true || false class="c">// true  OR: at least one
!true         class="c">// false NOT

class="c">// short-circuit: return a VALUE, not just true/false
const name = userInput || "Guest";   class="c">// first truthy value → default
const port = process.env.PORT ?? 3000; class="c">// ?? only replaces null/undefined (0 and "" survive) ✓
user && user.login();                  class="c">// run right side only if left is truthy
user?.address?.city;                  class="c">// optional chaining: undefined instead of crash

```

? Chapter quiz

1. Which keyword should you use by default for variables?
2. What does "3" + 4 give? And "3" * 4?
3. Name five falsy values.
4. Why prefer === over ==?

Answers: 1) const. 2) "34" and 12. 3) false, 0, "", null, undefined, NaN. 4) === compares type and value; == converts types and gives surprising results.

69. JavaScript Reference B — Control Flow: Conditions and Loops

Programs make decisions (if this, do that) and repeat work (for each item, do this). These structures are the same in every language; master them here and you know them everywhere.

if, else if, else

Elzero JS #027-#030

Gamal JS

Angela \$15

Run code only when a condition is true. Chain alternatives with else if; else catches everything remaining. The condition is any expression converted to boolean.

if بتنفيذ الكود لو الشرط صح. else if بديل ثاني. else لو كل اللي فوق غلط.

EXAMPLE

```
const score = 85;

if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 70) {
  console.log("C");
} else {
  console.log("F");
}

class="c">// combine conditions
if (age >= 18 && hasId) { enter(); }
if (day === "fri" || day === "sat") { weekend(); }
if (!isLoggedIn) { showLogin(); }

class="c">// truthy checks
if (user.name) { }
if (items.length) { }

class="c">// nested
if (user) {
  if (user.isAdmin) { showAdminPanel(); }
}

class="c">// guard clause style (flatter, preferred)
if (!user) return;
if (!user.isAdmin) return;
showAdminPanel();
```

⚠ Common mistake

`if (x = 5)` assigns 5 to x and is always true. Comparison needs `===`. Also: `if (x === 5 || 6)` is always true — write `if (x === 5 || x === 6)`.

Ternary operator

Elzero JS #031

Angela §15

A one-line if/else that produces a value. Great for short choices; do not nest it more than once.

EXAMPLE

```
const label = age >= 18 ? "adult" : "minor";
class="c">//      condition ? value if true : value if false

const greeting = hour < 12 ? "Good morning" : hour < 18 ? "Good afternoon" : "Good evening";
btn.textContent = isPlaying ? "Pause" : "Play";
element.classList.toggle("dark", isDark); class="c">// often better than a ternary
```

switch

Elzero JS #032 #033

Angela §15

Compare one value against many exact cases. Each case needs `break` or execution 'falls through' into the next case. `default` runs when nothing matched.

EXAMPLE

```
switch (day) {
  case "sat":
  case "sun": class="c">// two cases, one block (intentional fall-through)
    console.log("Weekend");
    break;
  case "fri":
    console.log("Almost weekend");
    break;
  default:
    console.log("Work day");
}

class="c">// switch uses === ; a lookup object is often cleaner:
const colours = { error: "red", warn: "orange", ok: "green" };
const c = colours[status] ?? "gray";
```

for loop

Elzero JS #034 #035

Gamal JS

Angela §15

Repeat a block a known number of times. Three parts separated by semicolons: start; keep going while; step.

for بتكرر الكود عدد مرات معروف: ابدأ من؛ كمل طالما؛ زود.

EXAMPLE

```
for (let i = 0; i < 5; i++) {
  console.log(i);      class="c">// 0 1 2
3 4
}
class="c">// start condition step

for (let i = 10; i >= 0; i -= 2) { }
class="c">// 10 8 6 4 2 0

class="c">// loop over an array by index
const fruits = ["apple", "mango", "fig"];
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}

class="c">// nested loops: rows and columns
for (let r = 1; r <= 3; r++) {
  for (let c = 1; c <= 3; c++) {

console.log(class="v">`${r} x ${c} = ${r * c}
`);
  }
}
```

RESULT

```
0
1
2
3
4

0 apple
1 mango
2 fig
```

⚠ Common mistake

Off-by-one: `i <= arr.length` reads one past the end (undefined). Arrays start at 0, so the last index is `length - 1`; use `i < length`. Infinite loop: forgetting `i++` freezes the tab.

for...of and for...in

Elzero JS #036 #037

Angela §15

`for...of` gives each VALUE of an array (or string, Map, Set) — the loop you will use most.

`for...in` gives each KEY of an object.

EXAMPLE

```
const fruits = ["apple", "mango", "fig"];
for (const fruit of fruits) {
  console.log(fruit);      class="c">// apple mango fig
}
for (const ch of "hi") { }  class="c">// h i
for (const [i, f] of fruits.entries()) { console.log(i, f); }  class="c">// with index

const user = { name: "Ahmed", age: 25 };
for (const key in user) {
  console.log(key, user[key]);  class="c">// name Ahmed / age 25
}
class="c">// for arrays prefer for...of or forEach; for...in gives indexes as strings and inherited keys
```

while and do...while

Elzero JS #038 #039

Angela \$15

Repeat while a condition stays true, when you do not know how many times in advance. do...while runs the block at least once, then checks.

EXAMPLE

```
let lives = 3;
while (lives > 0) {
  play();
  lives--;
}

let input;
do {
  input = prompt("Type yes");
} while (input !== "yes");    class="c">// asks at least once, repeats until "yes"

class="c">// game loop / retry pattern
let attempts = 0;
while (!connected && attempts < 5) { tryConnect(); attempts++; }
```

⚠ Common mistake

A while loop whose condition never becomes false runs forever and freezes the page. Always make sure something inside the loop changes the condition.

break and continue, labels

Elzero JS #040 #041

Angela \$15

break exits the loop immediately. **continue** skips the rest of this iteration and goes to the next one. A label lets break/continue target an outer loop.

EXAMPLE

```
for (const n of [1, 2, 3, 4, 5, 6]) {
  if (n === 4) break;    class="c">// stop entirely at 4 → 1 2 3
  console.log(n);
}

for (const n of [1, 2, 3, 4, 5, 6]) {
  if (n % 2 === 0) continue;    class="c">// skip evens → 1 3 5
  console.log(n);
}

outer: for (const r of rows) {
  for (const c of cols) {
    if (c === target) break outer;    class="c">// leave both loops
  }
}
```

Practice: classic beginner exercises

Elzero JS trainings

Angela \$15

Do all of these in the console before moving on.

Try it yourself

1) Print the numbers 1–100, but print "Fizz" for multiples of 3, "Buzz" for multiples of 5, "FizzBuzz" for both. 2) Sum all numbers from 1 to 1000. 3) Given an array of prices, print only those above 50. 4) Print a 5-row triangle of stars. 5) Count down from 10 to 1 with a while loop.

Solution

```
class="c">// 1
for (let i = 1; i <= 100; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}
class="c">// 2
let sum = 0; for (let i = 1; i <= 1000; i++) sum += i; console.log(sum); class="c">// 500500
class="c">// 3
for (const p of prices) if (p > 50) console.log(p);
class="c">// 4
for (let i = 1; i <= 5; i++) console.log("*".repeat(i));
class="c">// 5
let n = 10; while (n >= 1) { console.log(n); n--; }
```

? Chapter quiz

1. What happens if you forget break in a switch case?
2. Which loop gives array values directly?
3. Difference between break and continue?
4. When would you use do...while instead of while?

Answers: 1) Execution falls through into the next case. 2) for...of. 3) break exits the loop; continue skips to the next iteration. 4) When the block must run at least once before the check.

70. JavaScript Reference C — Functions

A function is a named block of code you can run (call) as many times as you like, optionally giving it inputs (parameters) and getting an output (return value). Functions are how you avoid repeating yourself and how you split a program into understandable pieces.

Declaring and calling a function

Elzero JS #042-#046

Gamal JS

Angela \$15

Define once with `function name(parameters) { body }`. Run it by writing its name with parentheses. Inputs you pass when calling are arguments; the names inside the definition are parameters.

الفانكشن كود ليه اسم بتشغله وقت ما تحب. بتديله مدخلات (parameters) ويرجعلك نتيجة (return).

EXAMPLE

```
class="c"> // define
function greet(name) {
  console.log("Hello, " + name);
}
class="c"> // call
greet("Ahmed");
greet("Sara");

class="c"> // several parameters, in order
function add(a, b) {
  return a + b;
}
const total = add(2, 3);
console.log(add(10, 20));

function noReturn() { console.log("hi"); }
const r = noReturn();
```

class="c"> // name = parameter

class="c"> // "Ahmed" = argument → Hello, Ahmed

class="c"> // Hello, Sara

class="c"> // return sends the result OUT and stops the function

class="c"> // 5

class="c"> // 30

class="c"> // r is undefined

- Code after `return` in the same block never runs.
- A function without a return gives `undefined`.
- Parameters are local: they exist only inside the function.
- Function names are verbs: `calculateTotal`, `showError`, `getUser`.

Try it yourself

Write a function `isEven(n)` that returns true for even numbers, and `max(a, b)` that returns the bigger one without using `Math.max`.

Solution

```
function isEven(n) { return n % 2 === 0; }
function max(a, b) { return a > b ? a : b; }
console.log(isEven(4), max(3, 9));    class="c">// true 9
```

Default parameters, rest parameters, arguments

Elzero JS #047-#049

Angela §22

Parameters can have default values used when the caller omits them. The rest parameter `...name` collects any number of extra arguments into an array.

EXAMPLE

```
function greet(name = "friend", greeting = "Hello") {
  return class="v">`${greeting}, ${name}!`;
}
greet();                class="c">// Hello, friend!
greet("Ahmed");         class="c">// Hello, Ahmed!
greet("Ahmed", "Hi");   class="c">// Hi, Ahmed!
greet(undefined, "Hey"); class="c">// Hey, friend!  undefined triggers the default

function sum(...numbers) {          class="c">// rest: any count → array
  let total = 0;
  for (const n of numbers) total += n;
  return total;
}
sum(1, 2, 3, 4);         class="c">// 10

function log(first, ...others) { }  class="c">// rest must be last
```

Function expressions and arrow functions

Elzero JS #050-#052

Gamal JS

Angela §22

A function can be stored in a variable (expression) or written in the short arrow form. Arrow functions are the modern default for short functions and callbacks. Differences: arrows have no own `this` and are not hoisted.

	function declaration	arrow function
Hoisted	Yes	No
Own this	Yes (depends on how it is called)	No — inherits from surrounding code (good inside classes and callbacks)
arguments object	Yes	No (use ...rest)
Can be a constructor (new)	Yes	No
Best for	Top-level named functions	Callbacks, short helpers, methods that need outer this

EXAMPLE

```

class="c">// 1. declaration – hoisted (can be called before the line it is written)
function square(n) { return n * n; }

class="c">// 2. expression – not hoisted
const square2 = function (n) { return n * n; };

class="c">// 3. arrow function
const square3 = (n) => { return n * n; };
const square4 = n => n * n;      class="c">// one parameter: no parentheses; one expression:
implicit return
const add = (a, b) => a + b;
const sayHi = () => console.log("hi");      class="c">// no parameters: empty ( )
const makeUser = (name) => ({ name });      class="c">// returning an object literal needs ( )

class="c">// arrows shine as callbacks
[1, 2, 3].map(n => n * 2);      class="c">// [2, 4, 6]
button.addEventListener("click", () => console.log("clicked"));

```

Scope: global, function, block

Elzero JS #053 #054

Angela \$22

Scope is where a variable is visible. Variables declared inside a block `{ }` (with `let/const`) exist only inside it. Inner code can see outer variables; outer code cannot see inner ones. Avoid globals — they collide and are hard to debug.

EXAMPLE

```

const site = "Bakery";      class="c">// global: visible everywhere

function order() {
  const item = "bread";      class="c">// function scope: only inside order()
  if (true) {
    let qty = 2;      class="c">// block scope: only inside this if
    console.log(site, item, qty);      class="c">// ✓ all visible
  }
  console.log(qty);      class="c">// ✗ ReferenceError: qty is not defined
}
console.log(item);      class="c">// ✗ ReferenceError

class="c">// shadowing: inner variable with the same name hides the outer one
let x = 1;
{ let x = 2; console.log(x); }      class="c">// 2
console.log(x);      class="c">// 1

```

⚠ Common mistake

Forgetting `let/const` creates an accidental global: `count = 5;` inside a function leaks out. Use strict mode (`"use strict";` or modules) and it becomes an error.

Hoisting

Elzero JS #055 Angela §22

Before running code, JavaScript moves declarations to the top of their scope. Function declarations are fully hoisted (usable before their line). `let/const` are hoisted but not initialised — using them early throws an error (the 'temporal dead zone'). `var` is hoisted as undefined.

EXAMPLE

```
sayHi();                                class="c"> // ✓ works: function declarations are hoisted
function sayHi() { console.log("hi"); }
```



```
console.log(a);                          class="c"> // undefined (var hoisted without value)
var a = 5;
```



```
console.log(b);                          class="c"> // * ReferenceError: Cannot access before initialization
let b = 5;
```



```
sayBye();                                class="c"> // * TypeError: sayBye is not a function
var sayBye = function () {};
```

Closures

Elzero JS #056 Angela §22

A function remembers the variables of the place where it was created, even after that place has finished running. This 'backpack' of variables is a closure. It is how you make private state, counters, and function factories.

EXAMPLE

```
function makeCounter() {
  let count = 0;                                class="c"> // private: nobody outside can touch it
  return function () {
    count++;
    return count;
  };
}

const counter = makeCounter();
counter();   class="c"> // 1
counter();   class="c"> // 2
const other = makeCounter();
other();     class="c"> // 1 - its own count

class="c"> // factory
function multiplier(factor) { return n => n * factor; }
const double = multiplier(2);
const triple = multiplier(3);
double(5);   class="c"> // 10
triple(5);   class="c"> // 15
```

→ Every callback you write in an event listener is a closure over the variables around it.

→ Classic bug: `for (var i...) setTimeout(() => console.log(i))` prints the final value each time; with `let` each iteration gets its own `i`.

Callbacks and higher-order functions

Elzero JS #057 Angela §22

Functions are values: you can pass them to other functions and return them. A function passed as an argument is a callback; a function that takes or returns functions is higher-order. Array methods, event listeners and timers all use callbacks.

EXAMPLE

```
function repeat(times, action) {      class="c"> // action is a callback
  for (let i = 0; i < times; i++) action(i);
}
repeat(3, i => console.log("Run", i));

setTimeout(() => console.log("after 1s"), 1000);
button.addEventListener("click", handleClick);  class="c"> // pass the function, do NOT call it:
handleClick()
[3, 1, 2].sort((a, b) => a - b);                class="c"> // [1, 2, 3]
```

⚠ Common mistake

`addEventListener("click", handleClick())` calls the function immediately and passes its return value (undefined). Pass the name without parentheses, or wrap: `() => handleClick(id)`.

Recursion, IIFE, pure functions

Elzero JS #058 #059 Angela §22

Recursion: a function calling itself with a smaller problem, always with a base case to stop.
 IIFE: a function run immediately to create a private scope (less needed with modules).
 Pure functions: same input → same output, no side effects — easiest to test.

EXAMPLE

```

class="c">// recursion
function factorial(n) {
  if (n <= 1) return 1;      class="c">// base case
  return n * factorial(n - 1); class="c">// smaller problem
}
factorial(5);  class="c">// 120

function countFiles(folder) {
  let n = folder.files.length;
  for (const sub of folder.folders) n += countFiles(sub);  class="c">// tree walk
  return n;
}

class="c">// IIFE
(function () {
  const secret = 42;  class="c">// not global
})();

class="c">// pure vs impure
const pureAdd = (a, b) => a + b;      class="c">// pure
let total = 0; const impureAdd = n => total += n; class="c">// impure: changes outside state

```

? Chapter quiz

1. Parameter vs argument?
2. What does a function return if it has no return statement?
3. Write an arrow function that returns the square of a number.
4. What is a closure in one sentence?

Answers: 1) Parameter is the name in the definition; argument is the value passed when calling. 2) undefined.
 3) `const sq = n => n * n;` 4) A function that keeps access to the variables of the scope where it was created.

71. JavaScript Reference D — Strings, Numbers and Dates in Depth

Built-in methods for the values you use most. You do not memorise all of them; you learn they exist and look up the exact name. This chapter is the lookup table with examples.

String properties and methods

Elzero JS #060-#066

Angela §14

Strings are immutable: methods return a new string and never change the original. Indexing starts at 0.

EXAMPLE

```
const s = "Hello, World";
s.length           class="c"> 12
s[0] s.at(-1)      class="c"> "H" "d"
s.toUpperCase()     class="c"> "HELLO, WORLD"
s.toLowerCase()
s.trim()            class="c"> // remove spaces at both ends (trimStart, trimEnd)
s.includes("World") class="c"> // true
s.startsWith("He")  class="c"> // true
s.endsWith("!")     class="c"> // false
s.indexOf("o")      class="c"> // 4
(first) lastIndexOf → 8 -1 if missing
s.slice(0, 5)
class="c"> // "Hello" (start, end-exclusive; negatives from end)
s.slice(-5)          class="c"> // "World"
s.substring(7)       class="c"> // "World"
s.replace("World", "Giza") class="c"> // first match
s.replaceAll("l", "L") class="c"> // all
s.split(", ")        class="c"> // ["Hello", "World"] → array
s.split("")          class="c"> // every character
"a-b".split("-").join("+") class="c"> // "a+b"
s.repeat(2)
"5".padStart(3, "0") class="c"> // "005" (padEnd)
s.charAt(1) s.charCodeAt(1) class="c"> // "e" 101
s.concat("!")
class="c"> // prefer + or template literal
s.localeCompare("abc") class="c"> // sort helper
s.normalize()        class="c"> // unicode
```

RESULT

```
"Hello, World".slice(0,5) → "Hello"
" hi ".trim() → "hi"
"a,b,c".split(",") → ["a","b","c"]
"7".padStart(2,"0") → "07"
```

 Try it yourself

Write `capitalize(str)` that returns the string with its first letter uppercase, and `countWords(str)`.

Solution

```
const capitalize = s => s.charAt(0).toUpperCase() + s.slice(1);
const countWords = s => s.trim().split(/\s+/).length;
```

Number methods and formatting

Elzero JS #067 #068

Format numbers for display, check them, and parse them safely.

EXAMPLE

```
(1234.5678).toFixed(2)           class="c">// "1234.57" (string!)
(0.1 + 0.2).toFixed(2)           class="c">// "0.30"
(1234.5678).toPrecision(6)       class="c">// "1234.57"
(255).toString(16)               class="c">// "ff" base conversion
(255).toString(2)                class="c">// "11111111"
Number.isInteger(5)               class="c">// true
Number.isNaN(NaN)                class="c">// true (global isNaN converts first – avoid)
Number.parseFloat("3.5")
Number.MAX_SAFE_INTEGER

class="c">// display with thousands separators and currency
(1234567.891).toLocaleString("en-US")           class="c">// "1,234,567.891"
(1234.5).toLocaleString("ar-EG", { style: "currency", currency: "EGP" }) class="c">// "١٬٢٣٤,٥٠٠.٨٩١"
new Intl.NumberFormat("en-EG", { style: "currency", currency: "EGP" }).format(1234.5)
class="c">// "EGP 1,234.50"
(0.256).toLocaleString("en", { style: "percent" })           class="c">// "26%"
```

Date and time

Elzero JS #069

The built-in Date object. Months are 0-based (January = 0) — a famous trap. For anything serious (time zones, formatting) use the Intl API or a library like date-fns.

EXAMPLE

```

const now = new Date();
new Date(2026, 8, 7)
new Date("2026-09-07")
new Date("2026-09-07T14:30:00")
new Date(0)

now.getFullYear()
now.getMonth()
now.getDate()
now.getDay()
now.getHours() getMinutes() getSeconds()
now.getTime()
Date.now()

now.setDate(now.getDate() + 7);

class="c"> // formatting
now.toISOString()
now.toLocaleDateString("en-GB")
now.toLocaleDateString("ar-EG", { weekday: "long", day: "numeric", month: "long", year: "numeric" })
now.toLocaleTimeString("en-US", { hour: "2-digit", minute: "2-digit" })

class="c"> // difference in days
const days = Math.round((new Date("2026-12-31") - new Date()) / (1000 * 60 * 60 * 24));

class="c"> // countdown
setInterval(() => {
  const diff = target - Date.now();
  const h = Math.floor(diff / 3.6e6), m = Math.floor(diff % 3.6e6 / 6e4), s = Math.floor(diff % 6e4 / 1000);
  timer.textContent = ` ${h}:${m}:${s} `;
}, 1000);

```

Regular expressions (basics)

Elzero JS #070-#072

A pattern language for searching and validating text. Looks cryptic; learn the ten symbols below and you can handle emails, phones and simple parsing. Test patterns at regex101.com.

Symbol	Meaning
.	any character
\d \w \s	digit, word char, whitespace (capitals = NOT)
* + ?	0 or more, 1 or more, 0 or 1
{3} {2,5}	exactly 3, between 2 and 5
[abc] [a-z] [^0-9]	one of, range, not
^ \$	start, end of string
()	group, or
g i m flags	global (all matches), ignore case, multiline

EXAMPLE

```

const re = /\d{3}-\d{4}/;           class="c">// three digits, dash, four digits
re.test("555-1234")                 class="c">// true
"Call 555-1234 now".match(/\d+/g)   class="c">// ["555", "1234"]
"a1b2".replace(/\d/g, "#")         class="c">// "a#b#"

class="c">// common validators
const email = /^[^\\s@]+@[^\\s@]+\\.[^\\s@]+$/;
const egyptPhone = /^01[0125][0-9]{8}$/;
const strongPw = /^(?=.*[A-Z])(?=.*\\d){8,}$/;   class="c">// 8+, one uppercase, one digit
email.test("m@site.com");   class="c">// true

```

? Chapter quiz

1. Do string methods change the original string?
2. What month is new Date(2026, 0, 15)?
3. What does "42.5".padStart(6, "0") return?
4. Which method splits a string into an array?

Answers: 1) No, they return a new string. 2) January (months are 0-based). 3) "0042.5". 4) split().

72. JavaScript Reference E — Arrays and Objects in Depth

Arrays (ordered lists) and objects (labelled boxes) are how you represent real data: a list of products, a user profile, a shopping cart. Almost every line of real JavaScript touches one of them. The array methods `map`, `filter` and `reduce` are the most important things in this chapter.

Arrays: create, read, change

Elzero JS #073-#078

Gamal JS

Angela §16

An ordered collection. Elements are accessed by index starting at 0. Arrays can hold any mix of types, and can grow or shrink.

المصفوفة قائمة مرتبة. أول عنصر رقمه 0. length عدد العناصر.

EXAMPLE

```
const fruits = ["apple", "mango", "fig"];
fruits.length      class="c">// 3
fruits[0]          class="c">// "apple"
fruits[2]          class="c">// "fig"
fruits[3]          class="c">// undefined
(no error!)
fruits[fruits.length - 1] class="c">// last
→ "fig"
fruits.at(-1)      class="c">// "fig"
(modern)
fruits[1] = "banana"; class="c">// replace

class="c">// add / remove at the ends (MUTATE
the array)
fruits.push("kiwi"); class="c">// add to
end → returns new length
fruits.pop();       class="c">// remove
last → returns it
fruits.unshift("date"); class="c">// add to
start
fruits.shift();     class="c">// remove
first

class="c">// anywhere
fruits.splice(1, 1); class="c">//
remove 1 item at index 1
fruits.splice(1, 0, "pear"); class="c">//
insert at 1, remove 0
fruits.splice(0, 2, "a", "b"); class="c">//
replace first two

Array.isArray(fruits) class="c">// true
const mixed = [1, "two", true, null, [3, 4],
{ k: 1 }];
const empty = new Array(3).fill(0);
class="c">// [0, 0, 0]
Array.from({ length: 5 }, (_, i) => i + 1);
class="c">// [1, 2, 3, 4, 5]
```

RESULT

apple	mango	fig
[0]	[1]	[2]

length = 3, last index = 2

Finding and checking

Elzero JS #079-#081

Angela \$16

Search for elements or test conditions across the array.

EXAMPLE

```
const nums = [5, 12, 8, 130, 44];
nums.indexOf(8)           class="c">// 2    (-1 if not found) – uses ===
nums.includes(130)        class="c">// true
nums.find(n => n > 10)     class="c">// 12   first element that passes
nums.findIndex(n => n > 10) class="c">// 1
nums.findLast(n => n > 10) class="c">// 44
nums.some(n => n > 100)    class="c">// true – at least one
nums.every(n => n > 0)     class="c">// true – all

const users = [{ id: 1, name: "A" }, { id: 2, name: "B" }];
users.find(u => u.id === 2) class="c">// { id: 2, name: "B" }
users.some(u => u.name === "A") class="c">// true
```

Transforming: map, filter, reduce — the big three

Elzero JS #082-#086

Gamal JS

Angela \$22

These return a NEW array (or value) and leave the original untouched. **map**: transform every item (same length). **filter**: keep only items that pass a test. **reduce**: combine everything into one value. Chain them for data pipelines.

map يتحول كل عنصر (نفس العدد)، filter بتفلتر اللي ينجح في الشرط، reduce بتجمع الكل في قيمة واحدة.

EXAMPLE

```
const prices = [10, 25, 40, 5];

const doubled = prices.map(p => p * 2);      class="c">// [20, 50, 80, 10]
const expensive = prices.filter(p => p > 20);  class="c">// [25, 40]
const total = prices.reduce((sum, p) => sum + p, 0);  class="c">// 80
class="c">//
accumulator, current  initial value

class="c">// real data
const cart = [
  { name: "Bread", price: 5, qty: 2 },
  { name: "Cake", price: 40, qty: 1 },
  { name: "Tea", price: 10, qty: 3 },
];
const names = cart.map(item => item.name);
class="c">// ["Bread", "Cake", "Tea"]
const cheap = cart.filter(item => item.price < 20);  class="c">// Bread, Tea
const bill = cart.reduce((sum, item) => sum + item.price * item.qty, 0);  class="c">// 80
const html = cart.map(item => class="v">`<li>${item.name} × ${item.qty}</li>`).join("");

class="c">// chain
const cheapNames = cart.filter(i => i.price < 20).map(i => i.name.toUpperCase());

class="c">// reduce to an object (group / count)
const countByType = ["a", "b", "a"].reduce((acc, t) => { acc[t] = (acc[t] || 0) + 1; return acc; }, {});  class="c">// {a: 2, b: 1}
```

RESULT

```
[10, 25, 40, 5] → map(×2) → [20, 50, 80, 10]
[10, 25, 40, 5] → filter(>20) → [25, 40]
[10, 25, 40, 5] → reduce(sum) → 80
```

 Common mistake

Forgetting to `return` inside a multi-line map callback gives an array of undefined. Also: `reduce` without an initial value uses the first element as the accumulator — always pass the initial value.

 Try it yourself

Given `students = [{name:"A", grade: 85}, {name:"B", grade: 45}, {name:"C", grade: 92}]`:
 1) names of those who passed (≥ 50); 2) average grade; 3) an HTML list string of all names.

Solution

```
const passed = students.filter(s => s.grade >= 50).map(s => s.name);    class="c">// ["A", "C"]
const avg = students.reduce((s, x) => s + x.grade, 0) / students.length; class="c">// 74
const list = "<ul>" + students.map(s => class="v">`<li>${s.name}</li>`).join("") + "</ul>";
```

forEach, sort, reverse, slice, concat, spread, flat

Elzero JS #087-#090

Angela §16

More everyday array tools. Note which ones mutate (change the original) and which return a copy.

Mutates original	Returns new
push pop shift unshift splice sort reverse fill	map filter reduce slice concat flat join toSorted toReversed [...spread]

EXAMPLE

```
const arr = [3, 1, 2];
arr.forEach((n, i) => console.log(i, n));    class="c">// loop; returns nothing (use map when you
need a result)

arr.sort();                                class="c">// MUTATES; sorts as STRINGS by default → [1, 10, 2]
problem!
arr.sort((a, b) => a - b);                  class="c">// numbers ascending ✓
arr.sort((a, b) => b - a);                  class="c">// descending
users.sort((a, b) => a.name.localeCompare(b.name));    class="c">// by string property
users.sort((a, b) => a.age - b.age);          class="c">// by number property
arr.toSorted((a, b) => a - b); class="c">// modern: returns a sorted COPY

arr.reverse();                            class="c">// MUTATES (toReversed = copy)
arr.slice(1, 3);                          class="c">// copy of a part (start, end-exclusive) – original
untouched
arr.slice();                              class="c">// shallow copy of all
arr.concat([4, 5]);                       class="c">// new array joined
[...arr, 4, 5]                            class="c">// spread – same, modern ✓
[...a, ...b]                             class="c">// merge two arrays
const copy = [...arr];                   class="c">// copy (not the same reference)
arr.join(", ");                          class="c">// → string
[1, [2, [3]]].flat(Infinity);            class="c">// [1, 2, 3]
arr.flatMap(x => [x, x * 2]);
[...new Set([1, 1, 2])]                  class="c">// unique values → [1, 2]
arr.fill(0); arr.entries(); arr.keys(); arr.values();
```

⚠ Common mistake

`[10, 9, 1].sort()` → `[1, 10, 9]` because it compares as text. Always pass a compare function for numbers.

Objects: create, read, change

Elzero JS #091-#095

Gamal JS

Angela \$16

An unordered collection of key: value pairs (properties). Keys are strings (or symbols); values are anything, including arrays, other objects and functions (then called methods). Objects model 'a thing with attributes': a user, a product, a setting.

الكائن مجموعة من مفتاح: قيمة. يمثل حاجة لها صفات: مستخدم، منتج، إعدادات.

EXAMPLE

```
const user = {
  name: "Ahmed",
  age: 25,
  isAdmin: false,
  skills: ["html", "css"],
  address: { city: "Giza", zip:
"12511" },      class="c">// nested object
  greet() { return class="v">`Hi, I'm $
{this.name}`; }, class="c">// method; this =
the object
  "favourite colour":
"blue"
class="c">// key with space needs quotes
};

class="c">// read
user.name          class="c">//
"Ahmed"           dot notation
user["name"]       class="c">//
"Ahmed"           bracket notation (key as string)
const key = "age"; user[key] class="c">//
25                bracket when the key is in a variable
user.address.city  class="c">// "Giza"
user.skills[0]     class="c">// "html"
user.greet()       class="c">// "Hi,
I'm Ahmed"
user.email         class="c">//
undefined (no error)
user.address.street.name class="c">// ✖
TypeError: cannot read properties of undefined
user.address?.street?.name class="c">//
undefined, no crash ✓

class="c">// change / add / delete
user.age = 26;
user.email = "a@b.com"; class="c">// new
property
delete user.isAdmin;
"email" in user    class="c">// true
user.hasOwnProperty("age")
```

RESULT

```
user {
  name: "Ahmed",
  age: 25,
  skills: ["html", "css"],
  address: { city: "Giza" },
  greet: f
}
```

Object methods, keys/values/entries, spread, destructuring

Elzero JS #096-#100

Angela §22

Loop over an object, copy and merge objects, and pull properties into variables in one line.

EXAMPLE

```
const user = { name: "Ahmed", age: 25 };
Object.keys(user)      class="c">// ["name", "age"]
Object.values(user)    class="c">// ["Ahmed", 25]
Object.entries(user)   class="c">// [["name", "Ahmed"], ["age", 25]]
for (const [k, v] of Object.entries(user)) console.log(k, v);
Object.fromEntries([["a", 1]]) class="c">// { a: 1 }

class="c">// copy & merge (shallow)
const copy = { ...user };
const updated = { ...user, age: 26, city: "Giza" };   class="c">// later keys override
const merged = { ...defaults, ...options };
Object.assign(target, source);
const deep = structuredClone(user);   class="c">// deep copy incl. nested objects

class="c">// destructuring: pull properties into variables
const { name, age } = user;
const { name: userName, city = "Giza" } = user;   class="c">// rename, default
const { address: { city: town } } = user;         class="c">// nested
function show({ name, age }) { }                 class="c">// in parameters ✓ very common
const [first, , third, ...rest] = [1, 2, 3, 4, 5]; class="c">// arrays too: first=1, third=3,
rest=[4,5]
[a, b] = [b, a];   class="c">// swap

class="c">// freeze
Object.freeze(config);   class="c">// cannot change (shallow)

class="c">// shorthand when variable name = key name
const title = "Hi"; const post = { title, body };   class="c">// = { title: title, body: body }
class="c">// computed key
const field = "email"; const o = { [field]: "a@b.com" };
```

Reference vs value — the bug every beginner hits

Elzero JS #101

Angela §22

Primitives (numbers, strings, booleans) are copied by value. Objects and arrays are copied by reference: two variables point to the same thing, so changing one changes 'both'. To get an independent copy, clone it.

EXAMPLE

```

let a = 5;
let b = a;   b = 10;      class="c">// a is
still 5 (copied value)

const o1 = { x: 1 };
const o2 = o1;           class="c">// same
object!
o2.x = 99;
console.log(o1.x);       class="c">// 99 🤖

const o3 = { ...o1 };
class="c">// shallow copy → independent (top
level)
const arr2 = [...arr1];
const deep = structuredClone(o1);
class="c">// nested too

[1, 2] === [1, 2]        class="c">//
false – different objects
class="c">// compare contents:
JSON.stringify(a) === JSON.stringify(b) or a
library

```

RESULT

```

o1 ┌───┐
    │   │
    └───┴───> { x: 99 }
o2 ┌───┐
    │   │
    └───┴───> { x: 1 } (own copy)

```

- Passing an object to a function passes the reference: the function can modify your object. Pure functions return a new object instead.

JSON — sending and storing data

Elzero JS #102

Angela §30

JavaScript Object Notation is a text format that looks like a JS object literal. Every API and every database talks JSON. Convert object → text with `JSON.stringify` and text → object with `JSON.parse`.

JSON rule	Valid	Invalid
Keys in double quotes	<code>{"a": 1}</code>	<code>{a: 1}, {'a': 1}</code>
Strings in double quotes	<code>"hi"</code>	<code>'hi'</code>
No trailing commas	<code>[1, 2]</code>	<code>[1, 2,]</code>
No comments, functions, undefined	<code>null</code>	<code>undefined, () => {}</code>

EXAMPLE

```
const user = { name: "Ahmed", skills: ["html", "css"], active: true };

const text = JSON.stringify(user);
class="c">v">'{"name":"Ahmed","skills":["html","css"],"active":true}' ← keys in double
quotes, no functions, no undefined
JSON.stringify(user, null, 2);      class="c">v"> pretty-printed with 2-space indent

const back = JSON.parse(text);      class="c">v"> real object again
back.skills[0]                     class="c">v"> "html"
JSON.parse("{bad json}")           class="c">v"> * SyntaxError – wrap in try/catch

class="c">v"> localStorage only stores strings → JSON is the bridge
localStorage.setItem("user", JSON.stringify(user));
const saved = JSON.parse(localStorage.getItem("user") || "null");
localStorage.removeItem("user"); localStorage.clear();
class="c">v"> sessionStorage: same API, cleared when the tab closes
```

Map and Set

Elzero JS #103 #104

Two newer collections. **Set**: a list of unique values. **Map**: key-value pairs where keys can be any type (not only strings) and insertion order is kept.

EXAMPLE

```
const tags = new Set(["js", "css", "js"]); class="c">v"> {"js", "css"}
tags.add("html"); tags.has("css"); tags.delete("js"); tags.size;
const unique = [...new Set(array)];

const m = new Map();
m.set("name", "Ahmed"); m.set(42, "answer"); m.set(objKey, "any key type");
m.get("name"); m.has(42); m.delete(42); m.size;
for (const [k, v] of m) { }
new Map(Object.entries(obj)); Object.fromEntries(m);
```

? Chapter quiz

1. Which three array methods return a new array or value without changing the original?
2. How do you get the last element of an array?
3. What is the danger in `const b = a` when `a` is an object?
4. Convert an object to a JSON string — which function?

Answers: 1) `map`, `filter`, `reduce` (also `slice`, `concat`...). 2) `arr[arr.length - 1]` or `arr.at(-1)`. 3) `b` points to the same object; changing `b` changes `a`. 4) `JSON.stringify`.

73. JavaScript Reference F — The DOM: Selecting, Changing, Creating

The Document Object Model is the browser's live representation of your HTML as a tree of objects. Through the global `document`, JavaScript can find any element, read or change its text, attributes, styles and classes, create new elements and remove old ones. This is how static pages become applications.

What the DOM is

Elzero JS #105-#107

Gamal JS

Angela \$17

When the browser reads your HTML it builds a tree: document → html → head/body → every tag → text. Each node is a JavaScript object with properties (textContent, style, classList, children) and methods (append, remove, addEventListener). Changing the object changes the page instantly.

ال DOM شجرة من الكائنات يتمثل الصفحة. لما تغير الكائن، الصفحة بتتغير فوراً.

EXAMPLE

```
document                class="c">// the
whole page
document.documentElement class="c">// <html>
document.head
document.body
document.title = "New tab title";
document.URL
document.forms  document.images
document.links  class="c">// live collections
```

RESULT

```
document
└─ html
   └─ head
      └─ title → "text"
   └─ body
      └─ h1 → "text"
      └─ p → "text"
      └─ ul
         └─ li
            └─ li
```

Selecting elements

Elzero JS #108 #109

Gamal JS

Angela \$17

Find elements with CSS selectors. `querySelector` returns the first match (or null); `querySelectorAll` returns all matches as a NodeList you can loop with `forEach`. The older `getElementBy*` methods still work.

EXAMPLE

```

const title = document.querySelector("h1");
const btn   = document.querySelector("#save");
const first = document.querySelector(".card");
const inner = document.querySelector(".card .title");
const cards = document.querySelectorAll(".card");
cards.length;
cards.forEach(card => card.classList.add("ready"));
const arr = [...cards];
card.querySelector("img");

class="c">// older API
document.getElementById("save");
document.getElementsByClassName("card");
document.getElementsByTagName("li");

class="c">// first h1
class="c">// by id
class="c">// first .card
class="c">// any selector works
class="c">// NodeList of all

class="c">// real array → map/filter
class="c">// search INSIDE an element

class="c">// no # ; fastest
class="c">// live HTMLCollection (no forEach)

class="c">// checks
if (!btn) return;
el.matches(".active");
el.closest(".card");
handlers
class="c">// querySelector returns null when nothing matches
class="c">// does it match this selector?
class="c">// nearest ancestor (or itself) matching – great in event

```

⚠ Common mistake

Cannot read properties of null (reading 'addEventListener') — the element was not found: the selector is wrong, or the script ran before the HTML existed (move the script to the end of body or add defer).

Reading and changing content

Elzero JS #110 #111

Gamal JS

Angela \$17

Three properties for the content of an element. Use `textContent` for text (safe), `innerHTML` only for HTML you wrote yourself, never for user input.

EXAMPLE

```
const p = document.querySelector("p");
p.textContent      class="c">// read
text, all of it, no tags
p.textContent = "New
text";            class="c">// set
(any < > are shown literally – safe)
p.innerHTML = "Hello <strong>world</
strong>"; class="c">// parsed as HTML
p.innerHTML +=
"<br>more";        class="c">//
append (re-parses everything – slow in loops)
p.innerText      class="c">// like
textContent but respects CSS (hidden text
excluded); slower

input.value        class="c">// what the
user typed – for input, textarea, select
input.value = "";  class="c">// clear
checkbox.checked      class="c">// true/
false
select.value        class="c">// selected
option value
select.selectedIndex
```

RESULT

p.textContent = "a bold" → shows: a
bold
p.innerHTML = "a bold" → shows: a **bold**

⚠ Common mistake

`e1.innerHTML = userComment` lets an attacker inject `<script>` or `<img onerror=...>` — an XSS attack. Use `textContent` for anything that came from a user or an API.

Attributes and data attributes

Elzero JS #112 #113

Angela §17

Standard attributes are mirrored as properties (`img.src`, `a.href`, `input.disabled`). Any attribute can be read with `getAttribute/setAttribute`. Custom `data-*` attributes are read through `dataset`.

EXAMPLE

```
img.src = "photo2.jpg";
img.alt;
link.href = "https://...";
input.disabled = true;
input.placeholder = "Type...";

el.getAttribute("href");           class="c">// as written in HTML
el.setAttribute("aria-label", "Close");
el.removeAttribute("hidden");
el.hasAttribute("required");
el.id; el.className;               class="c">// whole class string

class="c">// <li data-id="7" data-price="20.5">
li.dataset.id      class="c">// "7" (always strings)
li.dataset.price   class="c">// "20.5" → Number(li.dataset.price)
li.dataset.status = "done"; class="c">// sets data-status="done"
```

Styles and classes

Elzero JS #114 #115

Gamal JS

Angela \$17

You can set inline styles from JS, but the maintainable way is to define classes in CSS and toggle them. `classList` has `add`, `remove`, `toggle`, `contains`, `replace`.

EXAMPLE

```
class="c">// inline style – property names in camelCase
box.style.backgroundColor = "gold";
box.style.width = "200px";
class="c">// units required
box.style.display = "none";
box.style.cssText = "color: red; font-size: 20px";
box.style.setProperty("--primary", "#f00"); class="c">// CSS variable

class="c">// read the REAL computed style (inline only shows what JS/HTML set)
getComputedStyle(box).width;

class="c">// classes ✓ preferred
box.classList.add("active", "big");
box.classList.remove("hidden");
box.classList.toggle("open");
class="c">// add if missing, remove if present → returns true/false
box.classList.toggle("dark", isDark);
class="c">// force: add when second arg is true
box.classList.contains("active");
box.classList.replace("old", "new");
box.className = "card featured";
class="c">// overwrite all
```

RESULT

Menu → click → `classList.toggle("open")` → Menu (open)

Tip: Keep JS for logic and CSS for appearance: JS decides when a class applies; CSS decides what it looks like (including transitions).

Creating, inserting, removing, cloning

Elzero JS #116-#119

Gamal JS

Angela §18

Build elements in JavaScript and put them in the page. Modern methods: append/prepend (multiple nodes or strings), before/after, remove, replaceWith. insertAdjacentHTML is the fast way to add a chunk of HTML.

EXAMPLE

```

class="c">// create
const li = document.createElement("li");
li.textContent = "New task";
li.classList.add("task");
li.dataset.id = 42;

class="c">// insert
list.append(li);           class="c">// as
last child (appendChild = older, one node
only)
list.prepend(li);         class="c">// as
first child
list.append(li1, li2, "text"); class="c">//
several
ref.before(li); ref.after(li); class="c">//
as siblings
ref.replaceWith(li);
parent.insertBefore(li, parent.firstChild);
class="c">// older

class="c">// HTML string at a position:
beforebegin | afterbegin | beforeend |
afterend
list.insertAdjacentHTML("beforeend",
class="v">`<li class="task">${escape(text)}</
li>`);

class="c">// remove
li.remove();
list.innerHTML = "";      class="c">//
remove all children (or
list.replaceChildren())

class="c">// clone
const copy = card.cloneNode(true);
class="c">// true = with children

class="c">// build a list from data
efficiently
const frag =
document.createDocumentFragment();
for (const item of items) {
  const li = document.createElement("li");
  li.textContent = item.name;
  frag.append(li);
}
list.append(frag);        class="c">// one
reflow instead of many

class="c">// or with template literal + map
(simple & common)
list.innerHTML = items.map(i =>
class="v">`<li>${i.name}</li>`).join("");

```

RESULT

beforebegin

afterbegin
 (existing children)
 beforeend

afterend

Try it yourself

Write `addTask(text)` that creates an `li` with the text and a delete button (×), appends it to `#list`, and make the button remove its `li` when clicked.

Solution

```
function addTask(text) {
  const li = document.createElement("li");
  li.textContent = text + " ";
  const del = document.createElement("button");
  del.textContent = "×";
  del.addEventListener("click", () => li.remove());
  li.append(del);
  document.querySelector("#list").append(li);
}
```

Traversing: parents, children, siblings

Elzero JS #120 #121

Angela §18

Move around the tree from an element you already have. Prefer the Element versions (they skip text nodes and comments).

EXAMPLE

```
el.parentElement           class="c"> // parent
(parentNode includes document)
el.children                 class="c"> //
HTMLCollection of element children
el.firstChild               el.lastElementChild
el.nextElementSibling
el.previousElementSibling
el.closest(".card")
class="c"> // walk UP to the nearest match
el.childElementCount
el.contains(other)          class="c"> // is
other inside el?

class="c"> // Node versions include text nodes
(usually not what you want)
el.childNodes               el.firstChild   el.nextSibling
el.nodeType                 el.nodeName
```

RESULT

```
parentElement
├─ previousElementSibling
├─ el — children[0], children[1]...
└─ nextElementSibling
```

Size, position and scrolling

Elzero JS #122

Measure elements and the window; scroll programmatically.

EXAMPLE

```

el.offsetWidth  el.offsetHeight  class="c">// incl. padding + border
el.clientWidth  el.clientHeight  class="c">// incl. padding, no border/scrollbar
el.scrollHeight  class="c">// full content height
el.getBoundingClientRect()  class="c">// {top, left, width, height...} relative to viewport
window.innerWidth  window.innerHeight
window.scrollTo  class="c">// how far the page is scrolled
window.scrollTo({ top: 0, behavior: "smooth" });
el.scrollTo({ behavior: "smooth", block: "start" });
el.scrollTop = 0;

```

? Chapter quiz

1. Which method returns the first element matching a CSS selector?
2. `textContent` or `innerHTML` for user input — and why?
3. How do you toggle a class named `active`?
4. Why is a `NodeList` from `querySelectorAll` not exactly an array?

Answers: 1) `document.querySelector`. 2) `textContent` — `innerHTML` would execute injected HTML/scripts (XSS). 3) `el.classList.toggle("active")`. 4) It lacks `map/filter`; convert with `[...list]` or `Array.from`.

74. JavaScript Reference G — Events

An event is something that happens: a click, a key press, a form submission, the page finishing loading, the window resizing. You attach a listener function to an element for an event type; the browser calls your function each time the event occurs and hands it an event object full of details.

addEventListener

Elzero JS #123-#125

Gamal JS

Angela §19

The standard way to react to events. Three parts: the element, the event name (a string, no 'on'), and the handler function. You can attach many listeners to one element and remove them later.

بتربط فانكشن بحدث على عنصر: لما الحدث يحصل، الفانكشن تنفذ.

EXAMPLE

```
const btn = document.querySelector("#save");

btn.addEventListener("click", function (event) {
  console.log("clicked!", event);
});
class="c">// arrow version
btn.addEventListener("click", () => save());
class="c">// named handler (can be removed later)
function onSave(e) { }
btn.addEventListener("click", onSave);
btn.removeEventListener("click", onSave);    class="c">// must be the SAME function reference
btn.addEventListener("click", onSave, { once: true });    class="c">// auto-remove after first run

class="c">// old ways (still seen): only one handler each, mixes HTML and JS
btn.onclick = () => {};
<button onclick="save()">    class="c">// avoid
```

⚠ Common mistake

`btn.addEventListener("onclick", ...)` — the event name is `click`, without `on`. And `addEventListener("click", save())` runs `save` immediately; pass `save` or `() => save()`.

The event object

Elzero JS #126

Angela §19

Your handler receives an object describing what happened: which element, where the mouse was, which key, and methods to stop default behaviour or propagation.

Event	Fires when	On
click / dblclick	mouse click / double	any element
mousedown / mouseup / mousemove	button pressed / released / moved	any
mouseenter / mouseleave	pointer enters / leaves (no bubbling)	any
mouseover / mouseout	same but bubbles from children	any
contextmenu	right click	any
keydown / keyup	key pressed / released	document, inputs
input	value changes on every keystroke	input, textarea, select
change	value committed (blur for text, immediately for checkbox/select)	form controls
submit	form sent	form
focus / blur	element gains / loses focus (no bubbling; focusin/focusout bubble)	inputs, buttons, links
load	fully loaded incl. images	window, img
DOMContentLoaded	HTML parsed (before images)	document
scroll	scrolled	window, scrollable elements
resize	window size changed	window
touchstart / touchmove / touchend	touch screens	any
pointerdown / pointermove / pointerup	unified mouse+touch+pen ✓	any
transitionend / animationend	CSS transition / animation finished	any
copy / paste / cut	clipboard	inputs, document
online / offline	network status	window

EXAMPLE

```

form.addEventListener("submit", (e) => {
  e.preventDefault();           class="c">// stop the browser's default action (page reload, link
  navigation, checkbox toggle)
  e.target                       class="c">// the element that triggered the event (deepest)
  e.currentTarget               class="c">// the element the listener is attached to
  e.type                        class="c">// "submit"
  e.timeStamp
});

document.addEventListener("keydown", (e) => {
  e.key                         class="c">// "a", "Enter", "Escape", "ArrowUp", " "
  e.code                       class="c">// physical key: "KeyA", "Space"
  e.ctrlKey e.shiftKey e.altKey e.metaKey
  if (e.key === "Escape") closeModal();
  if (e.ctrlKey && e.key === "s") { e.preventDefault(); save(); }
});

canvas.addEventListener("mousemove", (e) => {
  e.clientX e.clientY         class="c">// relative to viewport
  e.pageX   e.pageY           class="c">// relative to page
  e.offsetX e.offsetY         class="c">// relative to the element
  e.button   class="c">// 0 left, 1 middle, 2 right
});

```

Bubbling, capturing and delegation

Elzero JS #127 #128

Angela \$19

When you click a button inside a card inside a list, the click event fires on the button, then bubbles up to the card, the list, body, document. This lets you put ONE listener on a parent and handle clicks from any current or future child — event delegation. Essential for lists that change.

EXAMPLE

```

class="c"> // WITHOUT delegation: must add a
listener to every button, and new buttons get
none
document.querySelectorAll(".delete").forEach(b
=> b.addEventListener("click", del));

class="c"> // WITH delegation ✓ one listener on
the parent
list.addEventListener("click", (e) => {
  const btn = e.target.closest(".delete");
class="c"> // did the click land on/inside
a .delete?
  if (!btn || !list.contains(btn)) return;
  const li = btn.closest("li");
  li.remove();
});

class="c"> // stop bubbling (rarely needed)
inner.addEventListener("click", e =>
e.stopPropagation());

class="c"> // capturing phase (runs on the way
DOWN, before bubbling) – third argument true
parent.addEventListener("click", handler,
true);

class="c"> // close dropdown when clicking
anywhere else
document.addEventListener("click", (e) => {
  if (!dropdown.contains(e.target))
dropdown.classList.remove("open");
});

```

RESULT

click on <button>

↓ target: button

↑ bubbles to: li → ul (listener here handles it) → body → document

- Use `e.target.closest(selector)` rather than `e.target.matches` — the user may click an icon inside the button.
- `focus`, `blur`, `mouseenter`, `mouseleave` do not bubble; use `focusin/focusout`, `mouseover/mouseout` for delegation.

Forms: submit, input, validation from JS

Elzero JS #129

Angela \$19

Handle a form entirely in JavaScript: prevent the reload, read the values, validate, show errors, then send with `fetch` (next chapter).

EXAMPLE

```
const form = document.querySelector("#signup");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  class="c"> // read values: by name via form.elements, or FormData
  const name = form.elements.name.value.trim();
  const email = form.elements.email.value.trim();
  const data = Object.fromEntries(new FormData(form));  class="c"> // { name, email, ... } all
  fields at once ✓

  class="c"> // validate
  const errors = [];
  if (name.length < 2) errors.push("Name is too short");
  if (!/^^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/.test(email)) errors.push("Invalid email");
  const box = form.querySelector(".errors");
  box.textContent = errors.join(". ");
  if (errors.length) return;

  class="c"> // browser validation API
  if (!form.checkValidity()) { form.reportValidity(); return; }
  input.setCustomValidity("Username taken");  class="c"> // custom message; "" to clear

  class="c"> // send (chapter H)
  await fetch("/api/signup", { method: "POST", headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data) });
  form.reset();
});

class="c"> // live feedback while typing
form.elements.password.addEventListener("input", (e) => {
  meter.value = strength(e.target.value);
});

class="c"> // character counter
textarea.addEventListener("input", () => counter.textContent = class="c"> `v`$
{textarea.value.length}/200`);
```

Timers: setTimeout, setInterval, requestAnimationFrame

Elzero JS #130

Angela \$19

Run code later or repeatedly. Always keep the returned id so you can cancel.

EXAMPLE

```
const t = setTimeout(() => console.log("after 2s"), 2000);
clearTimeout(t);                                class="c"> // cancel before it fires

const i = setInterval(tick, 1000);              class="c"> // every second
clearInterval(i);                              class="c"> // stop (do this when the component is removed!)

class="c"> // debounce: run only after the user STOPS typing for 300ms (search boxes)
let timer;
input.addEventListener("input", () => {
  clearTimeout(timer);
  timer = setTimeout(() => search(input.value), 300);
});

class="c"> // smooth animation loop (60fps, pauses in background tabs)
function frame() {
  x += 2; box.style.transform = class="v"> `translateX(${x}px)`;
  if (x < 300) requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

Page load, window and document events

Elzero JS #131 Angela §19

Run code at the right time and react to the browser itself.

EXAMPLE

```
document.addEventListener("DOMContentLoaded", init);  class="c"> // HTML ready – not needed if
script has defer or is at end of body
window.addEventListener("load", () => {});           class="c"> // everything incl. images loaded
window.addEventListener("resize", () => {});         class="c"> // throttle it!
window.addEventListener("scroll", () => {
  header.classList.toggle("shrink", window.scrollY > 100);
});
window.addEventListener("beforeunload", (e) => { e.preventDefault(); });  class="c"> // "unsaved
changes?" prompt
window.addEventListener("hashchange", () => route(location.hash));

class="c"> // reveal on scroll (better than scroll events)
const io = new IntersectionObserver((entries) => {
  entries.forEach(en => { if (en.isIntersecting) en.target.classList.add("visible"); });
}, { threshold: 0.2 });
document.querySelectorAll(".reveal").forEach(el => io.observe(el));
```

Project walkthrough: the classic beginner apps

Elzero JS trainings Gamal JS projects Angela §17-§19

Angela's Dicee, Drum Kit and Simon; Elzero's and Gamal's trainings — all use the same handful of DOM + event techniques. Build every one of these before moving on.

EXAMPLE

```

class="c"> /* 1. DICEE – random images */
const n1 = Math.floor(Math.random() * 6) + 1;
document.querySelector(".img1").setAttribute("src", class="v">`images/dice${n1}.png`);

class="c"> /* 2. DRUM KIT – click or key plays a sound and animates */
document.querySelectorAll(".drum").forEach(btn => btn.addEventListener("click", function () {
  play(this.textContent); animate(this.textContent);
}));
document.addEventListener("keydown", e => { play(e.key); animate(e.key); });
function play(key) { const a = new Audio(class="v">`sounds/${key}.mp3`); a.play(); }
function animate(key) {
  const el = document.querySelector("." + key); if (!el) return;
  el.classList.add("pressed");
  setTimeout(() => el.classList.remove("pressed"), 100);
}

class="c"> /* 3. TO-DO LIST – add, delete, toggle done, persist */
let todos = JSON.parse(localStorage.getItem("todos") || "[]");
const render = () => {
  list.innerHTML = todos.map((t, i) => class="v">`
    <li class="${t.done ? "done" : ""}" data-i="${i}">
      <span>${t.text}</span> <button class="del">x</button>
    </li>`
    ).join("");
  localStorage.setItem("todos", JSON.stringify(todos));
};
form.addEventListener("submit", e => { e.preventDefault(); todos.push({ text: input.value, done:
false }); input.value = ""; render(); });
list.addEventListener("click", e => {
  const li = e.target.closest("li"); if (!li) return;
  const i = Number(li.dataset.i);
  if (e.target.matches(".del")) todos.splice(i, 1); else todos[i].done = !todos[i].done;
  render();
});
render();

class="c"> /* 4. SIMON GAME core idea */
const colours = ["red", "blue", "green", "yellow"];
let sequence = [], userIdx = 0;
function nextLevel() { sequence.push(colours[Math.floor(Math.random() * 4)]);
flash(sequence.at(-1)); userIdx = 0; }
document.querySelectorAll(".btn").forEach(b => b.addEventListener("click", () => {
  if (b.id === sequence[userIdx]) { userIdx++; if (userIdx === sequence.length)
setTimeout(nextLevel, 800); }
  else gameOver();
}));

class="c"> /* 5. TABS */
tabs.addEventListener("click", e => {
  const tab = e.target.closest("[data-tab]"); if (!tab) return;
  tabs.querySelectorAll(".active").forEach(t => t.classList.remove("active"));
  tab.classList.add("active");
  panels.forEach(p => p.hidden = p.id !== tab.dataset.tab);
});

class="c"> /* 6. DARK MODE toggle with memory */
const saved = localStorage.getItem("theme");
if (saved) document.body.classList.toggle("dark", saved === "dark");

```

```
toggle.addEventListener("click", () => {  
  const dark = document.body.classList.toggle("dark");  
  localStorage.setItem("theme", dark ? "dark" : "light");  
});
```

? Chapter quiz

1. What does `e.preventDefault()` do on a form submit?
2. What is event delegation and why use it?
3. Which event fires on every keystroke in an input?
4. How do you stop a `setInterval`?

Answers: 1) Stops the page from reloading so JS can handle the form. 2) One listener on a parent handles events from all children, including ones added later. 3) `input`. 4) `clearInterval(id)` with the id returned by `setInterval`.

75. JavaScript Reference H — Asynchronous JavaScript, fetch and APIs

Some things take time: downloading data, reading a file, waiting for a timer. JavaScript does not stop and wait — it continues and comes back when the result is ready. Promises and `async/await` are the tools for writing that clearly. `fetch` is how the browser talks to servers and APIs.

Synchronous vs asynchronous; the event loop

Elzero JS #132-#134

Gamal JS

Angela §22

JavaScript runs one line at a time on a single thread. Slow operations (network, timers) are handed to the browser, which notifies JavaScript via a queue when they finish. Meanwhile your code keeps running — so the order of console output can surprise you.

الجافاسكريبت بتشغل سطر واحد في المرة. العمليات البطيئة (إنترنت، تايمر) بتتسلم للمتصفح، ولما تخلص بيرجع ينفذ الكود بتاعها.

EXAMPLE

```
console.log("1");
setTimeout(() => console.log("2"), 0);
class="c">// even 0ms goes to the queue
console.log("3");
class="c">// output: 1 3 2

class="c">// why callbacks exist
function getData(callback) {
  setTimeout(() => callback({ name:
"Ahmed" })), 1000); class="c">// pretend
network
}
getData(user => console.log(user.name));
console.log("this prints first");
```

RESULT

```
log "1" → setTimeout → handed to browser
→ log "3" → stack empty → queue → log "2"
```

→ Callback hell: nesting callbacks inside callbacks for sequential steps becomes unreadable. Promises fix this.

Promises

Elzero JS #135-#138

Angela §22

A Promise is an object that represents a value that will arrive later. It is pending, then either fulfilled (with a value → `.then`) or rejected (with an error → `.catch`). `.finally` runs either way. `then`s chain, and each returns a new promise.

EXAMPLE

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => {
    Math.random() > 0.2 ? resolve("data ready") : reject(new Error("failed"));
  }, 1000);
});

p.then(value => {
  console.log(value);
  class="c">// "data ready"
  return value.toUpperCase();
  class="c">// passed to the next then
})
.then(upper => console.log(upper))
.catch(err => console.error(err.message))
  class="c">// any rejection above lands here
  .finally(() => hideSpinner());

class="c">// promisify a callback API
const wait = ms => new Promise(res =>
  setTimeout(res, ms));
wait(500).then(() => console.log("half a second later"));

class="c">// combinators
Promise.all([fetchUser(),
  fetchPosts()])      class="c">// wait for ALL; rejects if any fails → [user, posts]
Promise.allSettled([...])
class="c">// wait for all; never rejects; gives status per item
Promise.race([fetch(url),
  timeout(5000)])      class="c">// first to finish wins
Promise.any([...])
class="c">// first to FULFIL
```

RESULT

```
Promise { <pending> }
├─ resolve(value) → fulfilled → .then(value)
└─ reject(error) → rejected → .catch(error)
  → .finally()
```

async / await

Elzero JS #139 #140

Gamal JS

Angela \$22

Syntax sugar over promises that reads like normal top-to-bottom code. Mark a function `async` (it now always returns a promise); inside it, `await` pauses until a promise settles and gives you the value. Errors are caught with ordinary try/catch.

async/await يتخلي الكود اللي بيستنى نتيجة يتقري زي الكود العادي سطر ورا سطر.

EXAMPLE

```

async function loadUser(id) {
  try {
    showSpinner();
    const res = await fetch(class="v">`/api/users/${id}`);      class="c">// pause here, no
blocking
    if (!res.ok) throw new Error(class="v">`HTTP ${res.status}`);  class="c">// fetch does NOT
reject on 404/500!
    const user = await res.json();                                class="c">// also async
    render(user);
    return user;                                                  class="c">// becomes the resolved value
  } catch (err) {
    showError(err.message);                                       class="c">// network error OR our throw
  } finally {
    hideSpinner();
  }
}
loadUser(1).then(u => console.log("done", u));

class="c">// sequential vs parallel
const a = await fetchA(); const b = await fetchB();              class="c">// one after another
(slow)
const [a2, b2] = await Promise.all([fetchA(), fetchB()]);        class="c">// at the same time ✓

class="c">// await in loops
for (const id of ids) { await process(id); }                     class="c">// one by one
await Promise.all(ids.map(id => process(id)));                    class="c">// all at once

class="c">// top-level await works in modules (<script type="module">)

```

⚠ Common mistake

Forgetting `await`: `const data = fetch(url)` gives a Promise object, not data. Also: `await` only works inside an async function (or a module).

fetch — GET, POST, headers, errors

Elzero JS #141-#144

Gamal JS

Angela \$30

The browser's built-in HTTP client. `fetch(url, options)` returns a promise of a Response; you then read the body as JSON, text, or blob. It only rejects on network failure — check `res.ok` yourself for HTTP errors.

Status	Meaning	What to do
200 OK / 201 Created	Success	Read the body
204 No Content	Success, empty body	Do not call .json()
400 Bad Request	You sent wrong data	Show validation message
401 Unauthorized	Not logged in / bad token	Redirect to login
403 Forbidden	Logged in but not allowed	
404 Not Found	Wrong URL or missing item	
429 Too Many Requests	Rate limited	Wait and retry
500 Server Error	Their bug	Show generic error, retry later

EXAMPLE

```

class="c"> // GET (default)
const res = await fetch("https://api.example.com/posts?limit=10");
res.status      class="c"> // 200
res.ok          class="c"> // true for 200–299
res.headers.get("content-type")
const posts = await res.json();    class="c"> // or res.text(), res.blob(), res.formData()

class="c"> // POST JSON
const res2 = await fetch("/api/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json", "Authorization": class="v"> `Bearer ${token}` },
  body: JSON.stringify({ title: "Hi", body: "Text" }),
});
const created = await res2.json();

class="c"> // PUT / PATCH / DELETE
await fetch(class="v"> `/api/posts/${id}`, { method: "DELETE" });
await fetch(class="v"> `/api/posts/${id}`, { method: "PATCH", headers: {"Content-
Type": "application/json"}, body: JSON.stringify({ done: true }) });

class="c"> // send a form incl. files
await fetch("/upload", { method: "POST", body: new FormData(form) });    class="c"> // no Content-
Type header: browser sets it

class="c"> // timeout / cancel
const ctrl = new AbortController();
setTimeout(() => ctrl.abort(), 5000);
await fetch(url, { signal: ctrl.signal });

class="c"> // reusable helper
async function api(path, options = {}) {
  const res = await fetch("https://api.example.com" + path, {
    headers: { "Content-Type": "application/json", ...options.headers },
    ...options,
    body: options.body ? JSON.stringify(options.body) : undefined,
  });
  if (!res.ok) throw new Error((await res.json()).message || res.statusText);
  return res.status === 204 ? null : res.json();
}
const user = await api("/users/1");
await api("/posts", { method: "POST", body: { title: "x" } });

```

Working with a real public API — weather app

Elzero JS #145

Gamal JS project

Angela \$30

Read the API documentation to learn the URL, required parameters (usually an API key) and the shape of the JSON. Then fetch, pick the fields you need, and put them in the DOM.

EXAMPLE

```

class="c"> // OpenWeatherMap: docs say GET /
data/2.5/weather?q={city}&appid={KEY}
&units=metric
const KEY = "your_key_here";    class="c"> //
in a real app keep keys on the server (Part 5)

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const city = input.value.trim();
  if (!city) return;
  out.textContent = "Loading...";
  try {
    const url = class="v"> `https:
api.openweathermap.org/data/2.5/weather?q=
${encodeURIComponent(city)}&appid=${KEY}
&units=metric`;
    const res = await fetch(url);
    if (res.status === 404) throw new
Error("City not found");
    if (!res.ok) throw new Error("Weather
service error");
    const data = await res.json();
    class="c"> // console.log(data) once and
read the structure: data.main.temp,
data.weather[0].description,
data.weather[0].icon
    out.innerHTML = class="v"> `
      <h2>${data.name}, ${data.sys.country}</
h2>
      
      <p class="temp">${
Math.round(data.main.temp)}°C</p>
      <p>${data.weather[0].description}</p>
      <p>Humidity ${data.main.humidity}% ·
Wind ${data.wind.speed} m/s</p>`;
    } catch (err) {
      out.textContent = err.message;
    }
  });

```

RESULT



- Free APIs to practise with: jsonplaceholder.typicode.com (fake posts/users), api.github.com/users/NAME, restcountries.com, open-meteo.com (weather, no key), dummyjson.com (products), pokeapi.co.
- `encodeURIComponent` makes user text safe inside a URL (spaces, &, Arabic).
- Never put secret API keys in front-end code — anyone can read them. Proxy through your own server (Part 5).

Try it yourself

Using <https://jsonplaceholder.typicode.com/users>, fetch the users and render a card for each with name, email and city. Show 'Loading...' first and an error message if it fails.

Solution

```
async function loadUsers() {
  const box = document.querySelector("#users");
  box.textContent = "Loading...";
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error("Failed to load");
    const users = await res.json();
    box.innerHTML = users.map(u => `
      <article class="card">
        <h3>${u.name}</h3>
        <p>${u.email}</p>
        <p>${u.address.city}</p>
      </article>` ).join("");
  } catch (e) { box.textContent = e.message; }
}
loadUsers();
```

Error handling: try/catch/finally, throw, custom errors

Elzero JS #146 #147

Angela §22

Errors stop the program unless caught. Wrap risky code in try; handle problems in catch; clean up in finally. Throw your own errors when input is invalid. Never swallow errors silently.

Built-in error	When
SyntaxError	Bad JSON, bad code
ReferenceError	Using an undeclared variable
TypeError	Calling something that is not a function; reading property of undefined/null
RangeError	Invalid array length, too much recursion
URIError	Bad decodeURI

EXAMPLE

```

try {
  const data = JSON.parse(text);           class="c">// may throw SyntaxError
  process(data);
} catch (err) {
  console.error(err.name, err.message);    class="c">// "SyntaxError" "Unexpected token..."
  console.error(err.stack);
  showToast("Could not read the file");
} finally {
  cleanup();                             class="c">// always runs
}

function divide(a, b) {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
}

class ValidationError extends Error {
  constructor(field, message) { super(message); this.name = "ValidationError"; this.field = field; }
}

try { validate(form); }
catch (e) { if (e instanceof ValidationError) markField(e.field, e.message); else throw e; }
class="c">// re-throw unknown errors

class="c">// unhandled promise rejections crash quietly – add a global net while developing
window.addEventListener("unhandledrejection", e => console.error("Unhandled:", e.reason));

```

Debugging like a professional

Elzero JS #004 #148

Angela \$22

Reading error messages and using the debugger saves hours. The message tells you the error type, a description, and the file + line number; click it in the console to jump to the code.

Tool	How
<code>console.log("label", value)</code>	Print with a label; log objects, not strings of objects
<code>console.table(array)</code>	Arrays of objects as a grid
debugger; statement	Pauses execution when DevTools is open; step through with F10/F11, inspect variables
Breakpoints	Sources tab → click a line number; conditional breakpoints on right click
Network tab	See every fetch: URL, status, request body, response body, timing
Elements tab	Live DOM; right-click an element → Break on → subtree modifications
Read the error	"Cannot read properties of undefined (reading 'name')" = something before <code>.name</code> is undefined; log that thing
Rubber duck	Explain the bug out loud line by line — you will find it

? Chapter quiz

1. Does fetch reject on a 404 response?
2. What does await do?
3. How do you run three fetches in parallel and wait for all?
4. Where should secret API keys live?

Answers: 1) No — it resolves with `res.ok` false; you must check and throw. 2) Pauses the async function until the promise settles and returns its value. 3) `await Promise.all([f1(), f2(), f3()])`. 4) On the server, never in browser code.

76. JavaScript Reference I — Modern JavaScript: Classes, Modules, Browser APIs, Patterns

ES6 (2015) and later versions modernised JavaScript. These features are what you will see in every codebase, tutorial and framework. Also included: the browser APIs you will actually use and the coding patterns that keep a growing app organised.

Classes and this

Elzero JS #149-#156

Gamal JS

Angela \$22

A class is a blueprint for objects with the same shape and behaviour. `constructor` runs when you create an instance with `new`. Methods are shared. `this` refers to the instance. Classes are the base of many frameworks and back-end code.

EXAMPLE

```

class Product {
  static count = 0;                                class="c">// belongs to the class, not instances
  #secret = "hidden";                               class="c">// private field

  constructor(name, price) {
    this.name = name;
    this.price = price;
    Product.count++;
  }
  get priceWithTax() { return this.price * 1.14; }    class="c">// getter: used like a property
  set discount(pct) { this.price *= 1 - pct / 100; }  class="c">// setter
  describe() { return class="v">`${this.name}: ${this.price} EGP`; }
  static compare(a, b) { return a.price - b.price; }
}

const bread = new Product("Bread", 5);
bread.describe();                                  class="c">// "Bread: 5 EGP"
bread.priceWithTax;                                class="c">// 5.7
bread.discount = 10;                                class="c">// price now 4.5
Product.count;                                     class="c">// 1
bread instanceof Product;                          class="c">// true

class="c">// inheritance
class Food extends Product {
  constructor(name, price, expiry) {
    super(name, price);                             class="c">// call parent constructor first
    this.expiry = expiry;
  }
  describe() { return super.describe() + class="v">` (exp ${this.expiry})`; }  class="c">//
  override + reuse
}

class="c">// this pitfalls
const fn = bread.describe; fn();                    class="c">// ✖ this is undefined (method detached)
const bound = bread.describe.bind(bread);           class="c">// ✓
btn.addEventListener("click", () => this.save());    class="c">// arrow keeps class this
btn.addEventListener("click", this.save.bind(this));

```

- Under the hood classes use prototypes; every object has a prototype chain up to `Object.prototype`. `Object.getPrototypeOf(bread) === Product.prototype`.
- Rules for `this`: in a method called as `obj.method()` → `obj`; in a plain function → `undefined` (strict) / `window`; in an arrow → whatever `this` was outside; with `bind`/`call`/`apply` → what you pass.

Modules: import and export

Elzero JS #157-#160

Angela \$22

Split code into files. Each file (module) has its own scope; you explicitly export what others may use and import what you need. In the browser, use `<script type="module">`; in Node, set `"type": "module"` in `package.json`. Bundlers (Vite) do this for you in React.

EXAMPLE

```

class="c">// utils.js
export const TAX = 0.14;
export function formatPrice(n) { return n.toFixed(2) + " EGP"; }
export default class Cart { }                                class="c">// one default per file

class="c">// main.js
import Cart, { TAX, formatPrice } from "./utils.js";          class="c">// default + named
import * as utils from "./utils.js";                          class="c">// namespace
import { formatPrice as fmt } from "./utils.js";              class="c">// rename
const { default: Lazy } = await import("./heavy.js");          class="c">// dynamic (on demand)

class="c">// index.html
<script type="module" src="main.js"></script>                  class="c">// deferred automatically, strict mode,
own scope

```

- Modules need a server (Live Server is fine); opening the file directly with file:// blocks imports.
- Relative paths must include ./ and the extension in the browser.

Destructuring, spread, rest, optional chaining, nullish — recap

Elzero JS #161-#165

Angela §22

Used everywhere in modern code, especially React. Quick reference.

EXAMPLE

```

const { name, age = 18, ...others } = user;
const [first, second = 0, ...rest] = arr;
function Card({ title, image, onDelete }) { }

const merged = { ...defaults, ...options };
const copy = [...arr];
Math.max(...numbers);
function f(...args) { }

user?.profile?.avatar ?? "/default.png";
user.list?.[0];
user.save?.();          class="c">// call only if it exists
x ||= 5;  x &&= y;  x ??= 10;  class="c">// logical assignment

class="c">// labelled template / tagged, numeric separators, BigInt
const big = 1_000_000;
const huge = 9007199254740993n;

```

Iterators, generators, Symbol

Elzero JS #166-#170

Advanced but appear in libraries. An iterable is anything for...of can loop; a generator function produces values lazily with `yield`.

EXAMPLE

```
function* idGen() { let id = 1; while (true) yield id++; }
const ids = idGen(); ids.next().value; class="c">// 1 then 2, 3...

const range = { from: 1, to: 3, *[Symbol.iterator]() { for (let i = this.from; i <= this.to; i++)
yield i; } }};
[...range] class="c">// [1, 2, 3]
for (const n of range) { }
```

Browser APIs you will use

Elzero JS #171-#180

Angela \$19

The browser exposes far more than the DOM. These are the common ones.

EXAMPLE

```
class="c">// storage
localStorage / sessionStorage (strings; JSON for objects) document.cookie = "theme=dark; max-
age=31536000; path="/;

class="c">// location & history
location.href location.pathname location.search location.hash
new URLSearchParams(location.search).get("page");
location.href = "/login"; location.reload();
history.pushState({}, "", "/new-url"); history.back();

class="c">// navigator
navigator.clipboard.writeText("copied!");
navigator.geolocation.getCurrentPosition(pos => console.log(pos.coords.latitude));
navigator.onLine; navigator.language; navigator.share({ title, url });

class="c">// observers
new IntersectionObserver(cb).observe(el); class="c">// element visible? (lazy load, infinite
scroll, reveal)
new ResizeObserver(cb).observe(el); class="c">// element size changed
new MutationObserver(cb).observe(el, { childList: true }); class="c">// DOM changed

class="c">// media & files
const file = input.files[0]; const url = URL.createObjectURL(file); img.src = url;
const reader = new FileReader(); reader.onload = e => preview.src = e.target.result;
reader.readAsDataURL(file);

class="c">// canvas 2D
const ctx = canvas.getContext("2d"); ctx.fillStyle = "red"; ctx.fillRect(10, 10, 100, 50);

class="c">// audio, notifications, fullscreen, vibration, Web Speech, WebSockets, Service Workers
(PWA) — look up as needed
new Audio("ding.mp3").play();
Notification.requestPermission().then(p => p === "granted" && new Notification("Hi"));
el.requestFullscreen();
const ws = new WebSocket("wss:"); class="c">//..."; ws.onmessage = e => console.log(e.data);
```

Organising a front-end app without a framework

Elzero JS #181-#188

Angela §22

Before React, learn to structure plain JS: keep state in one object, render from state, handle events centrally, and split into modules. This is exactly what frameworks automate.

EXAMPLE

```
class="c">// state.js
export const state = { todos: [], filter: "all" };

class="c">// render.js
export function render() {
  const visible = state.todos.filter(t => state.filter === "all" || (state.filter === "done" === t.done));
  list.innerHTML = visible.map(t => class="v"><li data-id="${t.id}" class="${t.done ? "done" : ""}>${escapeHtml(t.text)}</li>`).join("");
  counter.textContent = class="v">`${state.todos.filter(t => !t.done).length} left`;
}

class="c">// actions.js – the ONLY place state changes, then re-render
export function addTodo(text) { state.todos.push({ id: crypto.randomUUID(), text, done: false });
save(); render(); }
export function toggle(id) { const t = state.todos.find(t => t.id === id); t.done = !t.done;
save(); render(); }

class="c">// main.js – wire events
import { addTodo, toggle } from "./actions.js";
form.addEventListener("submit", e => { e.preventDefault(); addTodo(input.value); form.reset(); });
list.addEventListener("click", e => { const li = e.target.closest("li"); if (li)
toggle(li.dataset.id); });

class="c">// helpers
export const escapeHtml = s => s.replace(/&<"class="v">']/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"class="v">': "&quot;", "'": "&#39;" }[c]));
export const $ = (sel, root = document) => root.querySelector(sel);
export const $$ = (sel, root = document) => [...root.querySelectorAll(sel)];
```

→ The pattern 'state → render → events → change state → render again' is the core of React, Vue and every UI library. Learn it here in plain JS and React will feel natural.

Clean code checklist

Elzero JS #006

Angela §22

Rule	Example
Meaningful names	<code>getActiveUsers()</code> not <code>gau()</code>
Small functions, one job each	<code>validate()</code> , <code>save()</code> , <code>render()</code>
No magic numbers	<code>const MAX_RETRIES = 3</code>
const by default	let only when reassigning
Strict equality	<code>===</code> and <code>!==</code>
Early returns	<code>if (!user) return;</code>
Handle errors	<code>try/catch</code> around <code>await</code> ; check <code>res.ok</code>
No global variables	modules or an IIFE
Comment WHY, not WHAT	<code>// retry because the API is flaky on Mondays</code>
Format consistently	Prettier; 2-space indent; semicolons
Do not repeat yourself	extract a function the second time you copy code

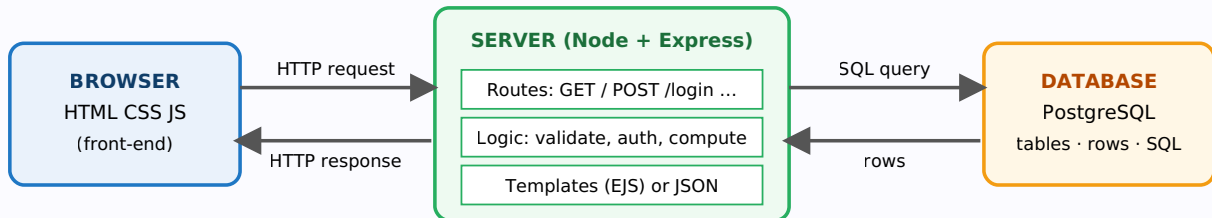
? Chapter quiz

1. What does `super()` do in a constructor?
2. How do you enable `import/export` in a browser script?
3. Which observer tells you when an element scrolls into view?
4. What is the single rule for where state should change in a plain-JS app?

Answers: 1) Calls the parent class constructor; required before using this in a subclass. 2) `script type="module"`. 3) `IntersectionObserver`. 4) Only in action functions, followed by a re-render.

77. How Back-end Works; Node.js & NPM

Angela §24-26



HTTP request = METHOD + URL + headers + (body). Response = STATUS CODE + headers + body.
 codes: 200 OK · 201 Created · 301 Redirect · 400 Bad request · 401 Unauthorized · 403 Forbidden · 404 Not found · 500 Server

Node.js lets you run JavaScript outside the browser — on a server. **NPM** (Node Package Manager) installs libraries ("packages") written by others.

```

# check install
node -v          npm -v
node app.js      # run a file
node             # REPL (interactive) - Ctrl+C twice to exit

# start a project
mkdir my-api && cd my-api
npm init -y      # creates package.json (project manifest)
npm install express # adds to dependencies + node_modules/
npm install -D nodemon # dev-only: auto-restarts server on save
npm uninstall express
npx nodemon app.js # run a package binary
  
```

```
// package.json (important parts)
{
  "name": "my-api",
  "type": "module",           // use import/
  export (ES modules)
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"   // npm run dev
  },
  "dependencies": { "express": "^4.19.0" }
}
// node_modules/ is huge → in .gitignore.
Others run `npm install` to rebuild it.
```

```
// Node built-in modules
import fs from "fs";           // file system
fs.writeFileSync("msg.txt", "Hello");
const txt = fs.readFileSync("msg.txt",
  "utf8");

import path from "path";
import { fileURLToPath } from "url";
const __dirname =
  path.dirname(fileURLToPath(import.meta.url));

// CommonJS (older style, still common):
const fs = require("fs");  module.exports =
{ fn };

// environment variables (secrets) – npm i
dotenv, create .env file
import "dotenv/config";
process.env.API_KEY;  process.env.PORT
```


78. Express.js — Server & Routing

Angela §27-29

```
// app.js – the smallest web server
import express from "express";
const app = express();
const port = 3000;

app.get("/", (req, res) => {           // when browser GETs "/"
  res.send("<h1>Hello from Express</h1>");
});

app.listen(port, () => console.log(`Server on http://localhost:${port}`));
```

HTTP METHOD (get/post/put/patch/delete) PATH — :id is a parameter

```
app.get("/users/:id", (req, res) => { ... });
```

req = incoming request (params, query, body, headers) res = what you send back

```
// ---- MIDDLEWARE: functions that run BEFORE your route, in order ----
app.use(express.static("public")); // serve /public files: css, images, client js
app.use(express.urlencoded({ extended: true })); // parse HTML form bodies → req.body
app.use(express.json()); // parse JSON bodies → req.body
app.use((req, res, next) => { // custom logger
  console.log(req.method, req.url);
  next(); // pass to the next handler – forget it and the
// request hangs!
});
// popular: morgan (logging), cors, cookie-parser, express-session, helmet

// ---- READING DATA FROM THE REQUEST ----
app.get("/users/:id", (req, res) => { // GET /users/42
  req.params.id; // "42" (route parameter)
});
app.get("/search", (req, res) => { // GET /search?q=cake&page=2
  req.query.q; req.query.page; // "cake", "2" (query string)
});
app.post("/contact", (req, res) => { // form or JSON body
  const { name, email } = req.body;
  res.redirect("/thanks");
});

// ---- SENDING RESPONSES ----
res.send("text or html");
res.json({ ok: true, data }); // API
res.status(404).send("Not found");
res.sendFile(__dirname + "/public/index.html");
res.render("home", { user }); // EJS template (next chapter)
res.redirect("/login");

// ---- ORGANISE ROUTES (routes/users.js) ----
import { Router } from "express";
const router = Router();
router.get("/", listUsers); router.post("/", createUser);
export default router;
// app.js: app.use("/users", usersRouter);

// ---- 404 + error handler (put LAST) ----
app.use((req, res) => res.status(404).render("404"));
app.use((err, req, res, next) => { console.error(err); res.status(500).send("Server error"); });
```

Postman (or Thunder Client in VS Code) lets you send GET/POST/PUT/DELETE requests to your server without a form — essential for testing APIs.

79. EJS Templating

Angela §29

EJS (Embedded JavaScript) = HTML files with JS inside, rendered on the server into final HTML. Lets you reuse headers/footers and insert data.

```
npm i ejs
// app.js
app.set("view engine", "ejs"); // looks in ./views/*.ejs automatically
app.get("/", (req, res) => {
  const todos = ["Learn EJS", "Build app"];
  res.render("index", { title: "My Todos", todos, user: null }); // pass data object
});
```

```
<!-- views/index.ejs -->
<%- include("partials/header", { title }) %>

<h1><%= title %></h1>

<% if (user) { %>
  <p>Welcome <%= user.name %></p>
<% } else { %>
  <a href="/login">Log in</a>
<% } %>

<ul>
  <% todos.forEach((t, i) => { %>
    <li><%= i + 1 %>. <%= t %></li>
  <% }) %>
</ul>

<form action="/add" method="post">
  <input name="todo" required>
  <button>Add</button>
</form>

<%- include("partials/footer") %>
```

Tag	Meaning
<%= value %>	Output, HTML-escaped (safe for user data) ☐
<%- html %>	Output raw HTML (for includes / trusted HTML)
<% code %>	Run JS, no output (if / loops)
<## comment %>	Comment
include("partials/x")	Insert another .ejs file

```
// folder layout
views/
  partials/
    header.ejs ← <!DOCTYPE> ... <body><nav>
    footer.ejs ← <footer>...</body></html>
  index.ejs
  404.ejs
public/
  styles.css ← href="/styles.css"
app.js
```

→ Alternatives with the same idea: Handlebars, Pug. React (ch. 48) moves rendering to the browser instead.

80. REST APIs — Build & Consume

Angela §30-32

A **REST API** is a server that speaks JSON instead of HTML, using HTTP methods as verbs on **resources** (nouns in the URL). The same API can serve a website, a mobile app, and other servers.

Action (CRUD)	Method	URL	Body?	Typical response
Create	POST	/api/posts	yes (new item)	201 + created item
Read all	GET	/api/posts ?page=2&tag=js	no	200 + array
Read one	GET	/api/posts/7	no	200 + item / 404
Update whole	PUT	/api/posts/7	yes (full item)	200
Update part	PATCH	/api/posts/7	yes (changed fields)	200
Delete	DELETE	/api/posts/7	no	204 / 200

```
// api.js — an in-memory REST API
import express from "express";
const app = express();
app.use(express.json());

let posts = [{ id: 1, title: "Hello", body: "First post" }];
let nextId = 2;

app.get("/api/posts", (req, res) => res.json(posts));

app.get("/api/posts/:id", (req, res) => {
  const post = posts.find(p => p.id === Number(req.params.id));
  if (!post) return res.status(404).json({ error: "Not found" });
  res.json(post);
});

app.post("/api/posts", (req, res) => {
  const { title, body } = req.body;
  if (!title) return res.status(400).json({ error: "title required" });
  const post = { id: nextId++, title, body };
  posts.push(post);
  res.status(201).json(post);
});

app.patch("/api/posts/:id", (req, res) => {
  const post = posts.find(p => p.id === Number(req.params.id));
  if (!post) return res.sendStatus(404);
  Object.assign(post, req.body); // merge changed fields
  res.json(post);
});

app.delete("/api/posts/:id", (req, res) => {
  posts = posts.filter(p => p.id !== Number(req.params.id));
  res.sendStatus(204);
});

app.listen(4000);
```

Consuming an API from a server (server → server)

```
npm i axios
import axios from "axios";
app.get("/joke", async (req, res) => {
  try {
    const r = await axios.get("https://official-joke-api.appspot.com/random_joke");
    res.render("joke", { joke: r.data }); // axios auto-parses JSON into .data
  } catch (e) { res.status(500).send(e.message); }
});

// axios.post(url, data, { headers: { Authorization: `Bearer ${TOKEN}` } })
// API auth styles: none · API key (?apiKey= or header) · Basic auth · Bearer token (OAuth/JWT)
```

CORS: a browser at `localhost:5173` calling an API at `localhost:4000` is blocked unless the API allows it: `npm i cors` → `app.use(cors())`. Server-to-server calls never hit CORS.

81. Databases: SQL & PostgreSQL

Angela §33-34

TABLE: users

id	name	email
1	Ahmed	a@mail.com
2	Sara	s@mail.com

↑ PRIMARY KEY (unique id per row)

TABLE: posts

id	title	user_id
1	Hello CSS	1
2	Grid tips	1
3	Node intro	2

↑ FOREIGN KEY user_id → users.id (one user HAS MANY pos)

Relational DB = tables linked by keys. Column = field, Row = record. SQL is the language to query them.

```
-- CREATE table (run once, e.g. in pgAdmin / psql)
CREATE TABLE users (
  id      SERIAL PRIMARY KEY,          -- auto-increment
  name     VARCHAR(50) NOT NULL,
  email    VARCHAR(100) UNIQUE NOT NULL,
  password TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);
CREATE TABLE posts (
  id      SERIAL PRIMARY KEY,
  title   TEXT NOT NULL,
  body    TEXT,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE
);
-- types: INTEGER, SERIAL, NUMERIC(10,2), TEXT, VARCHAR(n), BOOLEAN, DATE, TIMESTAMP, JSONB

-- CRUD
INSERT INTO users (name, email, password) VALUES ('Ahmed', 'a@mail.com', 'hash') RETURNING *;
SELECT * FROM users;
SELECT name, email FROM users WHERE id = 1;
SELECT * FROM posts WHERE title ILIKE '%css%' ORDER BY id DESC LIMIT 10 OFFSET 0;
UPDATE users SET name = 'Ahmed M.' WHERE id = 1;
DELETE FROM posts WHERE id = 3;
ALTER TABLE users ADD COLUMN avatar TEXT;
DROP TABLE posts;

-- JOIN: combine tables
SELECT posts.title, users.name
FROM posts
JOIN users ON posts.user_id = users.id;  -- INNER (only matches) | LEFT JOIN (all left rows)

-- aggregate
SELECT user_id, COUNT(*) FROM posts GROUP BY user_id HAVING COUNT(*) > 1;
-- COUNT SUM AVG MIN MAX · WHERE filters rows, HAVING filters groups
```

Postgres from Node (pg)

```
npm i pg
import pg from "pg";
const db = new pg.Pool({                // Pool = reusable connections
  connectionString: process.env.DATABASE_URL, // postgres://user:pass@host:5432/dbname
});

app.get("/posts", async (req, res) => {
  const result = await db.query("SELECT * FROM posts ORDER BY id DESC");
  res.render("posts", { posts: result.rows }); // rows = array of objects
});

app.post("/posts", async (req, res) => {
  const { title, body } = req.body;
  // □ PARAMETERISED query: $1 $2 – never build SQL with string concatenation (SQL injection!)
  await db.query("INSERT INTO posts (title, body, user_id) VALUES ($1, $2, $3)", [title, body,
  req.user.id]);
  res.redirect("/posts");
});
```

SQL injection: "... WHERE email = '" + email + "'" lets a user type ' OR 1=1 -- and read every row. Always use \$1 placeholders with a values array.

Alternatives you'll meet: MySQL (same SQL, small differences), SQLite (file-based, great for learning), MongoDB (NoSQL, JSON documents — Angela's older course used it with Mongoose). ORMs (Prisma, Sequelize) generate SQL for you; learn raw SQL first.

82. Authentication & Security

Angela §35

REGISTER



LOGIN



Every later request sends the cookie → server looks up the session → knows who you are (req.user / req.session.userId).


```

npm i bcrypt express-session passport passport-local
import bcrypt from "bcrypt";
import session from "express-session";

app.use(session({
  secret: process.env.SESSION_SECRET, // long random string in .env
  resave: false, saveUninitialized: false,
  cookie: { httpOnly: true, maxAge: 1000*60*60*24 }, // 1 day; add secure:true on https
}));

// REGISTER
app.post("/register", async (req, res) => {
  const { email, password } = req.body;
  const exists = await db.query("SELECT 1 FROM users WHERE email=$1", [email]);
  if (exists.rows.length) return res.status(400).send("Email already used");
  const hash = await bcrypt.hash(password, 10); // 10 = salt rounds (cost)
  const r = await db.query("INSERT INTO users (email, password) VALUES ($1,$2) RETURNING id",
[email, hash]);
  req.session.userId = r.rows[0].id; // log them in
  res.redirect("/secrets");
});

// LOGIN
app.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const r = await db.query("SELECT * FROM users WHERE email=$1", [email]);
  const user = r.rows[0];
  const ok = user && await bcrypt.compare(password, user.password);
  if (!ok) return res.status(401).send("Wrong email or password"); // same msg for both – don't
leak
  req.session.userId = user.id;
  res.redirect("/secrets");
});

// PROTECT a route with middleware
function requireLogin(req, res, next) {
  if (!req.session.userId) return res.redirect("/login");
  next();
}
app.get("/secrets", requireLogin, (req, res) => res.render("secrets"));

// LOGOUT
app.post("/logout", (req, res) => req.session.destroy(() => res.redirect("/")));

```

Levels of auth (Angela's "Secrets" app)

Level	Technique	Notes
1	Plain password in DB	❌ never
2	Encryption (reversible)	❌ key leak = all passwords leak
3	Hashing (md5/sha) — one-way	⚠️ fast → rainbow tables crack it
4	Salted, slow hash: bcrypt	✅ standard
5	Sessions + cookies (express-session / passport-local)	✅ stay logged in
6	OAuth 2.0 — "Sign in with Google" (passport-google-oauth20)	✅ you never handle their password
alt	JWT tokens (jsonwebtoken) — stateless, for APIs / mobile	sent as Authorization: Bearer <token>

Security checklist

- **Secrets in .env**, never in code or Git. Use **dotenv**.
- **Validate input** on the server (express-validator / zod) — the browser can be bypassed.
- **Parameterised SQL** (\$1) → no SQL injection. **Escape output** (EJS `<%=`) → no XSS.
- **HTTPS** in production; cookies **httpOnly** + **secure** + **sameSite**.
- **helmet** (security headers), **express-rate-limit** (brute force), **cors** with a whitelist.
- Generic error messages to users; detailed logs on the server.

83. Back-End Reference A — Node.js and npm from Zero

Until now JavaScript ran inside the browser. Node.js is a program that runs JavaScript on your computer or a server, with access to files, networks and databases. npm is its package manager: a huge library of ready-made code you can install with one command. Everything in the back-end part of this book runs on Node.

What Node.js is; running a file

Angela §24 §25

Node uses the same JavaScript language you know (variables, functions, promises) but without document or window — instead it has modules for files (fs), paths, HTTP servers, and the operating system. You run a file from the terminal.

Node.js يبشغل الجافاسكربت خارج المتصفح — على جهازك أو على السيرفر — عشان تعمل API وتتعامل مع قواعد البيانات والملفات.

```
EXAMPLE
# check the install
node -v          # v22.x
npm -v           # 10.x

# run a file
node index.js

# REPL: type JS interactively (Ctrl+C twice to exit)
node
> 2 + 2
4

# auto-restart on every save while developing (Node 18+)
node --watch index.js
```

- console.log prints to the terminal, not to a browser console.
- There is no `alert`, `document`, or `window` in Node.
- `process` is the global object about the running program: `process.argv` (command-line arguments), `process.env` (environment variables), `process.exit()`.

Modules in Node: require vs import

Angela §25

Node has two module systems. CommonJS (`require` / `module.exports`) is older and still very common in tutorials; ES modules (`import` / `export`) are the modern standard and match browser code. Pick one per project by setting `"type": "module"` in `package.json`.

	CommonJS	ES Modules
Import	<code>const x = require("x")</code>	<code>import x from "x"</code>
Export	<code>module.exports = {...}</code>	<code>export</code> / <code>export default</code>
<code>__dirname</code> , <code>__filename</code>	built-in	recreate from <code>import.meta.url</code>
Top-level <code>await</code>	No	Yes
Used in	older tutorials, many packages	modern projects, this book

EXAMPLE

```

class="c">// ----- CommonJS (default without "type": "module") -----
class="c">// math.js
function add(a, b) { return a + b; }
module.exports = { add };
class="c">// index.js
const { add } = require("./math");      class="c">// extension optional
const fs = require("fs");               class="c">// built-in
const express = require("express");     class="c">// from node_modules

class="c">// ----- ES Modules ("type": "module" in package.json) -----
class="c">// math.js
export function add(a, b) { return a + b; }
class="c">// index.js
import { add } from "./math.js";        class="c">// extension REQUIRED
import fs from "fs"; import express from "express";
import { fileURLToPath } from "url"; import path from "path";
const __dirname = path.dirname(fileURLToPath(import.meta.url)); class="c">// __dirname does not
exist in ESM

```

npm: package.json, install, scripts

Angela \$25

Every Node project has a `package.json` describing it and listing its dependencies. Installing a package downloads it into `node_modules` and records it. `node_modules` is huge and never committed to Git — anyone can recreate it with `npm install`.

EXAMPLE

```

npm init -y           # create package.json with defaults
npm install express   # add a dependency (also: npm i express)
npm install -D nodemon # dev-only dependency (-D / --save-dev)
npm install           # install everything listed (after git clone)
npm uninstall express
npm update
npm outdated
npx some-tool         # run a package binary without installing globally
npm run dev           # run a script from package.json
npm start             # shortcut for "start" script
npm ls --depth=0      # what is installed
npm audit fix         # patch known vulnerabilities

```

package.json explained

Angela §25

EXAMPLE

```

{
  "name": "bakery-api",
  "version": "1.0.0",
  "type": "module",                                     class="c">// use import/export
  "main": "index.js",
  "scripts": {
    "start": "node index.js",                           class="c">// production
    "dev": "node --watch index.js",                     class="c">// development: restarts on save
    "test": "node --test"
  },
  "dependencies": {                                     class="c">// needed to RUN
    "express": "^4.19.2",
    "pg": "^8.11.0"
  },
  "devDependencies": {                                 class="c">// needed only while DEVELOPING
    "nodemon": "^3.1.0"
  }
}

```

- `^4.19.2` = any 4.x.x $\geq$ 4.19.2 (no major jump). `~4.19.2` = only 4.19.x. Exact: `4.19.2`.
- package-lock.json pins the exact versions that were installed; commit it.
- Add `node_modules/` and `.env` to .gitignore before your first commit.

Built-in modules: fs, path, os, events

Angela §25

Node ships with modules for the operating system. fs reads and writes files (use the promise versions), path builds file paths that work on Windows and Linux.

EXAMPLE

```
import fs from "fs/promises";
import path from "path";
import os from "os";

class=c">// read & write text files
const text = await fs.readFile("notes.txt", "utf8");
await fs.writeFile("out.txt", "Hello\n"); class=c">// overwrite
await fs.appendFile("log.txt", class=v">`${new Date().toISOString()} visit\n`);

class=c">// JSON as a tiny database
const todos = JSON.parse(await fs.readFile("todos.json", "utf8"));
todos.push({ id: Date.now(), text: "new" });
await fs.writeFile("todos.json", JSON.stringify(todos, null, 2));

class=c">// folders
await fs.mkdir("uploads", { recursive: true });
const files = await fs.readdir("uploads");
await fs.unlink("old.txt"); class=c">// delete
const stat = await fs.stat("file.txt"); stat.size; stat.isDirectory();
try { await fs.access("x.txt"); } catch { class=c">/* does not exist */ }

class=c">// paths (never concatenate with "/" yourself)
path.join(__dirname, "public", "index.html") class=c">// C:\app\public\index.html or /app/
public/index.html
path.extname("photo.jpg") class=c">// ".jpg"
path.basename("/a/b/c.txt") class=c">// "c.txt"
path.resolve("uploads") class=c">// absolute path

os.platform(); os.cpus().length; os.homedir(); os.tmpdir();

class=c">// sync versions exist (readFileSync) – fine for startup config, not inside request
handlers
```

 Try it yourself

Write a script that reads a text file, counts its words and lines, and writes a report.json with the numbers.

Solution

```
import fs from "fs/promises";
const t = await fs.readFile(process.argv[2], "utf8");
const report = { lines: t.split("\n").length, words: t.trim().split(/\s+/).length, chars:
t.length };
await fs.writeFile("report.json", JSON.stringify(report, null, 2));
console.log(report);
class=c">// run: node count.js notes.txt
```

A raw HTTP server (to understand what Express hides)

Angela \$26

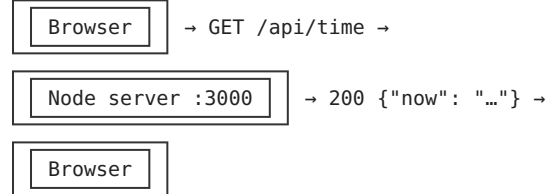
Node can serve web pages with the http module alone. You get a request object (URL, method, headers, body) and a response object (status, headers, body). Express is a thin layer that makes this pleasant.

EXAMPLE

```
import http from "http";

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);
  class="c"> // GET /about
  if (req.url === "/" ) {
    res.writeHead(200, { "Content-Type":
"text/html" });
    res.end("<h1>Home</h1>");
  } else if (req.url === "/api/time") {
    res.writeHead(200, { "Content-Type":
"application/json" });
    res.end(JSON.stringify({ now: new
Date() }));
  } else {
    res.writeHead(404); res.end("Not found");
  }
});
server.listen(3000, () =>
console.log("http: class="c"> //
localhost:3000));
```

RESULT



- Port 3000 is a convention for development. Only one program can use a port at a time — **EADDRINUSE** means something else is already listening.
- Stop the server with Ctrl+C in the terminal.

Environment variables and .env

Angela §31

Secrets (database passwords, API keys) and settings that differ per machine (port) must not be hard-coded. Put them in a .env file (ignored by Git) and read them from process.env.

EXAMPLE

```
# .env (never commit this file)
PORT=3000
DATABASE_URL=postgres://user:pass@localhost:5432/bakery
JWT_SECRET=change_me_to_a_long_random_string
WEATHER_KEY=abc123

# .gitignore
node_modules/
.env
```

- Node 20+: `node --env-file=.env index.js`. Older: `npm i dotenv` then `import "dotenv/config"` at the very top.

- Read: `const port = process.env.PORT || 3000;` — values are always strings.
- Commit a `.env.example` with the keys but fake values so teammates know what to set.
- On hosting platforms (Render, Railway, Vercel) you set these in the dashboard instead of a file.

? Chapter quiz

1. Which command creates package.json?
2. Why is node_modules not committed to Git?
3. Difference between dependencies and devDependencies?
4. Where should a database password live?

Answers: 1) `npm init -y`. 2) It is huge and fully reproducible from package.json with `npm install`. 3) dependencies are needed to run the app; devDependencies only while developing (nodemon, test tools). 4) In `.env` / environment variables, never in code.

84. Back-End Reference B — Express.js Complete

Express is the most used Node web framework. It handles routing (which code answers which URL), middleware (code that runs for every request: logging, parsing JSON, authentication), static files, templates and errors. Almost every Node tutorial and job uses it.

Hello Express

Angela §26

Install, create an app, define a route, listen on a port. Four lines and you have a web server.

Express إطار عمل بسيط عمل سيرفر: بتقوله لو حد طلب الرابط ده، رد عليه بكذا.

EXAMPLE

```
class="c">// npm init -y && npm i express
(set "type": "module")
import express from "express";
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.send("<h1>Hello from Express</h1>");
});

app.get("/about", (req, res) => {
  res.send("About page");
});

app.listen(PORT, () =>
console.log(class="v">`Server running on
http: class="c">//localhost:${PORT}`));
```

RESULT

localhost:3000

- `app.get(path, handler)` : when a GET request arrives for path, run handler.
- `req` = the request (what the client sent). `res` = the response you build.
- Every handler MUST send a response (send, json, render, redirect, end) or the browser waits forever.

HTTP methods, routes and route parameters

Angela §26 §29

Each route is a method + path. Dynamic segments start with a colon and appear in `req.params`. Query strings (`?page=2`) appear in `req.query`. The JSON body of POST/PUT requests appears in `req.body` after enabling the JSON parser.

Where data arrives	Example request	Access in Express
URL path parameter	GET /users/7	<code>req.params.id</code>
Query string	GET /users?role=admin	<code>req.query.role</code>
JSON body	POST /users {"name":"A"}	<code>req.body.name</code> (after <code>express.json()</code>)
Form body	POST from an HTML form	<code>req.body.name</code> (after <code>express.urlencoded()</code>)
Headers	Authorization: Bearer x	<code>req.headers.authorization</code> / <code>req.get("Authorization")</code>
Cookies	Cookie: token=abc	<code>req.cookies.token</code> (<code>cookie-parser</code>)
Uploaded file	multipart/form-data	<code>req.file</code> (<code>multer</code>)

EXAMPLE

```

app.use(express.json()); // parse JSON bodies ← required for req.body
app.use(express.urlencoded({ extended: true })); // parse HTML form bodies

app.get("/products", (req, res) => {
  const { page = 1, sort } = req.query; // /products?page=2&sort=price
  res.json({ page, sort });
});

app.get("/products/:id", (req, res) => { // /products/42
  const id = Number(req.params.id);
  const product = products.find(p => p.id === id);
  if (!product) return res.status(404).json({ error: "Not found" });
  res.json(product);
});

app.get("/users/:userId/orders/:orderId", (req, res) => { req.params.userId;
  req.params.orderId; });

app.post("/products", (req, res) => {
  const { name, price } = req.body; // JSON sent by the client
  if (!name || price == null) return res.status(400).json({ error: "name and price required" });
  const product = { id: Date.now(), name, price };
  products.push(product);
  res.status(201).json(product); // 201 Created
});

app.put("/products/:id", (req, res) => { // replace whole item */ });
app.patch("/products/:id", (req, res) => { // update some fields */ });
app.delete("/products/:id", (req, res) => {
  products = products.filter(p => p.id !== Number(req.params.id));
  res.status(204).end(); // No Content
});

app.all("/secret", handler); // any method
app.get("/files/*", handler); // wildcard
app.get(/^\/report-\d+$/, handler); // regex

```

Response methods

Angela §26

EXAMPLE

```
res.send("text or <b>html</b>")      class="c"> // sets Content-Type automatically
res.json({ ok: true })              class="c"> // JSON
res.status(404).send("Nope")        class="c"> // set status then body
res.sendStatus(204)                  class="c"> // status + default text
res.redirect("/login")               class="c"> // 302
res.redirect(301, "/new-url")
res.render("home", { title: "Hi" }) class="c"> // template (EJS)
res.sendFile(path.join(__dirname, "public", "index.html"))
res.download("report.pdf")
res.set("Cache-Control", "no-store"); res.type("png"); res.cookie("theme", "dark", { httpOnly:
true });
res.end()
```

⚠ Common mistake

Cannot set headers after they are sent — you responded twice, usually because you forgot `return` before `res.status(404).json(...)` and then the code continued to another `res.json`.

Middleware — the heart of Express

Angela §27

A middleware is a function `(req, res, next)` that runs for every request (or every request under a path) before the route. It can log, parse, authenticate, add data to req, or end the response early. Call `next()` to pass control to the next one. Order matters: middleware runs top to bottom.

الـ middleware فانكشن بتننفذ على كل طلب قبل الـ route: بتسجل، بتتحقق من تسجيل الدخول، بتقرأ البيانات. لازم تنادي next() عشان الطلب يكمل.

EXAMPLE

```

class="c"> // custom logger
function logger(req, res, next) {
  console.log(new Date().toISOString(),
    req.method, req.url);
  next();
}
class="c"> // continue; without this the
request hangs
}
app.use(logger);
class="c"> // for every request

class="c"> // built-in
app.use(express.json());
app.use(express.static("public"));
class="c"> // serve files in /public at /
(style.css, images...)
app.use("/assets",
  express.static("public")); class="c"> // or
under a prefix

class="c"> // third-party (npm i morgan cors
helmet cookie-parser)
import morgan from "morgan";
app.use(morgan("dev")); class="c"> //
request logs
import cors from "cors";
app.use(cors({ origin: "http://localhost:5173" })); // allow the React dev
server
import helmet from "helmet";
app.use(helmet()); class="c"> //
security headers
import cookieParser from "cookie-parser";
app.use(cookieParser());

class="c"> // only for some paths
app.use("/admin", requireAdmin);

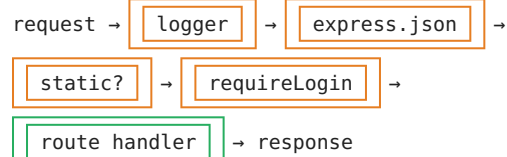
class="c"> // per-route middleware chain
app.get("/dashboard", requireLogin, (req, res)
=> res.render("dashboard", { user:
req.user }));

class="c"> // middleware that adds data
app.use((req, res, next) => { req.requestTime
= Date.now(); next(); });

class="c"> // middleware that ends early
function requireLogin(req, res, next) {
  if (!req.session?.userId) return
res.redirect("/login");
  next();
}

```

RESULT



→ Static files first, then API routes, then the 404 handler, then the error handler — that order.

- A request that never gets a response and never calls `next()` just hangs; the browser spins forever.

Organising with `express.Router` and folders

Angela §27 §29

Do not put 40 routes in one file. Group routes by resource in `routes/`, keep logic in `controllers/`, database code in `models/` or `db/`.

EXAMPLE

```
class="c">
// project layout
bakery-api/
├ index.js           ← app setup, middleware, mount routers, listen
├ routes/
│   ├── products.js
│   └── users.js
├ controllers/
│   └── products.js   ← functions (req, res) with the logic
├ db/
│   └── index.js      ← database connection
├ middleware/
│   └── auth.js
├ views/             ← EJS templates
├ public/             ← css, images, client js
├ .env .gitignore package.json

class="c">
// routes/products.js
import { Router } from "express";
import * as ctrl from "../controllers/products.js";
const router = Router();
router.get("/", ctrl.list);
router.get("/:id", ctrl.getOne);
router.post("/", ctrl.create);
router.put("/:id", ctrl.update);
router.delete("/:id", ctrl.remove);
export default router;

class="c">
// controllers/products.js
export async function list(req, res, next) {
  try { res.json(await db.getProducts()); } catch (e) { next(e); } }
// pass errors to error handler
}

class="c">
// index.js
import productsRouter from "../routes/products.js";
app.use("/api/products", productsRouter);
class="c">
// GET /api/products, GET /api/products/:id ...
```

Error handling and 404s

Angela §27

A 404 handler is a normal middleware placed after all routes. An error handler has FOUR parameters (err, req, res, next) and goes last. Async errors must be passed to next(err) or thrown inside a wrapper (Express 5 catches them automatically).

EXAMPLE

```
class="c">// 404 – after all routes
app.use((req, res) => {
  res.status(404).json({ error: class="v">`Route ${req.method} ${req.url} not found` });
  class="c">// or res.status(404).render("404")
});

class="c">// error handler – 4 params, LAST
app.use((err, req, res, next) => {
  console.error(err);
  const status = err.status || 500;
  res.status(status).json({ error: status === 500 ? "Something went wrong" : err.message });
});

class="c">// creating errors with status
class HttpError extends Error { constructor(status, msg) { super(msg); this.status = status; } }
if (!product) throw new HttpError(404, "Product not found"); class="c">// inside async handler
(Express 5) or wrap:

class="c">// async wrapper for Express 4
const wrap = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
router.get("/", wrap(async (req, res) => { res.json(await db.all()); }));
```

Serving a front-end + API from one Express app

Angela §27 §30

A common setup: Express serves your static HTML/CSS/JS (or the built React app) AND answers /api routes. The browser code calls the API with relative URLs, so there is no CORS problem.

EXAMPLE

```
app.use(express.json());
app.use(express.static(path.join(__dirname, "public"))); class="c">// index.html, app.js,
style.css

app.get("/api/todos", (req, res) => res.json(todos));
app.post("/api/todos", (req, res) => { todos.push(req.body); res.status(201).json(req.body); });

class="c">// SPA fallback: any other GET returns index.html so client-side routing works
app.get("*", (req, res) => res.sendFile(path.join(__dirname, "public", "index.html")));

class="c">// public/app.js
const todos = await (await fetch("/api/todos")).json(); class="c">// same origin → no CORS
```

File uploads, validation, rate limiting, compression

Angela §27

Common production add-ons, each a small npm package.

EXAMPLE

```
npm i multer express-validator express-rate-limit compression

# multer — uploads
import multer from "multer";
const upload = multer({ dest: "uploads/", limits: { fileSize: 2e6 } });
app.post("/avatar", upload.single("avatar"), (req, res) => res.json({ file: req.file.filename }));

# express-validator — input validation
import { body, validationResult } from "express-validator";
app.post("/signup",
  body("email").isEmail().normalizeEmail(),
  body("password").isLength({ min: 8 }),
  (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });
    // ...
  });

# rate limit — stop brute force
import rateLimit from "express-rate-limit";
app.use("/api/login", rateLimit({ windowMs: 15 * 60 * 1000, max: 10 }));

# compression — gzip responses
import compression from "compression"; app.use(compression());
```

Testing your API without a front-end

Angela \$29

Use Postman, Thunder Client (VS Code extension), or curl to send any method with any body and inspect the response.

EXAMPLE

```
curl http://localhost:3000/api/products
curl http://localhost:3000/api/products/1
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Cake","price":40}'
curl -X DELETE http://localhost:3000/api/products/1 -i # -i shows status + headers
```

- In Postman: pick the method, type the URL, Body → raw → JSON, Send. Save requests in a collection.
- Watch the server terminal at the same time: morgan shows each request and status.

? Chapter quiz

1. What must you enable to read `req.body` from JSON requests?
2. What happens if a middleware neither responds nor calls `next()`?
3. How many parameters does an error-handling middleware have?
4. Where do URL query strings (`?page=2`) appear?

Answers: 1) `app.use(express.json())`. 2) The request hangs forever. 3) Four: (`err`, `req`, `res`, `next`). 4) `req.query`.

86. Back-End Reference C — EJS

Templates and Server-Rendered Pages

Before single-page apps, and still today for many sites, the server builds the HTML with real data and sends the finished page. EJS (Embedded JavaScript) lets you write normal HTML with small JavaScript tags for variables, loops and conditions. Angela's Blog project and Secrets project use this approach.

Setup and first template

Angela §28

Install ejs, tell Express to use it, put templates in a views/ folder, and render with data.

EXAMPLE

```
class="c">// npm i ejs
app.set("view engine", "ejs");           class="c">// Express finds views/*.ejs
app.set("views", path.join(__dirname, "views"));
app.use(express.static("public"));

app.get("/", (req, res) => {
  res.render("home", {                   class="c">// views/home.ejs
    title: "Giza Bakery",
    products: [{ name: "Bread", price: 5 }, { name: "Cake", price: 40 }],
    user: null,
  });
});
```

EJS tags

Angela §28

Five tags do everything. `<%= %>` outputs a value safely (HTML-escaped). `<% %>` runs JavaScript without output (if, for). `<%- %>` outputs raw HTML (only for trusted content or includes).

`<%= %>` بتطبع قيمة بأمان، `<% %>` بتنفيذ كود من غير طباعة (زي if و for)، `<%- %>` بتطبع HTML خام.

Tag	Meaning
<%= value %>	Print value, HTML-escaped (safe)
<%- html %>	Print raw HTML (includes, trusted content)
<% code %>	Run JavaScript: if / for / const
<%# comment %>	Comment
<%- include("file", {data}) %>	Insert another template
<%_ _%> / -%>	Trim whitespace

EXAMPLE

```
<!-- views/home.ejs -->
<%- include("partials/header", { title }) %>

<h1><%= title %></h1>                                <!--
output, escaped -->

<% if (user) { %>                                     <!--
logic, no output -->
  <p>Welcome back, <%= user.name %></p>
<% } else { %>
  <a href="/login">Log in</a>
<% } %>

<ul>
  <% products.forEach(p => { %>

<li><%= p.name %> - <%= p.price.toFixed(2) %>
EGP</li>
  <% }) %>
</ul>
<% if (products.length === 0) { %><p>No
products yet.</p><% } %>

<%- post.htmlBody %>
<!-- raw HTML: only if YOU generated it -->
<%# this is a comment, not sent to the browser
%>

<%- include("partials/footer") %>
```

RESULT

Giza Bakery

[Log in](#)

- Bread — 5.00 EGP
- Cake — 40.00 EGP

⚠ Common mistake

Using a variable in the template you did not pass in render → **ReferenceError: user is not defined**. Always pass every variable (pass `user: null` when there is none), or use `locals.user`.

Layouts with partials

Angela \$28

Repeat the header, navigation and footer once by putting them in views/partials and including them in every page.

EXAMPLE

```
<!-- views/partials/header.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><%= title %> | Giza Bakery</title>
  <link rel="stylesheet" href="/css/style.css">    <!-- from public/ ; absolute path with / -->
</head>
<body>
<nav>
  <a href="/">Home</a> <a href="/products">Products</a>
  <% if (locals.user) { %><a href="/logout">Log out</a><% } %>
</nav>
<main>

<!-- views/partials/footer.ejs -->
</main>
<footer>© <%= new Date().getFullYear() %> Giza Bakery</footer>
<script src="/js/main.js"></script>
</body>
</html>
```

- Use root-relative URLs (`/css/style.css`) in templates so they work from any route depth like `/products/7`.
- `locals` holds all variables passed to render; `locals.user` is safe even when user was not passed.
- Make the logged-in user available everywhere:


```
app.use((req,res,next)=>{ res.locals.user = req.session.user; next(); }).
```

Forms with EJS: create, edit, delete

Angela §28 §29

HTML forms can only send GET or POST. For a classic server-rendered CRUD app you post to distinct URLs and redirect after success (Post-Redirect-Get pattern) so a refresh does not resubmit.

EXAMPLE

```

<!-- views/new-post.ejs -->
<form action="/posts" method="POST">
  <label>Title <input name="title" required></label>
  <label>Body <textarea name="body" rows="6" required></textarea></label>
  <button>Publish</button>
</form>

<!-- views/post.ejs - edit + delete -->
<form action="/posts/<%= post.id %>/edit" method="POST">
  <input name="title" value="<%= post.title %>">
  <textarea name="body"><%= post.body %></textarea>
  <button>Save</button>
</form>
<form action="/posts/<%= post.id %>/delete" method="POST" onsubmit="return confirm('Delete?')">
  <button class="danger">Delete</button>
</form>

<!-- server -->
app.use(express.urlencoded({ extended: true }));
app.post("/posts", (req, res) => { const { title, body } = req.body; posts.push({ id: Date.now(), title, body }); res.redirect("/"); });
app.post("/posts/:id/edit", (req, res) => { const p = posts.find(p => p.id == req.params.id); Object.assign(p, req.body); res.redirect("/posts/" + p.id); });
app.post("/posts/:id/delete", (req, res) => { posts = posts.filter(p => p.id != req.params.id); res.redirect("/"); });

```

- The `name` attribute of each input becomes the key in `req.body`.
- Alternative: `npm i method-override` to use PUT/DELETE from forms with `?_method=DELETE`.

Project: Blog website (Angela capstone) — full flow

Angela \$28 project

Routes and templates for a complete blog with create, read, update, delete, stored in memory (swap for a database in chapter E).

EXAMPLE

```

class="c">// index.js
let posts = [];
app.get("/", (req, res) => res.render("index",
{ title: "Blog", posts }));
app.get("/new", (req, res) =>
res.render("new", { title: "New post" }));
app.post("/posts", (req, res) =>
{ posts.unshift({ id: Date.now(), ...req.body,
date: new Date() }); res.redirect("/"); });
app.get("/posts/:id", (req, res) => {
  const post = posts.find(p => p.id ==
req.params.id);
  if (!post) return
res.status(404).render("404", { title: "Not
found" });
  res.render("post", { title: post.title,
post });
});
app.get("/posts/:id/edit", (req, res) =>
res.render("edit", { title: "Edit", post:
posts.find(p => p.id == req.params.id) }));
app.post("/posts/:id/edit", (req, res) =>
{ Object.assign(posts.find(p => p.id ==
req.params.id), req.body);
res.redirect(class="v">`/posts/$
{req.params.id}`); });
app.post("/posts/:id/delete", (req, res) =>
{ posts = posts.filter(p => p.id !=
req.params.id); res.redirect("/"); });

<!-- views/index.ejs -->
<% posts.forEach(p => { %>
  <article class="card">
    <h2><a href="/posts/<%= p.id %>"><%=
p.title %></a></h2>
    <time><%= p.date.toLocaleDateString("en-
GB") %></time>
    <p><%= p.body.slice(0, 120) %>...</p>
  </article>
<% }) %>

```

RESULT

My first post

07/09/2026

Today I learned EJS and built a blog...

Hello world

06/09/2026

Welcome to my blog...

? Chapter quiz

1. Which EJS tag prints a value safely?
2. How do you reuse a header across pages?
3. Why redirect after handling a POST form?
4. Which middleware parses HTML form submissions?

Answers: 1) <%= value %>. 2) Put it in views/partials/header.ejs and include it. 3) So a page refresh does not resubmit the form (Post-Redirect-Get). 4) express.urlencoded({ extended: true }).

87. Back-End Reference D — REST APIs and HTTP in Depth

An API (Application Programming Interface) is a server that answers with data (JSON) instead of pages, so any client — a React app, a mobile app, another server — can use it. REST is the set of conventions for designing those URLs and methods. Knowing HTTP well is what separates juniors from mid-level developers.

How HTTP works: request and response anatomy

Angela \$29 \$30

Every interaction is one request and one response, both plain text. Request: method, path, headers, optional body. Response: status code, headers, optional body.

كل تعامل على الويب عبارة عن طلب (method + رابط + body + headers) ورد (status + headers + body).

Header	Direction	Meaning
Content-Type	both	format of the body: application/json, text/html, multipart/form-data
Accept	request	formats the client can read
Authorization	request	credentials: Bearer token, Basic
Cookie / Set-Cookie	req / res	session data
Cache-Control	response	how long to cache
Location	response	where the new resource is (201) or redirect target (3xx)
Access-Control-Allow-Origin	response	CORS: which sites may call this API from a browser
User-Agent	request	browser/app name
Content-Length	both	body size in bytes

EXAMPLE

REQUEST

```
POST /api/products HTTP/1.1
Host: api.bakery.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
```

```
{"name": "Cake", "price": 40}
```

RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/products/17
Cache-Control: no-store
```

```
{"id": 17, "name": "Cake", "price": 40}
```

REST conventions

Angela §29

URLs name resources (nouns, plural); HTTP methods say what to do (verbs). Stateless: each request carries everything needed (token), the server keeps no memory between requests.

Action	Method	URL	Body	Success status
List all	GET	/api/products	—	200 + array
Get one	GET	/api/products/17	—	200 + object / 404
Create	POST	/api/products	new object	201 + created object
Replace	PUT	/api/products/17	full object	200
Update partially	PATCH	/api/products/17	changed fields	200
Delete	DELETE	/api/products/17	—	204 (no body)
Nested	GET	/api/users/3/orders	—	200
Filter / sort / page	GET	/api/products?category=bread&sort=-price&page=2&limit=20	—	200
Search	GET	/api/products?q=choc	—	200

- Nouns not verbs: `/products` ✓, `/getProducts` ✗.
- GET must never change data (safe); PUT/DELETE repeated give the same result (idempotent); POST creates each time.
- Version your API when public: `/api/v1/products`.

→ Return consistent JSON shapes: `{ data: [...], total: 120, page: 2 }` and errors as `{ error: "message" }`.

Status codes you must know

Angela §29

Code	Name	Use it when
200	OK	Successful GET/PUT/PATCH
201	Created	Successful POST that created something
204	No Content	Successful DELETE (no body)
301 / 302	Moved / Found	Permanent / temporary redirect
304	Not Modified	Cached copy is still good
400	Bad Request	Validation failed, malformed JSON
401	Unauthorized	No or invalid credentials (not logged in)
403	Forbidden	Logged in but not allowed
404	Not Found	No such route or item
409	Conflict	Duplicate (email already registered)
422	Unprocessable Entity	Valid JSON but semantic errors
429	Too Many Requests	Rate limit
500	Internal Server Error	Your bug / unhandled exception
502 / 503	Bad Gateway / Unavailable	Upstream down / maintenance

Building a complete REST API (in-memory)

Angela §29

A full products API with validation, filtering, pagination and correct status codes. Later chapters swap the array for a database.

EXAMPLE

```

import express from "express";
const app = express();
app.use(express.json());

let products = [{ id: 1, name: "Bread", price: 5, category: "bakery" }];
let nextId = 2;

function validate(body) {
  const errors = [];
  if (!body.name || typeof body.name !== "string") errors.push("name is required");
  if (typeof body.price !== "number" || body.price < 0) errors.push("price must be a positive number");
  return errors;
}

app.get("/api/products", (req, res) => {
  let result = [...products];
  const { category, q, sort, page = 1, limit = 10 } = req.query;
  if (category) result = result.filter(p => p.category === category);
  if (q) result = result.filter(p => p.name.toLowerCase().includes(q.toLowerCase()));
  if (sort) { const desc = sort.startsWith("-"); const key = sort.replace("-", "");
  result.sort((a, b) => (a[key] > b[key] ? 1 : -1) * (desc ? -1 : 1)); }
  const start = (page - 1) * limit;
  res.json({ data: result.slice(start, start + Number(limit)), total: result.length, page: Number(page) });
});

app.get("/api/products/:id", (req, res) => {
  const p = products.find(p => p.id === Number(req.params.id));
  if (!p) return res.status(404).json({ error: "Product not found" });
  res.json(p);
});

app.post("/api/products", (req, res) => {
  const errors = validate(req.body);
  if (errors.length) return res.status(400).json({ errors });
  const p = { id: nextId++, ...req.body };
  products.push(p);
  res.status(201).location(class="v">`/api/products/${p.id}`).json(p);
});

app.patch("/api/products/:id", (req, res) => {
  const p = products.find(p => p.id === Number(req.params.id));
  if (!p) return res.status(404).json({ error: "Product not found" });
  Object.assign(p, req.body);
  res.json(p);
});

app.delete("/api/products/:id", (req, res) => {
  const before = products.length;
  products = products.filter(p => p.id !== Number(req.params.id));
  if (products.length === before) return res.status(404).json({ error: "Product not found" });
  res.status(204).end();
});

app.use((req, res) => res.status(404).json({ error: "Not found" }));
app.use((err, req, res, next) => { console.error(err); res.status(500).json({ error: "Server

```

```
error" }); });
app.listen(3000);
```

Try it yourself

Add a `/api/products/stats` route that returns the count, the average price and the most expensive product. Then add PUT that requires all fields.

Solution

```
app.get("/api/products/stats", (req, res) => {  class="c"> // put BEFORE /:id or "stats" is
  treated as an id
  const avg = products.reduce((s, p) => s + p.price, 0) / (products.length || 1);
  const max = products.reduce((m, p) => p.price > m.price ? p : m, products[0]);
  res.json({ count: products.length, avg, max });
});
```

Calling other APIs from the server (axios / fetch)

Angela §30

The server can fetch third-party APIs too — and this is where secret keys belong. Your front-end calls your server; your server calls the paid API.

EXAMPLE

```
class="c"> // npm i axios    (or use built-in fetch in Node 18+)
import axios from "axios";

app.get("/api/weather/:city", async (req, res, next) => {
  try {
    const { data } = await axios.get("https://api.openweathermap.org/data/2.5/weather",
  {
    params: { q: req.params.city,appid: process.env.WEATHER_KEY, units: "metric" },
  });
    res.json({ city: data.name, temp: data.main.temp, desc: data.weather[0].description });
  } catch (err) {
    if (err.response?.status === 404) return res.status(404).json({ error: "City not found" });
    next(err);
  }
});

class="c"> // axios basics
axios.get(url, { params, headers })
axios.post(url, body, { headers })
axios.put / patch / delete
const api = axios.create({ baseURL: "https://api.x.com", headers: { Authorization:
class="v"> `Bearer ${KEY}` } });
class="c"> // axios throws on 4xx/5xx (unlike fetch) and parses JSON automatically
```

CORS explained

Angela §30

Browsers block JavaScript on site A from reading responses of site B unless B says it is allowed. If your React app on localhost:5173 calls your API on localhost:3000, that is cross-origin. Fix on the SERVER with the cors package (or a proxy). It is never a client-side bug.

EXAMPLE

```
class="c"> // server
import cors from "cors";
app.use(cors());
app.use(cors({ origin: ["http:localhost:5173", "https://myapp.com"], credentials:
true })); // specific

class="c"> // allow everyone (public API)

class="c"> // Vite dev proxy alternative (vite.config.js) → front-end calls /api, Vite forwards
to :3000
export default { server: { proxy: { "/api": "http:localhost:3000" } } };
```

⚠ Common mistake

Error in console: **No 'Access-Control-Allow-Origin' header is present.** It is the server that must send the header. Adding headers in fetch does not help.

Authentication for APIs: JWT

Angela §32

Stateless APIs use a JSON Web Token: on login the server signs a token containing the user id; the client stores it and sends it in the Authorization header; the server verifies the signature on every request. Details of hashing passwords are in chapter F.

EXAMPLE

```

class="c"> // npm i jsonwebtoken
import jwt from "jsonwebtoken";

class="c"> // login → issue token
app.post("/api/login", async (req, res) => {
  const user = await findUserAndCheckPassword(req.body.email, req.body.password);
  if (!user) return res.status(401).json({ error: "Invalid credentials" });
  const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn:
"7d" });
  res.json({ token });
});

class="c"> // middleware → verify
function auth(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: "Login required" });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ error: "Invalid or expired token" }); }
}
const adminOnly = (req, res, next) => req.user.role === "admin" ? next() :
res.status(403).json({ error: "Admins only" });

app.get("/api/me", auth, (req, res) => res.json(req.user));
app.delete("/api/products/:id", auth, adminOnly, remove);

class="c"> // client
localStorage.setItem("token", token);
fetch("/api/me", { headers: { Authorization: class="v"> `Bearer ${localStorage.getItem("token")}`
` } });

```

- A JWT is three base64 parts: header.payload.signature. Anyone can READ the payload (do not put secrets in it); only the server can VERIFY it.
- Safer storage for browsers: httpOnly cookie instead of localStorage (immune to XSS).

? Chapter quiz

1. Which method and status for creating a resource?
2. What does 401 mean versus 403?
3. Where is CORS fixed — client or server?
4. What is inside a JWT and who can read it?

Answers: 1) POST → 201 Created. 2) 401: not authenticated; 403: authenticated but not allowed. 3) Server. 4) header.payload.signature; anyone can read the payload, only the server can verify the signature.

88. Back-End Reference E — SQL and PostgreSQL

Real applications keep data in a database so it survives restarts and can be searched by millions of rows. SQL (Structured Query Language) is the standard language for relational databases; PostgreSQL is a free, powerful one used by Angela's course and countless companies. The same SQL works, with tiny changes, in MySQL and SQLite.

Relational databases in plain words

Angela §31

Data lives in tables (like spreadsheets): each row is one record, each column one field with a fixed type. Every row has a primary key (unique id). Tables link to each other through foreign keys (a column holding another table's id). This avoids duplicating data.

قاعدة البيانات جداول: كل صف سجل، كل عمود حقل. كل صف له id فريد، والجداول بتتربط عن طريق الـ id ده.

Term	Meaning
Table	A collection of rows with the same columns (users, products)
Row / record	One item
Column / field	One attribute with a type
Primary key (PK)	Unique identifier of a row, usually id SERIAL
Foreign key (FK)	Column referencing another table's PK
Schema	The set of tables and their columns
Query	A SQL command
Index	A lookup structure that makes searches on a column fast
Transaction	Several statements that succeed or fail together

Tools: psql and pgAdmin; connecting

Angela §31

PostgreSQL runs as a service. You talk to it with the psql terminal, the pgAdmin GUI, or from Node with the pg package. A connection string holds host, port, user, password and database name.

EXAMPLE

```
# terminal
psql -U postgres          # connect as the default superuser
\l                         # list databases
CREATE DATABASE bakery;
\c bakery                 # connect to it
\dt                       # list tables
\d products               # describe a table
\q                        # quit

# connection string format (in .env)
DATABASE_URL=postgres://USER:PASSWORD@HOST:5432/DBNAME
# free hosted Postgres for learning/deploying: Neon, Supabase, Railway, Render
```

Data types and CREATE TABLE

Angela §31

Define columns with types and constraints. Constraints protect data quality: NOT NULL, UNIQUE, DEFAULT, CHECK, REFERENCES.

Type	Use
SERIAL / BIGSERIAL	Auto id
INTEGER / BIGINT	Whole numbers
NUMERIC(10,2)	Money, exact decimals
REAL / DOUBLE PRECISION	Approximate decimals (science)
VARCHAR(n) / TEXT	Text (TEXT is fine in Postgres)
BOOLEAN	true / false
DATE / TIME / TIMESTAMPTZ	Dates; TIMESTAMPTZ stores time zone
JSONB	Flexible JSON data, indexable
UUID	Random unique ids (gen_random_uuid())
TEXT[]	Arrays

EXAMPLE

```

CREATE TABLE users (
  id          SERIAL PRIMARY KEY,           -- auto-incrementing integer
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) UNIQUE NOT NULL,
  password    TEXT NOT NULL,               -- the HASH, never plain
  role        VARCHAR(20) DEFAULT 'user' CHECK (role IN ('user', 'admin')),
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE products (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  price       NUMERIC(10, 2) NOT NULL CHECK (price >= 0), -- money: never FLOAT
  category    TEXT,
  in_stock    BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE orders (
  id          SERIAL PRIMARY KEY,
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, -- foreign key
  total       NUMERIC(10, 2) NOT NULL,
  status      TEXT DEFAULT 'pending',
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE order_items (               -- many-to-many: an order has many products, a product is in
many orders
  order_id    INTEGER REFERENCES orders(id) ON DELETE CASCADE,
  product_id  INTEGER REFERENCES products(id),
  qty         INTEGER NOT NULL DEFAULT 1,
  PRIMARY KEY (order_id, product_id)
);

ALTER TABLE products ADD COLUMN image_url TEXT;
ALTER TABLE products DROP COLUMN category;
ALTER TABLE products RENAME COLUMN name TO title;
DROP TABLE IF EXISTS temp_table;
CREATE INDEX idx_products_category ON products(category);

```

CRUD: INSERT, SELECT, UPDATE, DELETE

Angela §31

The four operations you will write every day.

INSERT إضافة، SELECT قراءة، UPDATE تعديل، DELETE حذف. متنساش WHERE في UPDATE و DELETE!

EXAMPLE

```
-- INSERT
INSERT INTO products (name, price, category)
VALUES ('Bread', 5.00, 'bakery');
INSERT INTO products (name, price, category)
VALUES
  ('Cake', 40, 'bakery'),
  ('Tea', 10, 'drinks')
RETURNING
*;
--
Postgres: give back the inserted rows (with
id)

-- SELECT
SELECT * FROM
products;
-- all
columns, all rows
SELECT name, price FROM
products;
-- some columns
SELECT * FROM products WHERE price < 20;
SELECT * FROM products WHERE category =
'bakery' AND in_stock = TRUE;
SELECT * FROM products WHERE name ILIKE
'%cake%';
-- case-insensitive contains
SELECT * FROM products WHERE price BETWEEN 5
AND 20;
SELECT * FROM products WHERE category IN
('bakery', 'drinks');
SELECT * FROM products WHERE image_url IS
NULL;
SELECT * FROM products ORDER BY price DESC,
name ASC;
SELECT * FROM products ORDER BY created_at
DESC LIMIT 10 OFFSET 20;
-- page 3 of 10
SELECT DISTINCT category FROM products;
SELECT name AS title, price * 1.14 AS
price_with_tax FROM products;
-- aliases,
expressions

-- UPDATE
UPDATE products SET price = 6, in_stock =
TRUE WHERE id = 1;
UPDATE products SET price = price * 1.10
WHERE category = 'bakery';
-- 10% raise

-- DELETE
DELETE FROM products WHERE id = 3;
DELETE FROM products WHERE in_stock = FALSE
AND created_at < NOW() - INTERVAL '1 year';
```

RESULT

```
SELECT name, price FROM products WHERE price < 20
ORDER BY price;
```

name	price
Bread	5.00
Tea	10.00

(2 rows)

⚠ Common mistake

`UPDATE products SET price = 0;` with no WHERE changes EVERY row. Same for DELETE. Write the WHERE first, run a SELECT with the same WHERE to check, then change SELECT to UPDATE/DELETE.

Aggregates and GROUP BY

Angela §31

Summarise many rows into one: counts, sums, averages, per group.

EXAMPLE

```
SELECT COUNT(*) FROM products;
SELECT COUNT(*), AVG(price), MIN(price),
MAX(price), SUM(price) FROM products;

SELECT category, COUNT(*) AS items,
ROUND(AVG(price), 2) AS avg_price
FROM products
GROUP BY category
HAVING COUNT(*) > 1 --
filter groups (WHERE filters rows before
grouping)
ORDER BY items DESC;

-- sales per day
SELECT DATE(created_at) AS day, SUM(total)
FROM orders GROUP BY day ORDER BY day;

-- top customers
SELECT user_id, SUM(total) AS spent FROM
orders GROUP BY user_id ORDER BY spent DESC
LIMIT 5;
```

RESULT

category	items	avg_price
bakery	2	22.50

JOINS — combining tables

Angela §31

Because data is split across tables, you join them back on the key columns. INNER JOIN keeps only matches; LEFT JOIN keeps all rows from the left table even without a match.

Join	Returns
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All left rows + matching right rows (NULL where none)
RIGHT JOIN	Mirror of LEFT
FULL OUTER JOIN	Everything from both
CROSS JOIN	Every combination (rare)

EXAMPLE

```
-- each order with the customer name
SELECT orders.id, users.name, orders.total
FROM orders
JOIN users ON users.id = orders.user_id;

-- with aliases and a filter
SELECT o.id, u.name, o.total, o.created_at
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.status = 'paid'
ORDER BY o.created_at DESC;

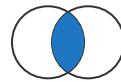
-- all users, including those with NO orders
(LEFT JOIN) and how many
SELECT u.name, COUNT(o.id) AS orders_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;

-- three tables: what was in order 10
SELECT p.name, oi.qty, p.price * oi.qty AS
line_total
FROM order_items oi
JOIN products p ON p.id = oi.product_id
WHERE oi.order_id = 10;

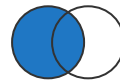
-- subquery
SELECT * FROM products WHERE price > (SELECT
AVG(price) FROM products);
SELECT * FROM users WHERE id IN (SELECT
user_id FROM orders WHERE total > 100);
```

RESULT

INNER JOIN



LEFT JOIN



Using PostgreSQL from Node with pg

Angela \$31

The pg package connects Node to Postgres. Use a Pool (reuses connections) and ALWAYS parameterised queries (\$1, \$2) — never build SQL by string concatenation, or attackers can inject SQL.

استخدم \$1 و \$2 للقيم في الاستعلام، متركبش SQL بالنص أبداً — ده اللي بيمنع SQL injection.

EXAMPLE

```

class="c"> // npm i pg
class="c"> // db/index.js
import pg from "pg";
export const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
class="c"> // hosted DBs often need: ssl: { rejectUnauthorized: false }
export const query = (text, params) => pool.query(text, params);

class="c"> // controllers/products.js
import { query } from "../db/index.js";

export async function list(req, res, next) {
  try {
    const { rows } = await query("SELECT * FROM products ORDER BY id");
    res.json(rows);
  } catch (e) { next(e); }
}

export async function getOne(req, res, next) {
  try {
    const { rows } = await query("SELECT * FROM products WHERE id = $1", [req.params.id]);
class="c"> // ✓ parameterised
    if (!rows[0]) return res.status(404).json({ error: "Not found" });
    res.json(rows[0]);
  } catch (e) { next(e); }
}

export async function create(req, res, next) {
  try {
    const { name, price } = req.body;
    const { rows } = await query(
      "INSERT INTO products (name, price) VALUES ($1, $2) RETURNING *", [name, price]);
    res.status(201).json(rows[0]);
  } catch (e) {
    if (e.code === "23505") return res.status(409).json({ error: "Already exists" });
class="c"> // unique violation
    next(e);
  }
}

export async function update(req, res, next) {
  const { rows } = await query("UPDATE products SET name = $1, price = $2 WHERE id = $3 RETURNING *", [req.body.name, req.body.price, req.params.id]);
  rows[0] ? res.json(rows[0]) : res.status(404).json({ error: "Not found" });
}

export async function remove(req, res, next) {
  const { rowCount } = await query("DELETE FROM products WHERE id = $1", [req.params.id]);
  rowCount ? res.status(204).end() : res.status(404).json({ error: "Not found" });
}

class="c"> // transaction: all or nothing
const client = await pool.connect();
try {
  await client.query("BEGIN");
  await client.query("UPDATE accounts SET balance = balance - $1 WHERE id = $2", [100, from]);
  await client.query("UPDATE accounts SET balance = balance + $1 WHERE id = $2", [100, to]);
  await client.query("COMMIT");
} catch (e) { await client.query("ROLLBACK"); throw e; } finally { client.release(); }

```

⚠ Common mistake

`query(`SELECT * FROM users WHERE email = '${email}'`)` — if email is `' OR 1=1 --` the attacker reads every user. This is SQL injection, the #1 web vulnerability. Parameters (\$1) make it impossible.

ORMs and query builders (Prisma, Sequelize, Knex) — awareness

Angela §31

An ORM maps tables to JavaScript objects so you write `prisma.product.findMany()` instead of SQL. Great productivity, but learn raw SQL first so you understand what they generate. Prisma is the modern favourite.

EXAMPLE

```
class="c">// Prisma example (npm i prisma @prisma/client; npx prisma init)
class="c">// schema.prisma
model Product { id Int @id @default(autoincrement()) name String price Decimal orders
OrderItem[] }

class="c">// code
import { PrismaClient } from "@prisma/client"; const prisma = new PrismaClient();
const cheap = await prisma.product.findMany({ where: { price: { lt: 20 } }, orderBy: { price:
"asc" } });
const p = await prisma.product.create({ data: { name: "Cake", price: 40 } });
await prisma.product.update({ where: { id: 1 }, data: { price: 6 } });
await prisma.product.delete({ where: { id: 1 } });
class="c">// npx prisma migrate dev → creates/updates tables from the schema
```

MongoDB in one page (the NoSQL alternative)

Angela §31

Older versions of Angela's course used MongoDB + Mongoose. Documents are JSON-like objects in collections; no fixed schema; good for flexible data. Concepts map 1:1 to SQL.

SQL	MongoDB
database	database
table	collection
row	document (JSON)
column	field
JOIN	\$lookup / embed the data
schema enforced by DB	schema in code (Mongoose)

EXAMPLE

```

class="c">// npm i mongoose
import mongoose from "mongoose";
await mongoose.connect(process.env.MONGO_URL);
const Product = mongoose.model("Product", new mongoose.Schema({ name: { type: String, required:
true }, price: Number }, { timestamps: true }));
await Product.create({ name: "Cake", price: 40 });
const cheap = await Product.find({ price: { $lt: 20 } }).sort({ price: 1 }).limit(10);
const one = await Product.findById(id);
await Product.findByIdAndUpdate(id, { price: 6 }, { new: true });
await Product.findByIdAndDelete(id);

```

? Chapter quiz

1. What is a foreign key?
2. What is wrong with: DELETE FROM users;
3. Why use \$1 parameters in pg queries?
4. INNER JOIN vs LEFT JOIN?

Answers: 1) A column that stores the primary key of a row in another table, linking them. 2) No WHERE — deletes every user. 3) Prevents SQL injection and handles quoting. 4) INNER keeps only matching rows; LEFT keeps all rows of the left table with NULLs where no match.

89. Back-End Reference F — Authentication and Security

Letting users register, log in, stay logged in and access only their own data — safely. Angela's Secrets project walks through six levels of security; this chapter covers all of them with modern packages: bcrypt hashing, sessions with Passport, OAuth with Google, plus the security habits every developer must have.

Level 1-2: never store plain passwords — hashing with bcrypt

Angela §32

A hash is a one-way scramble: you can turn a password into a hash, but not back. On login you hash what the user typed and compare hashes. bcrypt adds a random salt and is deliberately slow, so leaked hashes cannot be cracked quickly.

عمرک ما تخزن الباسورد زي ما هو. خزن الـ hash بتاعه (bcrypt)، ووقت الدخول قارن الـ hash اللي المستخدم كتبه بالمخزن.

EXAMPLE

```

class="c"> // npm i bcrypt
import bcrypt from "bcrypt";
const SALT_ROUNDS = 12;

class="c"> // register
app.post("/register", async (req, res) => {
  const { email, password } = req.body;
  if (password.length < 8) return res.status(400).json({ error: "Password too short" });
  const hash = await bcrypt.hash(password, SALT_ROUNDS);  class="c"> //
"$2b$12$N9qo8uL0ickgx2ZMRZoMye..."
  try {
    const { rows } = await query("INSERT INTO users (email, password) VALUES ($1, $2) RETURNING
id, email", [email.toLowerCase(), hash]);
    res.status(201).json(rows[0]);
  } catch (e) {
    if (e.code === "23505") return res.status(409).json({ error: "Email already registered" });
    throw e;
  }
});

class="c"> // login
app.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const { rows } = await query("SELECT * FROM users WHERE email = $1", [email.toLowerCase()]);
  const user = rows[0];
  const ok = user && await bcrypt.compare(password, user.password);  class="c"> // same generic
error for wrong email OR password
  if (!ok) return res.status(401).json({ error: "Invalid email or password" });
  class="c"> // → create session or JWT (next entries)
});

```

- Never say 'email not found' vs 'wrong password' — it lets attackers discover which emails exist.
- Never log passwords; never return the password hash in API responses (SELECT id, email, name ...).

Level 3: environment variables and secrets

Angela §32

Covered in chapter A: everything secret goes in .env, .env goes in .gitignore. If you ever commit a secret, rotate it (create a new one) — deleting the commit is not enough.

Level 4: sessions and cookies with express-session and Passport

Angela §32

For server-rendered apps: after login the server stores a session (user id) and gives the browser a cookie with the session id. The browser sends the cookie automatically on every

request. Passport is the standard library that plugs different login strategies (local password, Google, GitHub...) into this flow.

Cookie flag	Effect
httpOnly	JavaScript cannot read it (protects against XSS theft)
secure	Only sent over HTTPS
sameSite: "lax" "strict"	Not sent on cross-site requests (protects against CSRF)
maxAge	Expiry in ms

EXAMPLE

```

class="c">// npm i express-session passport
passport-local connect-pg-simple
import session from "express-session";
import passport from "passport";
import { Strategy as LocalStrategy } from
"passport-local";

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, secure:
process.env.NODE_ENV === "production",
maxAge: 1000 * 60 * 60 * 24 * 7 },
  class="c">// store: new
PgSession({ pool }) ← in production keep
sessions in the DB, not memory
}));
app.use(passport.initialize());
app.use(passport.session());

class="c">// how to check a password
passport.use(new
LocalStrategy({ usernameField: "email" },
async (email, password, done) => {
  try {
    const { rows } = await query("SELECT *
FROM users WHERE email = $1", [email]);
    const user = rows[0];
    if (!user || !(await
bcrypt.compare(password, user.password)))
return done(null, false, { message: "Invalid
credentials" });
    return done(null, user);
  } catch (e) { done(e); }
}));
class="c">// what to store in the session
(just the id) and how to get the user back
passport.serializeUser((user, done) =>
done(null, user.id));
passport.deserializeUser(async (id, done) => {
  const { rows } = await query("SELECT id,
email, name FROM users WHERE id = $1", [id]);
  done(null, rows[0] || false);
});

class="c">// routes
app.post("/login",
passport.authenticate("local",
{ successRedirect: "/secrets",
failureRedirect: "/login" }));
app.get("/logout", (req, res, next) =>
req.logout(err => err ? next(err) :
res.redirect("/")));

function ensureAuth(req, res, next)
{ req.isAuthenticated() ? next() :

```

```
res.redirect("/login"); }
app.get("/secrets", ensureAuth, async (req,
res) => {
  const { rows } = await query("SELECT secret
FROM secrets WHERE user_id = $1",
[req.user.id]);
  res.render("secrets", { secrets: rows, user:
req.user });
});
```

RESULT

1. POST /login (email+password) → server checks hash
2. server stores {userId: 7} in session store, replies Set-Cookie: sid=abc; HttpOnly
3. every next request: Cookie: sid=abc → server loads session → req.user
4. /logout destroys the session

Level 5: OAuth — Sign in with Google

Angela §32

Let users log in with an account they already have. You register your app in Google Cloud Console to get a client id and secret, then Passport handles the redirect dance. The user never gives you a password.

EXAMPLE

```
class="c">// npm i passport-google-oauth20
import { Strategy as GoogleStrategy } from
"passport-google-oauth20";
passport.use(new GoogleStrategy({
  clientId: process.env.GOOGLE_CLIENT_ID,
  clientSecret:
process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: "http://localhost:3000/auth/google/callback", //
must match Google console exactly
}, async (accessToken, refreshToken, profile,
done) => {
  const email = profile.emails[0].value;
  let { rows } = await query("SELECT * FROM
users WHERE email = $1", [email]);
  if (!rows[0]) ({ rows } = await
query("INSERT INTO users (email, name,
password) VALUES ($1, $2, class="v">'google')
RETURNING *", [email, profile.displayName]);
  done(null, rows[0]);
}));
app.get("/auth/google",
passport.authenticate("google", { scope:
["profile", "email"] }));
app.get("/auth/google/callback",
passport.authenticate("google",
{ failureRedirect: "/login" })), (req, res) =>
res.redirect("/secrets"));

<!-- login.ejs -->
<a class="btn" href="/auth/google">Sign in
with Google</a>
```

RESULT

Your site → /auth/google → Google login page → user approves → Google redirects to /auth/google/callback?code=... → Passport exchanges code for profile → your session is created

Sessions vs JWT — which to use

Angela §32

	Session + cookie	JWT
State	Server stores sessions	Stateless: token carries the data
Best for	Server-rendered sites (EJS), same domain	APIs used by SPA / mobile, multiple servers
Logout	Destroy session — instant	Token valid until expiry (use short expiry + refresh token or a blocklist)
Storage in browser	Cookie (automatic, httpOnly)	localStorage or httpOnly cookie
CSRF risk	Yes — use sameSite + CSRF token	No if in Authorization header
XSS risk	Low (httpOnly)	High if in localStorage

Authorisation: roles and ownership

Angela §32

Authentication = who are you. Authorisation = what may you do. Always check on the server that the logged-in user owns the resource or has the role; never trust a hidden field or the front-end.

EXAMPLE

```
class="c">// role check
const requireRole = (...roles) => (req, res, next) => roles.includes(req.user?.role) ? next() :
res.status(403).json({ error: "Forbidden" });
app.delete("/api/users/:id", auth, requireRole("admin"), removeUser);

class="c">// ownership check — the WHERE includes the user id
app.delete("/api/posts/:id", auth, async (req, res) => {
  const { rowCount } = await query("DELETE FROM posts WHERE id = $1 AND user_id = $2",
[req.params.id, req.user.id]);
  rowCount ? res.status(204).end() : res.status(404).json({ error: "Not found" }); class="c">//
404 not 403: do not reveal existence
});
```

Security checklist for every project

Angela §32

Threat	Defence
SQL injection	Parameterised queries (\$) / ORM
XSS (script injection)	textContent / <%= %> escaping; never innerHTML with user data; helmet CSP
CSRF	sameSite cookies; csrf tokens for forms; check Origin
Brute-force login	express-rate-limit; lock after N failures; bcrypt cost 12
Leaked secrets	env vars; .gitignore; rotate on leak
Password reset abuse	Random single-use token with expiry, sent by email; never send passwords
Insecure transport	HTTPS always (free via Let's Encrypt / hosting)
Bad input	Validate type/length/format on the server (express-validator, zod)
Info leaks	Generic error messages to client; details only in server logs
Outdated packages	npm audit; update regularly
Missing headers	helmet()
Large payloads / uploads	express.json({ limit: "100kb" }); multer limits; check mime types

Sending email (verification, reset)

Angela §32

EXAMPLE

```
class="c">// npm i nodemailer
import nodemailer from "nodemailer";
const transporter = nodemailer.createTransport({ host: process.env.SMTP_HOST, port: 587, auth:
{ user: process.env.SMTP_USER, pass: process.env.SMTP_PASS } });

class="c">// password reset flow
app.post("/forgot", async (req, res) => {
  const token = crypto.randomBytes(32).toString("hex");
  await query("UPDATE users SET reset_token = $1, reset_expires = NOW() + INTERVAL class="v">'1
hour' WHERE email = $2", [token, req.body.email]);
  await transporter.sendMail({ to: req.body.email, subject: "Reset your password", html:
class="v">`<a href="https:
res.json({ message: "If that email exists, a link was sent" });
  either way
});
```

? Chapter quiz

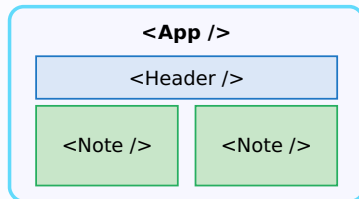
1. Why hash passwords instead of encrypting them?
2. Which cookie flag stops JavaScript from reading the cookie?
3. Authentication vs authorisation?
4. A user deletes `/api/posts/9` — what must the server check besides login?

Answers: 1) Hashing is one-way; even you cannot recover the passwords if the database leaks. 2) `httpOnly`. 3) Authentication = identity; authorisation = permissions. 4) That the post belongs to that user (or the user is admin).

90. React — Fundamentals & Hooks

Angela §36

React is a JS library for building UIs out of reusable **components**. Instead of manually changing the DOM, you describe what the UI should look like for a given **state**, and React updates the DOM when the state changes.



Components = functions that return JSX (HTML + JS)
 Data flows DOWN via **props** (parent → child).
 Each component can own **state**; change state → re-render. $UI = f(state)$

state changes

UI re-renders

```
# create a project (Vite – fast modern tool)
npm create vite@latest my-app -- --template react
cd my-app && npm install && npm run dev      # http://localhost:5173
# src/main.jsx mounts <App /> into <div id="root"> in index.html
```

JSX rules

```
function Welcome() {
  const name = "Ahmed";
  const isLoggedIn = true;
  return (
    <div className="card">                                {/* className not class; htmlFor not for */}
      <h1>Hello, {name}!</h1>                             {/* { } = any JS expression */}
      <p style={{ color: "red", fontSize: "20px" }}>inline style = object</p>
      {isLoggedIn ? <button>Logout</button> : <button>Login</button>}  {/* conditional */}
      {isLoggedIn && <p>Welcome back</p>}                    {/* show if true */}
      <img src={logo} alt="" />                             {/* self-close every void tag */}
    </div>                                                  {/* ONE root element (or <>...</> fragment) */}
  );
}
export default Welcome;
```

Props — passing data down

```
// Card.jsx
function Card({ name, img, phone }) { //
  // destructure props
  return (
    <div className="card">
      <img src={img} alt={name} />
      <h2>{name}</h2>
      <p>{phone}</p>
    </div>
  );
}
export default Card;
```

```
// App.jsx
import Card from "./Card";
import contacts from "./contacts"; // array
// of objects

function App() {
  return (
    <>
      <h1>My Contacts</h1>
      {contacts.map(c => ( //
        // render a LIST with map
        <Card key={c.id} //
          // key = unique, required
          name={c.name} img={c.imgURL}
          phone={c.phone} />
        )))}
    </>
  );
}
```

State — useState hook

current value (read) initial value

```
const [count, setCount] = useState(0);    count = 5 X (no re-render) setCount(5) ✓
```

updater fn — the ONLY way to change it

```

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>+</button>      /* pass a FUNCTION, don't call it */
      <button onClick={() => setCount(prev => prev - 1)}>-</button>
    </>
  );
}

// CONTROLLED INPUT – React state is the single source of truth
function Form() {
  const [form, setForm] = useState({ fName: "", email: "" });
  function handleChange(e) {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));    // copy old, override one key (never mutate!)
  }
  function handleSubmit(e) { e.preventDefault(); console.log(form); }
  return (
    <form onSubmit={handleSubmit}>
      <input name="fName" value={form.fName} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <button>Submit</button>
    </form>
  );
}

// LIST state (Keeper / todo app) – add & delete without mutation
const [notes, setNotes] = useState([]);
function addNote(note) { setNotes(prev => [...prev, note]); }
function deleteNote(id) { setNotes(prev => prev.filter((n, i) => i !== id)); }
// child → parent communication: pass the function DOWN as a prop, child calls it
<Note key={i} id={i} title={n.title} onDelete={deleteNote} />
// inside Note: <button onClick={() => props.onDelete(props.id)}>DELETE</button>

```


useEffect — side effects (fetching, timers, titles)

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => { // runs AFTER render
    fetch("/api/users")
      .then(r => r.json())
      .then(data => { setUsers(data); setLoading(false); });
  }, []); // [] = run once on mount. [id] = re-run when id changes. none = every render ( )

  useEffect(() => {
    const t = setInterval(tick, 1000);
    return () => clearInterval(t); // cleanup on unmount
  }, []);

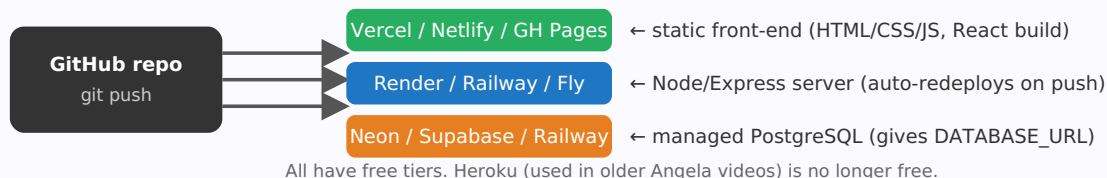
  if (loading) return <p>Loading...</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Next steps in React	Tool
Multiple pages	react-router-dom (<Routes><Route path="/about" element={<About/>}/>)
Share state across many components	useContext, or Zustand / Redux Toolkit
Data fetching / caching	TanStack Query (react-query)
Styling	CSS Modules, Tailwind CSS, styled-components, MUI
Full framework (SSR, routing, API routes)	Next.js

Connect it all: React front-end (`fetch("/api/posts")`) → Express API (ch. 45) → Postgres (ch. 46), with auth (ch. 47). In dev, tell Vite to proxy `/api` to `localhost:4000` in `vite.config.js`.

91. Deployment

Angela \$13, \$37



Static site → GitHub Pages / Netlify / Vercel

```
# GitHub Pages: push repo → Settings → Pages → Source: main / root → live in ~1 min
# Vercel/Netlify: "Import Git repository" → detects Vite → build: npm run build, output: dist
# React with Vite: relative asset paths – set base in vite.config.js if deploying to a sub-path
```

Express server → Render

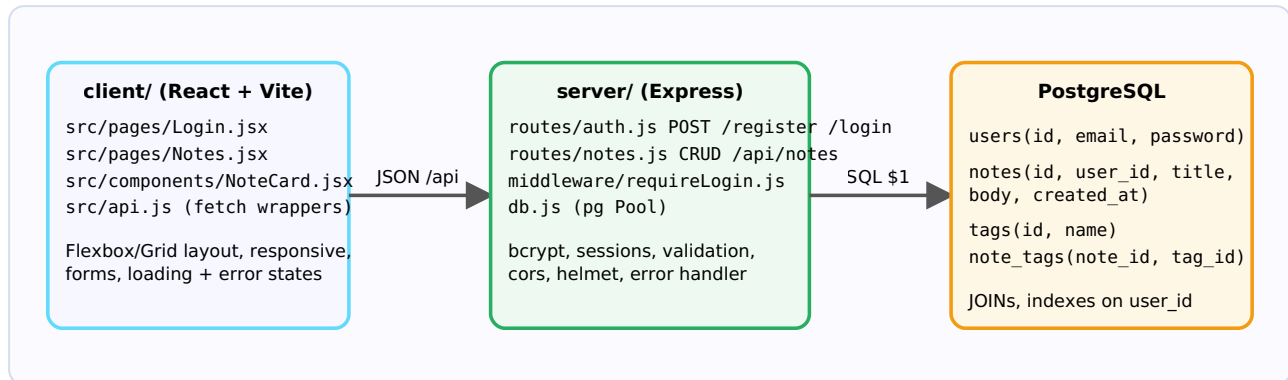
```
// 1. use the platform's port + host
const port = process.env.PORT || 3000;
app.listen(port, "0.0.0.0");
// 2. package.json: "start": "node app.js" and "engines": { "node": ">=20" }
// 3. .gitignore has node_modules and .env
// 4. Render → New Web Service → connect repo → Build: npm install, Start: npm start
// 5. Environment tab → add DATABASE_URL, SESSION_SECRET, API keys (values from your .env)
// 6. Postgres: create on Neon/Supabase → copy connection string → run your CREATE TABLE once
// pg on hosted DBs usually needs: ssl: { rejectUnauthorized: false }
```

Pre-launch checklist

- No secrets in the repo; all in env variables.
- `npm run build` works locally with no errors; test the production build (`npm run preview`).
- 404 page, favicon, `<title>` and meta description on every page.
- Run Lighthouse (DevTools → Lighthouse): performance, accessibility, SEO ≥ 90 .
- Custom domain (Namecheap/Cloudflare) → add DNS record the host gives you → HTTPS is automatic.

92. Full-Stack Capstone Project Plan

Build ONE complete app that proves every part of this book. Suggested: **"Study Notes"** — users register, create notes with tags, search them, and share a public link. Swap the theme for anything (recipes, workouts, bookstore).



Step	Do	Book chapters
1	Sketch screens on paper; write user stories ("as a user I can...")	—
2	Design the DB tables + relations; write CREATE TABLE	46
3	Build the Express API with Postman tests for every route	43, 45, 46
4	Add register/login/logout + protect note routes	47
5	Build the React UI page by page; connect with fetch	48, 39
6	Style: mobile-first, Grid/Flex, transitions, dark mode	23–33
7	Git commits throughout; README with screenshots + how to run	41
8	Deploy DB → API → client; set env vars; test live	49
9	Add to portfolio; write a short post about what you learned	—

93. Cheat Sheets

HTML skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1">
  <title>Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <script src="main.js" defer></script>
</body>
</html>
```

CSS reset + base

```
*, *::before, *::after { box-sizing: border-
box; margin: 0; padding: 0; }
html { font-size: 62.5%; scroll-behavior:
smooth; }
body { font: 1.6rem/1.6 system-ui, sans-serif;
color: #222; }
img { max-width: 100%; display: block; }
a { color: inherit; text-decoration: none; }
ul { list-style: none; }
.container { width: min(100% - 3rem, 1100px);
margin-inline: auto; }
:root { --primary: #1f77c4; --radius: 8px; }
```

Center anything

```
.parent { display: grid; place-items:
center; }
/* or */
.parent { display: flex; justify-content:
center; align-items: center; }
```

Flexbox

```
display:flex; flex-direction:row|column; flex-
wrap:wrap;
justify-content: start|center|end|space-
between|around|evenly;
align-items: stretch|start|center|end; gap:
1rem;
item: flex:1; flex:1 1 250px; align-self;
order;
```

Grid

```
display:grid;
grid-template-columns: repeat(auto-fit,
minmax(250px,1fr));
grid-template-areas: "a b" "c c"; gap:1rem;
item: grid-column: 1 / -1; grid-area: a;
```

Media queries

```
@media (min-width: 600px) { } /* tablet+ */
@media (min-width: 992px) { } /* desktop */
```

JS essentials

```
const el = document.querySelector(".x");
el.textContent = "hi";
el.classList.toggle("on");
el.addEventListener("click", e => { });
arr.map().filter().reduce().find().forEach()
const data = await (await fetch(url)).json();
JSON.stringify(obj); JSON.parse(str);
localStorage.setItem("k", v);
```

Express

```
import express from "express";
const app = express();
app.use(express.static("public"));
app.use(express.urlencoded({extended:true}));
app.use(express.json());
app.set("view engine","ejs");
app.get("/", (req,res)=>res.render("index",
{}));
app.post("/api/x",
(req,res)=>res.status(201).json(req.body));
app.listen(process.env.PORT||3000);
```

SQL

```
SELECT cols FROM t WHERE c=1 ORDER BY c DESC
LIMIT 10;
INSERT INTO t (a,b) VALUES ($1,$2) RETURNING
*;
UPDATE t SET a=$1 WHERE id=$2;
DELETE FROM t WHERE id=$1;
SELECT * FROM a JOIN b ON a.id=b.a_id;
```

Git

```
git init · git add . · git commit -m "msg"
git push · git pull · git clone url
git checkout -b feat · git merge feat
git status · git log --oneline · git diff
```

React

```
const [v, setV] = useState(init);
useEffect(() => { ...; return cleanup },
[deps]);
{list.map(i => <Item key={i.id} {...i} />)}
<input value={v} onChange={e =>
setV(e.target.value)} />
```

HTTP status codes

```
200 OK · 201 Created · 204 No Content
301/302 Redirect · 304 Not Modified
400 Bad Request · 401 Unauthorized · 403
Forbidden
404 Not Found · 409 Conflict · 422
Unprocessable
500 Server Error · 502 Bad Gateway · 503
Unavailable
```

Learn more (free, official)

```
developer.mozilla.org — HTML/CSS/JS reference
web.dev/learn — Google's courses
css-tricks.com — Flexbox & Grid guides
javascript.info — deep JS
expressjs.com · react.dev · postgresql.org/
docs
roadmap.sh/frontend · roadmap.sh/backend
```

— End of book. Now go build something. —

94. React Reference A — Thinking in Components, JSX, Props

React is a JavaScript library for building user interfaces out of components: small, reusable functions that return HTML-like markup. Instead of finding elements and changing them by hand (DOM chapter), you describe what the screen should look like for a given state, and React updates the DOM for you. It is the most in-demand front-end skill.

Creating a project with Vite

Angela §33

Gamal React course

Vite is the modern tool that creates a React project, runs a dev server with instant reload, and builds the optimised production files.

React مكتبة بتبني الواجهة من مكونات (components) صغيرة بتتكرر. بتوصف الشكل حسب الحالة (state) و React بتحدث الصفحة لوحدها.

EXAMPLE

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev          # http://localhost:5173
npm run build        # → dist/ folder to deploy

# project layout
my-app/
├─ index.html        ← has <div id="root"></div> and loads src/main.jsx
├─ src/
│  ├─ main.jsx       ← mounts <App /> into #root
│  ├─ App.jsx        ← your top component
│  ├─ App.css index.css
│  └─ components/    ← you create this
├─ public/           ← static files copied as-is
└─ package.json vite.config.js
```

→ Files with JSX use the .jsx extension.

→ Vite dev server proxy for your API: in vite.config.js add `server: { proxy: { "/api": "http://localhost:3000" } }`.

main.jsx and your first component

Angela §33

A component is a function whose name starts with a Capital letter and returns JSX. main.jsx renders the root component into the page once; everything else is components inside components.

EXAMPLE

```

class="c"> // src/main.jsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

class="c"> // src/App.jsx
function App() {
  return (
    <div className="app">
      <h1>Giza Bakery</h1>
      <p>Fresh every morning.</p>
    </div>
  );
}
export default App;

```

RESULT

Giza Bakery

Fresh every morning.

JSX rules

Angela §33 Gamal React

JSX looks like HTML but is JavaScript. A few differences matter.

HTML	JSX	Why
class="x"	className="x"	class is a JS keyword
for="id"	htmlFor="id"	for is a JS keyword
onclick="f()"	onClick={f}	camelCase; pass the function
style="color:red"	style={{ color: "red" }}	object; double braces = object inside JS
,	,	every tag must close
	{/* c */}	JS comment inside braces
tabindex, readonly	tabIndex, readOnly	camelCase attributes
text	{expression}	braces evaluate JavaScript: variables, ternaries, map — not if/for statements

EXAMPLE

```
function Card() {
  const name = "Bread";
  const price = 5;
  const inStock = true;
  const style = { color: "green", fontSize: 14 };
  class="c"> // object, camelCase, numbers = px

  return (
    <>                                     {class="c"> /* fragment: one parent
required, no extra div */
    <h2 className="title">{name}</h2>       {class="c"> /* class → className ; {} =
JavaScript expression */
    <p>Price: {price * 1.14} EGP</p>
    <label htmlFor="qty">Qty</label>       {class="c"> /* for → htmlFor */
    <input id="qty" type="number"           {class="c"> /* every tag closes: <input /
    >, <br />, <img /> */
    <p style={style}>{inStock ? "In stock" : "Sold out"}</p>
    {inStock && <button onClick={() => alert("added")}>Add</button>} {class="c"> /* onclick →
    onClick, value is a function */
    
    </>
  );
}
```

⚠ Common mistake

Returning two sibling elements without a wrapper → 'Adjacent JSX elements must be wrapped'. Use a fragment `<>....`. Also `{if (x) ...}` is invalid: use ternary or `&&`.

Props: passing data into components

Angela §33

Gamal React

Props are the arguments of a component, written like HTML attributes and received as one object (usually destructured). Data flows DOWN from parent to child. Props are read-only.

ال props زي ال attributes: الأب بيعت بيانات للابن. الابن بيقراها بس ومينفعش يغيرها.

EXAMPLE

```

class="c">// components/ProductCard.jsx
export default function ProductCard({ name,
price, image, onAdd, children }) {
  return (
    <article className="card">
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>{price} EGP</p>
      <button onClick={() => onAdd(name)}>Add
to cart</button>

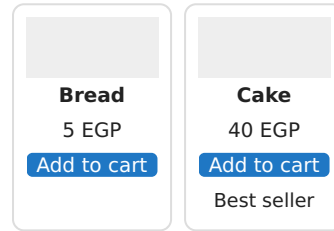
      {children}
    </article>
  );
}

class="c">// App.jsx
import ProductCard from "../components/
ProductCard";
function App() {
  const handleAdd = (name) =>
console.log("added", name);
  return (
    <main>
      <ProductCard name="Bread" price={5}
image="/bread.jpg" onAdd={handleAdd} />
      <ProductCard name="Cake" price={40}
image="/cake.jpg" onAdd={handleAdd}>
        <small>Best seller</small>
      </ProductCard>
    </main>
  );
}

class="c">// default values & spreading
function Button({ variant =
"primary", ...rest }) { return <button
className={class="v">btn ${variant}</button>
{...rest} /> }
<Button variant="danger" onClick={del}
disabled>Delete</Button>

```

RESULT



- Strings: `name="Bread"`. Anything else (numbers, booleans, objects, functions): `price={5}`.
- Callbacks as props (onAdd) are how children talk UP to parents.

Rendering lists and conditional rendering

Angela §33

Turn arrays into elements with `map` (never for loops in JSX). Every item needs a stable unique key so React can track changes. Show or hide with ternary, `&&` or early return.

EXAMPLE

```
const products = [{ id: 1, name: "Bread", price: 5 }, { id: 2, name: "Cake", price: 40 }];

function ProductList({ items, loading, error }) {
  if (loading) return <p>Loading...</p>;           class="c"> // early return
  if (error) return <p className="error">{error}</p>;
  if (items.length === 0) return <p>No products.</p>;

  return (
    <ul>
      {items.map(p => (
        <li key={p.id}>                               {class="c"} /* key = stable id, NOT the
index */)
        {p.name} - {p.price} EGP
        {p.price > 20 && <span className="badge">Premium</span>}
      </li>
      )}}
    </ul>
  );
}

class="c"> // filter + map
{items.filter(p => p.inStock).map(p => <ProductCard key={p.id} {...p} />)}
```

⚠ Common mistake

Using the array index as key: when items are reordered or deleted, React mixes up inputs and state between rows. Use database ids. Missing key → warning in console.

Styling React components

Angela §33

Several options; plain CSS files and CSS Modules are the simplest. Tailwind is very popular in jobs.

EXAMPLE

```

class="c"> // 1. global CSS: import './App.css'; then className="card"

class="c"> // 2. CSS Modules (scoped, no name clashes): Card.module.css
import styles from './Card.module.css';
<div className={styles.card}>

class="c"> // 3. inline (dynamic values)
<div style={{ width: class="v">`${percent}%`, background: color }} />

class="c"> // 4. conditional classes
<li className={class="v">`item ${done ? "done" : ""}`}>
class="c"> // npm i clsx → className={clsx("item", { done, active: id === current })}

class="c"> // 5. Tailwind (utility classes) — npm i -D tailwindcss @tailwindcss/vite
<button className="px-4 py-2 rounded bg-blue-600 text-white hover:bg-blue-700">Save</button>

class="c"> // 6. Bootstrap in React: npm i bootstrap; import "bootstrap/dist/css/
bootstrap.min.css" in main.jsx

```

? Chapter quiz

1. Why must component names start with a capital letter?
2. How do you write a class attribute in JSX?
3. What should you use as key when rendering a list?
4. Can a child component change its props?

Answers: 1) So React can tell components from HTML tags. 2) className. 3) A stable unique id (not the index). 4) No — props are read-only; the child calls a callback prop so the parent changes its state.

95. React Reference B — State, Events, Forms and Hooks

State is data that changes over time and that the UI depends on: the count, the typed text, the list of todos, whether a modal is open. When state changes, React re-renders the component. Hooks (functions starting with use) let function components have state and side effects.

useState

Angela §34

Gamal React

`const [value, setValue] = useState(initial)` . Read value; call setValue to change it and trigger a re-render. Never modify state directly — always create a new value.

useState بتديك قيمة وفانكشن لتغييرها. لما تغيرها React بتعيد رسم المكون. متعدلش على الـ state مباشرة.

EXAMPLE

```
import { useState } from "react";

function Counter() {
  const [count, setCount] =
    useState(0);      class="c"> // initial
    value 0

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count +
1)}>+1</button>
      <button onClick={() => setCount(c => c
+ 1)}>+1 (safe form)</button>  {class="c"> /*
use the function form when new value depends
on old */}
      <button onClick={() => setCount(0)}
>Reset</button>
    </div>
  );
}

class="c"> // several pieces of state
const [name, setName] = useState("");
const [isOpen, setIsOpen] = useState(false);
const [items, setItems] = useState([]);
const [user, setUser] = useState(null);

class="c"> // objects & arrays: make a NEW one
(immutability)
setUser({ ...user, age:
26 });      class="c"> //
✓ user.age = 26 * (no re-render)
setItems([...items,
newItem]);
class="c"> // add
setItems(items.filter(i => i.id !==
id));      class="c"> // remove
setItems(items.map(i => i.id === id ? { ...i,
done: !i.done } : i));  class="c"> // update
one
```

RESULT

You clicked **3** times

+1

Reset

- State updates are asynchronous and batched: logging count right after `setCount` shows the OLD value.
- Each component instance has its own state. Hooks must be called at the top level of the component, never inside if/loops.

⚠ Common mistake

`items.push(x); setItems(items)` — same array reference, React sees no change and does not re-render. Always `setItems([...items, x])`.

Handling events

Angela §34

EXAMPLE

```
function Form() {
  const handleClick = (e) => { console.log(e.target); };
  const handleKey = (e) => { if (e.key === "Enter") submit(); };

  return (
    <
      <button onClick={handleClick}>Click</button>           {class="c"}/* reference, not call
    */}
      <button onClick={() => remove(item.id)}>Delete</button>   {class="c"}/* need an argument →
    arrow */}
      <input onKeyDown={handleKey} onFocus={...} onBlur={...} onChange={...} />
      <form onSubmit={(e) => { e.preventDefault(); save(); }}>
      <div onMouseEnter={...} onMouseLeave={...} onScroll={...}>
    </>
  );
}
```

Controlled forms

Angela §34

Gamal React

In React the state is the source of truth: the input shows the state value and updates it on every change. One handler can serve all fields using the name attribute.

EXAMPLE

```
function Signup() {
  const [form, setForm] = useState({ name: "", email: "", plan: "free", agree: false });
  const [errors, setErrors] = useState({});

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const errs = {};
    if (form.name.trim().length < 2) errs.name = "Name too short";
    if (!/\S+@\S+\.\S+/.test(form.email)) errs.email = "Invalid email";
    if (!form.agree) errs.agree = "You must agree";
    setErrors(errs);
    if (Object.keys(errs).length) return;
    await fetch("/api/signup", { method: "POST", headers: { "Content-Type": "application/json" },
body: JSON.stringify(form) });
    setForm({ name: "", email: "", plan: "free", agree: false });
  };

  return (
    <form onSubmit={handleSubmit} noValidate>
      <input name="name" value={form.name} onChange={handleChange} placeholder="Name" />
      {errors.name && <small className="error">{errors.name}</small>}

      <input name="email" type="email" value={form.email} onChange={handleChange}
placeholder="Email" />
      {errors.email && <small className="error">{errors.email}</small>}

      <select name="plan" value={form.plan} onChange={handleChange}>
        <option value="free">Free</option><option value="pro">Pro</option>
      </select>

      <label><input name="agree" type="checkbox" checked={form.agree} onChange={handleChange} /> I
agree</label>
      {errors.agree && <small className="error">{errors.agree}</small>}

      <button disabled={!form.agree}>Sign up</button>
    </form>
  );
}
```

- `value` + `onChange` together = controlled. `value` alone makes a read-only input (React warns).
- Textarea uses `value` too (not children). Select uses `value` on the select, not selected on options.
- Uncontrolled alternative: `useRef` and read `ref.current.value` on submit; or `new FormData(e.target)`.

Lifting state up and sharing between components

Angela §34

When two components need the same data, move the state to their closest common parent and pass it down as props, with callback props to change it. This is the Keeper app pattern.

EXAMPLE

```
function App() {
  const [notes, setNotes] = useState([]);
  const addNote = (note) => setNotes(prev =>
[...prev, { ...note, id:
crypto.randomUUID() }]);
  const deleteNote = (id) => setNotes(prev =>
prev.filter(n => n.id !== id));

  return (
    <>
      <Header />
      <CreateArea onAdd={addNote} />
    </>
  );
}

function CreateArea({ onAdd }) {
  const [note, setNote] = useState({ title:
'', content: '' });
  const change = e => setNote({ ...note,
[e.target.name]: e.target.value });
  const submit = e => { e.preventDefault();
if (!note.title) return; onAdd(note);
setNote({ title: '', content: '' }); };
  return (
    <form onSubmit={submit}>
      <input name="title" value={note.title}
onChange={change} placeholder="Title" />
      <textarea name="content"
value={note.content} onChange={change}
rows={3} />
      <button>Add</button>
    </form>
  );
}

function Note({ title, content, onDelete }) {
  return <div className="note"><h3>{title}</h3><p>{content}</p><button onClick={onDelete}>X</button></div>;
}
```

RESULT

Keeper

Buy flour

5 kg for

Saturday



Call Sara

about the

order



useEffect: side effects, fetching data, cleanup

Angela §34

Gamal React

Rendering must be pure. Anything that touches the outside world — fetch, timers, subscriptions, document.title, localStorage — goes in useEffect. The dependency array controls WHEN it runs: [] once after first render; [x] whenever x changes; none → after every render (rare). Return a function to clean up.

useEffect للحاجات اللي خارج الرسم: جلب بيانات، تايمر، localStorage. المصفوفة بتحدد إمتى يتنفذ: [] مرة واحدة، [x] كل ما x تتغير.

EXAMPLE

```
import { useState, useEffect } from "react";

function Products({ category }) {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    fetch(class="v">`/api/products?category=${category}`, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error("Failed"); return r.json(); })
      .then(data => { setItems(data); setError(null); })
      .catch(e => { if (e.name !== "AbortError") setError(e.message); })
      .finally(() => setLoading(false));
    return () => controller.abort(); // cleanup: cancel if category changes or
    component unmounts
  }, [category]); // re-run when category changes

  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;
  return <ul>{items.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}

class="c">// other typical effects
useEffect(() => { document.title = class="v">`${count} clicks`; }, [count]);
useEffect(() => { localStorage.setItem("todos", JSON.stringify(todos)); }, [todos]);
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup timers!
}, []);

useEffect(() => {
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize", onResize);
}, []);

class="c">// async inside effect: define and call an inner function (effect itself cannot be
async)
useEffect(() => { (async () => { const r = await fetch(url); setData(await r.json()); })(); },
[url]);
```

⚠ Common mistake

Infinite loop: setting state inside an effect with no dependency array, or with a dependency that is a new object/array every render. Also: StrictMode runs effects twice in development — that is intentional to reveal missing cleanups.

useRef, useMemo, useCallback, useContext, useReducer**Angela §34**

The other built-in hooks you will meet.

EXAMPLE

```
class="c">// useRef – a box that survives re-renders without causing them; also DOM access
const inputRef = useRef(null);
useEffect(() => inputRef.current.focus(), []);
<input ref={inputRef} />
class="c">renders = useRef(0); renders.current++;      class="c">mutable value, no re-render

class="c">// useMemo – cache an expensive calculation
const sorted = useMemo(() => [...items].sort(byPrice), [items]);

class="c">// useCallback – keep the same function reference (for memoized children / effect deps)
const handleAdd = useCallback((item) => setItems(prev => [...prev, item]), []);

class="c">// useContext – share data with deep children without prop drilling
const ThemeContext = createContext("light");
function App() { const [theme, setTheme] = useState("light");
  return <ThemeContext.Provider value={{ theme, setTheme }}><Page /></ThemeContext.Provider>; }
function Button() { const { theme, setTheme } = useContext(ThemeContext); return <button
className={theme} onClick={() => setTheme(t => t === "light" ? "dark" : "light")}>Toggle</
button>; }

class="c">// useReducer – complex state with actions (mini Redux)
function reducer(state, action) {
  switch (action.type) {
    case "add": return [...state, action.todo];
    case "toggle": return state.map(t => t.id === action.id ? { ...t, done: !t.done } : t);
    case "remove": return state.filter(t => t.id !== action.id);
    default: return state;
  }
}
const [todos, dispatch] = useReducer(reducer, []);
dispatch({ type: "add", todo });
```

Custom hooks**Angela §34**

Extract reusable stateful logic into a function named useSomething. It uses other hooks inside and returns whatever you want.

EXAMPLE

```

class="c"> // hooks/useFetch.js
export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    let alive = true;
    setLoading(true);
    fetch(url).then(r => r.ok ? r.json() : Promise.reject(new Error(r.statusText)))
      .then(d => alive && setData(d)).catch(e => alive &&
setError(e.message)).finally(() => alive && setLoading(false));
    return () => { alive = false; };
  }, [url]);
  return { data, loading, error };
}
class="c"> // use
const { data: products, loading, error } = useFetch("/api/products");

class="c"> // hooks/useLocalStorage.js
export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => JSON.parse(localStorage.getItem(key)) ?? initial);
  useEffect(() => localStorage.setItem(key, JSON.stringify(value)), [key, value]);
  return [value, setValue];
}
const [todos, setTodos] = useLocalStorage("todos", []);

class="c"> // useToggle, useDebounce, useWindowSize, useOnClickOutside — same idea

```

? Chapter quiz

1. What happens when you call a state setter?
2. Why `setItems([...items, x])` and not `items.push(x)`?
3. What does an empty dependency array `[]` mean in `useEffect`?
4. When do you need a cleanup function?

Answers: 1) React schedules a re-render with the new value. 2) React compares references; push keeps the same array so no re-render happens. 3) Run once after the first render. 4) Timers, event listeners, subscriptions, in-flight fetches — anything that must stop when the component unmounts or deps change.

96. React Reference C — Routing, Data, Structure and a Full App

Real apps have multiple pages, talk to an API, keep the user logged in, and need a sensible folder structure. This chapter assembles those pieces into a complete front-end that connects to the Express API from the back-end part.

React Router: pages without reloads

Angela §34

A single-page app changes the URL and swaps components without a full page load. React Router maps paths to components.

EXAMPLE

```

class="c">// npm i react-router-dom
import { BrowserRouter, Routes, Route, Link, NavLink, useParams, useNavigate, Navigate, Outlet }
from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" end>Home</NavLink>                                {class="c"}/* NavLink gets
class "active" automatically */
        <NavLink to="/products">Products</NavLink>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/products" element={<Products />} />
        <Route path="/products/:id" element={<ProductDetail />} />    {class="c"}/* dynamic */
        <Route path="/dashboard" element={<RequireAuth><Layout /></RequireAuth>}>    {class="c"}/*
nested + protected */
        <Route index element={<Overview />} />
        <Route path="orders" element={<Orders />} />
      </Route>
      <Route path="/login" element={<Login />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </BrowserRouter>
);
}

function ProductDetail() {
  const { id } = useParams();                                class="c">// "/products/7" → "7"
  const navigate = useNavigate();
  const { data: p, loading } = useFetch(class="v">`/api/products/${id}`);
  if (loading) return <p>Loading...</p>;
  return <><h1>{p.name}</h1><button onClick={() => navigate(-1)}>Back</button></>;
}

function Layout() { return <><Sidebar /><Outlet /></>; }          class="c">// Outlet = where the
child route renders
function RequireAuth({ children }) { const { user } = useAuth(); return user ? children :
<Navigate to="/login" replace />; }

class="c">// query string
const [params, setParams] = useSearchParams(); params.get("page"); setParams({ page: 2 });

```

- Use Link/NavLink, never `<a href>` for internal links (that reloads the page).
- When deploying, configure the host to serve index.html for every path (Netlify: `_redirects` file with `/* /index.html 200`; Express: the `app.get("*")` fallback).

Talking to your Express API — an api module and auth context

Angela §34

Keep all fetch calls in one file; keep the logged-in user in a Context so any component can read it.

EXAMPLE

```
class="c">// src/api.js
const BASE = import.meta.env.VITE_API_URL || "";
class="c">// .env → VITE_API_URL=http://localhost:3000

async function request(path, { method = "GET", body, token } = {}) {
  const res = await fetch(BASE + path, {
    method,
    headers: { "Content-Type": "application/json", ...(token && { Authorization:
class="v">`Bearer ${token}` }) },
    body: body && JSON.stringify(body),
  });
  if (res.status === 204) return null;
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || res.statusText);
  return data;
}

export const api = {
  login: (creds) => request("/api/login", { method: "POST", body: creds }),
  products: () => request("/api/products"),
  addProduct: (p, token) => request("/api/products", { method: "POST", body: p, token }),
  deleteProduct: (id, token) => request(class="v">`/api/products/${id}`, { method: "DELETE",
token }),
};

class="c">// src/AuthContext.jsx
const AuthCtx = createContext(null);
export function AuthProvider({ children }) {
  const [token, setToken] = useLocalStorage("token", null);
  const user = useMemo(() => token ? JSON.parse(atob(token.split(".")[1])) : null, [token]);
class="c">// read JWT payload
  const login = async (creds) => { const { token } = await api.login(creds); setToken(token); };
  const logout = () => setToken(null);
  return <AuthCtx.Provider value={{ user, token, login, logout }}>{children}</AuthCtx.Provider>;
}

export const useAuth = () => useContext(AuthCtx);

class="c">// main.jsx: <AuthProvider><App /></AuthProvider>
class="c">// Login.jsx
const { login } = useAuth(); const navigate = useNavigate(); const [err, setErr] = useState("");
const submit = async (e) => { e.preventDefault(); try { await login(form); navigate("/dashboard"); } catch (e) { setErr(e.message); } };
```

Full CRUD page: products admin

Angela §34

Everything together: load on mount, add through a form, delete with optimistic update, show loading and errors.

EXAMPLE

```
function ProductsAdmin() {
  const { token } = useAuth();
  const [items, setItems] = useState([]);
  const [status, setStatus] =
    useState("loading");    class="c"> // loading
    | ready | error

  const [form, setForm] = useState({ name: "",
    price: "" });

  useEffect(() => {
    api.products().then(d =>
    { setItems(d.data ?? d);
    setStatus("ready"); }).catch(() =>
    setStatus("error"));
  }, []);

  const add = async (e) => {
    e.preventDefault();
    const created = await
    api.addProduct({ name: form.name, price:
    Number(form.price) }, token);
    setItems(prev => [...prev, created]);
    setForm({ name: "", price: "" });
  };

  const remove = async (id) => {
    const backup = items;
    setItems(prev => prev.filter(p => p.id !==
    id));    class="c"> // optimistic: update UI
    first
    try { await api.deleteProduct(id,
    token); } catch { setItems(backup);
    alert("Delete failed"); }
  };

  if (status === "loading") return <Spinner /
  >;
  if (status === "error") return <p
  className="error">Could not load products.</
  p>;

  return (
    <section>
      <form onSubmit={add} className="row">
        <input value={form.name} onChange={e
        => setForm({ ...form, name: e.target.value })}
        placeholder="Name" required />
        <input value={form.price} onChange={e
        => setForm({ ...form, price:
        e.target.value })} type="number" min="0"
        placeholder="Price" required />
        <button>Add</button>
      </form>
      <table>
        <thead><tr><th>Name</th><th>Price</
        th><th></th></tr></thead>
```



```

    <tbody>
      {items.map(p => (
        <tr key={p.id}><td>{p.name}</
td><td>{p.price} EGP</td><td><button
onClick={() => remove(p.id)}>Delete</button></
td></tr>
      ))}
    </tbody>
  </table>
</section>
);
}

```

RESULT

Name	Price	Add
Name	Price	
Bread	5 EGP	Delete
Cake	40 EGP	Delete

Project structure and conventions

Angela §34

EXAMPLE

```

src/
├─ main.jsx App.jsx index.css
├─ api.js           ← all HTTP calls
├─ components/      ← reusable UI: Button, Modal, Spinner, Navbar, ProductCard
├─ pages/           ← one per route: Home, Products, ProductDetail, Login, Dashboard,
NotFound
├─ hooks/           ← useFetch, useAuth, useLocalStorage
├─ context/         ← AuthContext, ThemeContext
├─ utils/           ← formatPrice, validators
├─ assets/          ← images, fonts

```

Rules of thumb

- One component per file, same name as the file, default export.
- Keep components small; if it scrolls, split it.
- State lives as low as possible, lifted only when shared.
- Derive values instead of storing them (total = items.reduce(...) – not a second state).
- Show three states for every async thing: loading, error, data.

Performance, ecosystem and what to learn next

Angela §34

Topic	One line
React.memo / useMemo / useCallback	Skip re-renders of expensive children; measure first
Code splitting	<code>const Page = lazy(() => import("./Page"));</code> wrap in
Error boundaries	Class component or <code>react-error-boundary</code> to catch render crashes
TanStack Query (React Query)	Caching, refetching, loading states for server data — replaces most <code>useEffect+fetch</code>
Zustand / Redux Toolkit	Global state when Context gets messy
React Hook Form + Zod	Forms with validation at scale
Next.js	React framework: file-based routing, server rendering, API routes — the most common React job stack
TypeScript	Types for props/state; strongly recommended after you are comfortable
Testing	Vitest + React Testing Library
DevTools	React Developer Tools browser extension: inspect props/state, profiler

? Chapter quiz

1. Which component do you use for internal links and why?
2. How do you read the `:id` from the URL?
3. Where should `fetch` calls live in a bigger app?
4. What are the three states every async UI must show?

Answers: 1) `Link/NavLink` — an tag reloads the whole page. 2) `useParams()`. 3) A single api module. 4) loading, error, data.

97. Walkthrough 1 — Building a Complete Restaurant Website Line by Line (HTML + CSS)

This chapter builds one real website from an empty folder to a finished responsive page — the way Gamal's and Elzero's final projects do. Every step shows the exact code to add and what you will see. Do not copy-paste: type it, save, refresh, look. If something differs from the picture, compare character by character — that is how you learn to debug.

Step 1 — Folder and skeleton

Gamal HTML #27

Elzero HTML

Angela \$2

Create a folder `nile-grill` with `index.html`, `css/style.css`, and an `images` folder. Start with the skeleton every page needs.

اعمل فولدر فيه index.html و css/style.css و images. ابدأ بالهيكل الأساسي.

EXAMPLE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Nile Grill – Egyptian Restaurant in Giza</title>
  <meta name="description" content="Family restaurant in Giza serving grills, koshari and fresh juices since 1998.">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

</body>
</html>
```

RESULT

Empty white page. Tab title: **Nile Grill — Egyptian Restaurant in Giza**

- The page is blank — correct. Structure first, content next.
- The Google Fonts link loads Poppins; the CSS will use it.

Step 2 — Header and navigation (HTML)

Gamal HTML #8 #20

Angela §2

EXAMPLE

```
<header class="site-header">
  <div class="container nav-wrap">
    <a href="#"
class="logo">Nile<span>Grill</span></a>
    <nav>
      <ul class="nav-links">
        <li><a href="#menu">Menu</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#gallery">Gallery</a></li>
      </ul>
    </nav>
  </div>
</header>
```

RESULT

NileGrill

- Menu
- About
- Gallery
- Contact
- Book a table

Unstyled: a bulleted list of blue links. Normal at this stage.

- The class names describe purpose (site-header, nav-links), not looks.
- The Book link has class btn so we can style it like a button later.

Step 3 — Hero, menu, about, gallery, contact, footer (HTML)

Gamal HTML #27

Elzero HTML

Angela §4

Write ALL the content now. A finished HTML page with zero CSS is a valid milestone.

EXAMPLE

```

<main>
  <section class="hero" id="home">
    <div class="container">
      <h1>Real Egyptian flavour, <br>grilled
fresh every day</h1>
      <p class="lead">Kofta, kebab, koshari
and fresh mango juice – in the heart of Giza
since 1998.</p>
      <a href="#menu" class="btn btn-lg">See
the menu</a>
    </div>
  </section>

  <section class="menu container" id="menu">
    <h2>Our menu</h2>
    <p class="sub">Prices in EGP. Everything
is halal and made daily.</p>
    <div class="cards">
      <article class="card">
        
        <div class="card-body">
          <h3>Kofta plate</h3>
          <p>Charcoal-grilled beef kofta,
rice, salad and tahini.</p>
          <span class="price">120</span>
        </div>
      </article>
      <article class="card">
        
        <div class="card-body">
          <h3>Koshari</h3>
          <p>Rice, lentils, pasta, chickpeas,
tomato sauce, crispy onions.</p>
          <span class="price">45</span>
        </div>
      </article>
      <article class="card">
        
        <div class="card-body">
          <h3>Mango juice</h3>
          <p>Fresh Alphonso mango, no sugar
added.</p>
          <span class="price">35</span>
        </div>
      </article>
    </div>
  </section>

  <section class="about" id="about">
    <div class="container two-col">
      
    <div>
        <h2>About us</h2>
        <p>Nile Grill started as a small cart
near the pyramids. Today we serve 500 guests a
day, but the recipes are still Grandma
Fatma's.</p>
        <ul class="ticks">
            <li>Family owned since 1998</li>
            <li>Fresh meat from local farms</li>
            <li>Free delivery in Giza</li>
        </ul>
    </div>
</div>
</section>

<section class="gallery container"
id="gallery">
    <h2>Gallery</h2>
    <div class="grid-gallery">
        
        
        
        
    </div>
</section>

<section class="contact" id="contact">
    <div class="container two-col">
        <div>
            <h2>Find us</h2>
            <address>12 Pyramids Road,
Giza<br>Daily 12:00 – 01:00<br><a href="tel:
+201141225852">+20 114 122 5852</a></address>
        </div>
        <form id="book" class="book-form"
action="#" method="POST">
            <h3>Book a table</h3>
            <label for="name">Name</label>
            <input id="name" name="name" required>
            <label for="phone">Phone</label>
            <input id="phone" name="phone"
type="tel" required>
            <div class="row">
                <div><label for="date">Date</
label><input id="date" name="date"
type="date" required></div>
                <div><label for="guests">Guests</
label><input id="guests" name="guests"
type="number" min="1" max="12" value="2"></
div>
            </div>
            <button class="btn"
type="submit">Reserve</button>
        </form>

```

```

    </div>
  </section>
</main>

<footer class="site-footer">
  <div class="container">
    <p>© 2026 Nile Grill · Made by Mahmoud
    Khattab</p>
  </div>
</footer>

```

RESULT

A long plain page: big heading, paragraphs, three broken-image icons with alt text, lists, a form with default grey inputs. **Everything readable, nothing pretty.** Validate it now at validator.w3.org.

- Notice every image has a meaningful alt, every input has a label, headings go h1 → h2 → h3 in order.
- Sections have ids so the nav links jump to them already — test the links!

Step 4 — CSS reset, variables, typography

Gamal CSS #1-#5

Elzero CSS #02 #22 #45

Angela §6

Open `css/style.css`. First lines of every project: reset, variables, base text.

EXAMPLE

```

/* 1. reset */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
img { max-width: 100%; display: block; }
ul { list-style: none; }
a { text-decoration: none; color: inherit; }
button, input { font: inherit; }

/* 2. variables */
:root {
  --red: #c0392b;
  --dark: #1e1e1e;
  --grey: #666;
  --light: #faf7f2;
  --white: #fff;
  --radius: 12px;
  --shadow: 0 8px 24px rgba(0,0,0,.08);
  --container: 1100px;
}

/* 3. base */
body { font-family: "Poppins", system-ui, sans-serif; color: var(--dark); line-height: 1.7; background: var(--white); }
h1 { font-size: clamp(2rem, 5vw, 3.5rem); line-height: 1.15; font-weight: 700; }
h2 { font-size: 2rem; margin-bottom: .5rem; }
h3 { font-size: 1.15rem; }
.sub, .lead { color: var(--grey); }
.lead { font-size: 1.15rem; max-width: 52ch; margin: 1rem 0 2rem; }
.container { max-width: var(--container); margin: 0 auto; padding: 0 20px; }
section { padding: 80px 0; }

```

RESULT

Font becomes Poppins, bullets disappear, links turn black, headings scale. Content is centred in a 1100px column.

- Variables mean you can rebrand the whole site by editing five lines.
- `section { padding: 80px 0 }` gives breathing room between sections at once.

Step 5 — Header with Flexbox and the button

Gamal CSS #40-#44

Elzero CSS #46-#48

Angela §8

EXAMPLE

```
.site-header { position: sticky; top: 0; z-index: 50; background: rgba(255,255,255,.92); backdrop-filter: blur(8px); border-bottom: 1px solid #eee; }
.nav-wrap { display: flex; align-items: center; justify-content: space-between; height: 70px; }
.logo { font-weight: 700; font-size: 1.4rem; }
.logo span { color: var(--red); }
.nav-links { display: flex; align-items: center; gap: 28px; }
.nav-links a { font-weight: 600; transition: color .2s; }
.nav-links a:hover { color: var(--red); }

.btn { display: inline-block; background: var(--red); color: var(--white) !important; padding: 10px 22px; border-radius: 999px; font-weight: 600; border: 0; cursor: pointer; transition: transform .15s, box-shadow .15s; }
.btn:hover { transform: translateY(-2px); box-shadow: 0 8px 18px rgba(192,57,43,.35); }
.btn-lg { padding: 14px 32px; font-size: 1.05rem; }
```

RESULT

NileGrill Menu About Gallery
Contact

Book a
table

- sticky + top:0 keeps the header while scrolling; the translucent background with blur is a modern touch.
- The !important on the button colour beats the `a { color: inherit }` rule — a rare justified use.

Step 6 — Hero with background image and overlay

Gamal CSS #16 #17

Elzero CSS #05 #06 #55

Angela \$7

EXAMPLE

```
.hero {
  min-height: 85vh;
  display: grid; align-items: center;
  background: linear-gradient(90deg,
    rgba(0,0,0,.75) 0%, rgba(0,0,0,.35) 60%,
    transparent), url(../images/hero.jpg) center/
  cover no-repeat;
  color: var(--white);
}
.hero h1 { max-width: 16ch; }
.hero .lead { color: #eee; }
```

RESULT

Real Egyptian flavour,
grilled fresh every day

Kofta, kebab, koshari... [See the menu](#)

- The CSS file is in css/, so the image path goes UP one level: `../images/hero.jpg`. The most common broken-image cause.
- The gradient darkens the left side so white text stays readable on any photo.

Step 7 — Menu cards with Grid

Gamal CSS #45 #46

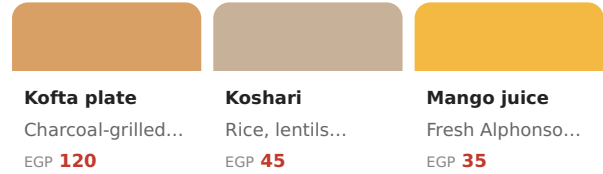
Elzero CSS #57-#63

Angela \$9

EXAMPLE

```
.cards { display: grid; grid-template-columns:
repeat(auto-fit, minmax(280px, 1fr)); gap:
28px; margin-top: 40px; }
.card { background: var(--white); border-
radius: var(--radius); overflow: hidden; box-
shadow: var(--shadow); transition: transform
.25s; }
.card:hover { transform: translateY(-6px); }
.card img { height: 200px; width: 100%;
object-fit: cover; }
.card-body { padding: 20px; position:
relative; }
.card-body p { color: var(--grey); font-size:
.95rem; margin: 6px 0 12px; }
.price { font-weight: 700; color: var(--red);
font-size: 1.2rem; }
.price::before { content: "EGP "; font-size:
.75rem; color: var(--grey); font-weight:
400; }
```

RESULT



- auto-fit + minmax = 3 cards on desktop, 2 on tablet, 1 on phone, with zero media queries.
- object-fit: cover keeps all images the same height without distortion.
- The ::before pseudo-element adds 'EGP' without touching HTML.

Step 8 — About, gallery, contact form

Gamal CSS #45-#47

Elzero CSS #57-#63

Angela \$9

EXAMPLE

```

.about { background: var(--light); }
.two-col { display: grid; grid-template-columns: 1fr 1fr; gap: 48px; align-items: center; }
.two-col img { border-radius: var(--radius); box-shadow: var(--shadow); }
.ticks li { padding-left: 28px; position: relative; margin: 8px 0; }
.ticks li::before { content: "✓"; position: absolute; left: 0; color: var(--red); }

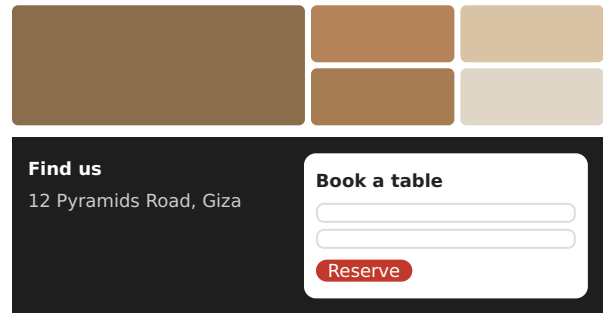
.grid-gallery { display: grid; grid-template-columns: repeat(4, 1fr); grid-auto-rows: 180px; gap: 12px; margin-top: 32px; }
.grid-gallery img { width: 100%; height: 100%; object-fit: cover; border-radius: 8px; }
.grid-gallery img:first-child { grid-column: span 2; grid-row: span 2; } /* one big featured photo */

.contact { background: var(--dark); color: var(--white); }
.contact address { font-style: normal; line-height: 2; color: #ccc; }
.contact address a { color: var(--white); text-decoration: underline; }
.book-form { background: var(--white); color: var(--dark); padding: 32px; border-radius: var(--radius); display: grid; gap: 8px; }
.book-form label { font-size: .85rem; font-weight: 600; margin-top: 6px; }
.book-form input { padding: 12px; border: 1px solid #ddd; border-radius: 8px; width: 100%; }
.book-form input:focus { outline: 2px solid var(--red); border-color: transparent; }
.book-form .row { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
.book-form .btn { margin-top: 12px; justify-self: start; }

.site-footer { background: #111; color: #888; text-align: center; padding: 24px 0; font-size: .9rem; }

```

RESULT



Step 9 — Responsive: media queries and the mobile menu

Gamal CSS #47

Elzero CSS media queries

Angela #9

Open DevTools, toggle the device toolbar (Ctrl+Shift+M) and shrink to 375px. Fix what breaks: the two-column sections, the gallery, the nav.

EXAMPLE

```

@media (max-width: 900px) {
  .two-col { grid-template-columns: 1fr; }
  .grid-gallery { grid-template-columns:
repeat(2, 1fr); grid-auto-rows: 140px; }
  section { padding: 56px 0; }
}

@media (max-width: 700px) {
  h2 { font-size: 1.6rem; }
  .book-form .row { grid-template-columns:
1fr; }
  .grid-gallery img:first-
child { grid-column: span 2; grid-row: span
1; }

  /* mobile nav: hide links, show a toggle */
  .nav-links { position: fixed; inset: 70px 0
auto 0; background: var(--white); flex-
direction: column; padding: 24px; gap: 18px;
transform: translateY(-120%); transition:
transform .3s; box-shadow: var(--shadow); }
  .nav-links.open { transform: none; }
  .menu-btn { display: block; }
}
.menu-btn { display: none; background: none;
border: 0; font-size: 1.6rem; cursor:
pointer; }

```

RESULT



- Add the toggle button in the HTML nav-wrap: `<button class="menu-btn" aria-label="Menu">☰</button>`.
- The JS to open it is three lines (next step).

Step 10 — Ten lines of JavaScript: menu toggle, active link, form message

Elzero JS DOM

Gamal JS

Angela \$17

EXAMPLE

```

class="c"> // js/main.js (add <script src="js/
main.js" defer></script> in head)
const menuBtn = document.querySelector(".menu-
btn");
const links = document.querySelector(".nav-
links");
menuBtn.addEventListener("click", () =>
links.classList.toggle("open"));
links.addEventListener("click", e => { if
(e.target.tagName === "A")
links.classList.remove("open"); });
class="c"> // close after choosing

document.querySelector(".book-
form").addEventListener("submit", e => {
  e.preventDefault();
  const name = e.target.name.value.trim();
  e.target.innerHTML = class="v">`<h3>Thank
you, ${name}!</h3><p>We will call you to
confirm.</p>`;
});

```

RESULT

Thank you, Ahmed!

We will call you to confirm.

- Note: the form content is replaced with text built from the user's name — since we put it in innerHTML, in a real site escape it or use.textContent. Good moment to remember XSS.
- Finish: run Lighthouse, fix contrast/alt warnings, push to GitHub, enable Pages. You have a live portfolio piece.

? Chapter quiz

1. Why is the image path in CSS ../images/... and not images/...?
2. Which single Grid line made the menu responsive with no media query?
3. What does position: sticky need to work?
4. Where did the 'EGP' text come from if it is not in the HTML?

Answers: 1) The CSS file lives in css/, so paths are relative to that folder. 2) grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)). 3) A top (or other offset) value. 4) The ::before pseudo-element with content.

98. Walkthrough 2 — Building a Quiz App Step by Step (JavaScript)

A full JavaScript application, built in small steps: data, rendering, events, scoring, timer, high scores in localStorage. It uses every JS concept from the reference chapters in one place. Type each step, run, and check the console when it does not work.

Step 1 — HTML shell and the data

Elzero JS #073 #091

Angela §20

EXAMPLE

```
<!-- index.html -->
<main class="quiz">
  <header><h1>Web Quiz</h1><div id="meta"><span id="progress">1 / 5</span> · <span id="timer">15</span></div></header>
  <h2 id="question">Loading...</h2>
  <div id="answers"></div>
  <button id="next" hidden>Next</button>
  <section id="result" hidden></section>
</main>
<script src="quiz.js" defer></script>
```

Step 2 — Questions as an array of objects

Elzero JS #091

Angela §16

EXAMPLE

```

class="c">// quiz.js
const questions = [
  { q: "What does HTML stand for?", options: ["Hyper Text Markup Language", "High Tech Modern Language", "Home Tool Markup Language"], answer: 0 },
  { q: "Which CSS property makes text bold?", options: ["font-style", "font-weight", "text-bold"], answer: 1 },
  { q: "Which keyword declares a constant?", options: ["var", "let", "const"], answer: 2 },
  { q: "querySelector returns...", options: ["all matches", "the first match", "a string"], answer: 1 },
  { q: "Which status code means Not Found?", options: ["200", "404", "500"], answer: 1 },
];

class="c">// state – the single source of truth
const state = { index: 0, score: 0, timeLeft: 15, timerId: null, locked: false };

class="c">// element references (once)
const $ = s => document.querySelector(s);
const els = { q: $("#question"), answers: $("#answers"), next: $("#next"), progress: $("#progress"), timer: $("#timer"), result: $("#result") };

```

- Data lives separately from the UI. Later you could load questions from an API with fetch and nothing else changes.
- `answer` stores the index of the correct option.

Step 3 — Render a question

Elzero JS #116-#119

Angela \$18

EXAMPLE

```

function renderQuestion() {
  const current = questions[state.index];
  state.locked = false;
  els.q.textContent = current.q;
  els.progress.textContent = `class="v">` +
    `${state.index + 1} / ${questions.length}`;
  els.answers.innerHTML =
    "";
  class="c">// clear old buttons
  current.options.forEach((text, i) => {
    const btn =
      document.createElement("button");
    btn.className = "answer";
    btn.textContent = text;
    btn.dataset.index =
      i;
    class="c">// remember which option this is
    els.answers.append(btn);
  });
  els.next.hidden = true;
  startTimer();
}
renderQuestion();

```

RESULT

Web Quiz 1 / 5 · 15s

What does HTML stand for?

Hyper Text Markup Language

High Tech Modern Language

Home Tool Markup Language

- `textContent` for the question text (safe); buttons are created with `createElement`.

→ dataset.index links each button to its option number without global variables.

Step 4 — Handle an answer with event delegation

Elzero JS #127 #128

Angela §19

EXAMPLE

```
els.answers.addEventListener("click", e => {
  const btn = e.target.closest(".answer");
  if (!btn || state.locked)
    return;
  class="c"> // ignore
  clicks outside buttons or after answering
  state.locked = true;
  clearInterval(state.timerId);

  const chosen = Number(btn.dataset.index);
  const correct =
    questions[state.index].answer;
  if (chosen === correct) state.score++;

  class="c"> // colour feedback
  [...els.answers.children].forEach((b, i) => {
    b.disabled = true;
    if (i === correct)
      b.classList.add("correct");
    else if (i === chosen)
      b.classList.add("wrong");
  });
  els.next.hidden = false;
});

els.next.addEventListener("click", () => {
  state.index++;
  state.index < questions.length ?
  renderQuestion() : showResult();
});
```

RESULT

Hyper Text Markup Language ✓

High Tech Modern Language ✗

Home Tool Markup Language

Next

→ One listener on the container handles all answer buttons, even though they are recreated each question.

→ **locked** prevents double answers — a state flag instead of removing listeners.

Step 5 — Countdown timer

Elzero JS #130

Angela §19

EXAMPLE

```
function startTimer() {
  clearInterval(state.timerId);           class="c">// never run two timers
  state.timeLeft = 15;
  els.timer.textContent = state.timeLeft;
  state.timerId = setInterval(() => {
    state.timeLeft--;
    els.timer.textContent = state.timeLeft;
    if (state.timeLeft <= 0) {
      clearInterval(state.timerId);
      state.locked = true;
      [...els.answers.children].forEach((b, i) => { b.disabled = true; if (i ===
questions[state.index].answer) b.classList.add("correct"); });
      els.next.hidden = false;
    }
  }, 1000);
}
```

⚠ Common mistake

Forgetting `clearInterval` before starting a new one makes the timer count down twice as fast after the second question. Always clear first.

Step 6 — Results and high score in `localStorage`

Elzero JS #102

Angela \$20

EXAMPLE

```
function showResult() {
  clearInterval(state.timerId);
  document.querySelector(".quiz
header").hidden = true;
  els.q.hidden = true; els.answers.hidden =
true; els.next.hidden = true;

  const best = Math.max(state.score,
Number(localStorage.getItem("bestScore") ||
0));
  localStorage.setItem("bestScore", best);

  const pct = Math.round(state.score /
questions.length * 100);
  els.result.hidden = false;
  els.result.innerHTML = `
    <h2>You scored ${state.score} / $
{questions.length} (${pct}%)</h2>
    <p>${pct === 100 ? "Perfect!" : pct >=
60 ? "Good job — review the ones you
missed." : "Keep studying and try again."}</p>
    <p>Best score: ${best}</p>
    <button id="restart">Play again</button>`;

  document.querySelector("#restart").addEventListener("click",
() => location.reload());
}
```

RESULT

You scored 4 / 5 (80%)

Good job — review the ones you missed.

Best score: 4

[Play again](#)

- localStorage stores strings — Number() when reading.
- Here innerHTML is fine because every value is a number or text we wrote ourselves.

Step 7 — Styles and stretch goals

Gamal CSS

Angela \$20

EXAMPLE

```
.quiz { max-width: 560px; margin: 40px auto; padding: 24px; font-family: system-ui; }
.quiz header { display: flex; justify-content: space-between; color: #666; }
#answers { display: grid; gap: 10px; margin: 20px 0; }
.answer { padding: 14px; border: 2px solid #ddd; border-radius: 10px; background: #fff; text-align: left; cursor: pointer; font-size: 1rem; transition: border-color .2s; }
.answer:hover:not(:disabled) { border-color: #1f77c4; }
.answer.correct { border-color: #27ae60; background: #eafaf1; }
.answer.wrong { border-color: #c0392b; background: #fdecea; }
#timer { font-weight: 700; }
```

Try it yourself

Stretch goals: 1) Shuffle questions and options each game (Fisher-Yates). 2) Load questions from <https://opentdb.com/api.php?amount=5&type=multiple> with fetch (decode HTML entities from the API). 3) Add categories with a select. 4) Animate the timer as a shrinking bar. 5) Store the last 5 scores with dates and show a table.

Solution

```
class="c">
// 1. shuffle
function shuffle(a) {
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

class="c">
// 2. load from API, then start
async function load() {
  const res = await fetch("https://opentdb.com/api.php?amount=5&type=multiple");
  const { results } = await res.json();
  const decode = s => new DOMParser().parseFromString(s, "text/html").body.textContent;
  questions.length = 0;
  results.forEach(r => {
    const options = shuffle([r.correct_answer, ...r.incorrect_answers].map(decode));
    questions.push({ q: decode(r.question), options, answer: options.indexOf(decode(r.correct_answer)) });
  });
  renderQuestion();
}
load();
```

? Chapter quiz

1. Why is the state kept in one object instead of many global variables?
2. What would happen without the locked flag?
3. Why must clearInterval be called at the start of startTimer?
4. Which pattern let one listener handle all answer buttons?

Answers: 1) Easier to reason about, reset and debug; mirrors how frameworks work. 2) A user could click several answers and score multiple times. 3) To stop the previous question's timer, avoiding two intervals running together. 4) Event delegation with e.target.closest.

99. Walkthrough 3 — Full-Stack Notes App: Express API + PostgreSQL + React, Step by Step

The complete chain: a database table, an Express REST API with authentication, and a React front-end that logs in, lists, creates and deletes notes. Each step is small and testable. When it works, you have built the architecture behind most web products.

Step 1 — Database

Angela \$31

EXAMPLE

```
CREATE DATABASE notes_app;
\c notes_app

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE notes (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  body TEXT DEFAULT '',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX ON notes(user_id);
```

- ON DELETE CASCADE: deleting a user deletes their notes automatically.
- The index on user_id makes 'notes of this user' fast.

Step 2 — API project setup

Angela \$25 \$26

EXAMPLE

```
mkdir notes-api && cd notes-api
npm init -y
npm i express pg bcrypt jsonwebtoken cors
npm i -D nodemon
# package.json: "type": "module", "scripts": "dev": "nodemon index.js", "start": "node index.js"

# .env
PORT=3000
DATABASE_URL=postgres://postgres:yourpass@localhost:5432/notes_app
JWT_SECRET=a_long_random_string_here

# .gitignore
node_modules
.env
```

Step 3 — db.js and index.js skeleton

Angela \$26 \$31

EXAMPLE

```

class="c">// db.js
import pg from "pg";
export const pool = new
pg.Pool({ connectionString:
process.env.DATABASE_URL });
export const q = (text, params) =>
pool.query(text, params);

class="c">// index.js
import "dotenv/config";
class="c">// or run: node --env-file=.env
index.js
import express from "express";
import cors from "cors";
import authRoutes from "./routes/auth.js";
import noteRoutes from "./routes/notes.js";

const app = express();
app.use(cors({ origin: "http:localhost:5173" }));
app.use(express.json());

app.get("/api/health", (req, res) =>
res.json({ ok: true }));
app.use("/api/auth", authRoutes);
app.use("/api/notes", noteRoutes);

app.use((req, res) =>
res.status(404).json({ error: "Not found" }));
app.use(err, req, res, next) =>
{ console.error(err); res.status(err.status
|| 500).json({ error: err.message || "Server
error" }); });

app.listen(process.env.PORT || 3000, () =>
console.log("API on http:localhost:3000"));

```

RESULT

```

$ npm run dev
API on http://localhost:3000

$ curl localhost:3000/api/health
{"ok":true}

```

→ Test the health route before writing anything else. If this fails, fix the setup first.

Step 4 — Auth routes: register and login with JWT

Angela §32

EXAMPLE

```

class="c"> // routes/auth.js
import { Router } from "express";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import { q } from "../db.js";
const r = Router();

r.post("/register", async (req, res, next) => {
  try {
    const { email = "", password = "" } = req.body;
    if (!email.includes("@") || password.length < 8) return res.status(400).json({ error: "Valid email and 8+ char password required" });
    const hash = await bcrypt.hash(password, 12);
    const { rows: [user] } = await q("INSERT INTO users (email, password) VALUES ($1, $2) RETURNING id, email", [email.toLowerCase(), hash]);
    res.status(201).json(user);
  } catch (e) {
    if (e.code === "23505") return res.status(409).json({ error: "Email already registered" });
    next(e);
  }
});

r.post("/login", async (req, res, next) => {
  try {
    const { email = "", password = "" } = req.body;
    const { rows: [user] } = await q("SELECT * FROM users WHERE email = $1", [email.toLowerCase()]);
    if (!user || !(await bcrypt.compare(password, user.password))) return res.status(401).json({ error: "Invalid email or password" });
    const token = jwt.sign({ id: user.id, email: user.email }, process.env.JWT_SECRET, { expiresIn: "7d" });
    res.json({ token });
  } catch (e) { next(e); }
});
export default r;

class="c"> // middleware/auth.js
import jwt from "jsonwebtoken";
export function auth(req, res, next) {
  const token = (req.headers.authorization || "").replace("Bearer ", "");
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }

```

```
    catch
  { res.status(401).json({ error: "Login
required" }); }
}
```

RESULT

```
$ curl -X POST localhost:3000/api/auth/register
-H "Content-Type: application/json" -d
'{"email":"a@b.com","password":"secret123"}'
{"id":1,"email":"a@b.com"}

$ curl -X POST localhost:3000/api/auth/login ...
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0:"}
```

Step 5 — Notes routes (owner-scoped CRUD)

Angela §29 §31

EXAMPLE

```

class="c">
// routes/notes.js
import { Router } from "express";
import { q } from "../db.js";
import { auth } from "../middleware/auth.js";
const r = Router();
r.use(auth);
login

class="c">
// everything below requires login

r.get("/", async (req, res, next) => {
  try {
    const { rows } = await q("SELECT * FROM notes WHERE user_id = $1 ORDER BY created_at DESC",
[req.user.id]);
    res.json(rows);
  } catch (e) { next(e); }
});

r.post("/", async (req, res, next) => {
  try {
    const { title, body = "" } = req.body;
    if (!title?.trim()) return res.status(400).json({ error: "Title required" });
    const { rows: [note] } = await q("INSERT INTO notes (user_id, title, body) VALUES ($1, $2,
$3) RETURNING *", [req.user.id, title.trim(), body]);
    res.status(201).json(note);
  } catch (e) { next(e); }
});

r.patch("/:id", async (req, res, next) => {
  try {
    const { title, body } = req.body;
    const { rows: [note] } = await q(
      "UPDATE notes SET title = COALESCE($1, title), body = COALESCE($2, body) WHERE id = $3 AND
user_id = $4 RETURNING *",
      [title, body, req.params.id, req.user.id]);
    note ? res.json(note) : res.status(404).json({ error: "Not found" });
  } catch (e) { next(e); }
});

r.delete("/:id", async (req, res, next) => {
  try {
    const { rowCount } = await q("DELETE FROM notes WHERE id = $1 AND user_id = $2",
[req.params.id, req.user.id]);
    rowCount ? res.status(204).end() : res.status(404).json({ error: "Not found" });
  } catch (e) { next(e); }
});
export default r;

```

- Every query includes `user_id = req.user.id`: users can only ever see or change their own notes. This is authorisation done in the WHERE clause.
- `COALESCE($1, title)` keeps the old value when the client did not send that field — a clean PATCH.

Step 6 — React project and api helper

Angela \$33 \$34

EXAMPLE

```
npm create vite@latest notes-web -- --template react
cd notes-web && npm i && npm i react-router-dom
# .env → VITE_API_URL=http://localhost:3000
npm run dev # http://localhost:5173
```

Step 7 — api.js, auth hook, Login page

Angela §34

EXAMPLE

```

class="c"> // src/api.js
const BASE = import.meta.env.VITE_API_URL;
export async function api(path, { method =
"GET", body } = {}) {
  const token = localStorage.getItem("token");
  const res = await fetch(BASE + path, {
    method,
    headers: { "Content-Type": "application/
json", ...(token && { Authorization:
class="v">`Bearer ${token}` }) },
    body: body && JSON.stringify(body),
  });
  if (res.status === 204) return null;
  const data = await res.json().catch(() =>
({}));
  if (!res.ok) throw new Error(data.error ||
"Request failed");
  return data;
}

```

```

class="c"> // src/pages/Login.jsx
import { useState } from "react";
import { useNavigate } from "react-router-
dom";
import { api } from "../api";
export default function Login() {
  const [form, setForm] = useState({ email:
"", password: "" });
  const [error, setError] = useState("");
  const [mode, setMode] = useState("login");
  const navigate = useNavigate();
  const change = e => setForm({ ...form,
[e.target.name]: e.target.value });

  async function submit(e) {
    e.preventDefault(); setError("");
    try {
      if (mode === "register") await api("/
api/auth/register", { method: "POST", body:
form });
      const { token } = await api("/api/auth/
login", { method: "POST", body: form });
      localStorage.setItem("token", token);
      navigate("/");
    } catch (err) { setError(err.message); }
  }

  return (
    <form onSubmit={submit} className="card">
      <h1>{mode === "login" ? "Log in" :
"Create account"}</h1>
      <input name="email" type="email"
value={form.email} onChange={change}
placeholder="Email" required />
      <input name="password" type="password"
value={form.password} onChange={change}
placeholder="Password (8+)" required

```

```
minLength={8} />
    {error && <p className="error">{error}</p>}
  <button>{mode === "login" ? "Log in" : "Register"}</button>
  <button type="button" className="link"
    onClick={() => setMode(mode === "login" ? "register" : "login")}>
    {mode === "login" ? "No account? Register" : "Have an account? Log in"}
  </button>
</form>
);
}
```

RESULT

Log in[No account? Register](#)

Step 8 — Notes page: list, add, delete

Angela §34

EXAMPLE

```

class="c"> // src/pages/Notes.jsx
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { api } from "../api";

export default function Notes() {
  const [notes, setNotes] = useState([]);
  const [status, setStatus] =
    useState("loading");
  const [form, setForm] = useState({ title:
    "", body: "" });
  const navigate = useNavigate();

  useEffect(() => {
    api("/api/notes").then(d => { setNotes(d);
    setStatus("ready"); })
    .catch(e => { if (e.message === "Login
    required") navigate("/login"); else
    setStatus("error"); });
  }, [navigate]);

  async function add(e) {
    e.preventDefault();
    const note = await api("/api/notes",
    { method: "POST", body: form });
    setNotes([note, ...notes]);
    setForm({ title: "", body: "" });
  }

  async function remove(id) {
    const prev = notes;
    setNotes(notes.filter(n => n.id !==
    id));
    class="c"> // optimistic
    try { await api(class="v"> `/api/notes/$
    {id}`, { method: "DELETE" }); } catch
    { setNotes(prev); }
  }

  function logout()
  { localStorage.removeItem("token");
  navigate("/login"); }

  if (status === "loading") return <p>Loading...
  </p>;
  if (status === "error") return <p
  className="error">Could not load notes.</p>;

  return (
    <main className="notes">
      <header><h1>My notes</h1><button
      onClick={logout} className="link">Log out</
      button></header>
      <form onSubmit={add} className="card">
        <input value={form.title} onChange={e
        => setForm({ ...form, title:
        e.target.value }}} placeholder="Title"

```

```

required />
    <textarea value={form.body}
onChange={e => setForm({ ...form, body:
e.target.value })} placeholder="Write
something..." rows={3} />
    <button>Add note</button>
  </form>
  {notes.length === 0 && <p
className="muted">No notes yet – add your
first one above.</p>}
  <ul className="grid">
    {notes.map(n => (
      <li key={n.id} className="card">
        <h3>{n.title}</h3>
        <p>{n.body}</p>
        <small>{new
Date(n.created_at).toLocaleDateString()}</
small>
        <button onClick={() =>
remove(n.id)} aria-label="Delete">✕</button>
      </li>
    ))}
  </ul>
</main>
);
}

class="c"> // src/App.jsx
import { BrowserRouter, Routes, Route } from
"react-router-dom";
import Login from "../pages/Login"; import
Notes from "../pages/Notes";
export default function App() {
  return <BrowserRouter><Routes><Route
path="/" element={<Notes />} /><Route path="/
login" element={<Login />} /></Routes></
BrowserRouter>;
}

```

RESULT

My notes[Log out](#)

Shopping ✕

Flour, eggs, mango

07/09/2026

Idea ✕

Quiz app v2 with API

06/09/2026

- Three states (loading / error / ready), optimistic delete, redirect to login on 401 — the same patterns you will use in every React app.
- Test flow: register → login → add two notes → refresh (still there: database!) → delete one → log out → try / → redirected.

Step 9 — Deploy the three pieces

Angela \$35

EXAMPLE

1. Database: neon.tech → create project → run the CREATE TABLE statements in its SQL editor → copy connection string.
2. API: push notes-api to GitHub → render.com Web Service → env vars DATABASE_URL, JWT_SECRET, and CORS origin of your future front-end URL → note the API URL.
In db.js add ssl: { rejectUnauthorized: false } if Neon requires it.
3. Front-end: push notes-web → netlify.com → build npm run build, publish dist → env VITE_API_URL=https://your-api.onrender.com → add public/_redirects with "/* /index.html 200".
4. Update cors({ origin: "https://your-site.netlify.app" }) in the API and redeploy.
5. Open the live site on your phone. Register, add a note. Done: a real full-stack app on the internet.

 Try it yourself

Extend the app: edit notes inline (PATCH), search box filtering by title, pin favourite notes (add a boolean column and a toggle), and show a toast on errors instead of alert.

Solution

```
-- SQL
ALTER TABLE notes ADD COLUMN pinned BOOLEAN DEFAULT FALSE;

class="c">// API: PATCH already accepts any field via COALESCE – add pinned to the UPDATE:
class="c">// SET title = COALESCE($1,title), body = COALESCE($2,body), pinned =
COALESCE($3,pinned)

class="c">// React toggle
async function togglePin(n) {
  const updated = await api(class="v">`/api/notes/${n.id}`, { method: "PATCH", body:
{ pinned: !n.pinned } });
  setNotes(notes.map(x => x.id === n.id ? updated : x).sort((a, b) => b.pinned - a.pinned));
}

class="c">// search
const [qtext, setQ] = useState("");
const visible = notes.filter(n => n.title.toLowerCase().includes(qtext.toLowerCase()));
```

? Chapter quiz

1. Where in the API is it guaranteed that a user cannot delete another user's note?
2. Why does the React app send the token in every request, and where does it get it?
3. What does the front-end do when the API answers 401?
4. Why keep VITE_API_URL in an environment variable?

Answers: 1) In the SQL WHERE clause: id = \$1 AND user_id = \$2 with the id from the verified JWT. 2) The API is stateless; api.js reads the token from localStorage and adds the Authorization header. 3) Redirects to /login. 4) It differs between local development and production; changing it must not require code changes.

100. JavaScript Mini-Projects Gallery — 15 Small Apps with Full Code

Each mini-project is complete: HTML, the CSS that matters, and all the JavaScript, in under 60 lines. They mirror the trainings in Elzero's and Gamal's JavaScript courses and Angela's early JS projects. Build one per day; after two weeks you will write DOM code without thinking.

1. Counter with limits

Elzero JS DOM

Angela §17

EXAMPLE

```
<!-- HTML --> <div class="counter"><button
id="dec">-</button><span id="val">0</
span><button id="inc">+</button><button
id="reset">Reset</button></div>

class="c">
// JS
let count = 0; const MIN = 0, MAX = 10;
const val = document.querySelector("#val");
function render() { val.textContent = count;
dec.disabled = count <= MIN; inc.disabled =
count >= MAX; val.style.color = count ===
MAX ? "red" : ""; }
document.querySelector("#inc").onclick = () =>
{ count++; render(); };
document.querySelector("#dec").onclick = () =>
{ count--; render(); };
document.querySelector("#reset").onclick = ()
=> { count = 0; render(); };
render();
```

RESULT



→ State (count) → render() → UI. Never read the number back from the DOM.

2. Tip calculator

Angela §15

EXAMPLE

```
<form id="tip"><input id="bill" type="number" step="0.01" placeholder="Bill"><select
id="pct"><option value="10">10%</option><option value="15" selected>15%</option><option
value="20">20%</option></select><input id="people" type="number" min="1" value="1"><output
id="out"></output></form>

const f = document.querySelector("#tip");
f.addEventListener("input", () => {
  const bill = Number(f.bill.value), pct = Number(f.pct.value), people = Math.max(1,
  Number(f.people.value));
  if (!bill) return f.out.value = "";
  const tip = bill * pct / 100, total = bill + tip;
  f.out.value = `Tip ${tip.toFixed(2)} · Total ${total.toFixed(2)} · Each ${(total /
  people).toFixed(2)} `;
});
```

- The input event on the form catches changes from all fields (bubbling).
- form.elements are reachable as f.bill, f.pct via their id/name.

3. Digital clock with greeting

Elzero JS Date

Angela \$19

EXAMPLE

```
function tick() {
  const now = new Date();
  document.querySelector("#clock").textContent
= now.toLocaleTimeString("en-GB");
  document.querySelector("#date").textContent
= now.toLocaleDateString("en-GB", { weekday:
"long", day: "numeric", month: "long" });
  const h = now.getHours();
  document.querySelector("#greet").textContent
= h < 12 ? "Good morning" : h < 18 ? "Good
afternoon" : "Good evening";
}
tick(); setInterval(tick, 1000);
```

RESULT

14:32:07

Monday, 7 September

Good afternoon

4. Colour flipper / palette generator

Elzero JS trainings

EXAMPLE

```
const hex = () => "#" + Math.floor(Math.random() * 0xffffffff).toString(16).padStart(6, "0");
document.querySelector("#flip").addEventListener("click", () => {
  const c = hex();
  document.body.style.background = c;
  document.querySelector("#code").textContent = c;
});
document.querySelector("#code").addEventListener("click", async e => {
  await navigator.clipboard.writeText(e.target.textContent);
  e.target.textContent += " ✓ copied"; setTimeout(() => e.target.textContent =
e.target.textContent.replace(" ✓ copied", ""), 1000);
});
```

→ toString(16) converts to hexadecimal; padStart guarantees 6 digits.

5. Accordion / FAQ

Elzero JS DOM

Gamal JS

EXAMPLE

```
<div class="acc"><button class="q">Do you deliver?</button><div class="a">Yes, in Giza.</div><button class="q">Opening hours?</button><div class="a">12:00-01:00 daily.</div></div>

class="c"/* CSS */ .a { display: none; padding: 8px 12px; } .a.open { display: block; } .q.active { color: crimson; }

document.querySelector(".acc").addEventListener("click", e => {
  const q = e.target.closest(".q"); if (!q) return;
  const answer = q.nextElementSibling;
  const wasOpen = answer.classList.contains("open");
  document.querySelectorAll(".a.open").forEach(a => a.classList.remove("open"));
class="c">// close others
  document.querySelectorAll(".q.active").forEach(b => b.classList.remove("active"));
  if (!wasOpen) { answer.classList.add("open"); q.classList.add("active"); }
});
```

6. Tabs

Elzero JS DOM

Gamal JS

EXAMPLE

```
<div class="tabs"><button data-tab="menu" class="active">Menu</button><button data-tab="about">About</button></div>
<section id="menu" class="panel">...</section><section id="about" class="panel" hidden>...</section>

document.querySelector(".tabs").addEventListener("click", e => {
  const btn = e.target.closest("[data-tab]"); if (!btn) return;
  document.querySelectorAll(".tabs button").forEach(b => b.classList.toggle("active", b === btn));
  document.querySelectorAll(".panel").forEach(p => p.hidden = p.id !== btn.dataset.tab);
});
```

7. Modal dialog (native dialog element)

Angela §18

EXAMPLE

```
<button id="open">Book a table</button>
<dialog id="dlg"><form method="dialog"><h3>Booking</h3><input placeholder="Name"
required><menu><button value="cancel">Cancel</button><button value="ok">Confirm</button></menu></form></dialog>

const dlg = document.querySelector("#dlg");
document.querySelector("#open").onclick = () => dlg.showModal();
class="c">// traps focus, Esc closes, backdrop
dlg.addEventListener("close", () => console.log("closed with", dlg.returnValue));
dlg.addEventListener("click", e => { if (e.target === dlg) dlg.close("backdrop"); });
class="c">// click outside closes

class="c">/* CSS */ dialog::backdrop { background: rgba(0,0,0,.5); } dialog { border: 0; border-radius: 12px; padding: 24px; }
```

→ The native dialog gives accessibility (focus trap, Escape) for free — prefer it over a hand-made div overlay.

8. Image slider / carousel

Gamal JS project

Elzero JS trainings

EXAMPLE

```
<div class="slider"><button id="prev"></button><button id="next"></button><div id="dots"></div></div>

const images = ["images/1.jpg", "images/2.jpg", "images/3.jpg"];
let i = 0, timer;
const img = document.querySelector("#slide"), dots = document.querySelector("#dots");
dots.innerHTML = images.map( (_, n) => class="v"><button data-n="${n}"></button>` ).join("");
function show(n) {
  i = (n + images.length) % images.length;          class="c">// wrap around both directions
  img.src = images[i];
  dots.querySelectorAll("button").forEach((d, n) => d.classList.toggle("on", n === i));
}
document.querySelector("#next").onclick = () => show(i + 1);
document.querySelector("#prev").onclick = () => show(i - 1);
dots.onclick = e => e.target.dataset.n && show(Number(e.target.dataset.n));
document.addEventListener("keydown", e => { if (e.key === "ArrowRight") show(i + 1);
if (e.key === "ArrowLeft") show(i - 1); });
const auto = () => timer = setInterval(() => show(i + 1), 3000);
auto();
document.querySelector(".slider").addEventListener("mouseenter", () => clearInterval(timer));
document.querySelector(".slider").addEventListener("mouseleave", auto);
show(0);
```

→ The modulo trick `(n + len) % len` makes prev from 0 go to the last image.

9. Live search / filter a list

Elzero JS arrays

Angela \$22

EXAMPLE

```
const products = [{ name: "Kofta", cat: "grill" }, { name: "Koshari", cat: "veg" }, { name: "Kebab", cat: "grill" }, { name: "Mango juice", cat: "drink" }];
const list = document.querySelector("#list"), search = document.querySelector("#search"), cat = document.querySelector("#cat");
function render() {
  const q = search.value.trim().toLowerCase(), c = cat.value;
  const visible = products.filter(p => p.name.toLowerCase().includes(q) && (c === "all" || p.cat === c));
  list.innerHTML = visible.length ? visible.map(p => `<li class="${p.cat}">${p.name} <small>${p.cat}</small></li>`).join("") : "<li class='empty'>No results</li>";
  document.querySelector("#count").textContent = `<span>${visible.length} of ${products.length}</span>`;
}
search.addEventListener("input", render); cat.addEventListener("change", render); render();
```

10. Form validation with live feedback

Elzero JS forms

Angela \$19

EXAMPLE

```
const rules = {
  name: v => v.trim().length >= 2 || "At least 2 characters",
  email: v => /^[S+@S+\.S+$/].test(v) || "Enter a valid email",
  phone: v => /^01[0125]\d{8}$/.test(v) || "Egyptian mobile: 01xxxxxxxxx",
  password: v => v.length >= 8 && /\d/.test(v) || "8+ characters with a number",
};
const form = document.querySelector("#signup");
function validateField(input) {
  const res = rules[input.name](input.value);
  const msg = input.nextElementSibling;
  after each input
  msg.textContent = res === true ? "" : res;
  input.classList.toggle("invalid", res !== true);
  return res === true;
}
form.addEventListener("input", e => rules[e.target.name] && validateField(e.target));
form.addEventListener("submit", e => {
  e.preventDefault();
  const ok = [...form.querySelectorAll("input")].map(validateField).every(Boolean);
  class="c"> map first so ALL get messages
  if (ok) { alert("Registered!"); form.reset(); }
});
```

- Rules as an object of functions: adding a field = adding one line.
- map then every — using every alone would stop at the first failure and leave later fields unmarked.

11. Shopping cart (add, quantity, remove, total, persist)

Elzero JS

Gamal JS project

Angela §22

EXAMPLE

```
const catalog = [{ id: 1, name: "Kofta",
price: 120 }, { id: 2, name: "Koshari",
price: 45 }, { id: 3, name: "Juice", price:
35 }];
let cart =
JSON.parse(localStorage.getItem("cart") ||
"{}");
const save = () =>
localStorage.setItem("cart",
JSON.stringify(cart));

document.querySelector("#catalog").innerHTML =
catalog.map(p => `<li>${p.name} - $
${p.price} <button data-add="${p.id}">Add</
button></li>`).join("");

function render() {
  const rows = Object.entries(cart).map(([id,
qty]) => {
    const p = catalog.find(x => x.id ===
Number(id));
    return `<tr><td>${p.name}</
td><td><button data-dec="${id}">-</button> $
{qty} <button data-inc="${id}">+</button></
td><td>${p.price * qty}</td><td><button data-
del="${id}">x</button></td></tr>`;
  });
  document.querySelector("#cart
tbody").innerHTML = rows.join("") || "<tr><td
colspan=4>Cart is empty</td></tr>";
  const total =
Object.entries(cart).reduce((s, [id, q]) => s
+ catalog.find(x => x.id === Number(id)).price
* q, 0);
  document.querySelector("#total").textContent
= total + " EGP";
  document.querySelector("#badge").textContent
= Object.values(cart).reduce((a, b) => a + b,
0);
  save();
}
document.addEventListener("click", e => {
  const d = e.target.dataset;
  if (d.add) cart[d.add] = (cart[d.add] || 0)
+ 1;
  if (d.inc) cart[d.inc]++;
  if (d.dec) { cart[d.dec]--; if (cart[d.dec]
<= 0) delete cart[d.dec]; }
  if (d.del) delete cart[d.del];
  if (d.add || d.inc || d.dec || d.del)
render();
});
render();
```

RESULT

Item	Qty	Sum	
Kofta	- 2 +	240	×
Juice	- 1 +	35	×

Total: 275 EGP

→ The cart is a map of id → quantity; catalog data is looked up, never duplicated into the cart.

→ One document-level listener with data-* attributes handles every button.

12. Typing speed test

Elzero JS strings

Angela \$19

EXAMPLE

```
const text = "the quick brown fox jumps over the lazy dog";
const target = document.querySelector("#target"), input = document.querySelector("#input"), out = document.querySelector("#out");
target.innerHTML = [...text].map(ch => `<span>${ch}</span>`).join("");
let start = null;
input.addEventListener("input", () => {
  if (!start) start = Date.now();
  const typed = input.value;
  [...target.children].forEach((span, i) => {
    span.className = typed[i] === null ? "" : typed[i] === text[i] ? "ok" : "bad";
  });
  if (typed === text) {
    const minutes = (Date.now() - start) / 60000;
    const wpm = Math.round(text.split(" ").length / minutes);
    out.textContent = `Done! ${wpm} words per minute`;
    input.disabled = true;
  }
});
```

13. Memory card game

Angela \$20

EXAMPLE

```
const icons = ["", "", "", "", "", ""];
const deck = [...icons, ...icons].sort(() => Math.random() - 0.5);
const board = document.querySelector("#board");
board.innerHTML = deck.map((ic, i) => `<button class="card" data-i="${i}"><span>${ic}</span></button>`).join("");
let first = null, lock = false, matched = 0, moves = 0;
board.addEventListener("click", e => {
  const card = e.target.closest(".card");
  if (!card || lock || card.classList.contains("flipped")) return;
  card.classList.add("flipped");
  if (!first) { first = card; return; }
  moves++; document.querySelector("#moves").textContent = moves;
  const same = deck[first.dataset.i] === deck[card.dataset.i];
  if (same) { matched += 2; first = null; if (matched === deck.length) setTimeout(() => alert(`You won in ${moves} moves!`), 300); }
  else { lock = true; setTimeout(() => { first.classList.remove("flipped"); card.classList.remove("flipped"); first = null; lock = false; }, 800); }
});

class="c"/* CSS */ .card span { visibility: hidden; } .card.flipped span { visibility: visible; }
#board { display: grid; grid-template-columns: repeat(4, 70px); gap: 8px; }
```

14. Pomodoro timer with progress ring

Angela §19

EXAMPLE

```
const WORK = 25 * 60, BREAK = 5 * 60;
let mode = "work", left = WORK, id = null;
const time = document.querySelector("#time"), ring = document.querySelector("#ring"), startBtn =
document.querySelector("#start");
function render() {
  const m = String(Math.floor(left / 60)).padStart(2, "0"), s = String(left % 60).padStart(2,
"0");
  time.textContent = `class="v">` + `${m}:${s}`; document.title = `class="v">` + `${m}:${s} - ${mode}`;
  const total = mode === "work" ? WORK : BREAK;
  ring.style.setProperty("--p", `class="v">` + `${(1 - left / total) * 100}%`); `class="c">` + `// conic-
gradient(var(--c) var(--p), #eee 0)
}
function toggle() {
  if (id) { clearInterval(id); id = null; startBtn.textContent = "Start"; return; }
  startBtn.textContent = "Pause";
  id = setInterval(() => {
    left--; render();
    if (left <= 0) { clearInterval(id); id = null; new Audio("ding.mp3").play(); mode = mode ===
"work" ? "break" : "work"; left = mode === "work" ? WORK : BREAK; startBtn.textContent = "Start";
render(); }
  }, 1000);
}
startBtn.onclick = toggle;
document.querySelector("#reset").onclick = () => { clearInterval(id); id = null; left = mode ===
"work" ? WORK : BREAK; startBtn.textContent = "Start"; render(); };
render();
```

15. Infinite scroll feed from an API

Elzero JS fetch

Angela §30

EXAMPLE

```
const feed = document.querySelector("#feed"), sentinel = document.querySelector("#sentinel");
let page = 1, loading = false, done = false;
async function loadMore() {
  if (loading || done) return;
  loading = true; sentinel.textContent = "Loading...";
  try {
    const res = await fetch(class="v">`https: class="c">`//jsonplaceholder.typicode.com/posts?
    _page=${page}&_limit=10`);
    const posts = await res.json();
    if (posts.length === 0) { done = true; sentinel.textContent = "No more posts"; return; }
    feed.insertAdjacentHTML("beforeend", posts.map(p => class="v">`<article><h3>${p.title}</
h3><p>${p.body}</p></article>`).join(""));
    page++; sentinel.textContent = "";
  } catch { sentinel.textContent = "Failed – scroll to retry"; }
  finally { loading = false; }
}
new IntersectionObserver(entries => entries[0].isIntersecting && loadMore(), { rootMargin:
"200px" }).observe(sentinel);
loadMore();
```

- IntersectionObserver fires when the sentinel element approaches the viewport — no scroll math.
- Flags (loading, done) prevent duplicate requests.

? Chapter quiz

1. In the counter, why call render() instead of updating the DOM inside each handler?
2. What does $(n + len) \% len$ achieve in the slider?
3. Why does the cart store only ids and quantities?
4. Which API replaces scroll-event maths for infinite scroll?

Answers: 1) One place draws the UI from state; handlers only change state — fewer bugs, easier features. 2) Wraps the index in both directions so prev from 0 goes to the last item. 3) Product data lives once in the catalog; the cart is just references, so price changes are automatic. 4) IntersectionObserver.

101. HTML & CSS Component Cookbook

— 20 UI Pieces with Code and Result

Buttons, badges, forms, navigation, cards, tooltips — the pieces every site is assembled from, each written in clean HTML and CSS you can paste into any project (no framework). Study how each uses the box model, Flexbox, pseudo-elements and transitions from the CSS reference.

1. Button system (primary, outline, ghost, sizes, disabled, loading)

Gamal CSS

Elzero CSS #39 #42

Angela \$7

EXAMPLE

```
.btn { display: inline-flex; align-items: center; gap: 8px; padding: 10px 18px; border-radius: 8px; border: 2px solid transparent; font-weight: 600; cursor: pointer; transition: .15s; background: #1f77c4; color: #fff; }
.btn:hover { background: #185f9d; }
.btn:focus-visible { outline: 3px solid #9cc7f0; outline-offset: 2px; }
.btn-outline { background: transparent; color: #1f77c4; border-color: currentColor; }
.btn-outline:hover { background: #eaf2fb; }
.btn-ghost { background: transparent; color: #1f77c4; }
.btn-sm { padding: 6px 12px; font-size: .85rem; }
.btn-lg { padding: 14px 26px; font-size: 1.1rem; }
.btn-disabled { opacity: .5; cursor: not-allowed; }
.btn.loading { color: transparent; position: relative; }
.btn.loading::after { content: ""; position: absolute; inset: 0; margin: auto; width: 18px; height: 18px; border: 2px solid #fff; border-top-color: transparent; border-radius: 50%; animation: spin .7s linear infinite; }
@keyframes spin { to { transform: rotate(360deg); } }
```

RESULT



2. Badges and pills

Angela \$11

EXAMPLE

```
.badge { display: inline-block; padding: 2px 10px; border-radius: 999px; font-size: .75rem; font-weight: 600; }
.badge-success { background: #e6f7ec; color: #1b7a3d; }
.badge-danger { background: #fdecea; color: #b3261e; }
.badge-dot::before { content: ""; display: inline-block; width: 6px; height: 6px; border-radius: 50%; background: currentColor; margin-right: 6px; }
```

RESULT



3. Text input with floating label

Gamal CSS #37

Elzero CSS #34

Angela \$7

EXAMPLE

```
<div class="field"><input id="em"
placeholder=" " required><label
for="em">Email</label></div>

.field { position: relative; }
.field
input { width: 100%; padding: 18px 12px 6px; border: 1px solid #ccc; border-radius: 8px; font: inherit; }
.field
label { position: absolute; left: 12px; top: 12px; color: #888; transition: .15s; pointer-events: none; }
.field input:focus + label, .field
input:not(:placeholder-shown) + label { top: 4px; font-size: .7rem; color: #1f77c4; }
.field input:focus { outline: 2px solid #1f77c4; border-color: transparent; }
```

RESULT



→ placeholder=" " (a space) makes :placeholder-shown work as an 'is empty' check.

4. Custom checkbox and toggle switch

Elzero CSS #36

Angela \$7

EXAMPLE

```

<label class="check"><input
type="checkbox"><span></span> Remember me</
label>
<label class="switch"><input
type="checkbox"><span></span></label>

.check input { position: absolute; opacity:
0; }
.check span { display: inline-block; width:
18px; height: 18px; border: 2px solid #999;
border-radius: 4px; vertical-align: middle;
margin-right: 6px; }
.check input:checked + span { background:
#1f77c4; border-color: #1f77c4; background-
image: url("data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' viewBox='0
0 24 24' fill='none' stroke='white' stroke-
width='3'%3E%3Cpath d='M5 13l4 4L19 7'/%3E%3C/
svg%3E"); }
.check input:focus-visible + span { outline:
2px solid #9cc7f0; }

.switch input { position: absolute; opacity:
0; }
.switch span { display: inline-block; width:
44px; height: 24px; background: #ccc; border-
radius: 12px; position: relative; transition:
.2s; }
.switch span::after { content: ""; position:
absolute; top: 2px; left: 2px; width: 20px;
height: 20px; background: #fff; border-radius:
50%; transition: .2s; }
.switch input:checked + span { background:
#1f77c4; }
.switch input:checked + span::after
{ transform: translateX(20px); }

```

RESULT

☒ Remember me ☒

→ Hide the real input with opacity, not display:none, so it stays keyboard-accessible.

5. Dropdown menu (hover + focus)

Gamal CSS #37

Elzero CSS #34

Angela \$7

EXAMPLE

```
<li class="has-menu"><a href="#">Products</a><ul class="menu"><li><a href="#">Grills</a></li><li><a href="#">Drinks</a></li></ul></li>

.has-menu { position: relative; }
.menu { position: absolute; top: 100%; left: 0; min-width: 180px; background: #fff; box-shadow: 0 10px 30px rgba(0,0,0,.12); border-radius: 8px; padding: 8px 0; list-style: none; opacity: 0; visibility: hidden; transform: translateY(8px); transition: .2s; }
.has-menu:hover .menu, .has-menu:focus-within .menu { opacity: 1; visibility: visible; transform: none; }
.menu a { display: block; padding: 8px 16px; } .menu a:hover { background: #f3f6fa; }
```

RESULT

Products ▼

Grills

Drinks

→ :focus-within opens it for keyboard users too.

6. Tooltip with pseudo-element

Elzero CSS #36

Angela §7

EXAMPLE

```
<button data-tip="Add to favourites">♥</button>

[data-tip] { position: relative; }
[data-tip]::after { content: attr(data-tip); position: absolute; bottom: calc(100% + 8px); left: 50%; transform: translateX(-50%) translateY(4px); background: #222; color: #fff; padding: 6px 10px; border-radius: 6px; font-size: .8rem; white-space: nowrap; opacity: 0; pointer-events: none; transition: .15s; }
[data-tip]:hover::after, [data-tip]:focus-visible::after { opacity: 1; transform: translateX(-50%); }
```

RESULT

Add to favourites



7. Alert / toast

Angela §11

EXAMPLE

```
.alert { display: flex; gap: 12px; align-items: flex-start; padding: 14px 16px; border-radius: 10px; border-left: 4px solid; }
.alert-info { background: #eaf2fb; border-color: #1f77c4; }
.alert-success { background: #e6f7ec; border-color: #1b7a3d; }
.alert-error { background: #fdecea; border-color: #b3261e; }

.toast { position: fixed; bottom: 24px; right: 24px; background: #222; color: #fff; padding: 12px 18px; border-radius: 10px; animation: toast-in .3s ease-out, toast-out .3s 2.7s forwards; }
@keyframes toast-in { from { transform: translateY(20px); opacity: 0; } }
@keyframes toast-out { to { opacity: 0; } }
```

RESULT

✓ Order saved

✗ Payment failed

8. Avatar, avatar group, and status dot

Angela §7

EXAMPLE

```
.avatar { width: 40px; height: 40px; border-radius: 50%; object-fit: cover; border: 2px solid #fff; }
.avatars { display: flex; }
.avatars .avatar:not(:first-child) { margin-left: -12px; }
.status { position: relative; display: inline-block; }
.status::after { content: ""; position: absolute; right: 0; bottom: 0; width: 10px; height: 10px; border-radius: 50%; background: #1b7a3d; border: 2px solid #fff; }
```

RESULT



9. Breadcrumbs and pagination

Angela §11

EXAMPLE

```
.crumbs { display: flex; gap: 8px; list-style: none; font-size: .9rem; color: #666; }
.crumbs li + li::before { content: ">"; margin-right: 8px; }
.pager { display: flex; gap: 6px; list-style: none; }
.pager a { display: grid; place-items: center; min-width: 36px; height: 36px; border-radius: 8px; border: 1px solid #ddd; }
.pager a.current { background: #1f77c4; color: #fff; border-color: #1f77c4; }
```

RESULT

Home > Menu > **Grills**

10. Progress bar and skeleton loader

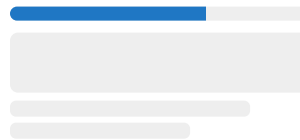
Angela §11

EXAMPLE

```
.progress { height: 10px; background: #eee; border-radius: 5px; overflow: hidden; }
.progress > div { height: 100%; background: #1f77c4; width: var(--value, 0%); transition: width .4s; }

.skeleton { background: linear-gradient(90deg, #eee 25%, #f5f5f5 50%, #eee 75%); background-size: 200% 100%; animation: shimmer 1.2s infinite; border-radius: 6px; }
@keyframes shimmer { to { background-position: -200% 0; } }
.skeleton.text { height: 14px; margin: 6px 0; }
.skeleton.img { aspect-ratio: 16/9; }
```

RESULT



11. Responsive table → cards on mobile

Gamal CSS #23

Elzero CSS #33

Angela §9

EXAMPLE

```
@media (max-width: 600px) {
  table, thead, tbody, tr, th, td { display:
  block; }
  thead { display: none; }
  tr { border: 1px solid #ddd; border-radius:
  8px; padding: 8px; margin-bottom: 10px; }
  td { display: flex; justify-content: space-
  between; padding: 6px 0; }
  td::before { content: attr(data-label);
  font-weight: 600; color: #666; }
}
<!-- HTML: <td data-label="Price">120</td> -->
```

RESULT

Item	Kofta
Price	120
Qty	2

12. Testimonial card with quote mark

Elzero CSS #36

Angela §11

EXAMPLE

```
.quote { position: relative; padding: 32px
24px 24px; background: #fff; border-radius:
16px; box-shadow: 0 8px 24px
rgba(0,0,0,.06); }
.quote::before { content: "\201C"; position:
absolute; top: -10px; left: 16px; font-size:
5rem; line-height: 1; color: #1f77c4; opacity:
.25; font-family: Georgia, serif; }
.quote footer { display: flex; align-items:
center; gap: 12px; margin-top: 16px; font-
size: .9rem; color: #666; }
```

RESULT

“Best koshari in Giza. We
come every Friday.

 Sara M.

13. Image hover overlay (gallery caption)

Gamal CSS #30

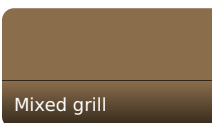
Elzero CSS #30 #42

Angela §7

EXAMPLE

```
.tile { position: relative; overflow: hidden;
border-radius: 12px; }
.tile img { display: block; width: 100%;
transition: transform .4s; }
.tile .cap { position: absolute; inset: auto 0
0 0; padding: 16px; color: #fff; background:
linear-gradient(transparent, rgba(0,0,0,.7));
transform: translateY(100%); transition:
.3s; }
.tile:hover img { transform: scale(1.06); }
.tile:hover .cap, .tile:focus-within .cap
{ transform: none; }
```

RESULT



14. Stepper / progress steps

Angela §8

EXAMPLE

```
.steps { display: flex; list-style: none;
counter-reset: step; }
.steps li { flex: 1; text-align: center;
position: relative; color: #999; font-size:
.85rem; }
.steps li::before { counter-increment: step;
content: counter(step); display: grid; place-
items: center; width: 32px; height: 32px;
margin: 0 auto 6px; border-radius: 50%;
background: #eee; font-weight: 700; }
.steps li:not(:last-child)::after { content:
""; position: absolute; top: 16px; left: 50%;
width: 100%; height: 2px; background: #eee; z-
index: -1; }
.steps li.done::before, .steps li.done::after
{ background: #1f77c4; color: #fff; }
.steps li.active::before { background:
#1f77c4; color: #fff; box-shadow: 0 0 4px
#dbe7f7; }
```

RESULT

1 2 3
Cart Address Pay

→ CSS counters number the steps automatically — add an li and it rennumbers.

15. Sticky call-to-action bar on mobile

Elzero CSS #30

Angela §9

EXAMPLE

```
.cta-bar { display: none; }
@media (max-width: 700px) {
.cta-bar { display: flex; gap: 12px;
position: fixed; bottom: 0; left: 0; right: 0;
padding: 12px 16px calc(12px + env(safe-area-
inset-bottom)); background: #fff; box-shadow:
0 -4px 16px rgba(0,0,0,.08); z-index: 40; }
.cta-bar .btn { flex: 1; }
body { padding-bottom: 80px; } /* so
content is not hidden behind it */
}
```

RESULT



→ env(safe-area-inset-bottom) respects the iPhone home bar.

16. Accordion with only HTML + CSS (details)

Angela §3

EXAMPLE

```

details { border: 1px solid #e5e5e5; border-radius: 10px; padding: 0 16px; margin-bottom: 8px; }
summary { list-style: none; cursor: pointer; padding: 14px 0; font-weight: 600; display: flex; justify-content: space-between; }
summary::-webkit-details-marker { display: none; }
summary::after { content: "+"; font-size: 1.3em; transition: transform .2s; }
details[open] summary::after { transform: rotate(45deg); }
details p { padding-bottom: 14px; color: #555; }

```

RESULT

17. Search bar with icon and clear button

Angela \$7

EXAMPLE

```

<form class="search" role="search"><input type="search" placeholder="Search dishes..." aria-label="Search"><button aria-label="Search"></button></form>

.search { display: flex; align-items: center; border: 1px solid #ccc; border-radius: 999px; padding: 4px 4px 4px 16px; max-width: 420px; }
.search:focus-within { border-color: #1f77c4; box-shadow: 0 0 0 3px #dbe7f7; }
.search input { flex: 1; border: 0; outline: 0; font: inherit; background: none; }
.search button { border: 0; background: #1f77c4; color: #fff; width: 36px; height: 36px; border-radius: 50%; cursor: pointer; }

```

RESULT

18. Notification bell with count

Elzero CSS #30 #31

Angela \$7

EXAMPLE

```

.bell { position: relative; display: inline-grid; place-items: center; width: 40px; height: 40px; border-radius: 50%; }
.bell:hover { background: #f3f3f3; }
.bell .count { position: absolute; top: 4px; right: 4px; min-width: 18px; height: 18px; padding: 0 5px; border-radius: 9px; background: #c0392b; color: #fff; font-size: .7rem; display: grid; place-items: center; }

```

RESULT



19. Two-tone section divider (wave / slant)

Angela \$7

EXAMPLE

```
.slant { position: relative; background:
#1e1e1e; color: #fff; padding: 80px 0; clip-
path: polygon(0 0, 100% 6%, 100% 100%, 0
94%); }
/* wave via inline SVG background */
.wave { background: #faf7f2 url("data:image/
svg+xml,%3Csvg xmlns='http://www.w3.org/2000/
svg' viewBox='0 0 1440 80'%3E%3Cpath
fill='%23ffffff' d='M0 40 C 360 80 1080 0 1440
40 L1440 0 L0 0 Z'/%3E%3C/svg%3E") top/100%
80px no-repeat; padding-top: 100px; }
```

RESULT



20. Print-friendly invoice

Angela \$9

EXAMPLE

```
.invoice { max-width: 800px; margin: auto;
padding: 40px; }
.invoice
table { width: 100%; border-collapse:
collapse; } .invoice td, .invoice th
{ padding: 10px; border-bottom: 1px solid
#eee; text-align: left; }
.invoice td:last-child, .invoice th:last-
child { text-align: right; }
.invoice tfoot td { font-weight: 700; border-
top: 2px solid #222; }
@media print { nav, .no-print { display:
none !important; } .invoice { padding: 0; }
body { font-size: 12pt; } @page { margin:
20mm; } }
```

RESULT

INVOICE #1042

Kofta × 2	240
Juice × 1	35
Total	275

? Chapter quiz

1. Why hide a custom checkbox's real input with opacity instead of display: none?
2. Which pseudo-class opens a dropdown for keyboard users?
3. How does the responsive table show column names on mobile?
4. What CSS feature numbers the stepper automatically?

Answers: 1) display: none removes it from keyboard focus and screen readers; opacity keeps it functional.
 2) :focus-within. 3) td::before with content: attr(data-label). 4) CSS counters (counter-reset / counter-increment / counter()).

102. Tools Reference A — Git and GitHub Complete

Git records the history of your project so you can go back, experiment safely, and work with others. GitHub hosts your repositories online and is your public portfolio — every employer will look at it. You will use ten commands 95% of the time; they are all here with the exact situations where you need them.

The mental model: working folder → staging → commits

Angela §23

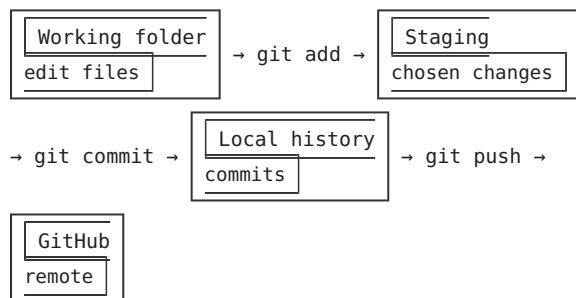
You edit files in the working folder. You choose which changes to include (stage them). You take a snapshot (commit) with a message. The chain of commits is the history. A remote (GitHub) is a copy of that history online.

Git يحفظ نسخ (commits) من مشروعك عشان ترجع لأي وقت. GitHub بيرفع النسخ دي أونلاين ويبقى معرض شغلك.

EXAMPLE

```
git init                # start
                        tracking this folder (once per project)
git status              # what
                        changed? ← run this constantly
git add index.html      # stage one
                        file
git add .               # stage
                        everything
git commit -m "Add product page" # snapshot
                        with message
git log --oneline       # history:
                        abc1234 Add product page
git diff               # unstaged
                        changes line by line
git diff --staged      # what will
                        be committed
```

RESULT



- First-time setup: `git config --global user.name "Your Name"` and `git config --global user.email "you@mail.com"`.
- Good commit messages: imperative, short, specific — 'Fix navbar overlap on mobile', not 'changes' or 'asdf'.
- Commit small and often; each commit should be one logical change.

.gitignore

Angela §23

Files Git must never track: dependencies, secrets, build output, editor junk. Create it BEFORE the first commit.

```
EXAMPLE
# .gitignore
node_modules/
.env
.env.*
!.env.example
dist/
build/
.DS_Store
Thumbs.db
*.log
.vscode/
coverage/
```

⚠ Common mistake

Committed `node_modules` or `.env` by mistake? `git rm -r --cached node_modules`, add it to `.gitignore`, commit. If a secret was pushed, change the secret — history is public.

GitHub: push an existing project, clone, pull

Angela §23

```
EXAMPLE
# 1. create an EMPTY repo on github.com (no README), copy its URL, then:
git remote add origin https://github.com/USER/REPO.git
git branch -M main
git push -u origin main          # first push; later just: git push

# get a project from GitHub
git clone https://github.com/USER/REPO.git
cd REPO && npm install

# get the latest changes from the remote
git pull

# see remotes
git remote -v
```

- Authentication: GitHub no longer accepts passwords for push — use a Personal Access Token or the GitHub CLI (`gh auth login`) or SSH keys.
- README.md at the repo root is shown on the repo page: describe the project, screenshot, how to run, live link.

Branches, merging and pull requests

Angela §23

A branch is a parallel line of work. Build each feature on its own branch, then merge into main. On GitHub, a pull request (PR) is the page where a branch is reviewed and merged.

EXAMPLE

```
git branch                # list; *
                           marks the current one
git switch -c feature/login
# create and switch (old: git checkout -b)
# ... commit work ...
git switch main
git merge feature/login    # bring it
                           into main
git branch -d feature/login # delete
                           merged branch

# team flow
git switch -c fix/navbar
git push -u origin fix/navbar # → on
GitHub: "Compare & pull request" → review →
Merge
git switch main && git pull    # update
                           local main afterwards
```

RESULT

```
main:    A — B ————— F (merge)
          \               /
feature:  C — D — E
```

Merge conflicts

Angela §23

When two branches changed the same lines, Git asks you to choose. The file gets markers; edit it to the final content, remove the markers, add, commit.

EXAMPLE

```
<<<<<<< HEAD
<h1>Giza Bakery</h1>
=====
<h1>Giza Bakery & Café</h1>
>>>>>> feature/renamed
```

→ decide, e.g. keep the second line, delete the three marker lines, then:

```
git add index.html
git commit -m "Merge feature/renamed"
```

- VS Code shows 'Accept Current / Incoming / Both' buttons above each conflict.
- Fewer conflicts: pull main often and keep branches short-lived.

Undo anything

Angela §23

Situation	Command
Discard changes in a file (not staged)	<code>git restore file.js</code>
Unstage a file (keep the change)	<code>git restore --staged file.js</code>
Fix the last commit message / add forgotten file	<code>git commit --amend</code>
Undo last commit but keep the changes	<code>git reset --soft HEAD~1</code>
Undo last commit and throw away changes	<code>git reset --hard HEAD~1</code> (dangerous)
Undo a pushed commit safely	<code>git revert abc1234</code> (makes a new opposite commit)
See an old version of a file	<code>git show abc1234:src/app.js</code>
Go look at an old commit	<code>git checkout abc1234</code> (then <code>git switch main</code> to come back)
Save work temporarily	<code>git stash</code> → later <code>git stash pop</code>
Find who changed a line	<code>git blame file.js</code>
I deleted a branch/commit by mistake	<code>git reflog</code> shows everything; <code>git switch -c rescue abc1234</code>

Forking and contributing to open source

Angela §23

EXAMPLE

```
# 1. Fork on GitHub (your own copy) 2. clone your fork
git clone https://github.com/YOU/project.git
git remote add upstream https://github.com/ORIGINAL/project.git
git switch -c fix-typo
# edit, commit
git push -u origin fix-typo      # → open a pull request to ORIGINAL
# keep your fork updated
git fetch upstream && git merge upstream/main
```

GitHub Pages: free hosting for static sites

Angela §23

Any repo with an `index.html` can be a live website in two minutes. Perfect for portfolios and HTML/CSS/JS projects.

EXAMPLE

```
# repo → Settings → Pages → Source: Deploy from branch → main / (root) → Save
# URL: https://USERNAME.github.io/REPO/
# Portfolio: name the repo USERNAME.github.io → https://USERNAME.github.io

# React (Vite) on GitHub Pages: set base in vite.config.js → base: "/REPO/", then
npm i -D gh-pages
# package.json scripts: "deploy": "vite build && gh-pages -d dist"
npm run deploy
```

? Chapter quiz

1. Order of the three steps to save a change in Git?
2. What file prevents node_modules from being committed?
3. Difference between git merge and a pull request?
4. How do you undo a commit that is already on GitHub?

Answers: 1) edit → git add → git commit. 2) .gitignore. 3) merge is the Git command; a PR is GitHub's review page that ends in a merge. 4) git revert then push.

103. Tools Reference B — Bootstrap 5 Complete

Bootstrap is a CSS framework: a big stylesheet of ready-made classes (grid, buttons, cards, navbars, modals) so you can build a clean responsive page fast without writing much CSS. Angela's TinDog project is built with it. Learn the grid and the spacing utilities first — they are 80% of daily use.

Setup and the container/row/col grid

Angela §10 §11

Link the CSS in head and the JS bundle before the closing body. The grid: a **container** centres content; a **row** holds columns; each **col** spans some of 12 units. Breakpoint prefixes (sm md lg xl xxl) make different layouts per screen size.

Bootstrap شوية classes جاهزة. الشبكة 12 عمود: container ثم row ثم col-md-6 يعني نص العرض على الشاشات المتوسطة وأكبر.

Prefix	Applies from	Typical device
(none) col-	0px	phones
sm	≥576px	large phones
md	≥768px	tablets
lg	≥992px	laptops
xl	≥1200px	desktops
xxl	≥1400px	large desktops

EXAMPLE

```

<link href="https://cdn.jsdelivr.net/npm/
bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/
bootstrap@5.3.3/dist/js/
bootstrap.bundle.min.js"></script>

<div class="container">                                <!--
container-fluid = full width -->
  <div class="row g-3">                                <!-- g-3
= gap between columns -->
    <div class="col-12 col-md-6 col-lg-4">1/3
on large, 1/2 on medium, full on phones</div>
    <div class="col-12 col-md-6 col-lg-4">...</
div>
    <div class="col-12 col-md-12 col-lg-4">...</
div>
  </div>
  <div class="row">
    <div class="col">auto</div><div
class="col">auto</div>
<!-- equal widths -->
    <div class="col-auto">as wide as content</
div>
  </div>
  <div class="row row-cols-1 row-cols-md-3">
<!-- every child becomes a column: 1 per row,
3 on md+ -->
    <div class="col">...</div>
  </div>
  <div class="row justify-content-center
align-items-center" style="min-height:200px">...
</div>
</div>

```

RESULT

Large screen

col-lg-4

col-lg-4

col-lg-4

Phone

col-12

col-12

col-12

- Mobile first: `col-md-6` means 'from md and up, half width' — below md it is full width.
- Columns must be direct children of a row; rows must be inside a container (or another col).

Spacing, sizing, display, flex utilities

Angela §10

Utility classes set one CSS property each. Pattern for spacing: {property}{side}-{size}: m = margin, p = padding; t r b l x y; sizes 0–5 (0, 4px, 8px, 16px, 24px, 48px) and auto.

Utility family	Examples
Spacing	m-0...m-5, p-*, mt-, mb-, ms-, me-, mx-, my-, gap-*, mx-auto, mt-md-4
Display	d-none d-block d-inline d-flex d-grid, d-{bp}-*
Flex	justify-content-{start,center,end,between,around}, align-items-*, flex-{row,column,wrap,fill,grow-1}, order-*
Text	text-{start,center,end}, fw-{bold,semibold,normal,light}, fs-1...6, text-{muted,primary,danger...}, text-uppercase, text-decoration-none
Colours	text-*, bg-*, border-*: primary secondary success danger warning info light dark white
Sizing	w-25 w-50 w-75 w-100 w-auto, h-*, vw-100 vh-100, mw-100
Borders/ shadow	border, border-0, border-top, rounded, rounded-circle, rounded-pill, shadow-sm shadow shadow-lg
Position	position-{static,relative,absolute,fixed,sticky}, top-0 start-50 translate-middle, fixed-top, sticky-top
Visibility	visible invisible, overflow-hidden, opacity-50, user-select-none

EXAMPLE

```

<div class="mt-3 mb-5 px-4 py-2 mx-auto">...</div>
<!-- margin-top 16px, bottom 48px, padding x 24px, y 8px, centred -->
<div class="w-100 w-md-50 h-100 mw-100 vh-100">           <!-- widths/heights in % -->
<div class="d-none d-md-block">desktop only</div>
<div class="d-block d-md-none">mobile only</div>
<div class="d-flex justify-content-between align-items-center gap-2 flex-wrap flex-column flex-md-row">
  <span class="text-center text-md-start">...</span>
  <div class="position-relative"><span class="position-absolute top-0 end-0 badge bg-danger">3</span></div>
             <!-- responsive image -->
  <div class="border border-2 border-primary rounded-3 shadow-sm overflow-hidden">
    <p class="fw-bold fs-4 text-uppercase text-muted lh-sm text-truncate">...</p>
  <div class="bg-primary text-white bg-opacity-75 p-3">...</div>
  <div class="ratio ratio-16x9"><iframe src="..."></iframe></div>

```

Typography, buttons, badges, alerts

Angela §11

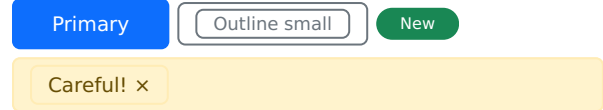
EXAMPLE

```

<h1 class="display-4 fw-bold">Big hero title</h1>
<!-- display-1...6 -->
<p class="lead">Intro paragraph slightly larger.</p>
<button class="btn btn-primary">Primary</button>
<button class="btn btn-outline-secondary btn-sm">Outline small</button>
<button class="btn btn-danger btn-lg w-100 disabled">Full width</button>
<a class="btn btn-link" href="#">Link button</a>
<span class="badge bg-success rounded-pill">New</span>
<div class="alert alert-warning alert-dismissible fade show" role="alert">
  Careful! <button class="btn-close" data-bs-dismiss="alert"></button>
</div>
<div class="spinner-border text-primary" role="status"></div>
<div class="progress"><div class="progress-bar" style="width:70%">70%</div></div>

```

RESULT



Navbar

Angela §11

The responsive navigation bar collapses into a hamburger on small screens — needs the JS bundle.

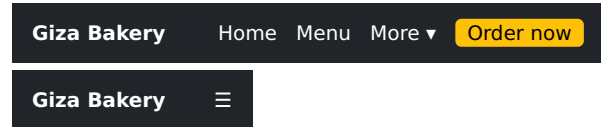
EXAMPLE

```

<nav class="navbar navbar-expand-lg bg-dark
navbar-dark sticky-top">
  <div class="container">
    <a class="navbar-brand fw-bold"
href="#">Giza Bakery</a>
    <button class="navbar-toggler" data-bs-
toggle="collapse" data-bs-target="#nav" aria-
controls="nav" aria-expanded="false" aria-
label="Toggle navigation">
      <span class="navbar-toggler-icon"></
span>
    </button>
    <div class="collapse navbar-collapse"
id="nav">
      <ul class="navbar-nav ms-
auto">                                <!-- ms-auto
pushes links right -->
        <li class="nav-item"><a class="nav-
link active" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-
link" href="#menu">Menu</a></li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle"
data-bs-toggle="dropdown" href="#">More</a>
          <ul class="dropdown-menu"><li><a
class="dropdown-item" href="#">About</a></
li></ul>
        </li>
        <li class="nav-item"><a class="btn
btn-warning ms-lg-3" href="#order">Order now</
a></li>
      </ul>
    </div>
  </div>
</nav>

```

RESULT



Cards, list groups, tables

Angela \$11

EXAMPLE

```

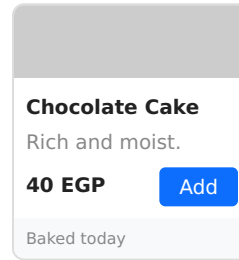
<div class="card h-100 shadow-sm">
  
  <div class="card-body d-flex flex-column">
    <h5 class="card-title">Chocolate Cake</h5>
    <p class="card-text text-muted">Rich and
    moist.</p>
    <div class="mt-auto d-flex justify-
    content-between align-items-center">
      <span class="fw-bold">40 EGP</span>
      <a href="#" class="btn btn-primary btn-
      sm">Add</a>
    </div>
  </div>
  <div class="card-footer text-muted
  small">Baked today</div>
</div>

<ul class="list-group">
  <li class="list-group-item d-flex justify-
  content-between">Bread <span class="badge bg-
  primary">5</span></li>
  <li class="list-group-item active">Cake</li>
</ul>

<table class="table table-striped table-hover
table-bordered align-middle">
  <thead class="table-dark"><tr><th>#</
th><th>Product</th><th>Price</th></tr></thead>
  <tbody><tr><td>1</td><td>Bread</td><td>5</
td></tr></tbody>
</table>
<div class="table-responsive">...wide table
scrolls on phones...</div>

```

RESULT



Forms

Angela §11

EXAMPLE

```

<form class="row g-3 needs-validation" novalidate>
  <div class="col-md-6">
    <label for="name" class="form-label">Name</label>
    <input id="name" class="form-control" required>
    <div class="invalid-feedback">Please enter your name.</div>
  </div>
  <div class="col-md-6">
    <label for="email" class="form-label">Email</label>
    <div class="input-group">
      <span class="input-group-text">@</span>
      <input id="email" type="email" class="form-control form-control-lg"
placeholder="you@mail.com">
    </div>
  </div>
  <div class="col-12">
    <select class="form-select"><option>Choose...</option></select>
  </div>
  <div class="col-12">
    <div class="form-check form-switch">
      <input class="form-check-input" type="checkbox" id="news">
      <label class="form-check-label" for="news">Subscribe</label>
    </div>
  </div>
  <div class="col-12 form-floating">
    <textarea class="form-control" id="msg" style="height:100px"></textarea>
    <label for="msg">Message</label>      <!-- floating label: label AFTER input -->
  </div>
  <div class="col-12"><button class="btn btn-primary">Send</button></div>
</form>

```

Interactive components: modal, dropdown, accordion, carousel, toast, offcanvas

Angela §11

They work with `data-bs-*` attributes and no JavaScript of your own (the bundle handles it). You can also control them from JS.

Component	Trigger attribute
Modal	<code>data-bs-toggle="modal" data-bs-target="#id"</code>
Dropdown	<code>data-bs-toggle="dropdown"</code>
Collapse / Accordion	<code>data-bs-toggle="collapse" data-bs-target="#id"</code>
Tabs / Pills	<code>data-bs-toggle="tab" data-bs-target="#pane"</code>
Tooltip / Popover	<code>data-bs-toggle="tooltip" title="..."</code> (needs JS init)
Offcanvas (side drawer)	<code>data-bs-toggle="offcanvas" data-bs-target="#id"</code>
Toast	JS: <code>new bootstrap.Toast(el).show()</code>
Carousel	<code>data-bs-ride="carousel" + data-bs-slide</code>

EXAMPLE

```

<!-- modal -->
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#orderModal">Order</button>
<div class="modal fade" id="orderModal" tabindex="-1">
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header"><h5 class="modal-title">Your order</h5><button class="btn-close"
data-bs-dismiss="modal"></button></div>
      <div class="modal-body">...</div>
      <div class="modal-footer"><button class="btn btn-secondary" data-bs-dismiss="modal">Close</
button><button class="btn btn-primary">Confirm</button></div>
    </div>
  </div>
</div>

<!-- accordion -->
<div class="accordion" id="faq">
  <div class="accordion-item">
    <h2 class="accordion-header"><button class="accordion-button collapsed" data-bs-
toggle="collapse" data-bs-target="#q1">Do you deliver?</button></h2>
    <div id="q1" class="accordion-collapse collapse" data-bs-parent="#faq"><div class="accordion-
body">Yes, in Giza.</div></div>
  </div>
</div>

<!-- carousel -->
<div id="hero" class="carousel slide" data-bs-ride="carousel">
  <div class="carousel-inner">
    <div class="carousel-item active"></div>
    <div class="carousel-item"></div>
  </div>
  <button class="carousel-control-prev" data-bs-target="#hero" data-bs-slide="prev"><span
class="carousel-control-prev-icon"></span></button>
  <button class="carousel-control-next" data-bs-target="#hero" data-bs-slide="next"><span
class="carousel-control-next-icon"></span></button>
</div>

<!-- control from JS -->
<script>
  const modal = new bootstrap.Modal("#orderModal"); modal.show(); modal.hide();
  new bootstrap.Toast("#saved").show();
  document.querySelectorAll("[data-bs-toggle=tooltip]").forEach(el => new bootstrap.Tooltip(el));
</script>

```

Customising Bootstrap and Bootstrap Icons

Angela §11

EXAMPLE

```

/* override AFTER the Bootstrap link, or set CSS variables */
:root { --bs-primary: #c0392b; --bs-body-font-family: "Cairo", sans-serif; }
.btn-primary { --bs-btn-bg: #c0392b; --bs-btn-border-color: #c0392b; --bs-btn-hover-bg: #a93226; }
/* dark mode: <html data-bs-theme="dark"> */

/* Icons */
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">
<i class="bi bi-cart3 fs-4"></i> <i class="bi bi-heart-fill text-danger"></i> <i class="bi bi-
whatsapp"></i>

/* with Sass (npm i bootstrap sass): change $primary before importing */
$primary: #c0392b;
@import "bootstrap/scss/bootstrap";

```

- Bootstrap is fast for dashboards and MVPs; many product sites use Tailwind or custom CSS. Knowing plain CSS (Part 2) is what makes you able to customise either.
- Alternatives: Tailwind CSS (utility-first), Bulma, Materialize, and component libraries for React (MUI, Chakra, shadcn/ui).

Project: TinDog landing page (Angela)

Angela \$11 project

A full landing page using only Bootstrap classes: navbar, hero with two columns, features row, testimonial carousel, pricing cards, footer.

EXAMPLE

```

<section class="bg-primary text-white py-5">
  <div class="container">
    <div class="row align-items-center">
      <div class="col-lg-6 text-center text-
lg-start">
        <h1 class="display-4 fw-bold
mb-4">Meet new and interesting dogs nearby.</
h1>
        <a class="btn btn-dark btn-lg me-2
mb-2"><i class="bi bi-apple"></i> Download</a>
        <a
class="btn btn-outline-light btn-lg mb-2"><i
class="bi bi-google-play"></i> Download</a>
      </div>
      <div class="col-lg-6 text-center"></div>
    </div>
  </div>
</section>

<section class="py-5 text-center">
  <div class="container">
    <div class="row g-4">
      <div class="col-md-4"><i class="bi bi-
check-circle fs-1 text-primary"></i><h3
class="h5 mt-3">Easy to use</h3><p
class="text-muted">So easy, your dog could do
it.</p></div>
      <div class="col-md-4"><i class="bi bi-
bullseye fs-1 text-primary"></i><h3 class="h5
mt-3">Elite clientele</h3><p class="text-
muted">Only the finest dogs.</p></div>
      <div class="col-md-4"><i class="bi bi-
heart fs-1 text-primary"></i><h3 class="h5
mt-3">Guaranteed</h3><p class="text-muted">Or
your money back.</p></div>
    </div>
  </div>
</section>

<section class="py-5 bg-light">
  <div class="container">
    <h2 class="text-center mb-5">A plan for
every dog's needs</h2>
    <div class="row g-4 justify-content-
center">
      <div class="col-md-6 col-lg-4">
        <div class="card h-100 text-center">
          <div class="card-header py-3"><h3
class="h5 mb-0">Chihuahua</h3></div>
          <div class="card-body"><p
class="display-6">Free</p><ul class="list-
unstyled"><li>5 matches per day</li><li>10
messages</li></ul><button class="btn btn-
outline-primary w-100">Sign up</button></div>
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
    <!-- more cards -->
  </div>
</div>
</section>

<footer class="py-4 bg-dark text-white-50
text-center small">
  <i class="bi bi-twitter mx-2"></i><i
class="bi bi-facebook mx-2"></i><i class="bi
bi-instagram mx-2"></i>
  <p class="mt-2 mb-0">© 2026 TinDog</p>
</footer>

```

RESULT

Meet new and
interesting dogs
nearby.

Download

✓
Easy

⊙
Elite

♥
Guaranteed

Chihuahua

Free

Sign up

© 2026 TinDog

? Chapter quiz

1. How many columns does the Bootstrap grid have?
2. What does col-md-6 mean on a phone?
3. Which class centres a block horizontally?
4. What must be loaded for the navbar hamburger and modals to work?

Answers: 1) 12. 2) Full width — md classes apply from 768px up; below, it falls back to col-12. 3) mx-auto (with a width). 4) bootstrap.bundle.min.js.

104. Tools Reference C — Deployment, Domains and Going Live

A project on your laptop is invisible. Deployment puts it on the internet with a real URL. The right service depends on what you built: static files, a Node server, a database, or all three.

What kind of app do you have?

Angela §35

You built...	Deploy on (free tiers)	Notes
HTML/CSS/JS only (static)	GitHub Pages, Netlify, Vercel, Cloudflare Pages	Drag-and-drop or connect the repo; instant
React/Vite app (static build)	Netlify, Vercel, Cloudflare Pages, GitHub Pages	Build command <code>npm run build</code> , publish <code>dir dist</code>
Node/Express server	Render, Railway, Fly.io, Koyeb	Needs start script and PORT from env
PostgreSQL database	Neon, Supabase, Railway, Render	Copy the connection string into env vars
Full stack (React + Express + DB)	Front on Netlify/Vercel + API on Render + DB on Neon; OR one Render service serving the built React from Express	Set CORS / VITE_API_URL accordingly
Next.js	Vercel	Zero config

Preparing a Node app for production

Angela §35

EXAMPLE

```

class="c">// 1. read the port from the environment – hosts assign it
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log("Listening on", PORT));

class="c">// 2. package.json
"scripts": { "start": "node index.js" },      class="c">// hosts run npm start
"engines": { "node": ">=20" }

class="c">// 3. all secrets via env vars; NODE_ENV=production set by the host
class="c">// 4. serve the React build (optional single-service setup)
app.use(express.static(path.join(__dirname, "client/dist")));
app.get("*", (req, res) => res.sendFile(path.join(__dirname, "client/dist/index.html")));

class="c">// 5. security & logs
app.use(helmet()); app.use(morgan("combined")); app.set("trust proxy", 1);  class="c">// trust
proxy for secure cookies behind the host

```

- Test locally with `NODE_ENV=production PORT=8080 npm start`.
- Database migrations: run your CREATE TABLE statements once against the hosted DB (psql with the connection string, or a migration script).

Deploy an Express API on Render (step by step)

Angela §35

EXAMPLE

1. Push the project to GitHub (with .gitignore excluding node_modules and .env).
2. render.com → New → Web Service → connect the repo.
3. Settings: Runtime Node · Build command: npm install · Start command: npm start · Instance: Free.
4. Environment → add DATABASE_URL, JWT_SECRET, etc. (copy from your .env).
5. Create Web Service → watch the log → "Listening on 10000" → URL https://your-app.onrender.com
6. Every git push to main redeploys automatically.

Database: neon.tech → New project → copy the connection string (postgres://...?sslmode=require) → paste as DATABASE_URL.

In pg Pool add: ssl: { rejectUnauthorized: false } when the host requires it.

Free tier note: Render free services sleep after inactivity; the first request takes ~30 s. Fine for portfolios.

Deploy a React app on Netlify / Vercel

Angela §35

EXAMPLE

Netlify: app.netlify.com → Add new site → Import from Git → pick repo
 Build command: `npm run build` Publish directory: `dist`
 Environment variables: `VITE_API_URL=https://your-api.onrender.com`
 For React Router: add `public/_redirects` containing: `/* /index.html 200`

Vercel: vercel.com → New Project → import repo → framework preset Vite → Deploy.
 Rewrites for Router: `vercel.json { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }`

Both give a preview URL for every branch/PR and HTTPS automatically.
 Remember to allow the front-end origin in the API's `cors()` options.

Custom domains, HTTPS, DNS in plain words

Angela §35

A domain (mybakery.com) is rented yearly from a registrar (Namecheap, GoDaddy, Cloudflare). DNS records point the name to your host. HTTPS certificates are free and automatic on all the hosts above.

Record	Purpose	Example
A	name → IPv4 address	@ → 75.2.60.5
CNAME	name → another name	www → your-site.netlify.app
MX	email servers	@ → mail provider
TXT	verification / SPF	@ → "v=spf1 ..."

- Changes take minutes to hours to propagate.
- Always redirect `http` → `https` and `www` → root (or the reverse) — hosts have a toggle.

Performance and SEO checklist before launch

Angela §35

Check	How
Lighthouse score	DevTools → Lighthouse → run on production URL; aim 90+
Images	Compress (Squoosh), correct size, loading="lazy", modern formats (WebP/AVIF), width/height set
Fonts	Limit to 2 families; font-display: swap; preconnect to Google Fonts
Minify / bundle	Vite does it; for plain sites minify CSS/JS
Caching	Static assets with hashes → long Cache-Control; HTML short
Meta tags	title, description, viewport, Open Graph (og:title, og:image) for link previews
Accessibility	alt text, labels, contrast, keyboard navigation, headings order
404 page	Custom, with a link home
Analytics / monitoring	Plausible or Google Analytics; UptimeRobot; Sentry for errors
robots.txt & sitemap.xml	Let search engines index

Docker in one page (awareness)

Angela §35

Docker packages your app with its exact environment so it runs identically anywhere. Not required for your first deployments, but you will meet it in jobs.

EXAMPLE

```
# Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "index.js"]

# build & run
docker build -t bakery-api .
docker run -p 3000:3000 --env-file .env bakery-api

# docker-compose.yml runs app + postgres together for local dev
```

? Chapter quiz

1. Where does an Express app get its port in production?
2. Which file makes React Router work on Netlify?
3. Where do you set DATABASE_URL on Render?
4. What does a CNAME record do?

Answers: 1) process.env.PORT. 2) public/_redirects with /* /index.html 200. 3) Environment variables in the service settings. 4) Points one name at another name (www → your-site.netlify.app).

105. Projects Workbook — 12 Guided Projects from Beginner to Full Stack

Reading is not learning; building is. These projects are ordered by difficulty and map to the courses (Gamal's and Elzero's trainings, Angela's capstones). For each: what to build, the concepts it practises, a step plan, and stretch goals. Put every one on GitHub with a README and a live link — that is your portfolio.

Project 1 — Personal profile page (HTML only)

Gamal HTML #27

Elzero HTML

Angela \$2

A single page about you: heading, photo, paragraph, list of skills, a table of education, links to your socials, a contact form (not working yet).

EXAMPLE

Concepts: document structure, headings, p, img, ul/ol, table, a, form controls, semantic tags (header, main, section, footer).

Steps:

1. index.html with doctype, head (title, meta charset, viewport), body.
2. header with your name (h1) and a nav with 3 anchor links to sections (#about, #skills, #contact).
3. section#about: img (alt!), 2 paragraphs, a blockquote.
4. section#skills: ul of skills; ol of your learning plan; table of education (thead/tbody).
5. section#contact: form with name, email, message textarea, submit button; each input has a label.
6. footer with © year and links.
7. Validate at validator.w3.org; fix every error.

Stretch: add a favicon; add a details/summary FAQ; embed a YouTube video with iframe.

Project 2 — Style the profile + responsive layout (CSS)

Gamal CSS #47 projects

Elzero CSS

Angela \$6 \$9

EXAMPLE

Concepts: selectors, box model, colours, fonts, Flexbox, Grid, media queries, hover/transition, CSS variables.

Steps:

1. style.css linked; * { box-sizing: border-box } ; :root variables for colours and fonts.
2. Google font; body line-height 1.6; max-width container centred.
3. Sticky header with flex nav (space-between); links hover underline transition.
4. About: two-column flex (image | text) → stack under 768px.
5. Skills: grid of cards (auto-fit, minmax(150px, 1fr)) with shadow and hover lift.
6. Form styled: full-width inputs, focus outline, primary button.
7. Dark mode with prefers-color-scheme.

Stretch: hamburger menu with the checkbox trick; animated skill bars with @keyframes.

Project 3 — Landing page clone (TinDog with Bootstrap)

Angela \$11 project

EXAMPLE

Concepts: Bootstrap grid, navbar, cards, carousel, utilities, icons.

Steps: navbar with brand + collapse · hero (row, 2 cols, buttons) · features (3 cols) ·

testimonial carousel · pricing cards (3 plans, middle highlighted) · footer.

Stretch: replace Bootstrap with your own CSS Grid version to prove you understand what it does.

Project 4 — Dicee + Drum Kit (JS basics, DOM, events)

Angela \$16 \$17 \$19

Elzero JS trainings

EXAMPLE

Dicee: two dice images; on Refresh button (and on load) pick `Math.random` 1–6 for each, set `img src`, show "Player 1 wins" / "Draw" in `h1`.

Drum Kit: 7 buttons w a s d j k l; click or key press → play matching sound (new `Audio`), add `.pressed` class for 100 ms.

Concepts: variables, `Math`, `querySelectorAll`, `addEventListener`, `classList`, `setTimeout`, template literals, `this/e.key`.

Stretch: score counter with `localStorage`; keyboard focus styles; a "record and replay" sequence.

Project 5 — Simon Game (state, arrays, timing)

Angela \$20

EXAMPLE

Rules: game shows a growing random sequence of 4 colours; player repeats it by clicking; wrong click → game over → press any key to restart.

State: `gamePattern[]`, `userPattern[]`, `level`, `started`.

Steps: `nextSequence()` pushes a colour and flashes it · click handler pushes to `userPattern` and calls `checkAnswer(index)` · if all matched, `setTimeout(nextSequence, 1000)` · on error add `.game-over` class to body, reset.

Concepts: arrays, comparison, functions, `setTimeout`, `classList`, game loop thinking.

Stretch: high score in `localStorage`; increasing speed; sounds; mobile touch.

Project 6 — To-Do App with `localStorage` (complete front-end app)

Elzero JS

Gamal JS project

EXAMPLE

Features: add, edit (double-click), delete, mark done, filter all/active/done, clear completed, count left, persist in `localStorage`, keyboard Enter to add, empty state message.

Architecture: state array of `{id, text, done}`; `render()` rebuilds the list from state; event delegation on the `ul`; `save()` after every change.

Concepts: everything from JS chapters A–G; JSON; delegation; immutability basics.

Stretch: drag to reorder; due dates; dark mode; export/import JSON.

Project 7 — Weather / Movie search app (fetch, async, APIs)

Angela \$30 **Gamal JS project**

EXAMPLE

Weather: form with city → fetch OpenWeatherMap (or open-meteo with geocoding) → show temp, icon, description, 5-day forecast cards; handle loading, 404, network errors; remember last city.

Movies (alternative): search OMDb/TMDB → grid of posters → click → modal with details.

Concepts: async/await, fetch, JSON, error handling, DOM rendering from data, debounce on input, URLSearchParams.

Stretch: geolocation to auto-detect city; favourites list; charts with Chart.js.

Project 8 — Blog website with Express + EJS (server-side CRUD)

Angela \$28 capstone

EXAMPLE

Routes: GET / (list) · GET /new · POST /posts · GET /posts/:id · GET /posts/:id/edit · POST /posts/:id/edit · POST /posts/:id/delete.

Views: partials header/footer, index, new, post, edit, 404.

Steps: npm init, express + ejs, static folder, in-memory array, all routes, forms with PRG redirect, styling.

Concepts: Node, npm, Express routing, middleware, urlencoded bodies, EJS tags/partial, HTTP methods.

Stretch: markdown support (marked package); image upload (multer); search; pagination.

Project 9 — REST API + PostgreSQL (Book Notes / Products API)

Angela \$29 \$31 capstone

EXAMPLE

Build /api/books: full CRUD in Postgres with pg, validation, filtering (?q=&sort=), pagination, correct status codes, error handler, morgan logs, tested with Postman/curl.

Tables: books(id, title, author, rating, notes, read_at, cover_url). Optional: fetch cover from Open Library covers API by ISBN.

Concepts: SQL DDL/DML, pg Pool, parameterised queries, REST design, environment variables, Router/controllers structure.

Stretch: JOIN with an authors table; full-text search; export CSV; OpenAPI (Swagger) docs.

Project 10 — Secrets: authentication (bcrypt, sessions, Google OAuth)

Angela \$32 capstone

EXAMPLE

Pages: register, login, /secrets (protected, shows the user's secrets), submit a secret, logout.
 Users in Postgres with hashed passwords; sessions with express-session + Passport local; "Sign in with Google" button via passport-google-oauth20.
 Security: bcrypt 12 rounds; generic errors; httpOnly cookies; rate limit login; validation.
 Concepts: everything in Back-End Reference F.
 Stretch: email verification; password reset by email; JWT version of the same API for a React client.

Project 11 — Keeper app + Products dashboard (React)

Angela \$33 \$34 capstone

Gamal React course

EXAMPLE

Keeper: Header, Footer, CreateArea (controlled form), Note components; add and delete notes; state lifted to App; persist with useLocalStorage.
 Dashboard: React Router pages (Home, Products, ProductDetail, Login, Admin); api.js talking to Project 9's API; AuthContext with JWT; protected route; CRUD table with loading/error/optimistic updates; search with debounce.
 Concepts: components, props, state, effects, custom hooks, router, context, forms.
 Stretch: React Query; TypeScript; charts; role-based UI; unit tests.

Project 12 — Capstone: full-stack e-commerce (Giza Bakery)

Angela \$36 capstone

EXAMPLE

Front: React + Router – product grid with filters, product page, cart (Context + localStorage), checkout form, order history, admin panel.
 Back: Express REST API – products, orders, users, JWT auth with roles, image upload, PostgreSQL with users/products/orders/order_items, validation, rate limiting, helmet.
 Deploy: front on Netlify, API on Render, DB on Neon; custom domain; Lighthouse 90+.
 Deliverables: GitHub repo with README (screenshots, architecture diagram, how to run, live links), 5-minute demo video.
 This project alone demonstrates every skill an employer expects from a junior full-stack developer.

Portfolio and README template

Angela \$35

EXAMPLE

```
# Project Name
One-sentence description. **Live demo:** https://... **API:** https://...

## Screenshots
![home](docs/home.png)

## Features
- Browse and search products
- Cart persisted in localStorage
- JWT authentication with admin role

## Tech stack
React, React Router, Vite · Node, Express, PostgreSQL · Deployed on Netlify + Render + Neon

## Run locally
git clone ... && cd ... && npm install
cp .env.example .env # fill values
npm run dev

## What I learned
Three honest sentences about the hardest part.
```

? Chapter quiz

1. Which project first uses a database?
2. Why should every project be on GitHub with a README?
3. What three states must every fetch-based UI handle?
4. What is the capstone?

Answers: 1) Project 9 (REST API + PostgreSQL). 2) It is your portfolio; employers read code and READMEs. 3) Loading, error, data. 4) Project 12: full-stack e-commerce deployed with a custom domain.

106. Career Kit — Interview Questions, Cheat Sheets and What Next

The last chapter turns knowledge into a job. Interview questions juniors are actually asked (with short model answers), one-page cheat sheets for each technology, common error messages decoded, and a plan for the months after this book.

HTML & CSS interview questions

[Gamal all](#)[Elzero all](#)[Angela all](#)

Question	Short answer
What is semantic HTML and why use it?	Tags that describe meaning (header, nav, article). Better accessibility, SEO and readability.
Difference between div and span?	div is block-level (new line, full width); span is inline (inside text).
What does the viewport meta tag do?	Tells mobile browsers to use the device width and 1× zoom so responsive CSS works.
Explain the box model.	Content + padding + border + margin. box-sizing: border-box makes width include padding and border.
What is specificity?	How the browser decides which rule wins: inline > id > class/attribute/pseudo-class > element. !important overrides all.
Block vs inline vs inline-block?	Block: full width, new line, respects width/height. Inline: flows in text, ignores width/height and vertical margins. Inline-block: inline flow but respects box sizes.
Flexbox vs Grid?	Flex: one dimension (row OR column), content-driven. Grid: two dimensions, layout-driven.
How do you centre a div?	Parent display:flex; justify-content:center; align-items:center. Or grid; place-items:center. Or margin:auto with width.
position values?	static (default), relative (offset from itself), absolute (relative to nearest positioned ancestor), fixed (viewport), sticky (hybrid).
rem vs em vs px?	px absolute; em relative to parent font-size; rem relative to root font-size (predictable — use for spacing/text).
What are pseudo-classes and pseudo-elements?	States like :hover, :nth-child; generated parts like ::before, ::after, ::placeholder.
Mobile-first?	Write base CSS for small screens, then min-width media queries add complexity for larger screens.
How to make an image responsive?	max-width:100%; height:auto; plus srcset/picture for different sizes.
What is the cascade?	The algorithm combining origin, importance, specificity and source order to pick the winning declaration.
Accessibility basics?	alt text, labels for inputs, semantic tags, colour contrast, keyboard operability, focus styles, ARIA only when needed.

JavaScript interview questions

Elzero JS

Gamal JS

Angela \$14-\$22

Question	Short answer
var vs let vs const?	var function-scoped, hoisted as undefined, re-declarable; let/const block-scoped, TDZ; const cannot be reassigned.
== vs ===?	== converts types before comparing; === compares type and value. Use ===.
What is hoisting?	Declarations are processed before code runs: function declarations fully; var as undefined; let/const exist but are inaccessible until their line.
Explain closures.	A function keeps access to the variables of its creating scope even after that scope finished. Enables private state, counters, callbacks.
What is this?	The object a function is called on. Plain call: undefined/window; method: the object; arrow: inherited from outer scope; bind/call/apply set it explicitly.
Difference between null and undefined?	undefined: declared but not assigned (language). null: intentional empty (developer).
What is the event loop?	JS runs one thing at a time; async tasks (timers, fetch) finish in the browser and their callbacks are queued; the loop runs them when the call stack is empty.
Promise vs async/await?	Same mechanism; async/await is syntax that reads sequentially and uses try/catch.
map vs forEach?	map returns a new array of results; forEach returns undefined (side effects only).
What is event delegation?	One listener on a parent handles events from children via bubbling and e.target; works for dynamically added children.
Shallow vs deep copy?	Spread/Object.assign copy one level (nested objects shared); structuredClone copies everything.
Truthy/falsy values?	Falsy: false, 0, -0, 0n, "", null, undefined, NaN; all else truthy (including [] and {}).
What does JSON.stringify drop?	Functions, undefined, symbols; Dates become strings.
localStorage vs sessionStorage vs cookies?	local: persists, 5MB, JS only. session: per tab. cookies: 4KB, sent to server with every request, can be httpOnly.
Debounce vs throttle?	Debounce: run after activity stops (search input). Throttle: run at most every N ms (scroll).
What is a higher-order function?	One that takes or returns a function (map, filter, addEventListener).
Arrow functions differences?	No own this/arguments, cannot be constructors, implicit return, not hoisted.
What is the DOM?	Browser's object tree of the page that JS can read and change.

Back-end, React and general questions

Angela §23-§35

Question	Short answer
What is REST?	Conventions for APIs: resources as nouns in URLs, HTTP methods as verbs, stateless, JSON, standard status codes.
GET vs POST vs PUT vs PATCH vs DELETE?	Read / create / replace / partial update / remove.
What is middleware in Express?	Function (req, res, next) that runs before route handlers: logging, parsing, auth. next() passes control.
How do you handle errors in Express?	4-argument middleware at the end; pass async errors with next(err).
What is CORS?	Browser security blocking cross-origin reads; the server must send Access-Control-Allow-Origin.
Session vs JWT?	Session: server stores state, cookie holds id. JWT: signed token holds the data, stateless.
Why hash passwords? Why bcrypt?	One-way so leaks do not reveal passwords; bcrypt is salted and slow against brute force.
What is SQL injection and how to prevent it?	Attacker's input becomes SQL; prevent with parameterised queries/ORM.
Primary vs foreign key?	PK uniquely identifies a row; FK references another table's PK.
INNER vs LEFT JOIN?	INNER: matching rows only; LEFT: all left rows plus matches.
What is npm? package.json vs package-lock?	Package manager; declared dependencies vs exact installed versions.
What is a component in React?	Function returning JSX; receives props; may hold state.
Props vs state?	Props: read-only inputs from parent. State: internal, changed with setState → re-render.
Why keys in lists?	Let React track items across renders; use stable ids.
What does useEffect do? Dependency array?	Runs side effects after render; [] once, [x] when x changes; return cleanup.
Controlled component?	Input whose value is driven by state with onChange.
What is the virtual DOM?	In-memory tree React diffs against the previous one to apply minimal real DOM updates.
Lifting state up?	Moving shared state to the closest common parent.
What is Git branching / a pull request?	Parallel lines of work; PR = proposal to merge with review.
Explain what happens when you type a URL and press Enter.	DNS lookup → TCP/TLS connection → HTTP request → server responds HTML → browser parses, requests CSS/JS/images, builds DOM/CSSOM, renders, runs JS.

Common error messages decoded

Elzero JS #004

Error	Meaning	Fix
Uncaught ReferenceError: x is not defined	Variable/function name does not exist in scope	Typo? Declared? Script order?
TypeError: Cannot read properties of null (reading 'addEventListener')	querySelector found nothing	Check selector; run script after HTML (defer)
TypeError: Cannot read properties of undefined (reading 'name')	Something before .name is undefined	Log the object; optional chaining; check API shape
TypeError: x is not a function	Calling a non-function (typo, wrong import, called result)	Check the name and import
SyntaxError: Unexpected token	Missing bracket/quote/ comma, or JSON.parse on HTML	Check matching brackets; log the response text
Uncaught (in promise)	A rejected promise had no catch	Add try/catch or .catch
CORS policy: No 'Access-Control-Allow-Origin'	Server did not allow your origin	cors() on server / dev proxy
404 (Not Found) for a file	Wrong path/case	Check relative path, case sensitivity on Linux servers
EADDRINUSE: address already in use :3000	Another process uses the port	Stop it (Ctrl+C) or change PORT
Cannot find module 'express'	Not installed	npm install
Cannot set headers after they are sent	Responded twice	return after res.send/json
ECONNREFUSED 5432	Database not running / wrong host	Start Postgres; check DATABASE_URL
relation "users" does not exist	Table not created in this DB	Run the CREATE TABLE
Warning: Each child in a list should have a unique key	Missing key prop	key={item.id}
Too many re-renders	setState called during render	Move into handler/effect; onClick={() => set(x)} not onClick={set(x)}
Objects are not valid as a React child	Rendering an object	Render its fields or JSON.stringify
fatal: not a git repository	Not in a repo folder	cd into it or git init
! [rejected] main -> main (fetch first)	Remote has commits you lack	git pull --rebase then push

One-page cheat sheet: HTML

Gamal HTML

Elzero HTML

EXAMPLE

```
<!DOCTYPE html><html lang="en"><head><meta charset="UTF-8"><meta name="viewport"
content="width=device-width, initial-scale=1"><title>T</title><link rel="stylesheet"
href="style.css"></head><body>...<script src="app.js" defer></script></body></html>
```

Text: h1–h6 p br hr strong em small mark sub sup blockquote q cite abbr code pre span

Lists: ul ol li dl dt dd Links: <a href target rel download> Images: picture figure figcaption

Semantic: header nav main section article aside footer address time details summary dialog

Tables: table caption thead tbody tfoot tr th td (colspan rowspan scope)

Forms: form(action method) label input(type name id value placeholder required min max step pattern) textarea select option button(type) fieldset legend datalist

Input types: text email password number tel url date time checkbox radio file range color hidden search

Media: video audio source track iframe canvas svg

Attributes: id class style title hidden data-* tabindex aria-* lang dir

Entities: write < to show < , > to show > , & to show & , for a non-breaking space, © for ©

One-page cheat sheet: CSS

Gamal CSS

Elzero CSS

EXAMPLE

```
/* selectors */ * el .c #i el.c a b (desc) a > b a + b a ~ b [attr=v] :hover :focus :active :nth-child(2n) :first-child :last-child :not() :is() :has() ::before ::after ::placeholder
/* box */ width height max-width min-height padding border border-radius margin box-sizing:border-box outline box-shadow overflow display(block inline inline-block flex grid none) visibility
/* text */ color font-family font-size font-weight font-style line-height text-align text-decoration text-transform letter-spacing white-space text-overflow direction
/* bg */ background(-color -image -size:cover -position -repeat) linear-gradient() opacity filter
/* position */ position(relative absolute fixed sticky) top right bottom left z-index
/* flex */ display:flex flex-direction flex-wrap justify-content align-items align-content gap flex:1 order align-self
/* grid */ display:grid grid-template-columns:repeat(auto-fit,minmax(200px,1fr)) grid-template-rows gap grid-column:1/-1 grid-area grid-template-areas place-items
/* fx */ transition:all .3s transform:translate() scale() rotate() @keyframes animation cursor
/* responsive */ @media (max-width:768px){} @media (prefers-color-scheme:dark){}
clamp(1rem,2vw,2rem) vw vh rem em % container queries
/* vars */ :root{--c:#333} color:var(--c)    /* reset */ *{margin:0;padding:0;box-sizing:border-box} img{max-width:100%;display:block}
```

One-page cheat sheet: JavaScript

Elzero JS

Gamal JS

EXAMPLE

```
const let ; typeof ; === !== ; && || ?? ?. ; class="v">`template ${x}` ; if/else switch ?: ; for
for...of for...in while ; break continue
function f(a=1,...r){} const f=(a)=>a*2 ; return ; closures ; try{}catch(e){}finally{} throw new
Error()
Strings: length toUpperCase trim includes startsWith indexOf slice split replace(All) padStart
repeat at
Numbers: Number() parseInt parseFloat toFixed Math.round/floor/ceil/random/max/min isNaN
Arrays: push pop shift unshift splice slice concat [...a] indexOf includes find findIndex some
every filter map reduce forEach sort((a,b)=>a-b) reverse join flat Array.from Array.isArray
Objects: {k:v} o.k o["k"] Object.keys/values/entries {...o} destructuring {a,b}=o structuredClone
JSON.stringify/parse
DOM: querySelector(All) getElementById.textContent.innerHTML.value.classList.add/remove/toggle
style.x.setAttribute dataset createElement append remove closest parentElement children
Events: el.addEventListener("click", e => {}) e.target e.preventDefault() e.key ; delegation ;
DOMContentLoaded submit input change keydown
Async: setTimeout setInterval Promise .then .catch async/await fetch(url,{method,headers,body})
res.ok res.json() Promise.all
Storage: localStorage.setItem/getItem/removeItem (strings → JSON)
Classes: class A extends B { constructor(){super()} method(){static s(){get g(){#private }
Modules: export / export default / import x, {y} from "./f.js" ; <script type="module">
```

One-page cheat sheet: Node, Express, SQL, Git

Angela \$23-\$32

EXAMPLE

```
NODE/NPM: npm init -y · npm i pkg · npm i -D pkg · npm run dev · node --watch index.js ·
process.env.X · import fs from "fs/promises"
EXPRESS: const app=express(); app.use(express.json()); app.use(express.static("public")); app.get/
post/put/patch/delete(path,(req,res)=>{}); req.params req.query req.body req.headers; res.json()
res.status() res.send() res.render() res.redirect(); app.use((req,res)=>404);
app.use((err,req,res,next)=>500); Router(); app.listen(PORT)
EJS: <%= v %> <%- html %> <% code %> <%- include("partials/x") %> ; app.set("view engine","ejs")
SQL: CREATE TABLE t(id SERIAL PRIMARY KEY, col TYPE NOT NULL UNIQUE DEFAULT, fk INT REFERENCES
o(id)); INSERT INTO t(c) VALUES($1) RETURNING *; SELECT c FROM t WHERE x=$1 ORDER BY c DESC LIMIT
10 OFFSET 0; UPDATE t SET c=$1 WHERE id=$2; DELETE FROM t WHERE id=$1; JOIN o ON o.id=t.fk; GROUP
BY; COUNT/SUM/AVG; LIKE/ILIKE; BETWEEN; IN; IS NULL
PG: new pg.Pool({connectionString}); const {rows}=await pool.query(sql,[params])
AUTH: bcrypt.hash(pw,12) bcrypt.compare(pw,hash) · jwt.sign(payload,secret,{expiresIn})
jwt.verify(token,secret) · express-session + passport · cookies httpOnly secure sameSite
HTTP: 200 201 204 301 400 401 403 404 409 429 500 · headers Content-Type Authorization · CORS on
server
GIT: git init · status · add · commit -m "" · log --oneline · branch · switch -c b · merge b ·
remote add origin url · push -u origin main · pull · clone · restore · reset · revert · stash
· .gitignore
REACT: function C({p}){const [s,setS]=useState(0); useEffect(()=>{...;return cleanup},{deps});
return <div onClick={()=>setS(s+1)}>{s}</div> · map+key · controlled inputs · lift state ·
Context · Router: BrowserRouter Routes Route Link useParams useNavigate
```

After this book: the next 6 months

Angela \$36

Month	Focus	Output
1	Rebuild Projects 1–6 without looking; daily 30 min of JS exercises (Elzero assignments, Codewars 8–7 kyu)	6 repos live on GitHub Pages
2	Projects 7–9; learn Postman, Git branching daily; read MDN for every new API	REST API deployed on Render + Neon
3	Projects 10–11; TypeScript basics; React Query; testing basics	Full React front-end connected to your API
4	Capstone Project 12; performance & accessibility polish; write README + demo video	Portfolio site linking all projects
5	Next.js or advanced Node; Docker basics; contribute one small open-source PR; mock interviews with the question lists above	LinkedIn + CV updated; 20 applications/week
6	Freelance small sites (Upwork, Mostaql, Khamsat) or internship; keep shipping; specialise (front-end, back-end, or full-stack)	First paid work / junior role

- Consistency beats intensity: 1–2 hours every day beats 12 hours on Friday.
- When stuck for more than 30 minutes: read the error, search it, ask in Elzero/Gamal community groups or Stack Overflow with a minimal example.
- Teach what you learn — write short posts or explain to a friend; it is the fastest way to make it stick.
- Arabic resources used in this book: Elzero Web School (HTML one video, CSS 88 videos / one video, JavaScript 188 videos / one video parts 1–2), Abdelrahman Gamal (HTML & CSS 81 videos, JavaScript, React). English: Dr. Angela Yu's Complete Full-Stack Web Development Bootcamp. MDN Web Docs for reference on everything.

? Chapter quiz

1. What is the difference between props and state in one line?
2. How do you prevent SQL injection?
3. Name the falsy values.
4. What is the very first thing to do when you see an error?

Answers: 1) Props come from the parent and are read-only; state is owned by the component and changes with a setter. 2) Parameterised queries or an ORM. 3) false, 0, -0, 0n, "", null, undefined, NaN. 4) Read the message and the line number carefully before changing anything.

107. Exercise Workbook A — HTML Drills

(30 exercises with solutions)

Type every solution yourself, open it in the browser and look at the result before reading the answer. Each drill practises one idea from the HTML chapters; together they cover everything a junior is expected to write from memory.

Drill 1: Create a minimal valid page with the title 'My Shop' and a heading tha

Note: The title appears on the browser tab; the h1 appears on the page.

Try it yourself

Create a minimal valid page with the title 'My Shop' and a heading that says Welcome.

Solution

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Shop</title>
</head>
<body>
  <h1>Welcome</h1>
</body>
</html>
```

Drill 2: Write three paragraphs

Note: strong = important (bold); em = stressed (italic). Use b and i only for visual styling without meaning.

Try it yourself

Write three paragraphs. Make one word bold for importance and one word italic for emphasis.

Solution

```
<p>Fresh bread is baked <strong>every morning</strong>.</p>
<p>We use <em>only</em> local flour.</p>
<p>Open daily from 7 am.</p>
```

Drill 3: Create an unordered shopping list with 4 items and an ordered recipe w

Note: ul = bullets, ol = numbers. Only li may be a direct child.

 Try it yourself

Create an unordered shopping list with 4 items and an ordered recipe with 3 steps.

Solution

```
<ul>
  <li>Flour</li><li>Yeast</li><li>Salt</li><li>Water</li>
</ul>
<ol>
  <li>Mix the ingredients</li>
  <li>Let the dough rise</li>
  <li>Bake for 30 minutes</li>
</ol>
```

Drill 4: Nest a list: 'Drinks' with sub-items Tea and Coffee under it

Note: The inner ul goes INSIDE the li, after its text.

 Try it yourself

Nest a list: 'Drinks' with sub-items Tea and Coffee under it.

Solution

```
<ul>
  <li>Drinks
    <ul>
      <li>Tea</li>
      <li>Coffee</li>
    </ul>
  </li>
  <li>Cakes</li>
</ul>
```


Drill 5: Add a link to https://developer

Note: External links need the full https:// URL; internal links use relative paths. rel=noopener with target=_blank is a security habit.

Try it yourself

Add a link to https://developer.mozilla.org that opens in a new tab, and a link to your own about.html page.

Solution

```
<a href="https://developer.mozilla.org" target="_blank" rel="noopener">MDN</a>
<a href="about.html">About us</a>
```

Drill 6: Add a link that jumps to a section further down the same page

Note: The href with # targets an element whose id matches.

Try it yourself

Add a link that jumps to a section further down the same page.

Solution

```
<a href="#contact">Go to contact</a>
...
<section id="contact"><h2>Contact</h2></section>
```

Drill 7: Insert an image images/cake

Note: alt describes the picture for screen readers and when the file fails. width/height prevent layout jumps.

Try it yourself

Insert an image images/cake.jpg, 300 pixels wide, with a good alt text, lazy loaded.

Solution

```

```

Drill 8: Wrap an image with a caption

Note: figure + figcaption tie the caption to the image semantically.

Try it yourself

Wrap an image with a caption.

Solution

```
<figure>
  
  <figcaption>Our shop in Giza, opened 2015</figcaption>
</figure>
```

Drill 9: Build a 3-column table: Product, Price, In stock — with a header row a

Note: th for headers, td for data; thead/tbody group rows.

Try it yourself

Build a 3-column table: Product, Price, In stock — with a header row and two data rows.

Solution

```
<table>
  <thead>
    <tr><th>Product</th><th>Price</th><th>In stock</th></tr>
  </thead>
  <tbody>
    <tr><td>Bread</td><td>5 EGP</td><td>Yes</td></tr>
    <tr><td>Cake</td><td>40 EGP</td><td>No</td></tr>
  </tbody>
</table>
```

Drill 10: Make a table cell span two columns

Note: colspan merges horizontally, rowspan merges vertically.

Try it yourself

Make a table cell span two columns.

Solution

```
<tr><td colspan="2">Total</td><td>45 EGP</td></tr>
```

Drill 11: Create a contact form with name, email, message and a submit button

Note: label for = input id links them (clicking the label focuses the field). name is what the server receives.

 Try it yourself

Create a contact form with name, email, message and a submit button. Every field must have a label.

Solution

```
<form action="/contact" method="POST">
  <label for="name">Name</label>
  <input id="name" name="name" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <label for="msg">Message</label>
  <textarea id="msg" name="message" rows="5"></textarea>

  <button type="submit">Send</button>
</form>
```

Drill 12: Add a group of radio buttons for size (Small/Medium/Large) where only

Note: Radios with the SAME name form one group. checked preselects.

 Try it yourself

Add a group of radio buttons for size (Small/Medium/Large) where only one can be picked, with Medium preselected.

Solution

```
<fieldset>
  <legend>Size</legend>
  <label><input type="radio" name="size" value="s"> Small</label>
  <label><input type="radio" name="size" value="m" checked> Medium</label>
  <label><input type="radio" name="size" value="l"> Large</label>
</fieldset>
```

Drill 13: Add checkboxes for extras (Nuts, Cream) and a dropdown for delivery ti

Note: Checkboxes allow many; select allows one (or many with multiple).

 Try it yourself

Add checkboxes for extras (Nuts, Cream) and a dropdown for delivery time.

Solution

```
<label><input type="checkbox" name="extras" value="nuts"> Nuts</label>
<label><input type="checkbox" name="extras" value="cream"> Cream</label>

<label for="time">Delivery time</label>
<select id="time" name="time">
  <option value="">Choose...</option>
  <option value="am">Morning</option>
  <option value="pm" selected>Afternoon</option>
</select>
```

Drill 14: Add a number input for quantity between 1 and 10, a date input, and a

Note: min/max/step give free validation and the right mobile keyboard.

 Try it yourself

Add a number input for quantity between 1 and 10, a date input, and a range slider.

Solution

```
<input type="number" name="qty" min="1" max="10" value="1">
<input type="date" name="day" min="2026-09-07">
<input type="range" name="sugar" min="0" max="100" step="10">
```

Drill 15: Require a phone number that matches an Egyptian mobile pattern

Note: pattern is a regular expression; title becomes the error hint.

Try it yourself

Require a phone number that matches an Egyptian mobile pattern.

Solution

```
<input type="tel" name="phone" pattern="01[0125][0-9]{8}" placeholder="01xxxxxxxx" required
title="11 digits starting with 01">
```

Drill 16: Make a file upload that accepts only images

Note: Uploads require enctype multipart/form-data on the form.

Try it yourself

Make a file upload that accepts only images.

Solution

```
<form method="POST" enctype="multipart/form-data">
  <input type="file" name="photo" accept="image/*">
</form>
```

Drill 17: Structure a blog page with semantic tags: header with nav, main with t

Note: Semantic tags say WHAT a block is. Screen readers and search engines use them.

Try it yourself

Structure a blog page with semantic tags: header with nav, main with two articles, an aside, a footer.

Solution

```
<header>
  <h1>My Blog</h1>
  <nav><a href="/">Home</a> <a href="/about">About</a></nav>
</header>
<main>
  <article><h2>Post one</h2><p>...</p></article>
  <article><h2>Post two</h2><p>...</p></article>
</main>
<aside><h3>About me</h3></aside>
<footer>© 2026</footer>
```

Drill 18: Show a quotation with its source, and an abbreviation with its full fo**Note:** Hover over abbr shows the title. **Try it yourself**

Show a quotation with its source, and an abbreviation with its full form.

Solution

```
<blockquote cite="https://example.com">Bread is the king of the table.</blockquote>
<p>Made with <abbr title="Hypertext Markup Language">HTML</abbr>.</p>
```

Drill 19: Display a code snippet exactly as typed, preserving spaces and line br**Note:** pre keeps whitespace; code marks it as code. **Try it yourself**

Display a code snippet exactly as typed, preserving spaces and line breaks.

Solution

```
<pre><code>function hi() {
  console.log("hi");
}</code></pre>
```

Drill 20: Write H₂O and x² using HTML

Note: sub = subscript, sup = superscript.

Try it yourself

Write H₂O and x² using HTML.

Solution

```
<p>H<sub>2</sub>O and x<sup>2</sup></p>
```

Drill 21: Add an expandable FAQ item that is closed by default

Note: No JavaScript needed. Add the open attribute to start expanded.

Try it yourself

Add an expandable FAQ item that is closed by default.

Solution

```
<details>
  <summary>Do you deliver?</summary>
  <p>Yes, within Giza and Cairo.</p>
</details>
```

Drill 22: Embed a YouTube video and a video file with controls

Note: Use the embed URL for YouTube, not the watch URL.

Try it yourself

Embed a YouTube video and a video file with controls.

Solution

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID" title="Video"
allowfullscreen></iframe>
<video controls width="400" poster="thumb.jpg">
  <source src="promo.mp4" type="video/mp4">
  Your browser does not support video.
</video>
```

Drill 23: Add a favicon and a description meta tag for search engines**Note:** Both go inside head. **Try it yourself**

Add a favicon and a description meta tag for search engines.

Solution

```
<link rel="icon" href="favicon.ico">
<meta name="description" content="Fresh bread and cakes in Giza, delivered daily.">
```

Drill 24: Mark a paragraph as Arabic, right to left**Note:** dir=rtl flips text direction; lang helps screen readers and fonts. Put both on html for a fully Arabic site. **Try it yourself**

Mark a paragraph as Arabic, right to left.

Solution

```
<p lang="ar" dir="rtl">مرحباً بكم في مخبزنا</p>
```

Drill 25: Create a button that does nothing yet (for JavaScript later) and a but**Note:** Inside a form, a button without type submits — use type=button when you do not want that.

Try it yourself

Create a button that does nothing yet (for JavaScript later) and a button that resets a form.

Solution

```
<button type="button" id="load">Load more</button>
<button type="reset">Clear</button>
```

Drill 26: Add a date/time that machines can read

Note: datetime holds the machine format; the text is for humans.

Try it yourself

Add a date/time that machines can read.

Solution

```
<p>Published <time datetime="2026-09-07T09:00">7 September 2026</time></p>
```

Drill 27: Hide an element from everyone, and hide an element visually but keep i

Note: hidden = display:none. The sr-only pattern keeps text for assistive tech.

Try it yourself

Hide an element from everyone, and hide an element visually but keep it for screen readers.

Solution

```
<p hidden>Not shown at all</p>
<span class="sr-only">Menu</span>
<style>.sr-only{position:absolute;width:1px;height:1px;overflow:hidden;clip:rect(0 0 0 0);white-space:nowrap}</style>
```

Drill 28: Store custom data on an element for JavaScript

Note: data-* attributes are the standard way to attach data.

 Try it yourself

Store custom data on an element for JavaScript.

Solution

```
<li data-id="42" data-category="cakes">Chocolate cake</li>
<!-- JS: li.dataset.id → "42" -->
```

Drill 29: Write a comment in HTML and explain why comments are not private

Note: Anyone can view page source and read comments. Never put secrets in them.

 Try it yourself

Write a comment in HTML and explain why comments are not private.

Solution

```
<!-- TODO: add prices -->
```

Drill 30: Fix this broken markup: <p>Hello world</p> <p>ite

Note: Tags must close in reverse order of opening; ul may only contain li; images need alt and quoted attributes.

 Try it yourself

Fix this broken markup:

Hello **world**

item

Solution

```
<p>Hello <strong>world</strong></p>
<ul><li>item</li></ul>

```

108. Exercise Workbook B — CSS Drills

(30 exercises with solutions)

Each drill gives you HTML you can assume exists and asks for the CSS. Write it, refresh, compare. The last ten are small layout challenges combining Flexbox, Grid and media queries.

Drill 1: Make all paragraphs dark grey, 18px, with 1

Note: Set the family once on body; it inherits to everything.

Try it yourself

Make all paragraphs dark grey, 18px, with 1.6 line height, and the page font Arial.

Solution

```
body { font-family: Arial, sans-serif; }
p { color: #333; font-size: 18px; line-height: 1.6; }
```

Drill 2: Style only elements with class

Note: A class selector starts with a dot.

Try it yourself

Style only elements with class .note: yellow background, padding 12px, left border 4px orange.

Solution

```
.note {
  background: #fff8c4;
  padding: 12px;
  border-left: 4px solid orange;
}
```

Drill 3: Make the element with id main-title centred, uppercase, with 2px letter

Note: id selector starts with #. Prefer classes for styling; ids are fine for unique headings.

Try it yourself

Make the element with id main-title centred, uppercase, with 2px letter spacing.

Solution

```
#main-title {  
  text-align: center;  
  text-transform: uppercase;  
  letter-spacing: 2px;  
}
```

Drill 4: Remove underlines from links, make them blue, and underline on hover w

Note: transition on the base rule so it animates both in and out.

Try it yourself

Remove underlines from links, make them blue, and underline on hover with a smooth colour change.

Solution

```
a { color: #1f77c4; text-decoration: none; transition: color .2s; }  
a:hover { text-decoration: underline; color: #0d4f8b; }
```

Drill 5: Make every second table row light grey and highlight a row on hover

Note: nth-child(even) = 2, 4, 6...; odd also works; 3n selects every third.

Try it yourself

Make every second table row light grey and highlight a row on hover.

Solution

```
tr:nth-child(even) { background: #f3f3f3; }
tr:hover { background: #e0ecf9; }
```

Drill 6: Make a 200×200 box with a 2px solid border, 20px padding, 10px rounded

Note: Without border-box the box would be 244px wide (200 + 40 padding + 4 border).

Try it yourself

Make a 200×200 box with a 2px solid border, 20px padding, 10px rounded corners and a soft shadow — and its total width must stay 200px.

Solution

```
.box {
  box-sizing: border-box;
  width: 200px; height: 200px;
  padding: 20px;
  border: 2px solid #333;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0,0,0,.15);
}
```

Drill 7: Centre a 600px wide container horizontally on the page

Note: margin auto left/right centres a block with a width. max-width lets it shrink on phones.

Try it yourself

Centre a 600px wide container horizontally on the page.

Solution

```
.container { max-width: 600px; margin: 0 auto; padding: 0 16px; }
```

Drill 8: Make a perfect circle avatar 80px from any image, cropped without dist

Note: object-fit: cover crops instead of squashing.

Try it yourself

Make a perfect circle avatar 80px from any image, cropped without distortion.

Solution

```
.avatar { width: 80px; height: 80px; border-radius: 50%; object-fit: cover; }
```

Drill 9: Make a button: blue background, white text, no border, 10px 20px paddi

Note: Two values in padding: vertical then horizontal.

Try it yourself

Make a button: blue background, white text, no border, 10px 20px padding, pointer cursor, darker on hover, slightly pressed on active.

Solution

```
.btn {
  background: #1f77c4; color: #fff; border: 0;
  padding: 10px 20px; border-radius: 6px; cursor: pointer;
  transition: background .2s, transform .1s;
}
.btn:hover { background: #185f9d; }
.btn:active { transform: translateY(1px); }
```

Drill 10: Put a full-screen background image that covers the area, centred, not

Note: A gradient of two identical semi-transparent colours acts as an overlay layer above the image.

Try it yourself

Put a full-screen background image that covers the area, centred, not repeating, with a dark overlay under white text.

Solution

```
.hero {
  min-height: 100vh;
  background: linear-gradient(rgba(0,0,0,.5), rgba(0,0,0,.5)), url(hero.jpg) center/cover no-repeat;
  color: #fff;
  display: grid; place-items: center;
}
```

Drill 11: Create a navigation bar: links in a row, space between logo and links,

Note: Flex on the bar spreads logo and menu; flex on the ul lines up the links.

Try it yourself

Create a navigation bar: links in a row, space between logo and links, vertically centred, 60px tall.

Solution

```
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 60px; padding: 0 24px;
}
.nav ul { display: flex; gap: 24px; list-style: none; margin: 0; padding: 0; }
```

Drill 12: Centre one element exactly in the middle of the screen (both axes)

Note: Both work; grid is shorter.

Try it yourself

Centre one element exactly in the middle of the screen (both axes).

Solution

```
.wrapper { display: flex; justify-content: center; align-items: center; min-height: 100vh; }
/* or */
.wrapper { display: grid; place-items: center; min-height: 100vh; }
```

Drill 13: Make three cards in a row of equal width with 20px gaps that stack on

Note: flex: 1 shares the space equally.

Try it yourself

Make three cards in a row of equal width with 20px gaps that stack on screens under 700px.

Solution

```
.cards { display: flex; gap: 20px; }
.card { flex: 1; }
@media (max-width: 700px) {
  .cards { flex-direction: column; }
}
```

Drill 14: Make a responsive card grid: as many 250px-minimum columns as fit, grow

Note: No media query needed — the single most useful Grid line.

Try it yourself

Make a responsive card grid: as many 250px-minimum columns as fit, growing to fill.

Solution

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

Drill 15: Build a page layout with header, sidebar (250px), content and footer u

Note: Areas let you reorder visually without touching HTML.

Try it yourself

Build a page layout with header, sidebar (250px), content and footer using Grid areas; sidebar under content on mobile.

Solution

```
.page {
  display: grid; min-height: 100vh;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas: "header header" "sidebar main" "footer footer";
}
header { grid-area: header; } aside { grid-area: sidebar; } main { grid-area: main; } footer
{ grid-area: footer; }
@media (max-width: 768px) {
  .page { grid-template-columns: 1fr; grid-template-areas: "header" "main" "sidebar"
"footer"; }
}
```

Drill 16: Make a sticky header that stays at the top when scrolling, above other

Note: sticky needs a top value; z-index keeps it over content.

Try it yourself

Make a sticky header that stays at the top when scrolling, above other content.

Solution

```
header { position: sticky; top: 0; z-index: 100; background: #fff; }
```

Drill 17: Place a red notification badge on the top-right corner of an icon

Note: The parent must be positioned (relative) for absolute children to anchor to it.

Try it yourself

Place a red notification badge on the top-right corner of an icon.

Solution

```
.icon { position: relative; display: inline-block; }
.badge {
  position: absolute; top: -6px; right: -6px;
  background: red; color: #fff; font-size: 12px;
  min-width: 18px; height: 18px; border-radius: 9px;
  display: grid; place-items: center;
}
```

Drill 18: Make a footer that stays at the bottom of the screen when the page is

Note: main grows to fill; footer is pushed to the bottom. Not fixed positioning.

Try it yourself

Make a footer that stays at the bottom of the screen when the page is short, but moves down when content is long.

Solution

```
body { min-height: 100vh; display: flex; flex-direction: column; margin: 0; }
main { flex: 1; }
```

Drill 19: Truncate a long title to one line with an ellipsis

Note: All three properties are required for the single-line version.

Try it yourself

Truncate a long title to one line with an ellipsis.

Solution

```
.title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
/* multi-line (3 lines): */
.excerpt { display: -webkit-box; -webkit-line-clamp: 3; -webkit-box-orient: vertical;
overflow: hidden; }
```

Drill 20: Make a loading spinner with pure CSS

Note: One coloured border side + rotation = spinner.

Try it yourself

Make a loading spinner with pure CSS.

Solution

```
.spinner {
  width: 40px; height: 40px;
  border: 4px solid #eee; border-top-color: #1f77c4;
  border-radius: 50%;
  animation: spin 1s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
```

Drill 21: Fade and slide a card up when it appears

Note: both keeps the start state before and the end state after.

Try it yourself

Fade and slide a card up when it appears.

Solution

```
.card { animation: rise .5s ease-out both; }
@keyframes rise {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: none; }
}
```

Drill 22: Make images never overflow their container and remove the small gap un

Note: Images are inline by default, which leaves a baseline gap; block removes it.

Try it yourself

Make images never overflow their container and remove the small gap under them.

Solution

```
img { max-width: 100%; height: auto; display: block; }
```

Drill 23: Define colour variables for a theme and a dark mode that switches auto

Note: Change the variables once; everything using them updates.

Try it yourself

Define colour variables for a theme and a dark mode that switches automatically.

Solution

```
:root { --bg: #fff; --text: #222; --primary: #1f77c4; }
@media (prefers-color-scheme: dark) {
  :root { --bg: #121212; --text: #eee; --primary: #64b5f6; }
}
body { background: var(--bg); color: var(--text); }
a { color: var(--primary); }
```

Drill 24: Style the first letter of the first paragraph as a big drop cap and se

Note: Pseudo-elements use two colons.

Try it yourself

Style the first letter of the first paragraph as a big drop cap and select text in yellow.

Solution

```
p:first-of-type::first-letter { font-size: 3em; float: left; line-height: 1; margin-right: 6px; }
::selection { background: yellow; }
```

Drill 25: Add a decorative arrow after every external link without changing the

Note: Attribute selector ^= means 'starts with'.

Try it yourself

Add a decorative arrow after every external link without changing the HTML.

Solution

```
a[href^="http"]::after { content: "↗"; font-size: .8em; }
```

Drill 26: Make a fluid heading that is never smaller than 24px, never larger than 48px

Note: clamp(min, preferred, max).

Try it yourself

Make a fluid heading that is never smaller than 24px, never larger than 48px, and scales with the screen.

Solution

```
h1 { font-size: clamp(24px, 5vw, 48px); }
```

Drill 27: Make form inputs full width, with padding, a visible focus ring, and a

Note: font: inherit makes inputs use the page font. :user-invalid only fires after interaction (fallback :invalid).

Try it yourself

Make form inputs full width, with padding, a visible focus ring, and a red border when invalid after typing.

Solution

```
input, textarea { width: 100%; padding: 10px; border: 1px solid #bbb; border-radius: 6px;
font: inherit; }
input:focus { outline: 2px solid #1f77c4; outline-offset: 2px; }
input:user-invalid { border-color: #c0392b; }
```

Drill 28: Create a mobile hamburger menu that opens without JavaScript

Note: The ~ sibling combinator reacts to the checkbox state.

Try it yourself

Create a mobile hamburger menu that opens without JavaScript.

Solution

```
.menu-toggle { display: none; }
@media (max-width: 768px) {
  .menu-toggle { display: block; }
  .nav-links { display: none; flex-direction: column; }
  #menu:checked ~ .nav-links { display: flex; }
}
/* HTML: <input type="checkbox" id="menu" hidden> <label for="menu" class="menu-toggle">☰</label> <ul class="nav-links">... */
```

Drill 29: Give two overlapping absolutely positioned boxes and put the first one

Note: Higher z-index is on top; only works on positioned elements.

Try it yourself

Give two overlapping absolutely positioned boxes and put the first one on top.

Solution

```
.a, .b { position: absolute; width: 100px; height: 100px; }
.a { background: red; z-index: 2; }
.b { background: blue; left: 40px; top: 40px; z-index: 1; }
```

Drill 30: Print styles: hide the navigation and show link URLs when the page is

Note: `attr()` reads an attribute into generated content.

Try it yourself

Print styles: hide the navigation and show link URLs when the page is printed.

Solution

```
@media print {  
  nav, .no-print { display: none; }  
  a::after { content: " (" attr(href) ")"; }  
  body { color: #000; background: #fff; }  
}
```

109. Exercise Workbook C — JavaScript Drills (40 exercises with solutions)

Solve in the browser console or a scratch file. Drills 1–20 are language basics, 21–30 arrays and objects, 31–40 DOM and async. Do not skip: interview screenings ask exactly these.

Drill 1: Swap two variables a and b without a temporary variable

Note: Array destructuring.

Try it yourself

Swap two variables a and b without a temporary variable.

Solution

```
[a, b] = [b, a];
```

Drill 2: Check if a number is even

Note: Remainder 0 when divided by 2.

Try it yourself

Check if a number is even.

Solution

```
const isEven = n => n % 2 === 0;
```

Drill 3: Convert Celsius to Fahrenheit

Note: Formula $F = C \times 9/5 + 32$.

Try it yourself

Convert Celsius to Fahrenheit.

Solution

```
const toF = c => c * 9 / 5 + 32;
```

Drill 4: Return the larger of two numbers without Math**Note:** Ternary. **Try it yourself**

Return the larger of two numbers without Math.max.

Solution

```
const max = (a, b) => a > b ? a : b;
```

Drill 5: Return 'child', 'teen' or 'adult' from an age**Note:** Guard clauses; the first matching return wins. **Try it yourself**

Return 'child', 'teen' or 'adult' from an age.

Solution

```
function group(age) {
  if (age < 13) return "child";
  if (age < 20) return "teen";
  return "adult";
}
```

Drill 6: Print the multiplication table of 7 from 1 to 10**Note:** for loop with template literal. **Try it yourself**

Print the multiplication table of 7 from 1 to 10.

Solution

```
for (let i = 1; i <= 10; i++) console.log(class="v">`7 x ${i} = ${7 * i}`);
```

Drill 7: Sum the digits of a number (e

Note: Convert to string, split to digits, add.

Try it yourself

Sum the digits of a number (e.g. 1234 → 10).

Solution

```
const sumDigits = n => String(n).split("").reduce((s, d) => s + Number(d), 0);
```

Drill 8: Reverse a string

Note: Strings have no reverse; arrays do.

Try it yourself

Reverse a string.

Solution

```
const reverse = s => s.split("").reverse().join("");
class="c">// or: [...s].reverse().join("")
```

Drill 9: Check if a word is a palindrome ignoring case and spaces

Note: Clean first, then compare with its reverse.

Try it yourself

Check if a word is a palindrome ignoring case and spaces.

Solution

```
const isPal = s => { const c = s.toLowerCase().replace(/[^a-z0-9]/g, ""); return c === [...c].reverse().join(""); };
```

Drill 10: Count vowels in a string

Note: match with g returns all matches or null.

 Try it yourself

Count vowels in a string.

Solution

```
const vowels = s => (s.match(/[aeiou]/gi) || []).length;
```

Drill 11: Capitalise the first letter of every word

Note: split → map → join is the standard text pipeline.

 Try it yourself

Capitalise the first letter of every word.

Solution

```
const titleCase = s => s.split(" ").map(w => w[0].toUpperCase() +  
w.slice(1).toLowerCase()).join(" ");
```

Drill 12: Generate a random integer between min and max inclusive

Note: Math.random gives [0,1); scale, floor, shift.

 Try it yourself

Generate a random integer between min and max inclusive.

Solution

```
const rand = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
```

Drill 13: Return the factorial of n (loop and recursion)

Note: Both give 120 for 5.

 Try it yourself

Return the factorial of n (loop and recursion).

Solution

```
const fact = n => { let r = 1; for (let i = 2; i <= n; i++) r *= i; return r; };
const factR = n => n <= 1 ? 1 : n * factR(n - 1);
```

Drill 14: Return the first n Fibonacci numbers

Note: Each number is the sum of the previous two.

 Try it yourself

Return the first n Fibonacci numbers.

Solution

```
function fib(n) { const a = [0, 1]; for (let i = 2; i < n; i++) a.push(a[i-1] + a[i-2]);
return a.slice(0, n); }
```

Drill 15: Check if a number is prime

Note: Only test divisors up to the square root.

 Try it yourself

Check if a number is prime.

Solution

```
function isPrime(n) {
  if (n < 2) return false;
  for (let i = 2; i <= Math.sqrt(n); i++) if (n % i === 0) return false;
  return true;
}
```

Drill 16: Write FizzBuzz for 1-15 as an array

Note: Check 15 first.

 Try it yourself

Write FizzBuzz for 1-15 as an array.

Solution

```
Array.from({ length: 15 }, (_, i) => { const n = i + 1; return n % 15 === 0 ? "FizzBuzz" : n % 3 === 0 ? "Fizz" : n % 5 === 0 ? "Buzz" : n; });
```

Drill 17: Format a number as Egyptian currency with two decimals

Note: Intl handles separators and symbols.

 Try it yourself

Format a number as Egyptian currency with two decimals.

Solution

```
const egp = n => n.toLocaleString("en-EG", { style: "currency", currency: "EGP" });
class="c">// egp(1234.5) → "EGP 1,234.50"
```

Drill 18: Return how many days until a given date

Note: Subtracting dates gives milliseconds; 86,400,000 ms in a day.

 Try it yourself

Return how many days until a given date.

Solution

```
const daysUntil = d => Math.ceil((new Date(d) - Date.now()) / 86400000);
```

Drill 19: Write a function that runs another function n times

Note: Higher-order function taking a callback.

 Try it yourself

Write a function that runs another function n times.

Solution

```
const times = (n, fn) => { for (let i = 0; i < n; i++) fn(i); };
times(3, i => console.log("hi", i));
```

Drill 20: Create a counter with private state using a closure

Note: c is invisible outside; only the returned functions can touch it.

 Try it yourself

Create a counter with private state using a closure.

Solution

```
function counter() { let c = 0; return { inc: () => ++c, get: () => c }; }
const k = counter(); k.inc(); k.inc(); k.get(); class="c">// 2
```

Drill 21: Find the largest number in an array

Note: Spread the array into Math.max.

 Try it yourself

Find the largest number in an array.

Solution

```
Math.max(...nums);
class="c">// or nums.reduce((m, n) => n > m ? n : m)
```

Drill 22: Remove duplicates from an array

Note: Set keeps unique values.

 Try it yourself

Remove duplicates from an array.

Solution

```
[...new Set(arr)]
```

Drill 23: Count how many times each word appears in a sentence

Note: reduce into an object.

Try it yourself

Count how many times each word appears in a sentence.

Solution

```
const counts = sentence.toLowerCase().split(/\s+/).reduce((acc, w) => { acc[w] = (acc[w] || 0) + 1; return acc; }, {});
```

Drill 24: Get the total price of a cart: [{price, qty}]

Note: Always pass the initial 0.

Try it yourself

Get the total price of a cart: [{price, qty}].

Solution

```
const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0);
```

Drill 25: Return only the names of users older than 18, sorted alphabetically

Note: filter → map → sort chain.

Try it yourself

Return only the names of users older than 18, sorted alphabetically.

Solution

```
users.filter(u => u.age > 18).map(u => u.name).sort((a, b) => a.localeCompare(b));
```

Drill 26: Group products by category into an object of arrays

Note: `||=` creates the array the first time.

Try it yourself

Group products by category into an object of arrays.

Solution

```
const byCat = products.reduce((acc, p) => { (acc[p.category] ||= []).push(p); return acc; }, {});
class="c">// or Object.groupBy(products, p => p.category)
```

Drill 27: Flatten `[[1,2],[3,[4]]]` to `[1,2,3,4]`

Note: Infinity flattens any depth.

Try it yourself

Flatten `[[1,2],[3,[4]]]` to `[1,2,3,4]`.

Solution

```
arr.flat(Infinity)
```

Drill 28: Split an array into chunks of size n

Note: slice windows.

Try it yourself

Split an array into chunks of size n.

Solution

```
const chunk = (a, n) => Array.from({ length: Math.ceil(a.length / n) }, (_, i) => a.slice(i * n, i * n + n));
```


Drill 29: Deep-clone an object and prove the copy is independent

Note: Spread would share the nested address object.

Try it yourself

Deep-clone an object and prove the copy is independent.

Solution

```
const copy = structuredClone(obj); copy.address.city = "X"; console.log(obj.address.city !== "X"); class="c">// true
```

Drill 30: Merge two objects where the second overrides, and remove a key without

Note: Rest in destructuring omits keys.

Try it yourself

Merge two objects where the second overrides, and remove a key without mutating.

Solution

```
const merged = { ...defaults, ...options };  
const { password, ...safeUser } = user; class="c">// safeUser has no password
```

Drill 31: Change the text and colour of #title when #btn is clicked

Note: Select, listen, change.

Try it yourself

Change the text and colour of #title when #btn is clicked.

Solution

```
document.querySelector("#btn").addEventListener("click", () => {
  const t = document.querySelector("#title");
  t.textContent = "Clicked!";
  t.style.color = "crimson";
});
```

Drill 32: Toggle a

Note: toggle returns the new state.

 Try it yourself

Toggle a `.dark` class on body with a button and remember it in localStorage.

Solution

```
const btn = document.querySelector("#theme");
if (localStorage.getItem("dark") === "1") document.body.classList.add("dark");
btn.addEventListener("click", () => {
  const on = document.body.classList.toggle("dark");
  localStorage.setItem("dark", on ? "1" : "0");
});
```

Drill 33: Render an array of names as list items inside #list

Note: map to strings, join, set innerHTML (safe here because the data is yours).

 Try it yourself

Render an array of names as list items inside `#list`.

Solution

```
const names = ["Ahmed", "Sara", "Omar"];
document.querySelector("#list").innerHTML = names.map(n => `<li>${n}</li>`).join("");
```

Drill 34: Live character counter for a textarea with max 200

Note: input fires on every keystroke.

Try it yourself

Live character counter for a textarea with max 200.

Solution

```
const ta = document.querySelector("textarea"), out = document.querySelector("#count");
ta.addEventListener("input", () => out.textContent = `v">`${ta.value.length}/200``);
```

Drill 35: Delete any list item when its × button is clicked, including items add

Note: Event delegation on the parent.

Try it yourself

Delete any list item when its × button is clicked, including items added later.

Solution

```
list.addEventListener("click", e => {
  if (e.target.matches(".del")) e.target.closest("li").remove();
});
```

Drill 36: Validate a form: show an error under the email field if invalid, other

Note: FormData collects all named fields.

Try it yourself

Validate a form: show an error under the email field if invalid, otherwise log the data and reset.

Solution

```
form.addEventListener("submit", e => {
  e.preventDefault();
  const data = Object.fromEntries(new FormData(form));
  const err = form.querySelector(".error");
  if (!/^\S+@\S+\.\S+$/.test(data.email)) { err.textContent = "Invalid email"; return; }
  err.textContent = "";
  console.log(data); form.reset();
});
```

Drill 37: Show a countdown from 10 to 0 in #timer, one number per second, then s

Note: Always clear the interval.

Try it yourself

Show a countdown from 10 to 0 in #timer, one number per second, then show 'Go!'.

Solution

```
let n = 10; const el = document.querySelector("#timer");
el.textContent = n;
const id = setInterval(() => {
  n--; el.textContent = n > 0 ? n : "Go!";
  if (n <= 0) clearInterval(id);
}, 1000);
```

Drill 38: Fetch https://jsonplaceholder

Note: Check res.ok; fetch does not throw on 404.

Try it yourself

Fetch https://jsonplaceholder.typicode.com/todos/1 and print its title; handle errors.

Solution

```
async function load() {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");
    if (!res.ok) throw new Error(res.status);
    const todo = await res.json();
    console.log(todo.title);
  } catch (e) { console.error("Failed:", e.message); }
}
load();
```

Drill 39: Fetch users and posts at the same time and print how many of each

Note: Promise.all runs them in parallel.

 Try it yourself

Fetch users and posts at the same time and print how many of each.

Solution

```
const [users, posts] = await Promise.all([
  fetch("https://jsonplaceholder.typicode.com/users").then(r => r.json()),
  fetch("https://jsonplaceholder.typicode.com/posts").then(r => r.json()),
]);
console.log(users.length, posts.length);
```

Drill 40: Write a debounce helper and use it on a search input

Note: Waits until typing pauses.

 Try it yourself

Write a debounce helper and use it on a search input.

Solution

```
const debounce = (fn, ms = 300) => { let t; return (...args) => { clearTimeout(t); t =
  setTimeout(() => fn(...args), ms); }; };
input.addEventListener("input", debounce(e => search(e.target.value)));
```

110. Exercise Workbook D — SQL and Back-End Drills (20 exercises with solutions)

Assume tables users(id, name, email, city, created_at), products(id, name, price, category, in_stock), orders(id, user_id, total, status, created_at). Write the SQL, then the Express route where asked.

Drill 1: Get all products cheaper than 20, most expensive first

Note: WHERE filters rows; ORDER BY sorts.

Try it yourself

Get all products cheaper than 20, most expensive first.

Solution

```
SELECT * FROM products WHERE price < 20 ORDER BY price DESC;
```

Drill 2: Get the names and emails of users from Giza

Note: Strings use single quotes in SQL.

Try it yourself

Get the names and emails of users from Giza.

Solution

```
SELECT name, email FROM users WHERE city = 'Giza';
```

Drill 3: Find products whose name contains 'cake', case-insensitive

Note: % is a wildcard; ILIKE ignores case (Postgres).

 Try it yourself

Find products whose name contains 'cake', case-insensitive.

Solution

```
SELECT * FROM products WHERE name ILIKE '%cake%';
```

Drill 4: Insert a new product and return its id

Note: RETURNING avoids a second query.

 Try it yourself

Insert a new product and return its id.

Solution

```
INSERT INTO products (name, price, category) VALUES ('Croissant', 12, 'bakery') RETURNING id;
```

Drill 5: Increase all drinks prices by 5%

Note: Never forget WHERE.

 Try it yourself

Increase all drinks prices by 5%.

Solution

```
UPDATE products SET price = price * 1.05 WHERE category = 'drinks';
```

Drill 6: Delete orders that are cancelled and older than 30 days

Note: Run a SELECT with the same WHERE first.

 Try it yourself

Delete orders that are cancelled and older than 30 days.

Solution

```
DELETE FROM orders WHERE status = 'cancelled' AND created_at < NOW() - INTERVAL '30 days';
```

Drill 7: Count the products in each category, only categories with more than 2

Note: HAVING filters groups.

 Try it yourself

Count the products in each category, only categories with more than 2.

Solution

```
SELECT category, COUNT(*) AS n FROM products GROUP BY category HAVING COUNT(*) > 2;
```

Drill 8: Average order total per user with the user's name

Note: JOIN then GROUP BY.

 Try it yourself

Average order total per user with the user's name.

Solution

```
SELECT u.name, ROUND(AVG(o.total), 2) AS avg_total  
FROM orders o JOIN users u ON u.id = o.user_id  
GROUP BY u.id, u.name;
```

Drill 9: List all users and their number of orders, including users with zero

Note: LEFT JOIN keeps users with no orders; COUNT(o.id) counts 0 for them.

 Try it yourself

List all users and their number of orders, including users with zero.

Solution

```
SELECT u.name, COUNT(o.id) AS orders
FROM users u LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name ORDER BY orders DESC;
```

Drill 10: The 5 most recent paid orders with customer email

Note: LIMIT after ORDER BY.

 Try it yourself

The 5 most recent paid orders with customer email.

Solution

```
SELECT o.id, o.total, o.created_at, u.email
FROM orders o JOIN users u ON u.id = o.user_id
WHERE o.status = 'paid'
ORDER BY o.created_at DESC LIMIT 5;
```

Drill 11: Products more expensive than the average price

Note: Subquery in WHERE.

 Try it yourself

Products more expensive than the average price.

Solution

```
SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products);
```

Drill 12: Users who never ordered

Note: Two classic ways.

 Try it yourself

Users who never ordered.

Solution

```
SELECT * FROM users WHERE id NOT IN (SELECT DISTINCT user_id FROM orders);
-- or: LEFT JOIN ... WHERE o.id IS NULL
```

Drill 13: Page 3 of products, 20 per page

Note: OFFSET = (page – 1) × limit.

 Try it yourself

Page 3 of products, 20 per page.

Solution

```
SELECT * FROM products ORDER BY id LIMIT 20 OFFSET 40;
```

Drill 14: Create a reviews table linked to users and products with a rating 1-5

Note: UNIQUE pair = one review per user per product.

 Try it yourself

Create a reviews table linked to users and products with a rating 1-5.

Solution

```
CREATE TABLE reviews (
  id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  product_id INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  body TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (user_id, product_id)
);
```

Drill 15: Total revenue per month this year

Note: DATE_TRUNC buckets timestamps.

 Try it yourself

Total revenue per month this year.

Solution

```
SELECT DATE_TRUNC('month', created_at) AS month, SUM(total)
FROM orders WHERE status = 'paid' AND created_at >= DATE_TRUNC('year', NOW())
GROUP BY month ORDER BY month;
```

Drill 16: Express route: GET /api/products? category=&minPrice= using pg safely

Note: Build the parameter list dynamically; never interpolate values into the SQL string.

 Try it yourself

Express route: GET /api/products?category=&minPrice= using pg safely.

Solution

```
app.get('/api/products', async (req, res, next) => {
  try {
    const { category, minPrice = 0 } = req.query;
    const params = [minPrice];
    let sql = 'SELECT * FROM products WHERE price >= $1';
    if (category) { params.push(category); sql += ` AND category = ${params.length}`; }
    const { rows } = await pool.query(sql + ' ORDER BY id', params);
    res.json(rows);
  } catch (e) { next(e); }
});
```

Drill 17: Express route: POST /api/orders that creates an order and its items in

Note: If any insert fails, nothing is saved.

 Try it yourself

Express route: POST /api/orders that creates an order and its items in one transaction.

Solution

```
app.post('/api/orders', auth, async (req, res, next) => {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const { rows: [order] } = await client.query('INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING *', [req.user.id, req.body.total]);
    for (const it of req.body.items)
      await client.query('INSERT INTO order_items (order_id, product_id, qty) VALUES ($1, $2, $3)', [order.id, it.productId, it.qty]);
    await client.query('COMMIT');
    res.status(201).json(order);
  } catch (e) { await client.query('ROLLBACK'); next(e); }
  finally { client.release(); }
});
```

Drill 18: Express: protect PATCH /api/orders/:id so only the owner or an admin c

Note: Authorisation happens on the server, every time.

 Try it yourself

Express: protect PATCH /api/orders/:id so only the owner or an admin can change it.

Solution

```
app.patch('/api/orders/:id', auth, async (req, res) => {
  const { rows: [order] } = await pool.query('SELECT * FROM orders WHERE id = $1', [req.params.id]);
  if (!order) return res.status(404).json({ error: 'Not found' });
  if (order.user_id !== req.user.id && req.user.role !== 'admin') return
  res.status(403).json({ error: 'Forbidden' });
  const { rows: [updated] } = await pool.query('UPDATE orders SET status = $1 WHERE id = $2 RETURNING *', [req.body.status, order.id]);
  res.json(updated);
});
```

Drill 19: Express: a middleware that logs method, URL, status and duration of ev

Note: The finish event fires when the response is sent.

 Try it yourself

Express: a middleware that logs method, URL, status and duration of every request.

Solution

```
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => console.log(req.method, req.originalUrl, res.statusCode, `${Date.now() - start}ms`));
  next();
});
```

Drill 20: Express: register route with validation, hashing and duplicate-email h

Note: Validate → hash → insert → map DB errors to HTTP codes.

 Try it yourself

Express: register route with validation, hashing and duplicate-email handling.

Solution

```
app.post('/api/register', async (req, res, next) => {
  const { email = '', password = '' } = req.body;
  if (!/^S+@S+\.S+$/i.test(email) || password.length < 8) return
  res.status(400).json({ error: 'Invalid email or password too short' });
  try {
    const hash = await bcrypt.hash(password, 12);
    const { rows: [user] } = await pool.query('INSERT INTO users (email, password)
VALUES ($1, $2) RETURNING id, email', [email.toLowerCase(), hash]);
    res.status(201).json(user);
  } catch (e) {
    if (e.code === '23505') return res.status(409).json({ error: 'Email already
registered' });
    next(e);
  }
});
```

111. Glossary — 128 Terms Explained Simply (English + Arabic)

Every technical word used in this book, in alphabetical order, with a one-line meaning and a short Arabic hint. When a chapter uses a word you do not know, look here first.

Term	Meaning	بالعربي
API	Application Programming Interface — a set of URLs a program can call to get or change data (usually JSON).	الواجهة التي يبرامج بيكم بيها برنامج ثاني
Argument	The actual value you pass to a function when calling it: greet("Ahmed").	القيمة التي بتبعها للفانكشن
Array	An ordered list of values, written with square brackets: [1, 2, 3].	قائمة مرتبة من القيم
Async / asynchronous	Work that finishes later (network, timers) while the rest of the program continues.	شغل بيخلص بعدين من غير ما يوقف البرنامج
Attribute	Extra information on an HTML tag: href, src, class, id.	معلومة إضافية على الوسم
Authentication	Proving who you are (login).	تسجيل الدخول: إثبات الهوية
Authorisation	Checking what you are allowed to do (admin vs user).	الصلاحيات: مسموحلك تعمل إيه
Backend / server side	The part that runs on the server: database, business rules, authentication.	الجزء الذي شغال على السيرفر
bcrypt	Library that hashes passwords slowly and with a salt.	مكتبة لتشفير الباسورد باتجاه واحد
Block element	HTML element that starts on a new line and fills the width (div, p, h1).	عنصر بياخد سطر لوحده
Boolean	A true/false value.	قيمة صح أو غلط
Bootstrap	CSS framework with ready-made classes and components.	إطار عمل CSS جاهز
Box model	Content + padding + border + margin of every element.	طبقات كل عنصر: المحتوى، الحشو، الإطار، الهامش
Breakpoint	A screen width where a media query changes the layout.	عرض الشاشة الذي التصميم بيتغير عنده
Browser	Program that shows web pages: Chrome, Firefox, Safari, Edge.	المتصفح
Bug	A mistake in code that causes wrong behaviour.	خطأ في الكود
Bundle / bundler	Tool (Vite, webpack) that combines many files into optimised ones for production.	أداة تجميع الملفات للنشر
Cache	A saved copy of data to avoid fetching it again.	نسخة محفوظة عشان السرعة
Callback	A function passed to another function to be called later.	فانكشن بتتبع لفانكشن ثانية عشان تنفذ بعدين
Cascade	The CSS rules for deciding which style wins.	قواعد تحديد أي ستايل يكسب
Class (CSS)	A reusable name you put on elements to style them: class="card".	اسم بتحطه على عناصر عشان تنسقها

Term	Meaning	بالعربي
Class (JS)	A blueprint for creating objects with the same properties and methods.	قالب لإنشاء كائنات
CLI / terminal	Text window where you type commands (npm, git, node).	شاشة الأوامر
Client	The user's browser or app that sends requests.	المتصفح اللي يطلب
Closure	A function that remembers variables from where it was created.	فانكشن فاكدة المتغيرات اللي حوالها
Commit	A saved snapshot in Git with a message.	نسخة محفوظة في Git
Component	A reusable piece of UI (React function returning JSX).	قطعة واجهة قابلة لإعادة الاستخدام
Console	DevTools panel where console.log prints and errors appear.	لوحة الرسائل والأخطاء
Cookie	Small text the server stores in the browser and receives back with every request.	بيانات صغيرة بيخزنها السيرفر في المتصفح
CORS	Browser rule blocking cross-origin API reads unless the server allows them.	حماية المتصفح من طلبات نطاق ثاني
CRUD	Create, Read, Update, Delete — the four data operations.	إنشاء، قراءة، تعديل، حذف
CSS	Cascading Style Sheets — the language for how pages look.	لغة تنسيق الشكل
Database	Organised storage for data that survives restarts (PostgreSQL, MySQL, MongoDB).	قاعدة البيانات
Debugging	Finding and fixing bugs using logs, breakpoints and reading errors.	تتبع الأخطاء وإصلاحها
Declaration (CSS)	One property: value pair, e.g. color: red;.	خاصية وقيمتها
Dependency	A package your project needs (listed in package.json).	حزمة مشروعك محتاجها
Deploy	Put your app on a server so the world can use it.	نشر المشروع أونلاين
DevTools	Browser developer tools (F12): Elements, Console, Network, Sources.	أدوات المطور في المتصفح
DNS	System translating domain names into IP addresses.	تحويل اسم الموقع لعنوان IP
DOM	Document Object Model — the tree of objects representing the page that JS can change.	شجرة عناصر الصفحة اللي الجافاسكريبت بتغيرها
Domain	A website's name: example.com.	اسم الموقع
EJS	Template language mixing HTML with JavaScript on the server.	قوالب HTML بجافاسكريبت على السيرفر
Element	One HTML tag with its content: Hi .	وسم بمحتواه

Term	Meaning	بالعربي
Endpoint	One URL + method of an API: GET /api/products.	رابط واحد في الـ API
Environment variable	A setting read from the system (process.env.PORT), used for secrets.	إعداد خارج الكود للأسرار
ES6 / ES2015+	Modern JavaScript versions: let/const, arrows, classes, modules, promises.	إصدارات الجافاسكريبت الحديثة
Event	Something that happens: click, keydown, submit, load.	حدث
Event loop	Mechanism letting JS run async callbacks when the main work is done.	آلية تشغيل الأكواد غير المتزامنة
Express	Node framework for building servers and APIs.	إطار عمل لعمل سيرفر بـ Node
fetch	Browser function for HTTP requests; returns a promise.	دالة طلب بيانات من سيرفر
Flexbox	CSS layout for arranging items in a row or column.	تخطيط صف أو عمود
Foreign key	Column holding another table's primary key to link rows.	عمود يربط جدول بجدول ثاني
Framework	Large tool giving structure to your app (React, Express, Bootstrap).	إطار عمل
Frontend / client side	The part in the browser: HTML, CSS, JS, React.	الجزء اللي في المتصفح
Function	A named, reusable block of code that can take inputs and return an output.	كود باسم ينفذ وقت ما تناديه
Git	Version control system tracking changes to code.	نظام تتبع تغييرات الكود
GitHub	Website hosting Git repositories; your portfolio.	موقع استضافة مشاريع Git
Grid	CSS layout in two dimensions (rows and columns).	تخطيط صفوف وأعمدة
Hash	One-way scramble of data (passwords).	تشفير باتجاه واحد
Hoisting	JS moving declarations to the top of their scope before running.	رفع التعريفات لأول النطاق
Hook	React function starting with use that adds state or effects (useState, useEffect).	دوال React الخاصة
HTML	HyperText Markup Language — the structure and content of pages.	لغة هيكل الصفحة
HTTP / HTTPS	Protocol browsers and servers use to talk; S = encrypted.	بروتوكول التواصل؛ S يعني مشفر
Immutability	Not changing existing data; creating new copies instead (React state).	عدم تعديل البيانات وإنشاء نسخة جديدة
Index (DB)	Structure that makes searching a column fast.	فهرس لتسريع البحث
Index (array)	Position of an item, starting at 0.	رقم مكان العنصر ويبدأ من 0

Term	Meaning	بالعربي
Inline element	HTML element flowing inside text (span, a, strong).	عنصر جوه السطر
IP address	Numeric address of a computer on the network.	عنوان الجهاز على الشبكة
JavaScript	Programming language of the browser (and Node on servers).	لغة برمجة المتصفح
JSON	Text format for data: {"name": "A"}.	صيغة نصية لتبادل البيانات
JSX	HTML-like syntax inside JavaScript used by React.	كتابة HTML جوه جافاسكريبت
JWT	JSON Web Token — signed token proving identity in stateless APIs.	توكن موقع لإثبات الهوية
Key (React)	Unique prop on list items so React can track them.	معرف فريد لعناصر القائمة
Library	Collection of ready code you call (React, axios).	مكتبة كود جاهز
localStorage	Browser storage for strings that persists across visits.	تخزين في المتصفح يفضل موجود
Media query	CSS rule applied only at certain screen sizes.	قاعدة CSS لمقاس شاشة معين
Method	A function stored in an object (arr.push, user.greet).	فانكشن جوه كائن
Middleware	Express function running between request and route handler.	كود بيتنفذ قبل الرد على الطلب
Module	A file that exports code for other files to import.	ملف يبصّر كود لملفات تانية
Node.js	Runtime running JavaScript outside the browser.	تشغيل الجافاسكريبت على السيرفر
npm	Node package manager for installing libraries.	مدير حزم Node
null / undefined	Empty on purpose / not assigned yet.	فاضي عمدًا / لسه ماتحددش
Object	Collection of key: value pairs: { name: "A" }.	كائن من مفاتيح وقيم
ORM	Library mapping database tables to code objects (Prisma).	ربط الجداول بكائنات
Parameter	The named input in a function definition: function greet(name).	اسم المدخل في تعريف الفانكشن
Parse	Turn text into structured data (JSON.parse).	تحويل نص لبيانات
Port	Number identifying a service on a machine (3000, 5432, 80, 443).	رقم الخدمة على الجهاز
PostgreSQL	Free, powerful relational database.	قاعدة بيانات علائقية
Primary key	Unique id column of a table.	المعرف الفريد للصف
Promise	Object representing a future value (pending → fulfilled/rejected).	وعد بقيمة هتيجي بعدين

Term	Meaning	بالعربي
Props	Inputs passed to a React component.	مدخلات مكون React
Pseudo-class	Selector for a state: :hover, :focus, :nth-child.	محدد لحالة العنصر
Query string	Data after ? in a URL: ?page=2&sort=price.	البيانات بعد علامة الاستفهام في الرابط
React	Library for building UIs from components.	مكتبة بناء الواجهات
Refactor	Improve code structure without changing behaviour.	تحسين الكود من غير تغيير النتيجة
Regex	Regular expression — pattern for matching text.	نمط للبحث في النصوص
Render	Produce the visible output (browser painting; React returning JSX; EJS building HTML).	إنتاج الشكل النهائي
Repository (repo)	A project tracked by Git.	مشروع متتبع بـ Git
Request / response	What the client sends / what the server answers.	الطلب / الرد
Responsive	Layout that adapts to any screen size.	تصميم متجاوب مع كل الشاشات
REST	Conventions for designing HTTP APIs around resources.	قواعد تصميم الـ API
Route	A URL pattern the server or router answers.	مسار/رابط يبرد عليه السيرفر
Scope	Where a variable is visible.	نطاق رؤية المتغير
Selector	The part of a CSS rule choosing which elements to style.	الجزء اللي يختار العناصر
Semantic HTML	Tags that describe meaning: header, nav, article.	وسوم بتوصف المعنى
Server	Computer/program answering requests over the network.	الخادم
Session	Server-side memory of a logged-in user, referenced by a cookie.	جلسة المستخدم على السيرفر
SPA	Single Page Application — one HTML page whose content JS swaps (React).	تطبيق صفحة واحدة
Specificity	Weight of a selector deciding which CSS rule wins.	وزن المحدد
SQL	Language for querying relational databases.	لغة قواعد البيانات
SQL injection	Attack inserting SQL through inputs; prevented by parameters.	هجوم عن طريق حقن SQL
SSR	Server-Side Rendering — HTML built on the server (EJS, Next.js).	بناء الصفحة على السيرفر
State	Data that changes and drives the UI.	بيانات بتتغير ويتحرك الواجهة
Static site	Pages served as fixed files with no server logic.	موقع ملفات ثابتة

Term	Meaning	بالعربي
String	Text value in quotes.	نص
Syntax	The grammar rules of a language.	قواعد كتابة اللغة
Tag	The angle-bracket markers of HTML: opening and closing.	الوسم
Template literal	Backtick string with <code>\${expressions}</code> .	نص بعلامة backtick ويحيط متغيرات
this	The object a function is currently working on.	الكائن الحالي
Token	A string proving identity (JWT) or authorising API access.	رمز إثبات هوية
Transaction	Group of database statements that succeed or fail together.	مجموعة عمليات تنجح أو تفشل معاً
URL	Address of a resource: <code>https://site.com/path?query#hash</code> .	الرابط
Validation	Checking data is correct (format, length, required) — in browser and on server.	التحقق من البيانات
Variable	A named container for a value (let, const).	متغير
Viewport	The visible area of the browser window.	المساحة الظاهرة من الصفحة
Vite	Fast build tool and dev server for React and modern JS.	أداة بناء وتشغيل سريعة
XSS	Cross-Site Scripting — injecting scripts through user content; prevented by escaping.	حقن سكريبت ضار عبر محتوى المستخدم
z-index	CSS stacking order of positioned elements.	ترتيب الطبقات فوق بعض